

HINT:
The File Format

HINT: The File Format

Version 1.4

**Reflowable
Output
for T_EX**

Für meine Mutter

MARTIN RUCKERT *Munich University of Applied Sciences*

Second edition

The author has taken care in the preparation of this book, but makes no expressed or implied warranty of any kind and assumes no responsibility for errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

Ruckert, Martin.
HINT: The File Format
Includes index.
ISBN 979-854992684-4

Internet page <http://hint.userweb.mwn.de/hint/format.html> may contain current information about this book, downloadable software, and news.

Copyright © 2019, 2021 by Martin Ruckert

All rights reserved. Printed by Kindle Direct Publishing. This publication is protected by copyright, and permission must be obtained prior to any prohibited reproduction, storage in a retrieval system, or transmission in any form or by any means, electronic, mechanical, photocopying, recording, or likewise. To obtain permission to use material from this work, please submit a written request to Martin Ruckert, Hochschule München, Fakultät für Informatik und Mathematik, Lothstrasse 64, 80335 München, Germany.

ruckert@cs.hm.edu

ISBN-13: 979-854992684-4

First printing: August 2019

Second edition: August 2021

Last commit: Mon Jan 19 09:58:39 2026

Preface

Late in summer 2017, with my new C based `cweb` implementation of `TEX`[9] in hand[20][17][18], I started to write the first prototype of the `HINT` viewer. I basically made two copies of `TEX`: In the first copy, I replaced the *build_page* procedure by an output routine which used more or less the printing routines already available in `TEX`. This was the beginning of the `HINT` file format. In the second copy, I replaced `TEX`'s main loop by an input routine that would feed the `HINT` file more or less directly to `TEX`'s *build_page* procedure. And after replacing `TEX`'s *ship_out* procedure by a modified rendering routine of a dvi viewer that I had written earlier for my experiments with `TEX`'s Computer Modern fonts[16], I had my first running `HINT` viewer. My sabbatical during the following Fall term gave me time for “rapid prototyping” various features that I considered necessary for reflowable `TEX` output[19].

The textual output format derived from the original `TEX` debugging routines proved to be insufficient when I implemented a “page up” button because it did not support reading the page content “backwards”. As a consequence, I developed a compact binary file format that could be parsed easily in both directions. The `HINT` short file format was born. I stopped an initial attempt at eliminating the old textual format because it was so much nicer when debugging. Instead, I converted the long textual format into the short binary format as a preliminary step in the viewer. This was not a long term solution. When opening a big file, as produced from a 1000 pages `TEX` file, the parsing took several seconds before the first page would appear on screen. This delay, observed on a fast desktop PC, is barley tolerable, and the delay one would expect on a low-cost, low-power, mobile device seemed prohibitive. The consequence is simple: The viewer will need an input file in the short format; and to support debugging (or editing), separate programs are needed to translate the short format into the long format and back again. But for the moment, I did not bother to implement any of this but continued with unrestricted experimentation.

With the beginning of the Spring term 2018, I stopped further experiments with the `HINT` viewer and decided that I had to write down a clean design of the `HINT` file format. Or of both file formats? Professors are supposed to do research, and hence I tried an experiment: Instead of writing down a traditional language specification, I decided to stick with the “literate programming” paradigm[10] and write the present book. It describes and implements the `stretch` and `shrink` programs translating one file format into the other. As a side effect, it contains the underlying language specification. Whether this experiment is a success as a

language specification remains to be seen, and you should see for yourself. But the only important measure for the value of a scientific experiment is how much you can learn from it—and I learned a lot.

The whole project turned out to be much more difficult than I had expected. Early on, I decided that I would use a recursive descent parser for the short format and an $\text{LR}(k)$ parser for the long format. Of course, I would use `lex/flex` and `yacc/bison` to generate the $\text{LR}(k)$ parser, and so I had to extend the `cweb` tools[11] to support the corresponding source files.

About in mid May, after writing down about 100 pages, the first problems emerged that could not be resolved with my current approach. I had started to describe font definitions containing definitions of the interword glue and the default hyphen, and the declarative style of my exposition started to conflict with the sequential demands of writing an output file. So it was time for a first complete redesign. Two more passes over the whole book were necessary to find the concepts and the structure that would allow me to go forward and complete the book as you see it now.

While rewriting was on its way, many “nice ideas” were pruned from the book. For example, the initial idea of optimizing the `HINT` file while translating it was first reduced to just gathering statistics and then disappeared completely. The added code and complexity was just too distracting.

What you see before you is still a snapshot of the `HINT` file format because its development is still under way. We will know what features are needed for a reflowable $\text{T}_{\text{E}}\text{X}$ file format only after many people have started using the format. To use the format, the end-user will need implementations, and the implementer will need a language specification. The present book is the first step in an attempt to solve this “chicken or egg” dilemma.

München
August 20, 2019

Martin Ruckert

Contents

	Preface	v
	Contents	vii
1	Introduction	1
1.1	Glyphs	1
1.2	Scanning the Long Format	2
1.3	Parsing the Long Format	3
1.4	Writing the Short Format	4
1.5	Parsing the Short Format	8
1.6	Writing the Long Format	10
2	Data Types	13
2.1	Integers	13
2.2	Strings	14
2.3	Character Codes	16
2.4	Floating Point Numbers	20
2.5	Fixed Point Numbers	26
2.6	Dimensions	26
2.7	Extended Dimensions	28
2.8	Stretch and Shrink	31
3	Simple Nodes	35
3.1	Penalties	35
3.2	Languages	36
3.3	Rules	37
3.4	Kerns	40
3.5	Glue	42
4	Lists	47
4.1	Plain Lists	49
4.2	Texts	52
5	Composite Nodes	61
5.1	Boxes	61
5.2	Extended Boxes	64
5.3	Leaders	68
5.4	Baseline Skips	70
5.5	Ligatures	72
5.6	Discretionary breaks	74

5.7	Paragraphs	77
5.8	Mathematics	79
5.9	Adjustments	81
5.10	Tables	81
6	Extensions	85
6.1	Images	85
6.2	Positions, Outlines, Links, and Labels	96
6.3	Colors	110
6.4	Rotation	116
6.5	Unknown Extensions	116
7	Replacing TeX's Page Building Process	121
7.1	Stream Definitions	125
7.2	Stream Content	128
7.3	Page Template Definitions	129
7.4	Page Ranges	130
8	File Structure	137
8.1	Banner	137
8.2	Long Format Files	139
8.3	Short Format Files	140
8.4	Mapping a Short Format File to Memory	142
8.5	Compression	144
8.6	Reading Short Format Sections	146
8.7	Writing Short Format Sections	146
9	Directory Section	149
9.1	Directories in Long Format	149
9.2	Directories in Short Format	154
10	Definition Section	161
10.1	Maximum Values	162
10.2	Definitions	166
10.3	Parameter Lists	170
10.4	Fonts	172
10.5	References	176
11	Defaults	179
11.1	Integers	179
11.2	Dimensions	181
11.3	Extended Dimensions	181
11.4	Glue	182
11.5	Baseline Skips	183
11.6	Labels	184
11.7	Streams	184
11.8	Page Templates	184
11.9	Page Ranges	184
11.10	List, Texts, and Parameters	185
11.11	Colors	185

12	Content Section	187
13	Processing the Command Line	189
14	Error Handling and Debugging	195
	Appendix	197
A	Traversing Short Format Files	197
A.1	Lists	199
A.2	Glyphs	199
A.3	Penalties	199
A.4	Kerns	200
A.5	Extended Dimensions	200
A.6	Language	200
A.7	Rules	201
A.8	Glue	201
A.9	Boxes	201
A.10	Extended Boxes	202
A.11	Leaders	203
A.12	Baseline Skips	203
A.13	Ligatures	203
A.14	Discretionary breaks	204
A.15	Paragraphs	204
A.16	Mathematics	204
A.17	Adjustments	204
A.18	Tables	205
A.19	Images	205
A.20	Links	205
A.21	Streams	206
A.22	Colors	206
B	Reading Short Format Files Backwards	207
B.1	Floating Point Numbers	208
B.2	Extended Dimensions	208
B.3	Stretch and Shrink	209
B.4	Glyphs	209
B.5	Penalties	210
B.6	Kerns	210
B.7	Language	210
B.8	Rules	210
B.9	Glue	211
B.10	Boxes	212
B.11	Extended Boxes	213
B.12	Leaders	214
B.13	Baseline Skips	214
B.14	Ligatures	215
B.15	Discretionary breaks	215
B.16	Paragraphs	216
B.17	Mathematics	216

B.18	Images	216
B.19	Links and Labels	217
B.20	Colors	218
B.21	Plain Lists, Texts, and Parameter Lists	218
B.22	Adjustments	219
B.23	Tables	219
B.24	Stream Nodes	220
B.25	References	220
B.26	Unknown nodes	220
C	Code and Header Files	223
C.1	<code>basetypes.h</code>	223
C.2	<code>format.h</code>	224
C.3	<code>tables.c</code>	225
C.4	<code>get.h</code>	226
C.5	<code>get.c</code>	227
C.6	<code>put.h</code>	227
C.7	<code>put.c</code>	229
C.8	<code>lexer.l</code>	229
C.9	<code>parser.y</code>	230
C.10	<code>shrink.c</code>	231
C.11	<code>stretch.c</code>	232
C.12	<code>skip.c</code>	234
D	Format Definitions	237
D.1	Reading the Long Format	237
D.2	Writing the Long Format	238
D.3	Reading the Short Format	239
D.4	Writing the Short Format	240
	Crossreference of Code	243
	References	249
	Index	251

1 Introduction

This book defines a file format for reflowable text. Actually it describes two file formats: a long format that optimizes readability for human beings, and a short format that optimizes readability for machines and the use of storage space. Both formats use the concept of nodes and lists of nodes to describe the file content. Programs that process these nodes will likely want to convert the compressed binary representation of a node—the short format—or the lengthy textual representation of a node—the long format—into a convenient internal representation. So most of what follows is just a description of these nodes: their short format, their long format and sometimes their internal representation. Where as the description of the long and short external format is part of the file specification, the description of the internal representation is just informational. Different internal representations can be chosen based on the individual needs of the program.

While defining the format, I illustrate the processing of long and short format files by implementing two utilities: **shrink** and **stretch**. **shrink** converts the long format into the short format and **stretch** goes the other way.

There is also a prototype viewer for this file format and a special version of `TeX`[8] to produce output in this format. Both are not described here; a survey describing them can be found in [19].

1.1 Glyphs

Let's start with a simple and very common kind of node: a node describing a character. Because we describe a format that is used to display text, we are not so much interested in the character itself but we are interested in the specific glyph. In typography, a glyph is a unique mark to be placed on the page representing a character. For example the glyph representing the character 'a' can have many forms among them 'a', 'a', or 'a'. Such glyphs come in collections, called fonts, representing every character of the alphabet in a consistent way.

The long format of a node describing the glyph 'a' might look like this: “<glyph 97 *1>”. Here “97” is the character code which happens to be the ASCII code of the letter 'a' and “*1” is a font reference that stands for “Computer Modern Roman 10pt”. Reference numbers, as you can see, start with an asterisk reminiscent of references in the C programming language. The asterisk enables us to distinguish between ordinary numbers like “1” and references like “*1”.

To make this node more readable, we will see in section 2.3 that it is also possible to write “<glyph 'a' *1 (cmr10)>”. The latter form uses a comment “(cmr10)”, enclosed in parentheses, to give an indication of what kind of font happens to be font 1, and it uses “'a'”, the character enclosed in single quotes to denote the

ASCII code of ‘a’. But let’s keep things simple for now and stick with the decimal notation of the character code.

The rest is common for all nodes: a keyword, here “`glyph`”, and a pair of pointed brackets “`<...>`”.

Internally, we represent a glyph by the font number and the character number or character code. To store the internal representation of a glyph node, we define an appropriate structure type, named after the node with an uppercase first letter.

```
< hint types 1i > ≡ (1)
    typedef struct { uint32_t c; uint8_t f; } Glyph;
```

Used in 551dli, 553dlili, and 560dlx.

Let us now look at the program `shrink` and see how it will convert the long format description to the internal representation of the glyph and finally to a short format description.

1.2 Scanning the Long Format

First, `shrink` reads the input file and extracts a sequence of tokens. This is called “scanning”. We generate the procedure to do the scanning using the program `flex`[12] which is the GNU version of the common UNIX tool `lex`[13].

The input to `flex` is a list of pattern/action rules where the pattern is a regular expression and the action is a piece of C code. Most of the time, the C code is very simple: it just returns the right token number to the parser which we consider shortly.

The code that defines the tokens will be marked with a line ending in “`--- ==>`”. This symbol stands for “*Reading the long format*”. These code sequences define the syntactical elements of the long format and at the same time implement the reading process. All sections where that happens are preceded by a similar heading and for reference they are conveniently listed together starting on page 237.

Reading the Long Format:

--- ==>

```
< symbols 2ii > ≡ (2)
%token START "<"
%token END ">"
%token GLYPH "glyph"
%token < u > UNSIGNED
%token < u > REFERENCE
```

Used in 556dlvi.

You might notice that a small caps font is used for `START`, `END` or `GLYPH`. These are “terminal symbols” or “tokens”. Next are the scanning rules which define the connection between tokens and their textual representation.

```
< scanning rules 3iii > ≡ (3)
"<"          SCAN_START; return START;
">"          SCAN_END; return END;
glyph        return GLYPH;
```

```

0|[1-9][0-9]*  SCAN_UDEC(yytext); return UNSIGNED;
\*(0|[1-9][0-9]*)  SCAN_UDEC(yytext + 1); return REFERENCE;
[[:space:]]      ;
\([^()\n]*\)\n  ;

```

Used in 555dlv.

As we will see later, the macros starting with `SCAN_...` are scanning macros. Here `SCAN_UDEC` is a macro that converts the decimal representation that did match the given pattern to an unsigned integer value; it is explained in section 2.1. The macros `SCAN_START` and `SCAN_END` are explained in section 4.2.

The action “;” is a “do nothing” action; here it causes spaces or comments to be ignored. Comments start with an opening parenthesis and are terminated by a closing parenthesis or the end of line character. The pattern “[^()\\n]” is a negated character class that matches all characters except parentheses and the newline character. These are not allowed inside comments. For detailed information about the patterns used in a `flex` program, see the `flex` user manual[12].

1.3 Parsing the Long Format

Next, the tokens produced by the scanner are assembled into larger entities. This is called “parsing”. We generate the procedure to do the parsing using the program `bison`[12] which is the GNU version of the common UNIX tool `yacc`[13].

The input to `bison` is a list of parsing rules, called a “grammar”. The rules describe how to build larger entities from smaller entities. For a simple glyph node like “<glyph 97 *1>”, we need just these rules:

Reading the Long Format:

— — — $\Rightarrow$

<symbols 2ii > + $\equiv$ (4)

%**type** < u > start

%**type** < c > glyph

<parsing rules 5v > $\equiv$ (5)

glyph: UNSIGNED REFERENCE

{ \$\$c = \$1; REF(font_kind, \$2); \$\$f = \$2; };

content_node: start GLYPH glyph END { hput_tags(\$1, hput_glyph(&(\$3))); };

start: START { HPUTNODE; \$\$ = (uint32_t)(hpos++ - hstart); }

Used in 556dlvi.

You might notice that a slanted font is used for *glyph*, *content_node*, or *start*. These are “nonterminal symbols” and occur on the left hand side of a rule. On the right hand side of a rule you find nonterminal symbols, as well as terminal symbols and C code enclosed in braces.

Within the C code, the expressions `$1` and `$2` refer to the variables on the parse stack that are associated with the first and second symbol on the right hand side of the rule. In the case of our glyph node, these will be the values 97 and 1, respectively, as produced by the macro `SCAN_UDEC`. `$$` refers to the variable associated with the left hand side of the rule. These variables contain the internal

representation of the object in question. The type of the variable is specified by a mandatory **token** or optional **type** clause when we define the symbol. In the above **type** clause for *start* and *glyph*, the identifiers *u* and *c* refer to the **union** declaration of the parser (see page 231) where we find **uint32_t** *u* and **Glyph** *c*. The macro **REF** tests a reference number for its valid range.

Reading a node is usually split into the following sequence of steps:

- Reading the node specification, here a *glyph* consisting of an **UNSIGNED** value and a **REFERENCE** value.
- Creating the internal representation in the variable **\$\$** based on the values of **\$1**, **\$2**, ... Here the character code field *c* is initialized using the **UNSIGNED** value stored in **\$1** and the font field *f* is initialized using **\$2** after checking the reference number for the proper range.
- A *content_node* rule explaining that *start* is followed by **GLYPH**, the keyword that directs the parser to *glyph*, the node specification, and a final **END**.
- Parsing *start*, which is defined as the token **START** will assign to the corresponding variable *p* on the parse stack the current position *hpos* in the output and increments that position to make room for the start byte, which we will discuss shortly.
- At the end of the *content_node* rule, the **shrink** program calls a *hput...* function, here *hput_glyph*, to write the short format of the node as given by its internal representation to the output and return the correct tag value.
- Finally the *hput_tags* function will add the tag as a start byte and end byte to the output stream.

Now let's see how writing the short format works in detail.

1.4 Writing the Short Format

A content node in short form begins with a start byte. It tells us what kind of node it is. To describe the content of a short **HINT** file, 32 different kinds of nodes are defined. Hence the kind of a node can be stored in 5 bits and the remaining bits of the start byte can be used to contain a 3 bit “info” value.

We define an enumeration type to give symbolic names to the kind-values. The exact numerical values are of no specific importance; we will see in section 4.2, however, that the assignment chosen below, has certain advantages.

Because the usage of kind-values in content nodes is slightly different from the usage in definition nodes, we define alternative names for some kind-values. To display readable names instead of numerical values when debugging, we define two arrays of strings as well. Keeping the definitions consistent is achieved by creating all definitions from the same list of identifiers using different definitions of the macro **DEF_KIND**.

```

< hint basic types 6vi > ≡
#define DEF_KIND (C,D,N) C##_kind = N
typedef enum { <kinds 9ix > , <alternative kind names 10x > } Kind;
#undef DEF_KIND

```

Used in 548dxlviii.

```

⟨ define content_name and definition_name 7vii ⟩ ≡ (7)
#define DEF_KIND (C, D, N) #C
  const char *content_name[32] = { ⟨ kinds 9ix ⟩ } ;
#undef DEF_KIND
#define DEF_KIND (C, D, N) #D
  const char *definition_name[#20] = { ⟨ kinds 9ix ⟩ } ;
#undef DEF_KIND

```

Used in 549dxlix.

```

⟨ print content_name and definition_name 8viii ⟩ ≡ (8)
  printf("const_char_*content_name[32]={");
  for (k = 0; k ≤ 31; k++) { printf("\'%s\'", content_name[k]);
    if (k < 31) printf(", ");
  }
  printf("};\n\n"); printf("const_char_*definition_name[32]={");
  for (k = 0; k ≤ 31; k++) { printf("\'%s\'", definition_name[k]);
    if (k < 31) printf(", ");
  }
  printf("};\n\n");

```

Used in 549dxlix.

```

⟨ kinds 9ix ⟩ ≡ (9)
  DEF_KIND(list, list, 0),
  DEF_KIND(param, param, 1),
  DEF_KIND(range, range, 2),
  DEF_KIND(xdimen, xdimen, 3),
  DEF_KIND(adjust, adjust, 4),
  DEF_KIND(glyph, font, 5),
  DEF_KIND(kern, dimen, 6),
  DEF_KIND(glue, glue, 7),
  DEF_KIND(ligature, ligature, 8),
  DEF_KIND(disc, disc, 9),
  DEF_KIND(language, language, 10),
  DEF_KIND(rule, rule, 11),
  DEF_KIND(image, image, 12),
  DEF_KIND(leaders, leaders, 13),
  DEF_KIND(baseline, baseline, 14),
  DEF_KIND(hbox, hbox, 15),
  DEF_KIND(vbox, vbox, 16),
  DEF_KIND(par, par, 17),
  DEF_KIND(math, math, 18),
  DEF_KIND(table, table, 19),
  DEF_KIND(item, item, 20),
  DEF_KIND(hset, hset, 21),
  DEF_KIND(vset, vset, 22),
  DEF_KIND(hpack, hpack, 23),

```

```

DEF_KIND(vpack, vpack, 24),
DEF_KIND(stream, stream, 25),
DEF_KIND(page, page, 26),
DEF_KIND(link, label, 27),
DEF_KIND(color, color, 28),
DEF_KIND(undefined1, undefined1, 29),
DEF_KIND(undefined2, undefined2, 30),
DEF_KIND(penalty, int, 31)

```

Used in [6vi](#) and [7vii](#).

For a few kind-values we have alternative names; we will use them to express different intentions when using them.

```

⟨ alternative kind names 10x ⟩ ≡ (10)
    font_kind = glyph_kind, int_kind = penalty_kind, unknown_kind = penalty_kind,
    dimen_kind = kern_kind, label_kind = link_kind, outline_kind = link_kind

```

Used in [6vi](#).

The info values can be used to represent numbers in the range 0 to 7; for an example see the *hput_glyph* function later in this section. Mostly, however, the individual bits are used as flags indicating the presence or absence of immediate parameter values. If the info bit is set, it means the corresponding parameter is present as an immediate value; if it is zero, it means that there is no immediate parameter value present, and the node specification will reveal what value to use instead. In some cases there is a common default value that can be used, in other cases a one byte reference number is used to select a predefined value.

To make the binary representation of the info bits more readable, we define an enumeration type.

```

⟨ hint basic types 6vi ⟩ +≡ (11)
    typedef enum { b000 = 0, b001 = 1, b010 = 2, b011 = 3, b100 = 4, b101 = 5,
                   b110 = 6, b111 = 7 } Info;

```

After the start byte follows the node content and it is the purpose of the start byte to reveal the exact syntax and semantics of the node content. Because we want to be able to read the short form of a HINT file in forward direction and in backward direction, the start byte is duplicated after the content as an end byte.

We store a kind and an info value in one byte and call this a tag.

```

⟨ hint basic types 6vi ⟩ +≡ (12)
    typedef uint8_t Tag;

```

The following macros are used to assemble and disassemble tags:

```

⟨ hint macros 13xiii ⟩ ≡ (13)
#define KIND(T) (((T) >> 3) & #1F)
#define NAME(T) content_name[KIND(T)]
#define INFO(T) ((T) & #7)
#define TAG(K, I) (((K) << 3) | (I))

```

Used in [548dxlviii](#) and [553dliii](#).

Writing a short format HINT file is implemented by a collection of *hput...* functions; they follow most of the time the same schema:

- First, we define a variable for *info*.
- Then follows the main part of the function body, where we decide on the output format, do the actual output and set the *info* value accordingly.
- We combine the info value with the kind-value and return the correct tag.
- The tag value will be passed to *hput_tags* which generates debugging information, if requested, and stores the tag before and after the node content.

After these preparations, we turn our attention again to the *hput_glyph* function. The font number in a glyph node is between 0 and 255 and fits nicely in one byte, but the character code is more difficult: we want to store the most common character codes as a single byte and less frequent codes with two, three, or even four byte. Naturally, we use the *info* bits to store the number of bytes needed for the character code.

Writing the Short Format:

⇒ ...

```

⟨ put functions 14xiv ⟩ ≡ (14)
static uint8_t hput_n(uint32_t n)
{ if (n ≤ #FF) { HPUT8(n); return 1; }
  else if (n ≤ #FFFF) { HPUT16(n); return 2; }
  else if (n ≤ #FFFFFF) { HPUT24(n); return 3; }
  else { HPUT32(n); return 4; }
}
Tag hput_glyph(Glyph *g)
{ Info info;
  info = hput_n(g→c); HPUT8(g→f);
  return TAG(glyph_kind, info);
}

```

Used in 554dliv and 557dlvii.

The *hput_tags* function is called after the node content has been written to the stream. It gets a the position of the start byte and the tag. With this information it writes the start byte at the given position and the end byte at the current stream position.

```

⟨ put functions 14xiv ⟩ +≡ (15)
void hput_tags(uint32_t pos, Tag tag)
{ DBGTAG(tag, hstart + pos); DBGTAG(tag, hpos); HPUTX(1);
  *(hstart + pos) = *(hpos++) = tag; }

```

The variables *hpos* and *hstart*, the macros HPUT8, HPUT16, HPUT24, HPUT32, and HPUTX are all defined in section 8.3; they put 8, 16, 24, or 32 bits into the output stream and check for sufficient space in the output buffer. The macro DBGTAG writes debugging output; its definition is found in section 14.

Now that we have seen the general outline of the `shrink` program, starting with a long format file and ending with a short format file, we will look at the program `stretch` that reverses this transformation.

1.5 Parsing the Short Format

The inverse of writing the short format with a `hput...` function is reading the short format with a `hget...` function.

The schema of `hget...` functions reverse the schema of `hput...` functions. Here is the code for the initial and final part of a get function:

```

⟨read the start byte a 16xvi ⟩ ≡ (16)
    Tag a, z; /* the start and the end byte */
    uint32_t node_pos = (uint32_t)(hpos - hstart);
    if (hpos ≥ hend)
        QUIT("Attempt to read a start byte at the end of the section");
    HGETTAG(a);

```

Used in 18xviii, 94xciv, 121cxi, 139cxxxix, 146cxlvi, 158clviii, 167clxvii, 202ccii, 313cccxi, 369ccclxi

```

⟨read and check the end byte z 17xvii ⟩ ≡ (17)
    HGETTAG(z); if (a ≠ z)
        QUIT("Tag mismatch [%s,%d] != [%s,%d] at 0x%x to "SIZE_F"\n",
            NAME(a), INFO(a), NAME(z), INFO(z), node_pos, hpos - hstart - 1);

```

Used in 18xviii, 94xciv, 121cxi, 139cxxxix, 146cxlvi, 158clviii, 167clxvii, 202ccii, 313cccxi, 369ccclxi

The central routine to parse the content section of a short format file is the function `hget_content_node` which calls `hget_content` to do most of the processing.

`hget_content_node` will read a content node in short format and write it out in long format: It reads the start byte *a*, writes the START token using the function `hwrite_start`, and based on `KIND(a)`, it writes the node's keyword found in the `content_name` array. Then it calls `hget_content` to read the node's content and write it out. Finally it reads the end byte, checks it against the start byte, and finishes up the content node by writing the END token using the `hwrite_end` function. The function returns the tag byte so that the calling function might check that the content node meets its requirements.

`hget_content` uses the start byte *a*, passed as a parameter, to branch directly to the reading routine for the given combination of kind and info value. The reading routine will read the data and store its internal representation in a variable. All that the `stretch` program needs to do with this internal representation is writing it in the long format. As we will see, the call to the proper `hwrite...` function is included as final part of the the reading routine (avoiding another switch statement).

Reading the Short Format:

... $\Rightarrow$

```

⟨get functions 18xviii⟩ ≡
static void hget_content(Tag a);
Tag hget_content_node(void)
{ ⟨read the start byte a 16xvi⟩ hwrite_start();
  if (content_known[KIND(a)] & (1 ≪ INFO(a)))
    hwritef("%s", content_name[KIND(a)]);
  hget_content(a);
  ⟨read and check the end byte z 17xvii⟩
  hwrite_end(); return a;
}
static void hget_content(Tag a)
{ switch (a)
  { ⟨cases to get content 20xx⟩
    default:
      if (¬hget_unknown(a)) TAGERR(a);
      break;
  }
}

```

(18)

Used in 558dlviii and 560dlx.

We implement the code to read a glyph node in two stages. First we define a general reading macro `HGET_GLYPH(I, G)` that reads a glyph node with info value *I* into a **Glyph** variable *G*; then we insert this macro in the above switch statement for all cases where it applies. Knowing the function `hput_glyph`, the macro `HGET_GLYPH` should not be a surprise. It reverses `hput_glyph`, storing the glyph node in its internal representation. After that, the `stretch` program calls `hwrite_glyph` to produce the glyph node in long format.

Reading the Short Format:

... $\Rightarrow$

```

⟨get macros 19xix⟩ ≡
#define HGET_N(I, X)
  if ((I) ≡ 1) (X) = HGET8;
  else if ((I) ≡ 2) HGET16(X);
  else if ((I) ≡ 3) HGET24(X);
  else if ((I) ≡ 4) HGET32(X);
#define HGET_GLYPH(I, G) HGET_N(I, (G).c); (G).f = HGET8;
  REF_RNG(font_kind, (G).f);
  hwrite_glyph(&(G));

```

(19)

Used in 558dlviii and 560dlx.

Note that we allow a glyph to reference a font even before that font is defined. This is necessary because fonts usually contain definitions—for example the fonts hyphen character—that reference this or other fonts.

```

⟨ cases to get content 20xx ⟩ ≡
    case TAG(glyph_kind, 1): { Glyph g; HGET_GLYPH(1, g); } break;
    case TAG(glyph_kind, 2): { Glyph g; HGET_GLYPH(2, g); } break;
    case TAG(glyph_kind, 3): { Glyph g; HGET_GLYPH(3, g); } break;
    case TAG(glyph_kind, 4): { Glyph g; HGET_GLYPH(4, g); } break;

```

Used in 18xviii.

If this two stage method seems strange to you, consider what the C compiler will do with it. It will expand the `HGET_GLYPH` macro four times inside the switch statement. The macro is, however, expanded with a constant *I* value, so the expansion of the `if` statement in `HGET_GLYPH(1, g)`, for example, will become “`if (1 ≡ 1) ... else if (1 ≡ 2) ...`” and the compiler will have no difficulties eliminating the constant tests and the dead branches altogether. This is the most effective use of the switch statement: a single jump takes you to a specialized code to handle just the given combination of kind and info value.

Last not least, we implement the function *hwrite_glyph* to write a glyph node in long form—that is: in a form that is as readable as possible.

1.6 Writing the Long Format

The *hwrite_glyph* function inverts the scanning and parsing process we have described at the very beginning of this chapter. To implement the *hwrite_glyph* function, we use the function *hwrite_charcode* to write the character code. Besides writing the character code as a decimal number, this function can handle also other representations of character codes as fully explained in section 2.3. We split off the writing of the opening and the closing pointed bracket, because we will need this function very often and because it will keep track of the *nesting* of nodes and indent them accordingly. The *hwrite_range* and *hwrite_label* functions used in *hwrite_end* are discussed in section 7.4 and 6.2.

Writing the Long Format: ⇒ - - -

```

⟨ write functions 21xxi ⟩ ≡
    int nesting = 0;
    void hwrite_nesting(void)
    { int i;
      hwritec('\n');
      for (i = 0; i < nesting; i++) hwritec('␣');
    }
    void hwrite_start(void)
    { hwrite_nesting(); hwritec('<'); nesting++;
    }
    void hwrite_range(void);
    void hwrite_label(void);
    void hwrite_end(void)
    { nesting--; hwritec('>');
      if (section_no ≡ 2) {

```

```

    if (nesting == 0) hwrite_range();
    hwrite_label();
}
}
void hwrite_comment(char *str)
{ char c;
  if (str == NULL) return;
  hwritef("_(");
  while ((c = *str++) != 0)
    if (c == '(' ∨ c == ')') hwritec('_');
    else if (c == '\\n') hwritef("\\n(");
    else hwritec(c);
  hwritec(')');
}
void hwrite_charcode(uint32_t c);
void hwrite_ref(int n);
void hwrite_glyph(Glyph *g)
{ char *n = hfont_name[g→f];
  hwrite_charcode(g→c); hwrite_ref(g→f);
  if (n != NULL) hwrite_comment(n);
}

```

Used in 558dlviii and 560dlx.

The two primitive operations to write the long format file are defined as macros:

```

⟨ write macros 22xxii ⟩ ≡ (22)
#define hwritec(c) (hout ? putc(c, hout) : 0)
#define hwritef(...) (hout ? fprintf(hout, __VA_ARGS__) : 0)

```

Used in 558dlviii and 560dlx.

Now that we have completed the round trip of shrinking and stretching glyph nodes, we continue the description of the HINT file formats in a more systematic way.

2 Data Types

2.1 Integers

We have already seen the pattern/action rule for unsigned decimal numbers. It remains to define the macro `SCAN_UDEC` which converts a string containing an unsigned decimal number into an unsigned integer. We use the C library function `strtoul`:

Reading the long format:

```

< scanning macros 23xxiii > ≡
#define SCAN_UDEC(S) yylval.u = strtoul(S, NULL, 10)

```

Used in 555dlv.

Unsigned integers can be given in hexadecimal notation as well.

```

< scanning definitions 24xxiv > ≡
HEX          [0-9A-F]

```

Used in 555dlv.

```

< scanning rules 3iii > +≡
0x{HEX}+      SCAN_HEX(yyltext + 2); return UNSIGNED;

```

Note that the pattern above allows only upper case letters in the hexadecimal notation for integers.

```

< scanning macros 23xxiii > +≡
#define SCAN_HEX(S) yylval.u = strtoul(S, NULL, 16)

```

Last not least, we add rules for signed integers.

```

< symbols 2ii > +≡
%token < i > SIGNED
%type < i > integer

```

```

< scanning rules 3iii > +≡
[+-] (0|[1-9][0-9]*)  SCAN_DEC(yyltext); return SIGNED;

```

```

< scanning macros 23xxiii > +≡
#define SCAN_DEC(S) yylval.i = strtol(S, NULL, 10)

```

⟨ parsing rules 5v ⟩ +≡ (30)
`integer: SIGNED | UNSIGNED { RNG("number", $1, 0, #7FFFFFFF); };`

To preserve the “signedness” of an integer also for positive signed integers in the long format, we implement the function *hwrite_signed*.

Writing the long format:

⇒ - - -

⟨ write functions 21xxi ⟩ +≡ (31)
`void hwrite_signed(int32_t i)
{
 if (i < 0) hwritef("_-%d", -i);
 else hwritef("_+%d", +i);
}`

Reading and writing integers in the short format is done directly with the HPUT and HGET macros.

2.2 Strings

Strings are needed in the definition part of a HINT file to specify names of objects, and in the long file format, we also use them for file names. In the long format, strings are sequences of characters delimited by single quote characters; for example: “’Hello’” or “’cmr10-600dpi.tfm’”; in the short format, strings are byte sequences terminated by a zero byte. Because file names are system dependent, we do not allow arbitrary characters in strings but only printable ASCII codes which we can reasonably expect to be available on most operating systems. If your file names in a long format HINT file are supposed to be portable, you should probably be even more restrictive. For example you should avoid characters like “\” or “/” which are used in different ways for directories.

The internal representation of a string is a simple zero terminated C string. When scanning a string, we copy it to the *str_buffer* keeping track of its length in *str_length*. When done, we make a copy for permanent storage and return the pointer to the parser. To operate on the *str_buffer*, we define a few macros. The constant MAX_STR determines the maximum size of a string (including the zero byte) to be 2¹⁰ byte. This restriction is part of the HINT file format specification.

⟨ scanning macros 23xxiii ⟩ +≡ (32)
`#define MAX_STR (1 << 10) /* 210 Byte or 1kByte */
 static char str_buffer[MAX_STR];
 static int str_length;
#define STR_START (str_length = 0)
#define STR_PUT(C) (str_buffer[str_length++] = (C))
#define STR_ADD(C)
 STR_PUT(C); RNG("String_length", str_length, 0, MAX_STR - 1)
#define STR_END str_buffer[str_length] = 0
#define SCAN_STR yylval.s = str_buffer`

To scan a string, we switch the scanner to STR mode when we find a quote character, then we scan bytes in the range #20 to #7E, which is the range of

printable ASCII characters, until we find the closing single quote. Quote characters inside the string are written as two consecutive single quote characters.

Reading the long format:

— — — $\Rightarrow$

$\langle$ scanning definitions 24xxiv $\rangle + \equiv$ (33)
`%x STR`

$\langle$ symbols 2ii $\rangle + \equiv$ (34)
`%token < s > STRING`

$\langle$ scanning rules 3iii $\rangle + \equiv$ (35)
`, STR_START; BEGIN(STR);`
`< STR > {`
`, STR_END; SCAN_STR; BEGIN(INITIAL); return STRING;`
`,, STR_ADD('\'');`
`[\x20-\x7E] STR_ADD(yytext[0]);`
`. RNG("String_character", yytext[0], #20, #7E);`
`\n QUIT("Unterminated_string_in_line%d", yylineno);`
`}`

The function *hwrite_string* reverses this process; it must take care of the quote symbols.

Writing the long format:

$\Rightarrow$ — — —

$\langle$ write functions 21xxi $\rangle + \equiv$ (36)
`void hwrite_string(char *str)`
`{ hwritec(' ');`
`if (str  $\equiv$  NULL) hwritef("''");`
`else`
`{ hwritec('\');`
`while (*str  $\neq$  0)`
`{ if (*str  $\equiv$  '\') hwritec('\');`
`hwritec(*str);`
`str++;`
`}`
`hwritec('\');`
`}`

In the short format, a string is just a byte sequence terminated by a zero byte. This makes the function *hput_string*, to write a string, and the macro `HGET_STRING`, to read a string in short format, very simple. Note that after writing an unbounded string to the output buffer, the macro `HPUTNODE` will make sure that there is enough space left to write the remainder of the node.

Writing the short format:

⇒ ...

⟨ put functions 14xiv ⟩ +≡ (37)

```

void hput_string(char *str)
{
  char *s = str;
  if (s ≠ NULL) { do { HPUTX(1);
    HPUT8(*s);
  } while (*s++ ≠ 0);
  HPUTNODE;
}
else HPUT8(0);
}

```

Reading the short format:

... ⇒

⟨ shared get macros 38xxxviii ⟩ ≡ (38)

```

#define HGET_STRING(S) S = (char *) hpos;
while (hpos < hend ∧ *hpos ≠ 0) {
  RNG("String_character", *hpos, #20, #7E);
  hpos++;
}
hpos++;

```

Used in 551dli and 560dlx.

2.3 Character Codes

We have already seen in the introduction that character codes can be written as decimal numbers and section 2.1 adds the possibility to use hexadecimal numbers as well.

It is, however, in most cases more readable if we represent character codes directly using the characters themselves. Writing “a” is just so much better than writing “97”. To distinguish the character “9” from the number “9”, we use the common technique of enclosing characters within single quotes. So “'9'” is the character code and “9” is the number. Therefore we will define CHARCODE tokens and complement the parsing rules of section 1.3 with the following rule:

Reading the long format:

— — — ⇒

⟨ parsing rules 5v ⟩ +≡ (39)

```

glyph: CHARCODE REFERENCE
      { $$c = $1; REF(font_kind, $2); $$f = $2; };

```

If the character codes are small, we can represent them using ASCII character codes. We do not offer a special notation for very small character codes that map to the non-printable ASCII control codes; for them, the decimal or hexadecimal notation will suffice. For larger character codes, we use the multibyte encoding scheme known from UTF8 as follows. Given a character code *c*:

- Values in the range #00 to #7f are encoded as a single byte with a leading bit of 0.

$\langle \text{scanning definitions } 24_{\text{xxiv}} \rangle + \equiv$ (40)
`UTF8_1` $[\backslash\text{x00}-\backslash\text{x7F}]$

$\langle \text{scanning macros } 23_{\text{xxiii}} \rangle + \equiv$ (41)
`#define SCAN_UTF8_1(S)` $\text{yylval.u} = ((S)[0] \& \#7F)$

- Values in the range `#80` to `#7ff` are encoded in two byte with the first byte having three high bits 110, indicating a two byte sequence, and the lower five bits equal to the five high bits of *c*. It is followed by a continuation byte having two high bits 10 and the lower six bits equal to the lower six bits of *c*.

$\langle \text{scanning definitions } 24_{\text{xxiv}} \rangle + \equiv$ (42)
`UTF8_2` $[\backslash\text{xC0}-\backslash\text{xDF}] [\backslash\text{x80}-\backslash\text{xBF}]$

$\langle \text{scanning macros } 23_{\text{xxiii}} \rangle + \equiv$ (43)
`#define SCAN_UTF8_2(S)` $\text{yylval.u} = (((S)[0] \& \#1F) \ll 6) + ((S)[1] \& \#3F)$

- Values in the range `#800` to `#FFFF` are encoded in three byte with the first byte having the high bits 1110 indicating a three byte sequence followed by two continuation bytes.

$\langle \text{scanning definitions } 24_{\text{xxiv}} \rangle + \equiv$ (44)
`UTF8_3` $[\backslash\text{xEO}-\backslash\text{xEF}] [\backslash\text{x80}-\backslash\text{xBF}] [\backslash\text{x80}-\backslash\text{xBF}]$

$\langle \text{scanning macros } 23_{\text{xxiii}} \rangle + \equiv$ (45)
`#define SCAN_UTF8_3(S)`
 $\text{yylval.u} = (((S)[0] \& \#0F) \ll 12) + (((S)[1] \& \#3F) \ll 6) + ((S)[2] \& \#3F)$

- Values in the range `#1000` to `#1FFFFF` are encoded in four byte with the first byte having the high bits 11110 indicating a four byte sequence followed by three continuation bytes.

$\langle \text{scanning definitions } 24_{\text{xxiv}} \rangle + \equiv$ (46)
`UTF8_4` $[\backslash\text{xFO}-\backslash\text{xF7}] [\backslash\text{x80}-\backslash\text{xBF}] [\backslash\text{x80}-\backslash\text{xBF}] [\backslash\text{x80}-\backslash\text{xBF}]$

$\langle \text{scanning macros } 23_{\text{xxiii}} \rangle + \equiv$ (47)
`#define SCAN_UTF8_4(S)`
 $\text{yylval.u} = (((S)[0] \& \#07) \ll 18) + (((S)[1] \& \#3F) \ll 12) +$
 $((S)[2] \& \#3F) \ll 6 + ((S)[3] \& \#3F)$

In the long format file, we enclose a character code in single quotes, just as we do for strings. This is convenient but it has the downside that we must exercise special care when giving the scanning rules in order not to confuse character codes with strings. Further we must convert character codes back into strings in the rare case where the parser expects a string and gets a character code because the string was only a single character long.

Let's start with the first problem: The scanner might confuse a string and a character code if the first or second character of the string is a quote character which is written as two consecutive quotes. For example `'a''b'` is a string with three characters, `"a"`, `"'"`, and `"b"`. Two character codes would need a space to separate them like this: `'a' 'b'`.

⟨ symbols 2ii ⟩ +≡ (48)
%token < *u* > CHARCODE

⟨ scanning rules 3iii ⟩ +≡ (49)
 '','' STR_START; STR_PUT('\''); BEGIN(STR);
 '','','' SCAN_UTF8_1(*yytext* + 1); **return** CHARCODE;
 '[\x20-\x7E]','' STR_START; STR_PUT(*yytext*[1]); STR_PUT('\''); BEGIN(STR);
 '','','','' STR_START; STR_PUT('\''); STR_PUT('\''); BEGIN(STR);
 '{UTF8_1}' SCAN_UTF8_1(*yytext* + 1); **return** CHARCODE;
 '{UTF8_2}' SCAN_UTF8_2(*yytext* + 1); **return** CHARCODE;
 '{UTF8_3}' SCAN_UTF8_3(*yytext* + 1); **return** CHARCODE;
 '{UTF8_4}' SCAN_UTF8_4(*yytext* + 1); **return** CHARCODE;

If needed, the parser can convert character codes back to single character strings.

⟨ symbols 2ii ⟩ +≡ (50)
%type < *s* > *string*

⟨ parsing rules 5v ⟩ +≡ (51)
string: STRING | CHARCODE { **static char** *s*[2];
 RNG("String_element", \$1, #20, #7E); *s*[0] = \$1; *s*[1] = 0; \$\$ = *s*; };

The function *hwrite_charcode* will write a character code. While ASCII codes are handled directly, larger character codes are passed to the function *hwrite_utf8*. It returns the number of characters written.

Writing the long format:

⇒ - - -

⟨ write functions 21xxi ⟩ +≡ (52)
int *hwrite_utf8*(**uint32_t** *c*)
 { **if** (*c* < #80) { *hwritec*(*c*); **return** 1; }
else if (*c* < #800)
 { *hwritec*(#C0 | (*c* >> 6)); *hwritec*(#80 | (*c* & #3F)); **return** 2; }
else if (*c* < #10000)
 { *hwritec*(#E0 | (*c* >> 12));
hwritec(#80 | ((*c* >> 6) & #3F)); *hwritec*(#80 | (*c* & #3F));
return 3;
 }
else if (*c* < #200000)
 { *hwritec*(#F0 | (*c* >> 18)); *hwritec*(#80 | ((*c* >> 12) & #3F));
hwritec(#80 | ((*c* >> 6) & #3F)); *hwritec*(#80 | (*c* & #3F));
return 4;
 }
else RNG("character_code", *c*, 0, #1FFFFF);
return 0;
 }

```

void hwrite_charcode(uint32_t c)
{ if (c < #20) {
    if (option_hex) hwritef("_0x%02X", c);          /* non printable ASCII */
    else hwritef("_%u", c);
  }
  else if (c == '\') hwritef("_'\'');
  else if (c ≤ #7E) hwritef("_\\'%c\'", c);          /* printable ASCII */
  else if (option_utf8) { hwritef("_\\'"); hwrite_utf8(c); hwritec('\''); }
  else if (option_hex) hwritef("_0x%04X", c);
  else hwritef("_%u", c);
}

```

Reading the short format:

... $\Rightarrow$

```

⟨ shared get functions 53liii ⟩ ≡ (53)
#define HGET_UTF8C(X) (X) = HGET8; if ((X & #C0) ≠ #80)
    QUIT("UTF8_continuation_byte_expected_at_\"SIZE_F\"_got_0x%02X\\n",
        hpos - hstart - 1, X)
uint32_t hget_utf8(void)
{ uint8_t a;
  a = HGET8;
  if (a < #80) return a;
  else {
    if ((a & #E0) == #C0)
    { uint8_t b; HGET_UTF8C(b);
      return ((a & ~#E0) << 6) + (b & ~#C0);
    }
    else if ((a & #F0) == #E0)
    { uint8_t b, c; HGET_UTF8C(b); HGET_UTF8C(c);
      return ((a & ~#F0) << 12) + ((b & ~#C0) << 6) + (c & ~#C0);
    }
    else if ((a & #F8) == #F0)
    { uint8_t b, c, d; HGET_UTF8C(b); HGET_UTF8C(c); HGET_UTF8C(d);
      return ((a & ~#F8) << 18)
        + ((b & ~#C0) << 12) + ((c & ~#C0) << 6) + (d & ~#C0);
    }
    else QUIT("UTF8_byte_sequence_expected");
  }
}

```

Used in 552dlii, 558dlviii, and 560dlx.

Writing the short format:

⇒ ...

```

⟨ put functions 14xiv ⟩ +≡
void hput_utf8(uint32_t c)
{ HPUTX(4);
  if (c < #80) HPUT8(c);
  else if (c < #800) { HPUT8(#C0 | (c >> 6)); HPUT8(#80 | (c & #3F)); }
  else if (c < #10000)
  { HPUT8(#E0 | (c >> 12));
    HPUT8(#80 | ((c >> 6) & #3F)); HPUT8(#80 | (c & #3F));
  }
  else if (c < #200000)
  { HPUT8(#F0 | (c >> 18)); HPUT8(#80 | ((c >> 12) & #3F));
    HPUT8(#80 | ((c >> 6) & #3F)); HPUT8(#80 | (c & #3F));
  }
  else RNG("character_code", c, 0, #1FFFFFF);
}

```

(54)

2.4 Floating Point Numbers

You know a floating point numbers when you see it because it features a radix point. The optional exponent allows you to “float” the point.

Reading the long format:

— — — ⇒

```

⟨ symbols 2ii ⟩ +≡
%token < f > FPNUM
%type < f > number

```

(55)

```

⟨ scanning rules 3iii ⟩ +≡
[+-]?[0-9]+\.[0-9]+(e[+-]?[0-9])?  SCAN_DECFLOAT; return FPNUM;

```

(56)

The layout of floating point variables of type **double** or **float** typically follows the IEEE754 standard[6][7]. We use the following definitions:

```

⟨ hint basic types 6vi ⟩ +≡
#define FLT_M_BITS 23
#define FLT_E_BITS 8
#define FLT_EXCESS 127
#define DBL_M_BITS 52
#define DBL_E_BITS 11
#define DBL_EXCESS 1023

```

(57)

```

⟨ scanning macros 23xxiii ⟩ +≡
#define SCAN_DECFLOAT yylval.f = atof(yytext)

```

(58)

When the parser expects a floating point number and gets an integer number, it converts it. So whenever in the long format a floating point number is expected, an integer number will do as well.

```

⟨ parsing rules 5v ⟩ +≡
    number: UNSIGNED { $$ = (float64_t) $1; }
    | SIGNED { $$ = (float64_t) $1; }
    | FPNUM;

```

(59)

Unfortunately the decimal representation is not optimal for floating point numbers since even simple numbers in decimal notation like 0.1 do not have an exact representation as a binary floating point number. So if we want a notation that allows an exact representation of binary floating point numbers, we must use a hexadecimal representation. Hexadecimal floating point numbers start with an optional sign, then as usual the two characters “0x”, then follows a sequence of hex digits, a radix point, more hex digits, and an optional exponent. The optional exponent starts with the character “x”, followed by an optional sign, and some more hex digits. The hexadecimal exponent is given as a base 16 number and it is interpreted as an exponent with the base 16. As an example an exponent of “x10”, would multiply the mantissa by 16^{10} . In other words it would shift any mantissa 16 hexadecimal digits to the left. Here are the exact rules:

```

⟨ scanning rules 3iii ⟩ +≡
[+-]?0x{HEX}+\. {HEX}+(x[+-]?{HEX}+)?  SCAN_HEXFLOAT; return FPNUM;

```

(60)

```

⟨ scanning macros 23xxiii ⟩ +≡
#define SCAN_HEXFLOAT yylval.f = xtof(yytext)

```

(61)

There is no function in the C library for hexadecimal floating point notation so we have to write our own conversion routine. The function *xtof* converts a string matching the above regular expression to its binary representation. Its outline is very simple:

```

⟨ scanning functions 62lxii ⟩ ≡
float64_t xtof(char *x)
{ int sign, digits, exp;
  uint64_t mantissa = 0;
  DBG(DBGFLOAT, "converting %s:\n", x);
  ⟨ read the optional sign 63lxiii ⟩
  x = x + 2;
  ⟨ read the mantissa 64lxiv ⟩
  ⟨ normalize the mantissa 65lxv ⟩
  ⟨ read the optional exponent 66lxvi ⟩
  ⟨ return the binary representation 67lxvii ⟩
}

```

(62)

Used in 555dlv.

Now the pieces:

```

⟨ read the optional sign 63lxiii ⟩ ≡
if (*x ≡ '-') { sign = -1; x++; }
else if (*x ≡ '+') { sign = +1; x++; }
else sign = +1;

```

(63)

```
DBG(DBGFLOAT, "\t sign=%d\n", sign);
```

Used in 62lxii.

When we read the mantissa, we use the temporary variable *mantissa*, keep track of the number of digits, and adjust the exponent while reading the fractional part.

```
< read the mantissa 64lxiv > ≡ (64)
  digits = 0;
  while (*x == '0') x++; /* ignore leading zeros */
  while (*x != '.')
  { mantissa = mantissa << 4;
    if (*x < 'A') mantissa = mantissa + *x - '0';
    else mantissa = mantissa + *x - 'A' + 10;
    x++;
    digits++;
  }
  x++; /* skip "." */
  exp = 0;
  while (*x != 0 & *x != 'x')
  { mantissa = mantissa << 4;
    exp = exp - 4;
    if (*x < 'A') mantissa = mantissa + *x - '0';
    else mantissa = mantissa + *x - 'A' + 10;
    x++;
    digits++;
  }
  DBG(DBGFLOAT, "\tdigits=%d_mantissa=0x%" PRIx64 ",_exp=%d\n",
    digits, mantissa, exp);
```

Used in 62lxii.

To normalize the mantissa, first we shift it to place exactly one nonzero hexadecimal digit to the left of the radix point. Then we shift it right bit-wise until there is just a single 1 bit to the left of the radix point. To compensate for the shifting, we adjust the exponent accordingly. Finally we remove the most significant bit because it is not stored.

```
< normalize the mantissa 65lxv > ≡ (65)
  if (mantissa == 0) return 0.0;
  { int s;
    s = digits - DBL_M_BITS/4;
    if (s > 1) mantissa = mantissa >> (4 * (s - 1));
    else if (s < 1) mantissa = mantissa << (4 * (1 - s));
    exp = exp + 4 * (digits - 1);
    DBG(DBGFLOAT, "\tdigits=%d_mantissa=0x%" PRIx64 ",_exp=%d\n",
      digits, mantissa, exp);
    while ((mantissa >> DBL_M_BITS) > 1)
    { mantissa = mantissa >> 1; exp++; }
  }
```



```

DBG(DBGFLOAT, "\tdigits=%d\mantissa=0x%" PRIx64 ", \exp=%d\n",
    digits, mantissa, exp);
mantissa = mantissa & ~((uint64_t) 1 << DBL_M_BITS);
DBG(DBGFLOAT, "\tdigits=%d\mantissa=0x%" PRIx64 ", \exp=%d\n",
    digits, mantissa, exp);
}

```

Used in 62lxii.

In the printed representation, the exponent is an exponent with base 16. For example, an exponent of 2 shifts the hexadecimal mantissa two hexadecimal digits to the left, which corresponds to a multiplication by 16^2 .

(read the optional exponent 66lxvi) $\equiv$ (66)

```

if (*x == 'x')
{ int s;

  x++; /* skip the "x" */
  if (*x == '-') { s = -1; x++; }
  else if (*x == '+') { s = +1; x++; }
  else s = +1;
  DBG(DBGFLOAT, "\texpsign=%d\n", s);
  DBG(DBGFLOAT, "\texp=%d\n", exp);
  while (*x != 0) {
    if (*x < 'A') exp = exp + 4 * s * (*x - '0');
    else exp = exp + 4 * s * (*x - 'A' + 10);
    x++;
    DBG(DBGFLOAT, "\texp=%d\n", exp);
  }
}
RNG("Floating_point_exponent",
    exp, -DBL_EXCESS, DBL_EXCESS);

```

Used in 62lxii.

To assemble the binary representation, we use a **union** of a **float64_t** and **uint64_t**.

(return the binary representation 67lxvii) $\equiv$ (67)

```

{ union { float64_t d; uint64_t bits; } u;
  if (sign < 0) sign = 1; else sign = 0; /* the sign bit */
  exp = exp + DBL_EXCESS; /* the exponent bits */
  u.bits = ((uint64_t) sign << 63)
    | ((uint64_t) exp << DBL_M_BITS) | mantissa;
  DBG(DBGFLOAT, "\treturn%f\n", u.d);
  return u.d;
}

```

Used in 62lxii.

The inverse function is *hwrite_float64*. It strives to print floating point numbers as readable as possible. So numbers without fractional part are written as integers.

Numbers that can be represented exactly in decimal notation are represented in decimal notation. All other values are written as hexadecimal floating point numbers. We avoid an exponent if it can be avoided by using up to `MAX_HEX_DIGITS`. For the use with extended dimensions, floating point numbers should be printed as a suffix: without a leading space and with a mandatory sign.

Writing the long format:

⇒ - - -

```

⟨ write functions 21xxi ⟩ +≡ (68)
#define MAX_HEX_DIGITS 12
void hwrite_float64(float64_t d, bool suffix)
{ uint64_t bits, mantissa;
  int exp, digits;
  if (¬suffix) hwritec(' ');
  else if (d ≥ 0) hwritec('+');
  if (floor(d) ≡ d) { hwritef("%d", (int) d); return; }
  if (floor(10000.0 * d) ≡ 10000.0 * d) { hwritef("%g", d); return; }
  DBG(DBGFLOAT, "Writing_hexadecimal_float_uf\n", d);
  if (d < 0) { hwritec('-'); d = -d; }
  hwritef("0x");
  ⟨ extract mantissa and exponent 69lxix ⟩
  if (exp > MAX_HEX_DIGITS) ⟨ write large numbers 72lxxii ⟩
  else if (exp ≥ 0) ⟨ write medium numbers 73lxxiii ⟩
  else ⟨ write small numbers 74lxxiv ⟩
}

```

The extraction just reverses the creation of the binary representation.

```

⟨ extract mantissa and exponent 69lxix ⟩ ≡ (69)
{ union { float64_t d; uint64_t bits; } u;
  u.d = d; bits = u.bits;
}
mantissa = bits & (((uint64_t) 1 << DBL_M_BITS) - 1);
mantissa = mantissa + ((uint64_t) 1 << DBL_M_BITS);
exp = ((bits >> DBL_M_BITS) & ((1 << DBL_E_BITS) - 1)) - DBL_EXCESS;
digits = DBL_M_BITS + 1;
DBG(DBGFLOAT, "\tdigits=%d_mantissa=0x%" PRIx64 " _binary_exp=%d\n",
    digits, mantissa, exp);

```

Used in 68lxviii.

After we have obtained the binary exponent, we round it down, and convert it to a hexadecimal exponent.

```

⟨ extract mantissa and exponent 69lxix ⟩ +≡ (70)
{ int r;
  if (exp ≥ 0) { r = exp % 4;
    if (r > 0) { mantissa = mantissa << r; exp = exp - r; digits = digits + r; }
  }
}

```

```

    else {  $r = (-exp) \% 4$ ;
        if ( $r > 0$ ) {  $mantissa = mantissa \gg r$ ;  $exp = exp + r$ ;  $digits = digits - r$ ; }
    }
}
exp = exp / 4;
DBG(DBGFLOAT, "\tdigits=%d\mantissa=0x%" PRIx64 "\hex_exp=%d\n",
    digits, mantissa, exp);

```

In preparation for writing, we shift the mantissa to the left so that the leftmost hexadecimal digit of it will occupy the 4 leftmost bits of the variable *bits* .

```

⟨extract mantissa and exponent 69lxix⟩ +≡
    mantissa = mantissa << (64 - DBL_M_BITS - 4);    /* move leading digit to
    leftmost nibble */

```

If the exponent is larger than MAX_HEX_DIGITS, we need to use an exponent even if the mantissa uses only a few digits. When we use an exponent, we always write exactly one digit preceding the radix point.

```

⟨write large numbers 72lxixii⟩ +≡
{
    DBG(DBGFLOAT, "writing_large_number\n");
    hwritef("%X.", (uint8_t)(mantissa >> 60));
    mantissa = mantissa << 4;
    do { hwritef("%X", (uint8_t)(mantissa >> DBL_M_BITS) & #F);
        mantissa = mantissa << 4;
    } while (mantissa ≠ 0);
    hwritef("x%X", exp);
}

```

Used in 68lxviii.

If the exponent is small and non negative, we can write the number without an exponent by writing the radix point at the appropriate place.

```

⟨write medium numbers 73lxixiii⟩ +≡
{
    DBG(DBGFLOAT, "writing_medium_number\n");
    do { hwritef("%X", (uint8_t)(mantissa >> 60));
        mantissa = mantissa << 4;
        if (exp-- ≡ 0) hwritec(' ');
    } while (mantissa ≠ 0 ∨ exp ≥ -1);
}

```

Used in 68lxviii.

Last non least, we write numbers that would require additional zeros after the radix point with an exponent, because it keeps the mantissa shorter.

```

⟨write small numbers 74lxixiv⟩ +≡
{
    DBG(DBGFLOAT, "writing_small_number\n");
    hwritef("%X.", (uint8_t)(mantissa >> 60));
    mantissa = mantissa << 4;
    do { hwritef("%X", (uint8_t)(mantissa >> 60));
        mantissa = mantissa << 4;
    }
}

```

```

    } while (mantissa  $\neq$  0);
    hwritef("x-%X", -exp);
}

```

Used in 68lxviii.

Compared to the complications of long format floating point numbers, the short format is very simple because we just use the binary representation. Since 32 bit floating point numbers offer sufficient precision we use only the **float32_t** type. It is however not possible to just write **HPUT32**(*d*) for a **float32_t** variable *d* or **HPUT32**((**uint32_t**) *d*) because in the C language this would imply rounding the floating point number to the nearest integer. But we have seen how to convert floating point values to bit pattern before.

```

<put functions 14xiv> +≡
void hput_float32(float32_t d)
{ union { float32_t d; uint32_t bits; } u;
  u.d = d; HPUT32(u.bits);
}

```

(75)

```

<shared get functions 53liii> +≡
float32_t hget_float32(void)
{ union { float32_t d; uint32_t bits; } u;
  HGET32(u.bits);
  return u.d;
}

```

(76)

2.5 Fixed Point Numbers

TeX internally represents most real numbers as fixed point numbers or “scaled integers”. The type **Scaled** is defined as a signed 32 bit integer, but we consider it as a fixed point number with the binary radix point just in the middle with sixteen bits before and sixteen bits after it. To convert an integer into a scaled number, we multiply it by **ONE**; to convert a floating point number into a scaled number, we multiply it by **ONE** and **ROUND** the result to the nearest integer; to convert a scaled number to a floating point number we divide it by (**float64_t**) **ONE**.

```

<hint basic types 6vi> +≡
typedef int32_t Scaled;
#define ONE ((Scaled)(1 << 16))

```

(77)

```

<hint macros 13xiii> +≡
#define ROUND (X) ((int)((X) ≥ 0.0 ? floor((X) + 0.5) : ceil((X) - 0.5)))

```

(78)

Writing the long format:

⇒ - - -

```

<write functions 21xxi> +≡
void hwrite_scaled(Scaled x)
{ hwrite_float64(x/(float64_t) ONE, false);
}

```

(79)

2.6 Dimensions

In the long format, the dimensions of characters, boxes, and other things can be given in three units: **pt**, **in**, and **mm**.

Reading the long format:

— — — $\Rightarrow$

$\langle$ symbols 2ii $\rangle + \equiv$ (80)

%token DIMEN "dimen"

%token PT "pt"

%token MM "mm"

%token INCH "in"

%type $< d >$ dimension

$\langle$ scanning rules 3iii $\rangle + \equiv$ (81)

dimen **return** DIMEN;

pt **return** PT;

mm **return** MM;

in **return** INCH;

The unit **pt** is a printers point. The unit “**in**” stands for inches and we have $1\text{in} = 72.27\text{pt}$. The unit “**mm**” stands for millimeter and we have $1\text{in} = 25.4\text{mm}$.

The definition of a printers point given above follows the definition used in **T_EX** which is slightly larger than the official definition of a printer’s point which was defined to equal exactly 0.013837in by the American Typefounders Association in 1886[8].

We follow the tradition of **T_EX** and store dimensions as “scaled points” that is a dimension of d points is stored as $d \cdot 2^{16}$ rounded to the nearest integer. The maximum absolute value of a dimension is $(2^{30} - 1)$ scaled points.

$\langle$ hint basic types 6vi $\rangle + \equiv$ (82)

typedef Scaled Dimen;

#define MAX_DIMEN ((Dimen)(#3FFFFFFF))

$\langle$ parsing rules 5v $\rangle + \equiv$ (83)

dimension: number PT

{ $$$ = \text{ROUND}(\$1 * \text{ONE});$

RNG("Dimension", $$$$, $-\text{MAX_DIMEN}$, MAX_DIMEN); }

| number INCH

{ $$$ = \text{ROUND}(\$1 * \text{ONE} * 72.27);$

RNG("Dimension", $$$$, $-\text{MAX_DIMEN}$, MAX_DIMEN); }

| number MM

{ $$$ = \text{ROUND}(\$1 * \text{ONE} * (72.27/25.4));$

RNG("Dimension", $$$$, $-\text{MAX_DIMEN}$, MAX_DIMEN); };

When **stretch** is writing dimensions in the long format, for simplicity it always uses the unit “**pt**”.

Writing the long format:

⇒ — — —

⟨ write functions 21xxi ⟩ +≡ (84)

```

void hwrite_dimension(Dimen x)
{
  hwrite_scaled(x);
  hwritef("pt");
}

```

In the short format, dimensions are stored as 32 bit scaled point values without conversion.

Reading the short format:

... ⇒

⟨ get functions 18xviii ⟩ +≡ (85)

```

void hget_dimen(Tag a)
{
  if (INFO(a) ≡ b000) { uint8_t r;
    r = HGET8;
    REF(dimen_kind, r);
    hwrite_ref(r);
  }
  else { uint32_t d;
    HGET32(d);
    hwrite_dimension(d);
  }
}

```

Writing the short format:

⇒ ...

⟨ put functions 14xiv ⟩ +≡ (86)

```

Tag hput_dimen(Dimen d)
{
  HPUT32(d);
  return TAG(dimen_kind, b001);
}

```

2.7 Extended Dimensions

The dimension that is probably used most frequently in a $\text{\TeX}$ file is **hsize**: the horizontal size of a line of text. Common are also assignments like $\text{\hspace{0.5\hsize}\advance\hsize-10pt}$, for example to get two columns with lines almost half as wide as usual, leaving a small gap between left and right column. Similar considerations apply to **vsize**.

Because we aim at a reflowable format for $\text{\TeX}$ output, we have to postpone such computations until the values of **hsize** and **vsize** are known in the viewer. Until then, we do symbolic computations on linear functions of **hsize** and **vsize**. We call such a linear function $w + h \cdot \text{\hsize} + v \cdot \text{\vsize}$ an extended dimension and represent it by the three numbers w , h , and v .

⟨ hint basic types 6vi ⟩ +≡ (87)

```
typedef struct { Dimen w; float32_t h, v; } Xdimen;
```

Since very often a component of an extended dimension is zero, we store in the short format only the nonzero components and use the info bits to mark them: *b100* implies $w \neq 0$, *b010* implies $h \neq 0$, and *b001* implies $v \neq 0$.

Reading the long format: — — — $\Rightarrow$

⟨ symbols 2ii ⟩ +≡ (88)

```
%token XDIMEN "xdimen"
%token H "h"
%token V "v"
%type < xd > xdimen
```

⟨ scanning rules 3iii ⟩ +≡ (89)

```
xdimen      return XDIMEN;
h           return H;
v           return V;
```

⟨ parsing rules 5v ⟩ +≡ (90)

```
xdimen: dimension number H number V { $$w = $1; $$h = $2; $$v = $4; }
      | dimension number H { $$w = $1; $$h = $2; $$v = 0.0; }
      | dimension number V { $$w = $1; $$h = 0.0; $$v = $2; }
      | dimension { $$w = $1; $$h = 0.0; $$v = 0.0; };
xdimen_node: start XDIMEN xdimen END {
              hput_tags($1, hput_xdimen(&($3))); };
```

Writing the long format: $\Rightarrow$ — — —

⟨ write functions 21xxi ⟩ +≡ (91)

```
void hwrite_xdimen(Xdimen *x)
{ hwrite_dimension(x→w);
  if (x→h ≠ 0.0) { hwrite_float64(x→h, true); hwritec('h'); }
  if (x→v ≠ 0.0) { hwrite_float64(x→v, true); hwritec('v'); }
}

void hwrite_xdimen_node(Xdimen *x)
{ hwrite_start();
  hwritef("xdimen");
  hwrite_xdimen(x);
  hwrite_end();
}
```

Reading the short format:

... $\Rightarrow$

$\langle$ get macros 19xix $\rangle + \equiv$ (92)

```
#define HGET_XDIMEN(I, X)
  if ((I) & b100) HGET32((X).w); else (X).w = 0;
  if ((I) & b010) (X).h = hget_float32(); else (X).h = 0.0;
  if ((I) & b001) (X).v = hget_float32(); else (X).v = 0.0;
```

$\langle$ get functions 18xviii $\rangle + \equiv$ (93)

```
void hget_xdimen(Tag a, Xdimen *x)
{
  switch (a) {
    case TAG(xdimen_kind, b001): HGET_XDIMEN(b001, *x); break;
    case TAG(xdimen_kind, b010): HGET_XDIMEN(b010, *x); break;
    case TAG(xdimen_kind, b011): HGET_XDIMEN(b011, *x); break;
    case TAG(xdimen_kind, b100): HGET_XDIMEN(b100, *x); break;
    case TAG(xdimen_kind, b101): HGET_XDIMEN(b101, *x); break;
    case TAG(xdimen_kind, b110): HGET_XDIMEN(b110, *x); break;
    case TAG(xdimen_kind, b111): HGET_XDIMEN(b111, *x); break;
    default: QUIT("Extent expected got [%s,%d]", NAME(a), INFO(a));
  }
}
```

Note that the info value *b000*, usually indicating a reference, is not supported for extended dimensions. Most nodes that need an extended dimension offer the opportunity to give a reference directly without the start and end byte. An exception is the glue node, but glue nodes that need an extended width are rare.

$\langle$ get functions 18xviii $\rangle + \equiv$ (94)

```
void hget_xdimen_node(Xdimen *x)
{
   $\langle$  read the start byte a 16xvi  $\rangle$ 
  if (KIND(a)  $\equiv$  xdimen_kind) hget_xdimen(a, x);
  else QUIT("Extent expected at 0x%x got %s", node_pos, NAME(a));
   $\langle$  read and check the end byte z 17xvii  $\rangle$ 
}
```

Writing the short format:

$\Rightarrow$...

$\langle$ put functions 14xiv $\rangle + \equiv$ (95)

```
Tag hput_xdimen(Xdimen *x)
{
  Info info = b000;
  if (x  $\rightarrow$  w  $\equiv$  0  $\wedge$  x  $\rightarrow$  h  $\equiv$  0.0  $\wedge$  x  $\rightarrow$  v  $\equiv$  0.0) { HPUT32(0); info |= b100; }
  else {
    if (x  $\rightarrow$  w  $\neq$  0) { HPUT32(x  $\rightarrow$  w); info |= b100; }
    if (x  $\rightarrow$  h  $\neq$  0.0) { hput_float32(x  $\rightarrow$  h); info |= b010; }
    if (x  $\rightarrow$  v  $\neq$  0.0) { hput_float32(x  $\rightarrow$  v); info |= b001; }
  }
  return TAG(xdimen_kind, info);
}
```



```

}
void hput_xdimen_node(Xdimen *x)
{ uint32_t p = hpos++ - hstart;
  hput_tags(p, hput_xdimen(x));
}

```

2.8 Stretch and Shrink

In section 3.5, we will consider glue which is something that can stretch and shrink. The stretchability and shrinkability of the glue can be given in “pt” like a dimension, but there are three more units: **fil**, **fill**, and **filll**. A glue with a stretchability of 1 **fil** will stretch infinitely more than a glue with a stretchability of 1 **pt**. So if you stretch both glues together, the first glue will do all the stretching and the latter will not stretch at all. The “**fil**” glue has simply a higher order of infinity. You might guess that “**fill**” glue and “**filll**” glue have even higher orders of infinite stretchability. The order of infinity is 0 for **pt**, 1 for **fil**, 2 for **fill**, and 3 for **filll**.

The internal representation of a stretch is a variable of type **Stretch**. It stores the floating point value and the order of infinity separate as a **float64_t** and a **uint8_t**.

The short format tries to be space efficient and because it is not necessary to give the stretchability with a precision exceeding about six decimal digits, we use a single 32 bit floating point value. To write a **float32_t** value and an order value as one 32 bit value, we round the two lowest bit of the **float32_t** variable to zero using “round to even” and store the order of infinity in these bits. We define a union type **Stch** to simplify conversion.

```

⟨hint basic types 6vi ⟩ +≡ (96)
typedef enum { normal_o = 0, fil_o = 1, fill_o = 2, filll_o = 3 } Order;
typedef struct { float64_t f; Order o; } Stretch;
typedef union { float32_t f; uint32_t u; } Stch;

```

Writing the short format:

⇒ ...

```

⟨put functions 14xiv ⟩ +≡ (97)
void hput_stretch(Stretch *s)
{ uint32_t mantissa, lowbits, sign, exponent;
  Stch st;

  st.f = s->f;
  DBG(DBGFLOAT, "joining_ %f->%f (0x%X), %d:", s->f, st.f, st.u, s->o);
  mantissa = st.u & (((uint32_t) 1 << FLT_M_BITS) - 1);
  lowbits = mantissa & #7; /* lowest 3 bits */
  exponent = (st.u >> FLT_M_BITS) & (((uint32_t) 1 << FLT_E_BITS) - 1);
  sign = st.u & ((uint32_t) 1 << (FLT_E_BITS + FLT_M_BITS));
  DBG(DBGFLOAT, "s=%d_e=0x%x_m=0x%x", sign, exponent, mantissa);
  switch (lowbits) /* round to even */
  { case 0: break; /* no change */

```

```

case 1: mantissa = mantissa - 1; break;           /* round down */
case 2: mantissa = mantissa - 2; break;           /* round down to even */
case 3: mantissa = mantissa + 1; break;           /* round up */
case 4: break;                                     /* no change */
case 5: mantissa = mantissa - 1; break;           /* round down */
case 6: mantissa = mantissa + 1;           /* round up to even, fall through */
case 7: mantissa = mantissa + 1;           /* round up to even */
    if (mantissa ≥ ((uint32_t) 1 << FLT_M_BITS))
    { exponent++;                                     /* adjust exponent */
      RNG("Float32_exponent", exponent, 1, 2 * FLT_EXCESS);
      mantissa = mantissa >> 1;
    }
    break;
}
DBG(DBGFLOAT, "round_s=%d_e=0x%x_m=0x%x", sign, exponent, mantissa);
st.u = sign | (exponent << FLT_M_BITS) | mantissa | s→o;
DBG(DBGFLOAT, "float_f_hex_0x%x\n", st.f, st.u);
HPUT32(st.u);
}

```

Reading the short format:

... ⇒

```

⟨ get macros 19xix ⟩ +≡ (98)
#define HGET_STRETCH(S)
{ Stch st; HGET32(st.u); S.o = st.u & 3;
  st.u &= ~3;
  S.f = st.f; }

```

Reading the long format:

- - - ⇒

```

⟨ symbols 2ii ⟩ +≡ (99)
%token FIL "fil"
%token FILL "fill"
%token FILLL "filll"
%type < st > stretch
%type < o > order

```

```

⟨ scanning rules 3iii ⟩ +≡ (100)
fil      return FIL;
fill     return FILL;
filll    return FILLL;

```

```

⟨ parsing rules 5v ⟩ +≡ (101)
order: PT { $$ = normal_o; }
      | FIL { $$ = fil_o; } | FILL { $$ = fill_o; } | FILLL { $$ = filll_o; };
stretch: number order { $$.f = $1; $$.o = $2; };

```

Writing the long format:

$\Rightarrow$ - - -

$\langle$ write functions 21xxi $\rangle + \equiv$ (102)

```

void hwrite_order(Order o)
{
    switch (o) {
        case normal_o: hwritef("pt"); break;
        case fil_o: hwritef("fil"); break;
        case fill_o: hwritef("fill"); break;
        case filll_o: hwritef("filll"); break;
        default: QUIT("Illegal_order_%d",o); break;
    }
}

void hwrite_stretch(Stretch *s)
{ hwrite_float64(s→f,false);
  hwrite_order(s→o);
}

```


3 Simple Nodes

3.1 Penalties

Penalties are very simple nodes. They specify the cost of breaking a line or page at the present position. For the internal representation we use an **int32_t**. The full range of integers is, however, not used. Instead penalties must be between -20000 and +20000. (T_EX specifies a range of -10000 to +10000, but plain T_EX uses the value -20000 when it defines the supereject control sequence.) The more general node is called an integer node; it shares the same kind-value *int_kind* = *penalty_kind* but allows the full range of values. The info value of a penalty node is 1 or 2 and indicates the number of bytes used to store the integer. The info value 3 can be used for general integers (see section 10.2) that need four byte of storage.

Reading the long format:

— — — $\Rightarrow$

$\langle$ symbols 2ii $\rangle + \equiv$ (103)
 %token PENALTY "penalty"
 %token INTEGER "int"
 %type < i > penalty

$\langle$ scanning rules 3iii $\rangle + \equiv$ (104)
 penalty **return** PENALTY;
 int **return** INTEGER;

$\langle$ parsing rules 5v $\rangle + \equiv$ (105)
 penalty: integer { RNG("Penalty", \$1, -20000, +20000); \$\$ = \$1; };
 content_node: start PENALTY penalty END { hput_tags(\$1, hput_int(\$3)); };

Reading the short format:

... $\Rightarrow$

```

< cases to get content 20xx > +≡ (106)
  case TAG(penalty_kind, 1): { int32_t p; HGET_PENALTY(1, p); } break;
  case TAG(penalty_kind, 2): { int32_t p; HGET_PENALTY(2, p); } break;
  case TAG(penalty_kind, 3): { int32_t p; HGET_PENALTY(3, p); } break;

```

```

< get macros 19xix > +≡ (107)
#define HGET_PENALTY(I, P)
  if (I == 1) { int8_t n = HGET8; P = n; }
  else if (I == 2) { int16_t n; HGET16(n); RNG("Penalty", n, -20000, +20000);
    P = n; }
  else if (I == 3) { int32_t n; HGET32(n); RNG("Penalty", n, -20000, +20000);
    P = n; }
  hwrite_signed(P);

```

Writing the short format:

$\Rightarrow$...

```

< put functions 14xiv > +≡ (108)
Tag hput_int(int32_t n)
{ Info info;
  if (n >= 0)
  { if (n < #80) { HPUT8(n); info = 1; }
    else if (n < #8000) { HPUT16(n); info = 2; }
    else { HPUT32(n); info = 3; }
  }
  else
  { if (n >= -#80) { HPUT8(n); info = 1; }
    else if (n >= -#8000) { HPUT16(n); info = 2; }
    else { HPUT32(n); info = 3; }
  }
  return TAG(int_kind, info);
}

```

3.2 Languages

To render a HINT file on screen, information about the language is not necessary. Knowing the language is, however, very important for language translation and text to speech conversion which makes texts accessible to the visually-impaired. For this reason, HINT offers the opportunity to add this information and encourages authors to supply this information.

Language information by itself is not sufficient to decode text. It must be supplemented by information about the character encoding (see section 10.4).

To represent language information, the world wide web has set universally accepted standards. The Internet Engineering Task Force IETF has defined tags for identifying languages[15]: short strings like “en” for English or “de” for Deutsch, and longer ones like “sl-IT-nedis”, for the specific variant of the Nadiza dialect of

Slovenian that is spoken in Italy. We assume that any HINT file will contain only a small number of different languages and all language nodes can be encoded using a reference to a predefined node from the definition section (see section 10.5). In the definition section, a language node will just contain the language tag as given in [5] (see section 10.2).

Reading the long format:

— — — $\Rightarrow$

$\langle \text{symbols } 2\text{ii} \rangle + \equiv$ (109)
`%token LANGUAGE "language"`

$\langle \text{scanning rules } 3\text{iii} \rangle + \equiv$ (110)
`language            return LANGUAGE;`

When encoding language nodes in the short format, we use the info value *b000* for language nodes in the definition section and for language nodes in the content section that contain just a one-byte reference (see section 10.5). We use the info value 1 to 7 as a shorthand for the references *0 and *6 to the predefined language nodes.

Reading the short format:

$\dots \Rightarrow$

Writing the long format:

$\Rightarrow$ — — —

$\langle \text{cases to get content } 20\text{xx} \rangle + \equiv$ (111)
`case TAG(language_kind, 1): REF(language_kind, 0); hwrite_ref(0); break;`
`case TAG(language_kind, 2): REF(language_kind, 1); hwrite_ref(1); break;`
`case TAG(language_kind, 3): REF(language_kind, 2); hwrite_ref(2); break;`
`case TAG(language_kind, 4): REF(language_kind, 3); hwrite_ref(3); break;`
`case TAG(language_kind, 5): REF(language_kind, 4); hwrite_ref(4); break;`
`case TAG(language_kind, 6): REF(language_kind, 5); hwrite_ref(5); break;`
`case TAG(language_kind, 7): REF(language_kind, 6); hwrite_ref(6); break;`

Writing the short format:

$\Rightarrow \dots$

$\langle \text{put functions } 14\text{xiv} \rangle + \equiv$ (112)
`Tag hput_language(uint8_t n)`
`{`
`if (n < 7) return TAG(language_kind, n + 1);`
`HPUT8(n);`
`return TAG(language_kind, 0);`
`}`

3.3 Rules

Rules are simply black rectangles having a height, a depth, and a width. All of these dimensions can also be negative but a rule will not be visible unless its width is positive and its height plus depth is positive.

As a specialty, rules can have “running dimensions”. If any of the three dimensions is a running dimension, its actual value will be determined by running the rule up to the boundary of the innermost enclosing box. The width is never running in an horizontal list; the height and depth are never running in a vertical list. In the long format, we use a vertical bar “|” or a horizontal bar “_” (underscore character) to indicate a running dimension. Of course the vertical bar is meant to indicate a running height or depth while the horizontal bar stands for a running width. The parser, however, makes no distinction between the two and you can use either of them. In the short format, we follow T_EX and implement a running dimension by using the special value $-2^{30} = \text{\#C0000000}$.

```
<hint macros 13xiii> +≡ (113)
#define RUNNING_DIMEN \#C0000000
```

It could have been possible to allow extended dimensions in a rule node, but in most circumstances, the mechanism of running dimensions is sufficient and simpler to use. If a rule is needed that requires an extended dimension as its length, it is always possible to put it inside a suitable box and use a running dimension.

To make the short format encoding more compact, the first info bit *b100* will be zero to indicate a running height, bit *b010* will be zero to indicate a running depth, and bit *b001* will be zero to indicate a running width.

Because leaders (see section 5.3) may contain a rule node, we also provide functions to read and write a complete rule node. While parsing the symbol “rule” will just initialize a variable of type **Rule** (the writing is done with a separate routine), parsing a *rule_node* will always include writing it.

```
<hint types 1i> +≡ (114)
typedef struct { Dimen h, d, w; } Rule;
```

Reading the long format:

— — — $\implies$

```
<symbols 2ii> +≡ (115)
%token RULE "rule"
%token RUNNING "|"
%type < d > rule_dimension
%type < r > rule
```

```
<scanning rules 3iii> +≡ (116)
rule return RULE;
"|" return RUNNING;
"_" return RUNNING;
```

```
<parsing rules 5v> +≡ (117)
rule_dimension: dimension | RUNNING { $$ = RUNNING_DIMEN; };
```



```

rule:  rule_dimension rule_dimension rule_dimension
{ $$h = $1; $$d = $2; $$w = $3;
  if ($3 == RUNNING_DIMEN ^ ($1 == RUNNING_DIMEN ^ $2 ==
    RUNNING_DIMEN))
    QUIT("Incompatible_running_dimensions_0x%x_0x%x_0x%x",
      $1,$2,$3);
};

rule_node:  start RULE rule END { hput_tags($1,hput_rule(&($3))); };

content_node:  rule_node;

```

Writing the long format:

⇒ - - -

```

⟨ write functions 21xxi ⟩ += (118)
static void hwrite_rule_dimension(Dimen d, char c)
{ if (d == RUNNING_DIMEN) hwritef("_%c", c);
  else hwrite_dimension(d);
}

void hwrite_rule(Rule *r)
{ hwrite_rule_dimension(r→h, ' ');
  hwrite_rule_dimension(r→d, ' ');
  hwrite_rule_dimension(r→w, ' ');
}

```

Reading the short format:

... ⇒

```

⟨ cases to get content 20xx ⟩ += (119)
case TAG(rule_kind, b011):
{ Rule r; HGET_RULE(b011, r); hwrite_rule(&(r)); } break;
case TAG(rule_kind, b101):
{ Rule r; HGET_RULE(b101, r); hwrite_rule(&(r)); } break;
case TAG(rule_kind, b001):
{ Rule r; HGET_RULE(b001, r); hwrite_rule(&(r)); } break;
case TAG(rule_kind, b110):
{ Rule r; HGET_RULE(b110, r); hwrite_rule(&(r)); } break;
case TAG(rule_kind, b111):
{ Rule r; HGET_RULE(b111, r); hwrite_rule(&(r)); } break;

⟨ get macros 19xix ⟩ += (120)
#define HGET_RULE(I, R)
if ((I) & b100) HGET32((R).h); else (R).h = RUNNING_DIMEN;
if ((I) & b010) HGET32((R).d); else (R).d = RUNNING_DIMEN;
if ((I) & b001) HGET32((R).w); else (R).w = RUNNING_DIMEN;

```

```

⟨ get functions 18xviii ⟩ +≡ (121)
void hget_rule_node(void)
{
  ⟨ read the start byte a 16xvi ⟩
  if (KIND(a) ≡ rule_kind)
  {
    Rule r; HGET_RULE(INFO(a), r);
    hwrite_start(); hwritef("rule"); hwrite_rule(&r); hwrite_end();
  }
  else QUIT("Rule expected at 0x%x got %s", node_pos, NAME(a));
  ⟨ read and check the end byte z 17xvii ⟩
}

```

Writing the short format:

⇒ ...

```

⟨ put functions 14xiv ⟩ +≡ (122)
Tag hput_rule(Rule *r)
{
  Info info = b000;
  if (r→h ≠ RUNNING_DIMEN) { HPUT32(r→h); info |= b100; }
  if (r→d ≠ RUNNING_DIMEN) { HPUT32(r→d); info |= b010; }
  if (r→w ≠ RUNNING_DIMEN) { HPUT32(r→w); info |= b001; }
  return TAG(rule_kind, info);
}

```

3.4 Kerns

A kern is a bit of white space with a certain length. If the kern is part of a horizontal list, the length is measured in the horizontal direction, if it is part of a vertical list, it is measured in the vertical direction. The length of a kern is mostly given as a dimension but provisions are made to use extended dimensions as well.

The typical use of a kern is its insertion between two characters to make the natural distance between them a bit wider or smaller. In the latter case, the kern has a negative length. The typographic optimization just described is called “kerning” and has given the kern node its name. Kerns inserted from font information or math mode calculations are normal kerns, while kerns inserted from $\text{\TeX}$ ’s $\backslash\text{kern}$ or $\backslash/$ commands are explicit kerns. Kern nodes do not disappear at a line break unless they are explicit.

In the long format, explicit kerns are marked with an “!” sign and in the short format with the *b100* info bit. The two low order info bits are: 0 for a reference to a dimension, 1 for a reference to an extended dimension, 2 for an immediate dimension, and 3 for an immediate extended dimension node. To distinguish in the long format between a reference to a dimension and a reference to an extended dimension, the latter is prefixed with the keyword “*xdimen*” (see section 10.5).

```

⟨ hint types 1i ⟩ +≡ (123)
typedef struct { bool x; Xdimen d; } Kern;

```

Reading the long format:

— — — $\Rightarrow$

```

⟨ symbols 2ii ⟩ +≡ (124)
%token KERN "kern"
%token EXPLICIT "!"
%type < b > explicit
%type < kt > kern

```

```

⟨ scanning rules 3iii ⟩ +≡ (125)
kern          return KERN;
!             return EXPLICIT;

```

```

⟨ parsing rules 5v ⟩ +≡ (126)
explicit: { $$ = false; } | EXPLICIT { $$ = true; };
kern:    explicit xdimen { $$ .x = $1; $$ .d = $2; };
content_node: start KERN kern END { hput_tags($1, hput_kern(&($3))); }

```

Writing the long format:

$\Rightarrow$ — — —

```

⟨ write functions 21xxi ⟩ +≡ (127)
void hwrite_explicit(bool x)
{ if (x) hwritef("_!"); }
void hwrite_kern(Kern *k)
{ hwrite_explicit(k→x);
  if (k→d.h ≡ 0.0 ∧ k→d.v ≡ 0.0 ∧ k→d.w ≡ 0) hwrite_ref(zero_dimen_no);
  else hwrite_xdimen(&(k→d));
}

```

Reading the short format:

... $\Rightarrow$

```

⟨ cases to get content 20xx ⟩ +≡ (128)
case TAG(kern_kind, b010): { Kern k; HGET_KERN(b010, k); } break;
case TAG(kern_kind, b011): { Kern k; HGET_KERN(b011, k); } break;
case TAG(kern_kind, b110): { Kern k; HGET_KERN(b110, k); } break;
case TAG(kern_kind, b111): { Kern k; HGET_KERN(b111, k); } break;

```

```

⟨ get macros 19xix ⟩ +≡ (129)
#define HGET_KERN(I, K) K.x = (I) & b100;
if (((I) & b011) ≡ 2) { HGET32(K.d.w); K.d.h = K.d.v = 0.0; }
else if (((I) & b011) ≡ 3) hget_xdimen_node(&(K.d));
hwrite_kern(&k);

```

Writing the short format:

$\Rightarrow \dots$

```

⟨put functions 14xiv⟩ +≡ (130)
Tag hput_kern(Kern *k)
{ Info info;
  if (k→x) info = b100; else info = b000;
  if (k→d.h ≡ 0.0 ∧ k→d.v ≡ 0.0) {
    if (k→d.w ≡ 0) HPUT8(zero_dimen_no);
    else { HPUT32(k→d.w); info = info | 2; }
  }
  else { hput_xdimen_node(&(k→d)); info = info | 3; }
  return TAG(kern_kind, info);
}

```

3.5 Glue

We have seen in section 2.8 how to deal with stretchability and shrinkability and we will need this now. Glue has a natural width—which in general can be an extended dimension—and in addition it can stretch and shrink. It might have been possible to allow an extended dimension also for the stretchability or shrinkability of a glue, but this seems of little practical relevance and so simplicity won over generality. Even with that restriction, it is an understatement to regard glue nodes as “simple” nodes.

To use the info bits in the short format wisely, I collected some statistical data using the T_EXbook as an example. It turns out that about 99% of all the 58937 glue nodes (not counting the interword glues used inside texts) could be covered with only 43 predefined glues. So this is by far the most common case; we reserve the info value *b000* to cover it and postpone the description of such glue nodes until we describe references in section 10.5.

We expect the remaining cases to contribute not too much to the file size, and hence, simplicity is a more important aspect than efficiency when allocating the remaining info values.

Looking at the glues in more detail, we find that the most common cases are those where either one, two, or all three glue components are zero. We use the two lowest bits to indicate the presence of a nonzero stretchability or shrinkability and reserve the info values *b001*, *b010*, and *b011* for those cases where the width of the glue is zero. The zero glue, where all components are zero, is defined as a fixed, predefined glue instead of reserving a special info value for it. The cost of one extra byte when encoding it seems not too high a price to pay. After reserving the info value *b111* for the most general case of a glue, we have only three more info values left: *b100*, *b101*, and *b110*. Keeping things simple implies using the two lowest info bits—as before—to indicate a nonzero stretchability or shrinkability. For the width, three choices remain: using a reference to a dimension, using a reference to an extended dimension, or using an immediate value. Since references to glues are already supported, an immediate width seems best for glues that are not frequently reused, avoiding the overhead of references.

Here is a summary of the info bits and the implied layout of glue nodes in the short format:

- *b000*: reference to a predefined glue
- *b001*: zero width and nonzero shrinkability
- *b010*: zero width and nonzero stretchability
- *b011*: zero width and nonzero stretchability and shrinkability
- *b100*: nonzero width
- *b101*: nonzero width and nonzero shrinkability
- *b110*: nonzero width and nonzero stretchability
- *b111*: extended dimension and nonzero stretchability and shrinkability

$\langle \text{hint basic types } 6\text{vi} \rangle + \equiv$ (131)
typedef struct { **Xdimen** *w*; **Stretch** *p*, *m*; } **Glue**;

To test for a zero glue, we implement a macro:

$\langle \text{hint macros } 13\text{xiii} \rangle + \equiv$ (132)
#define ZERO_GLUE(*G*)
 ((*G*).*w.w* $\equiv$ 0 $\wedge$ (*G*).*w.h* $\equiv$ 0.0 $\wedge$ (*G*).*w.v* $\equiv$ 0.0 $\wedge$ (*G*).*p.f* $\equiv$ 0.0 $\wedge$ (*G*).*m.f* $\equiv$ 0.0)

Because other nodes (leaders, baselines, and fonts) contain glue nodes as parameters, we provide functions to read and write a complete glue node in the same way as we did for rule nodes. Further, such an internal *glue_node* has the special property that in the short format a node for the zero glue might be omitted entirely.

Reading the long format: - - - $\Rightarrow$

$\langle \text{symbols } 2\text{ii} \rangle + \equiv$ (133)
%token GLUE "glue"
%token PLUS "plus"
%token MINUS "minus"
%type < *g* > glue
%type < *b* > glue_node
%type < *st* > plus minus

$\langle \text{scanning rules } 3\text{iii} \rangle + \equiv$ (134)
glue **return** GLUE;
plus **return** PLUS;
minus **return** MINUS;

$\langle \text{parsing rules } 5\text{v} \rangle + \equiv$ (135)
plus: { $$$f = 0.0$; $$$o = 0$; }
 | PLUS *stretch* { $$$ = \2 ; };
minus: { $$$f = 0.0$; $$$o = 0$; }
 | MINUS *stretch* { $$$ = \2 ; };
glue: *xdimen plus minus* { $$$w = \1 ; $$$p = \2 ; $$$m = \3 ; };

```

content_node: start GLUE glue END {
    if (ZERO_GLUE($3)) { HPUT8(zero_skip_no);
        hput_tags($1, TAG(glue_kind, 0));
    }
    else hput_tags($1, hput_glue(&($3)));
};

glue_node: start GLUE glue END
    { if (ZERO_GLUE($3)) { hpos --; $$ = false; }
      else { hput_tags($1, hput_glue(&($3))); $$ = true; } };

```

Writing the long format:

$\Rightarrow$ — — —

```

⟨ write functions 21xxi ⟩ +≡ (136)
void hwrite_plus(Stretch *p)
{ if (p→f ≠ 0.0) { hwritef("_plus"); hwrite_stretch(p); }
}

void hwrite_minus(Stretch *m)
{ if (m→f ≠ 0.0) { hwritef("_minus"); hwrite_stretch(m); }
}

void hwrite_glue(Glue *g)
{ hwrite_xdimen(&(g→w)); hwrite_plus(&g→p); hwrite_minus(&g→m);
}

void hwrite_ref_node(Kind k, uint8_t n);

void hwrite_glue_node(Glue *g)
{ if (ZERO_GLUE(*g)) hwrite_ref_node(glue_kind, zero_skip_no);
  else { hwrite_start(); hwritef("glue"); hwrite_glue(g); hwrite_end(); }
}

```

Reading the short format:

$\dots \Rightarrow$

```

⟨ cases to get content 20xx ⟩ +≡ (137)
case TAG(glue_kind, b001):
    { Glue g; HGET_GLUE(b001, g); hwrite_glue(&g); } break;
case TAG(glue_kind, b010):
    { Glue g; HGET_GLUE(b010, g); hwrite_glue(&g); } break;
case TAG(glue_kind, b011):
    { Glue g; HGET_GLUE(b011, g); hwrite_glue(&g); } break;
case TAG(glue_kind, b100):
    { Glue g; HGET_GLUE(b100, g); hwrite_glue(&g); } break;
case TAG(glue_kind, b101):
    { Glue g; HGET_GLUE(b101, g); hwrite_glue(&g); } break;
case TAG(glue_kind, b110):
    { Glue g; HGET_GLUE(b110, g); hwrite_glue(&g); } break;
case TAG(glue_kind, b111):
    { Glue g; HGET_GLUE(b111, g); hwrite_glue(&g); } break;

```

```

⟨ get macros 19xix ⟩ +≡ (138)
#define HGET_GLUE(I, G){
    if ((I) ≠ b111) {
        if ((I) & b100) HGET32((G).w.w); else (G).w.w = 0;
    }
    if ((I) & b010) HGET_STRETCH((G).p) else (G).p.f = 0.0, (G).p.o = 0;
    if ((I) & b001) HGET_STRETCH((G).m) else (G).m.f = 0.0, (G).m.o = 0;
    if ((I) ≡ b111) hget_xdimen_node(&((G).w));
    else (G).w.h = (G).w.v = 0.0; }

```

The *hget_glue_node* can cope with a glue node that is omitted and will supply a zero glue instead.

```

⟨ get functions 18xviii ⟩ +≡ (139)
void hget_glue_node(void)
{ ⟨ read the start byte a 16xvi ⟩
  if (KIND(a) ≠ glue_kind) { hpos--;
    hwrite_ref_node(glue_kind, zero_skip_no); return; }
  if (INFO(a) ≡ b000) { uint8_t n = HGET8; REF(glue_kind, n);
    hwrite_ref_node(glue_kind, n); }
  else { Glue g; HGET_GLUE(INFO(a), g); hwrite_glue_node(&g); }
  ⟨ read and check the end byte z 17xvii ⟩
}

```

Writing the short format:

⇒ ...

```

⟨ put functions 14xiv ⟩ +≡ (140)
Tag hput_glue(Glue *g)
{ Info info = b000;
  if (ZERO_GLUE(*g)) { HPUT8(zero_skip_no); info = b000;
  }
  else if ((g→w.w ≡ 0 ∧ g→w.h ≡ 0.0 ∧ g→w.v ≡ 0.0)) {
    if (g→p.f ≠ 0.0) { hput_stretch(&g→p); info |= b010; }
    if (g→m.f ≠ 0.0) { hput_stretch(&g→m); info |= b001; }
  }
  else if (g→w.h ≡ 0.0 ∧ g→w.v ≡ 0.0 ∧ (g→p.f ≡ 0.0 ∨ g→m.f ≡ 0.0)) {
    HPUT32(g→w.w); info = b100;
    if (g→p.f ≠ 0.0) { hput_stretch(&g→p); info |= b010; }
    if (g→m.f ≠ 0.0) { hput_stretch(&g→m); info |= b001; }
  }
  else
  { hput_stretch(&g→p); hput_stretch(&g→m);
    hput_xdimen_node(&(g→w));
    info = b111;
  }
  return TAG(glue_kind, info);
}

```


4 Lists

When a node contains multiple other nodes, we package these nodes into a list node. It is important to note that list nodes never occur as individual nodes, they only occur as parts of other nodes. In total, we have three different types of lists: plain lists that use the kind-value *list_kind*, text lists that use the kind-value *list_kind* together with the info bit *b100*, and parameter lists that use the kind-value *param_kind*. A description of the first two types of lists follows here. Parameter lists are described in section 10.3.

Because lists are of variable size, it is not possible in the short format to tell from the kind and info bits of a tag byte the size of the list node. So advancing from the beginning of a list node to the next node after the list is not as simple as usual. To solve this problem, we store the size of the list immediately after the start byte and before the end byte. Alternatively we could require programs to traverse the entire list. The latter solution is more compact but inefficient for list with many nodes; our solution will cost some extra bytes, but the amount of extra bytes will only grow logarithmically with the size of the HINT file. It would be possible to allow both methods so that a HINT file could balance size and time trade-offs by making small lists—where the size can be determined easily by reading the entire list—without size information and making large lists with size information so that they can be skipped easily without reading them. But the added complexity seems too high a price to pay.

Now consider the problem of reading a content stream starting at an arbitrary position i in the middle of the stream. This situation occurs naturally when resynchronizing a content stream after an error has been detected, but implementing links poses a similar problem. We can inspect the byte at position i and see if it is a valid tag. If yes, we are faced with the problem of verifying that this is not a mere coincidence. So we determine the size s of the node. If the byte in question is a start byte, we should find a matching byte s bytes later in the stream; if it is an end byte, we should find the matching byte s bytes earlier in the stream; if we find no matching byte, this was neither a start nor an end byte. If we find exactly one matching byte, we can be quite confident (error probability $1/256$ if assuming equal probability of all byte values) that we have found a tag, and we know whether it is the beginning or the end tag. If we find two matching byte, we have most likely the start or the end of a node, but we do not know which of the two. To find out which of the two possibilities is true or to reduce the probability of an error, we can check the start and end byte of the node immediately preceding a start byte or immediately following an end byte in a similar way. By testing

two more byte, this additional check will reduce the error probability further to $1/2^{24}$ (under the same assumption as before). So checking more nodes is rarely necessary. This whole schema would, however, not work if we happen to find a tag byte that indicated either the begin or the end of a list without specifying the size of the list. Sure, we can verify the bytes before and after it to find out whether the byte following it is the begin of a node and the byte preceding it is the end of a node, but we still don't know if the byte itself starts a node list or ends a node list. Even reading along in either direction until finding a matching tag will not answer the question. The situation is better if we specify a size: we can read the suspected size after or before the tag and check if we find a matching tag and size at the position indicated.

In the short format, we use the two lower bits of the *info* value to indicate the number of byte used to store the list size: A list with $\text{info} \& \#3 = 1$ uses 1 byte, with $\text{info} \& \#3 = 2$ uses 2 byte, and with $\text{info} \& \#3 = 3$ uses 4 byte. The $\text{info} \& \#3$ value zero is reserved for references to predefined lists. An empty list is always represented using zero as the reference number. General predefined lists are currently implemented only for parameter lists.

Storing the list size immediately preceding the end tag creates a new problem: If we try to recover from an error, we might not know the size of the list and searching for the end of a list, we might be unable to tell the difference between the bytes that encode the list size and the start tag of a possible next node. If we parse the content backward, the problem is completely symmetric.

To solve the problem, we insert an additional byte immediately before the final size and after the initial size marking the size boundary. We choose the byte values $\#FF$, $\#FE$, and $\#FD$ which can not be confused with valid tag bytes and indicate that the size is stored using 1, 2, or 4 byte respectively. Under regular circumstances, these bytes are simply skipped. When searching for the list end (or start) these bytes would correspond to $\text{TAG}(\text{penalty_kind}, i)$ with $7 \geq i \geq 5$ and can not be confused with valid penalty nodes which use only the info values 0, 1, and 2. An empty list always uses the info value 0 and the reference value 0.

We are a bit lazy when it comes to the internal representation of a list. Since we need the representation as a short format byte sequence anyway, it consists of the position p of the start of the byte sequence combined with an integer s giving the size of the byte sequence. If the list is empty, s is zero.

```
< hint types 1i > +≡ (141)
typedef struct { Tag t; uint32_t p; uint32_t s; } List;
```

The major drawback of this choice of representation is that it ties together the reading of the long format and the writing of the short format; these are no longer independent. So starting with the present section, we have to take the short format representation of a node into account already when we parse the long format representation.

In the long format, we may start a list node with an estimate of the size needed to store the list in the short format. We do not want to require the exact size because this would make editing of long format HINT files almost impossible. Of course this makes it also impossible to derive the exact s value of the internal

representation from the long format representation. Therefore we start by parsing the estimate of the list size and use it to reserve the necessary number of byte to store the size. Then we parse the *content_list*. As a side effect—and this is an important point—this will write the list content in short format into the output buffer. As mentioned above, whenever a node contains a list, we need to consider this side effect when we give the parsing rules. We will see examples for this in section 5.

The function *hput_list* will be called *after* the short format of the list is written to the output. Before we pass the internal representation of the list to the *hput_list* function, we update *s* and *p*. Further, we pass the position in the stream where the list size and its boundary mark is supposed to be. Before *hput_list* is called, space for the tag, the size, and the boundary mark is allocated based on the estimate. The function *hsize_bytes* computes the number of byte required to store the list size, and the function *hput_list_size* will later write the list size. If the estimate turns out to be wrong, the list data can be moved to make room for a larger or smaller size field.

If the long format does not specify a size estimate, a suitable default must be chosen. A statistical analysis shows that most plain lists need only a single byte to store the size; and even the total amount of data contained in these lists exceeds the amount of data stored in longer lists by a factor of about 3. Hence if we do not have an estimate, we reserve only a single byte to store the size of a list. The statistics looks different for lists stored as a text: The number of texts that require two byte for the size is slightly larger than the number of texts that need only one byte, and the total amount of data stored in these texts is larger by a factor of 2 to 7 than the total amount of data found in all other texts. Hence as a default, we reserve two byte to store the size for texts.

4.1 Plain Lists

Plain list nodes start and end with a tag of kind *list_kind*. Not uncommon are empty lists; these can be stored using *info* = 0 and a reference to the predefined empty list.

Writing the long format uses the fact that the function *hget_content_node*, as implemented in the **stretch** program, will output the node in the long format.

Reading the long format:

— — — $\Rightarrow$

$\langle$ symbols 2ii $\rangle + \equiv$ (142)

%**type** < l > list

%**type** < u > position content_list

$\langle$ parsing rules 5v $\rangle + \equiv$ (143)

position: { \$\$ = hpos - hstart; };

content_list: position | content_list content_node;

estimate: { hpos += 2; } | UNSIGNED { hpos += hsize_bytes(\$1) + 1; };

list: start estimate content_list END

{ \$\$t = TAG(list_kind, b010); \$\$p = \$3; \$\$s = (hpos - hstart) - \$3;
hput_tags(\$1, hput_list(\$1 + 1, &(\$\$))); };

Writing the long format:

⇒ - - -

```

< write functions 21xxi > += (144)
void hwrite_list(List *l)
{ uint32_t h = hpos - hstart, e = hend - hstart; /* save hpos and hend */
  hpos = l→p + hstart; hend = hpos + l→s;
  if (KIND(l→t) ≡ list_kind) {
    if (INFO(l→t) & b100) < write a text 155clv >
    else < write a list 145cxlv >
  }
  else QUIT("List expected got %s", content_name[KIND(l→t)]);
  hpos = hstart + h; hend = hstart + e; /* restore hpos and hend */
}

< write a list 145cxlv > ≡ (145)
{ if (l→s ≡ 0) hwritef("_<>");
  else
  { DBG(DBGNODE, "Write_list_at_0x%x_size=%u\n", l→p, l→s);
    hwrite_start(); if (section_no ≡ 2) hwrite_label();
    if (l→s > #FF) hwritef("%d", l→s);
    while (hpos < hend) hget_content_node();
    hwrite_end();
  }
}

```

Used in 144cxliv.

Reading the short format:

... ⇒

```

< shared get functions 53liii > += (146)
void hget_size_boundary(Info info)
{ uint32_t n;
  info = info & #3;
  if (info ≡ 0) return;
  n = HGET8;
  if (n ≠ #100 - info) QUIT("Non_matching_boundary_byte_0x%x_within\
    fo_value%d_at_0x%x", n, info, (uint32_t)(hpos - hstart - 1));
}

uint32_t hget_list_size(Info info)
{ uint32_t n = 0;
  info = info & #3;
  if (info ≡ 0) return 0;
  else if (info ≡ 1) n = HGET8;
  else if (info ≡ 2) HGET16(n);
  else if (info ≡ 3) HGET32(n);
  else QUIT("List_info%d must be 0, 1, 2, or 3", info);
  return n;
}

```

```

}
void hget_list(List *l)
{ if (KIND(*hpos)  $\neq$  list_kind  $\wedge$  KIND(*hpos)  $\neq$  param_kind)
  QUIT("List expected at 0x%x", (uint32_t)(hpos - hstart));
else {  $\langle$  read the start byte a 16xvi  $\rangle$ 
  l  $\rightarrow$  t = a;
  HGET_LIST(INFO(a), *l);
   $\langle$  read and check the end byte z 17xvii  $\rangle$ 
  DBG(DBGNODE, "Get list at 0x%x size=%u\n", l  $\rightarrow$  p, l  $\rightarrow$  s);
}
}

```

If a list has the info value zero, the list is the empty list. Other list references are currently not implemented.

```

 $\langle$  shared get macros 38xxxviii  $\rangle$  += (147)
#define HGET_LIST(I, L)
if (((I) & #3)  $\equiv$  0) { uint8_t n = HGET8; REF_RNG(KIND((L).t), n); (L).s = 0; }
else { (L).s = hget_list_size(I);
  hget_size_boundary(I);
  (L).p = hpos - hstart;
  hpos = hpos + (L).s;
  hget_size_boundary(I);
  { uint32_t s = hget_list_size(I);
    if (s  $\neq$  (L).s)
      QUIT("List sizes at 0x%x and \"SIZE_F\" do not match 0x%x \
        != 0x%x", node_pos + 1, hpos - hstart - I - 1, (L).s, s);
  }
}

```

Writing the short format:

$\Rightarrow \dots$

```

 $\langle$  put functions 14xiv  $\rangle$  += (148)
uint8_t hsize_bytes(uint32_t n)
{ if (n  $\equiv$  0) return 0;
  else if (n < #100) return 1;
  else if (n < #10000) return 2;
  else return 4;
}

void hput_list_size(uint32_t n, int i)
{ if (i  $\equiv$  0) return;
  else if (i  $\equiv$  1) HPUT8(n);
  else if (i  $\equiv$  2) HPUT16(n);
  else HPUT32(n);
}

```

```

Tag hput_list(uint32_t start_pos, List *l)
{
  if (l→s == 0) { hpos = hstart + start_pos; HPUT8(0);
    return TAG(KIND(l→t), INFO(l→t) & b100); }
  else
  {
    uint32_t list_end = hpos - hstart;
    int i = l→p - start_pos - 1; /* number of byte allocated for size */
    int j = hsize_bytes(l→s); /* number of byte needed for size */
    Info k;
    if (j == 4) k = 3;
    else k = j;
    DBG(DBGNODE, "Put_list_at_0x%x_size=%u\n", l→p, l→s);
    if (i > j & l→s > #100) j = i; /* avoid moving large lists */
    if (i != j)
    {
      int d = j - i;
      DBG(DBGNODE, "Moving_%u_byte_by_%d\n", l→s, d);
      if (d > 0) HPUTX(d);
      memmove(hstart + l→p + d, hstart + l→p, l→s);
      { adjust label positions after moving a list 258ccclviii }
      l→p = l→p + d; list_end = list_end + d;
    }
    hpos = hstart + start_pos; hput_list_size(l→s, j); HPUT8(#100 - k);
    hpos = hstart + list_end; HPUT8(#100 - k); hput_list_size(l→s, j);
    return TAG(KIND(l→t), k | (INFO(l→t) & b100));
  }
}

```

4.2 Texts

A Text is a list of nodes with a representation optimized for character nodes. In the long format, a sequence of characters like “Hello” is written “<glyph ’H’ *0> <glyph ’e’ *0> <glyph ’l’ *0> <glyph ’l’ *0> <glyph ’o’ *0>”, and even in the short format it requires 4 byte per character! As a text, the same sequence is written “Hello” in the long format and the short format requires usually just 1 byte per character. Indeed except the bytes with values from #00 to #20, which are considered control codes, all bytes and all UTF-8 multibyte sequences are simply considered character codes. They are equivalent to a glyph node in the “current font”. The current font is font number 0 at the beginning of a text and it can be changed using the control codes. We introduce the concept of a “current font” because we do not expect the font to change too often, and it allows for a more compact representation if we do not store the font with every character code. It has an important disadvantage though: storing only font changes prevents us from parsing a text backwards; we always have to start at the beginning of the text, where the font is known to be font number 0.

Defining a second format for encoding lists of nodes adds another difficulty to the problem we had discussed at the beginning of section 4. When we try to recover from an error and start reading a content stream at an arbitrary position, the first

thing we need to find out is whether at this position we have the tag byte of an ordinary node or whether we have a position inside a text.

Inside a text, character nodes start with a byte in the range #21–#F7. This is a wide range and it overlaps considerably with the range of valid tag bytes. It is however possible to choose the kind-values in such a way that the control codes do not overlap with the valid tag bytes that start a node. For this reason, the values *list_kind* $\equiv$ 0, *param_kind* $\equiv$ 1, *range_kind* $\equiv$ 2, *xdimen_kind* $\equiv$ 3, and *adjust_kind* $\equiv$ 4 were chosen on page 5. Lists, parameter lists, and extended dimensions occur only *inside* of content nodes, but are not content nodes in their own right; page ranges occur only in the definition section; so the values #00 to #1F are not used as tag bytes of content nodes. The value #20 would, as a tag byte, indicate an adjust node (*adjust_kind* $\equiv$ 4) with info value zero. Because there are no predefined adjustments, #20 is not used as a tag byte either. (An alternative choice would be to use the kind value 4 for paragraph nodes because there are no predefined paragraphs.)

The largest byte that starts an UTF8 code is #F7; hence, there are eight possible control codes, from #F8 to #FF, available. The first three values #F8, #F9, and #FA are actually used for penalty nodes with info values, 0, 1, and 2. The last three #FD, #FE, and #FF are used as boundary marks for the text size and therefore we can use only #FB and #FC as control codes.

In the long format, we do not provide a syntax for specifying a size estimate as we did for plain lists, because we expect text to be quite short. We allocate two byte for the size and hope that this will prove to be sufficient most of the time. Further, we will disallow the use of non-printable ASCII codes, because these are—by definition—not very readable, and we will give special meaning to some of the printable ASCII codes because we will need a notation for the beginning and ending of a text, for nodes inside a text, and the control codes.

Here are the details:

- In the long format, a text starts and ends with a double quote character “””. In the short format, texts are encoded similar to lists setting the info bit *b100*.
- Arbitrary nodes can be embedded inside a text. In the long format, they are enclosed in pointed brackets < ... > as usual. In the short format, an arbitrary node can follow the control code *txt_node* = #1E. Because text may occur in nodes, the scanner needs to be able to parse texts nested inside nodes nested inside nodes nested inside texts ... To accomplish this, we use the “stack” option of **flex** and include the pushing and popping of the stack in the macros **SCAN_START** and **SCAN_END**.
- The space character “_” with ASCII value #20 stands in both formats for the font specific interword glue node (control code *txt_glue*).
- The hyphen character “-” in the long format and the control code *txt_hyphen* = #1F in the short format stand for the font specific discretionary hyphenation node.
- In the long format, the backslash character “\” is used as an escape character. It is used to introduce notations for control codes, as described below, and to access the character codes of those ASCII characters that otherwise carry a

special meaning. For example “\” denotes the character code of the double quote character “””; and similarly “\\”, “\<”, “\>”, “_”, and “\–” denote the character codes of “\”, “<”, “>”, “_”, and “–” respectively.

- In the long format, a TAB-character (ASCII code #09) is silently converted to a space character (ASCII code #20); a NL-character (ASCII code #0A), together with surrounding spaces, TAB-characters, and CR-characters (ASCII code #0D), is silently converted to a single space character. All other ASCII characters in the range #00 to #1F are not allowed inside a text. This rule avoids the problems arising from “invisible” characters embedded in a text and it allows to break texts into lines, even with indentation, at word boundaries.

To allow breaking a text into lines without inserting spaces, a NL-character together with surrounding spaces, TAB-characters, and CR-characters is completely ignored if the whole group of spaces, TAB-characters, CR-characters, and the NL-character is preceded by a backslash character.

For example, the text “There is no more gas in the tank.” can be written as

```
"There is
→  no more gas
→  as in the tank."
```

To break long lines when writing a long format file, we use the variable *txt.length* to keep track of the approximate length of the current line.

- The control codes *txt.font* = #00, #01, #02, ..., and #07 are used to change the current font to font number 0, 1, 2, ..., and 7. In the long format these control codes are written \0, \1, \2, ..., and \7.
- The control code *txt.global* = #08 is followed by a second parameter byte. If the value of the parameter byte is *n*, it will set the current font to font number *n*. In the long format, the two byte sequence is written “\Fn\” where *n* is the decimal representation of the font number.
- The control codes #09, #0A, #0B, #0C, #0D, #0E, #0F, and #10 are also followed by a second parameter byte. They are used to reference the global definitions of penalty, kern, ligature, disc, glue, language, rule, and image nodes. The parameter byte contains the reference number. For example, the byte sequence #09 #03 is equivalent to the node <penalty *3>. In the long format these two-byte sequences are written, “\Pn\” (penalty), “\Kn\” (kern), “\Ln\” (ligature), “\Dn\” (disc), “\Gn\” (glue), “\Sn\” (speak or German “Sprache”), “\Rn\” (rule), and “\In\” (image), where *n* is the decimal representation of the parameter value.
- The control codes from *txt.local* = #11 to #1C are used to reference one of the 12 font specific parameters. In the long format they are written “\a”, “\b”, “\c”, ..., “\j”, “\k”, “\l”.
- The control code *txt.cc* = #1D is used as a prefix for an arbitrary character code represented as an UTF-8 multibyte sequence. Its main purpose is providing a method for including character codes less or equal to #20 which otherwise would be considered control codes. In the long format, the byte sequence is written “\Cn\” where *n* is the decimal representation of the character code.

- The control code *txt_node* = #1E is used as a prefix for an arbitrary node in short format. In the long format, it is written “<” and is followed by the node content in long format terminated by “>”.
- The control code *txt_hyphen* = #1F is used to access the font specific discretionary hyphen. In the long format it is simply written as “-”.
- The control code *txt_glue* = #20 is the space character, it is used to access the font specific interword glue. In the long format, we use the space character “_” as well.
- The control code *txt_ignore* = #FB is ignored, its position can be used in a link to specify a position between two characters. In the long format it is written as “\@”.
- The control code #FC is currently unused.

For the control codes, we define an enumeration type and for references, a reference type.

```
< hint types 1i > +≡ (149)
typedef enum {
    txt_font = #00, txt_global = #08, txt_local = #11, txt_cc = #1D,
    txt_node = #1E, txt_hyphen = #1F, txt_glue = #20, txt_ignore = #FB
} Txt;
```

Reading the long format: — — — ⇒

```
< scanning definitions 24xxiv > +≡ (150)
%x TXT
```

```
< symbols 2ii > +≡ (151)
%token TXT_START TXT_END TXT_IGNORE
%token TXT_FONT_GLUE TXT_FONT_HYPHEN
%token < u > TXT_FONT TXT_LOCAL
%token < rf > TXT_GLOBAL
%token < u > TXT_CC
%type < u > text
```

```
< scanning rules 3iii > +≡ (152)
\"          SCAN_TXT_START; return TXT_START;
< TXT > {
\"          SCAN_TXT_END; return TXT_END;
"<"        SCAN_START; return START;
">"        QUIT(">_not_allowed_in_text_mode");
\\\\"      yylval.u = '\\'; return TXT_CC;
\\\"        yylval.u = '\"'; return TXT_CC;
\\\"<"      yylval.u = '<'; return TXT_CC;
\\\">"      yylval.u = '>'; return TXT_CC;
```

```

\\ "_"          yyval.u = '_' ; return TXT_CC;
\\ "-"          yyval.u = '-' ; return TXT_CC;
\\ "@"          return TXT_IGNORE;
[_\\t\\r]*(\\n[_\\t\\r]*)+ return TXT_FONT_GLUE;
\\[_\\t\\r]*\\n[_\\t\\r]* ;
\\[0-7]         yyval.u = yytext[1] - '0' ; return TXT_FONT;
\\F[0-9]+\\      SCAN_REF(font_kind) ; return TXT_GLOBAL;
\\P[0-9]+\\      SCAN_REF(penalty_kind) ; return TXT_GLOBAL;
\\K[0-9]+\\      SCAN_REF(kern_kind) ; return TXT_GLOBAL;
\\L[0-9]+\\      SCAN_REF(ligature_kind) ; return TXT_GLOBAL;
\\D[0-9]+\\      SCAN_REF(disc_kind) ; return TXT_GLOBAL;
\\G[0-9]+\\      SCAN_REF(glue_kind) ; return TXT_GLOBAL;
\\S[0-9]+\\      SCAN_REF(language_kind) ; return TXT_GLOBAL;
\\R[0-9]+\\      SCAN_REF(rule_kind) ; return TXT_GLOBAL;
\\I[0-9]+\\      SCAN_REF(image_kind) ; return TXT_GLOBAL;
\\C[0-9]+\\      SCAN_UDEC(yytext + 2) ; return TXT_CC;
\\[a-1]         yyval.u = yytext[1] - 'a' ; return TXT_LOCAL;
"_"           return TXT_FONT_GLUE;
"-"           return TXT_FONT_HYPHEN;
{UTF8_1}      SCAN_UTF8_1(yytext) ; return TXT_CC;
{UTF8_2}      SCAN_UTF8_2(yytext) ; return TXT_CC;
{UTF8_3}      SCAN_UTF8_3(yytext) ; return TXT_CC;
{UTF8_4}      SCAN_UTF8_4(yytext) ; return TXT_CC;
}

```

⟨ scanning macros 23xxiii ⟩ +≡ (153)

```

#define SCAN_REF(K) yyval.rf.k = K; yyval.rf.n = atoi(yytext + 2)
    static int scan_level = 0;
#define SCAN_START yy_push_state(INITIAL); if (1 ≡ scan_level++)
    hpos0 = hpos;
#define SCAN_END
    if (scan_level--) yy_pop_state();
    elseQUIT("Too_many_>'_in_line%d", yylineno)
#define SCAN_TXT_START BEGIN(TXT)
#define SCAN_TXT_END BEGIN(INITIAL)

```

⟨ parsing rules 5v ⟩ +≡ (154)

```

list:  TXT_START position
    { hpos += 4;          /* start byte, two size byte, and boundary byte */
    } text TXT_END
    { $$t = TAG(list_kind, b110); $$p = $4; $$s = (hpos - hstart) - $4;
    hput_tags($2, hput_list($2 + 1, &($$))); };

```

```

text: position | text txt;
txt:  TXT_CC { hput_txt_cc($1); }
      | TXT_FONT { REF(font_kind,$1); hput_txt_font($1); }
      | TXT_GLOBAL { REF($1.k,$1.n); hput_txt_global(&($1)); }
      | TXT_LOCAL { RNG("Font_parameter", $1, 0, 11); hput_txt_local($1); }
      | TXT_FONT_GLUE { HPUTX(1); HPUT8(txt_glue); }
      | TXT_FONT_HYPHEN { HPUTX(1); HPUT8(txt_hyphen); }
      | TXT_IGNORE { HPUTX(1); HPUT8(txt_ignore); }
      | { HPUTX(1); HPUT8(txt_node); } content_node;

```

The following function keeps track of the position in the current line. If the line gets too long it will break the text at the next space character. If no suitable space character comes along, the line will be broken after any regular character.

Writing the long format:

⇒ — — —

```

⟨ write a text 155clv ⟩ ≡ (155)
{ if (l→s ≡ 0) hwritef("\ \\"");
  else
  { int pos = nesting + 20; /* estimate */
    hwritef("\ \");
    while (hpos < hend)
    { int i = hget_txt();
      if (i < 0) {
        if (pos++ < 70) hwritec(' ');
        else hwrite_nesting(), pos = nesting;
      }
      else if (i ≡ 1 ∧ pos ≥ 100)
      { hwritec(' '); hwrite_nesting(); pos = nesting; }
      else pos += i;
    }
    hwritec(' ');
  }
}

```

Used in 144cxliv.

The function returns the number of characters written because this information is needed in *hget_txt* below.

```

⟨ write functions 21xxi ⟩ +≡ (156)
int hwrite_txt_cc(uint32_t c)
{ if (c < #20) return hwritef("\ \C%d\ ", c);
  else switch (c) {
    case '\ ': return hwritef("\ \ \ ");
    case ' ': return hwritef("\ \ \ ");
    case '<': return hwritef("\ \<");
    case '>': return hwritef("\ \>");

```

```

    case '␣': return hwritef("\\␣");
    case '-': return hwritef("\\-");
    default: return option_utf8 ? hwrite_utf8(c) : hwritef("\\C%d\\", c);
  }
}

```

Reading the short format:

... $\Rightarrow$

```

⟨get macros 19xix⟩ +≡ (157)
#define HGET_GREF(K, S)
{ uint8_t n = HGET8; REF(K, n); return hwritef("\\S"%d\\", n); }

```

The function *hget.txt* reads a text element and writes it immediately. To enable the insertion of line breaks when writing a text, we need to keep track of the number of characters in the current line. For this purpose the function *hget.txt* returns the number of characters written. It returns -1 if a space character needs to be written providing a good opportunity for a break.

```

⟨get functions 18xviii⟩ +≡ (158)
int hget.txt(void)
{ if (*hpos ≥ #80 ∧ *hpos ≤ #F7) {
    if (option_utf8) return hwrite_utf8(hget_utf8());
    else return hwritef("\\C%d\\", hget_utf8());
  }
  else
  { uint8_t a;
    a = HGET8;
    switch (a) {
    case txt_font + 0: return hwritef("\\0");
    case txt_font + 1: return hwritef("\\1");
    case txt_font + 2: return hwritef("\\2");
    case txt_font + 3: return hwritef("\\3");
    case txt_font + 4: return hwritef("\\4");
    case txt_font + 5: return hwritef("\\5");
    case txt_font + 6: return hwritef("\\6");
    case txt_font + 7: return hwritef("\\7");
    case txt_global + 0: HGET_GREF(font_kind, "F");
    case txt_global + 1: HGET_GREF(penalty_kind, "P");
    case txt_global + 2: HGET_GREF(kern_kind, "K");
    case txt_global + 3: HGET_GREF(ligature_kind, "L");
    case txt_global + 4: HGET_GREF(disc_kind, "D");
    case txt_global + 5: HGET_GREF(glue_kind, "G");
    case txt_global + 6: HGET_GREF(language_kind, "S");
    case txt_global + 7: HGET_GREF(rule_kind, "R");
    case txt_global + 8: HGET_GREF(image_kind, "I");
    case txt_local + 0: return hwritef("\\a");
    case txt_local + 1: return hwritef("\\b");

```

```

    case txt_local + 2: return hwritef("\\c");
    case txt_local + 3: return hwritef("\\d");
    case txt_local + 4: return hwritef("\\e");
    case txt_local + 5: return hwritef("\\f");
    case txt_local + 6: return hwritef("\\g");
    case txt_local + 7: return hwritef("\\h");
    case txt_local + 8: return hwritef("\\i");
    case txt_local + 9: return hwritef("\\j");
    case txt_local + 10: return hwritef("\\k");
    case txt_local + 11: return hwritef("\\l");
    case txt_cc: return hwrite_txt_cc(hget_utf8());
    case txt_node:
    { int i;
      ⟨ read the start byte a 16xvi ⟩
      i = hwritef("<");
      i += hwritef("%s", content_name[KIND(a)]); hget_content(a);
      ⟨ read and check the end byte z 17xvii ⟩
      hwritec('>'); return i + 10; /* just an estimate */
    }
    case txt_hyphen: hwritec('-'); return 1;
    case txt_glue: return -1;
    case '<': return hwritef("\\<");
    case '>': return hwritef("\\>");
    case '": return hwritef("\\\"");
    case '-': return hwritef("\\-");
    case txt_ignore: return hwritef("\\@");
    default: hwritec(a); return 1;
  }
}

```

Writing the short format:

⇒ ...

```

⟨ put functions 14xiv ⟩ +≡ (159)
void hput_txt_cc(uint32_t c)
{ if (c ≤ #20) { HPUTX(2);
  HPUT8(txt_cc); HPUT8(c); }
  else hput_utf8(c);
}

void hput_txt_font(uint8_t f)
{ if (f < 8) HPUTX(1), HPUT8(txt_font + f);
  else QUIT("Use \\F%d\\ instead of \\%d for font, %d in a text", f,
    f, f);
}

```

```

void hput_txt_global(Ref * d)
{
    HPUTX(2);
    switch (d→k) {
        case font_kind: HPUT8(txt_global + 0); break;
        case penalty_kind: HPUT8(txt_global + 1); break;
        case kern_kind: HPUT8(txt_global + 2); break;
        case ligature_kind: HPUT8(txt_global + 3); break;
        case disc_kind: HPUT8(txt_global + 4); break;
        case glue_kind: HPUT8(txt_global + 5); break;
        case language_kind: HPUT8(txt_global + 6); break;
        case rule_kind: HPUT8(txt_global + 7); break;
        case image_kind: HPUT8(txt_global + 8); break;
        default:
            QUIT("Kind_%s_not_allowed_as_a_global_reference_in_a_text",
                NAME(d→k));
    }
    HPUT8(d→n);
}

void hput_txt_local(uint8_t n)
{
    HPUTX(1);
    HPUT8(txt_local + n);
}

```

5 Composite Nodes

The nodes that we consider in this section can contain one or more list nodes. When we implement the parsing routines for composite nodes in the long format, we have to take into account that parsing such a list node will already write the list node to the output. So we split the parsing of composite nodes into several parts and store the parts immediately after parsing them. On the parse stack, we will only keep track of the info value. This new strategy is not as transparent as our previous strategy used for simple nodes where we had a clean separation of reading and writing: reading would store the internal representation of a node and writing the internal representation to output would start only after reading is completed. The new strategy, however, makes it easier to reuse the grammar rules for the component nodes.

Another rule applies to composite nodes: in the short format, the subnodes will come at the end of the node, and especially a list node that contains content nodes comes last. This helps when traversing the content section as we will see in [appendix A](#).

5.1 Boxes

The central structuring elements of $\text{\TeX}$ are boxes. Boxes have a height h , a depth d , and a width w . The shift amount a shifts the contents of the box, the glue ratio r is a factor applied to the glue inside the box, the glue order o is its order of stretchability, and the glue sign s is -1 for shrinking, 0 for rigid, and $+1$ for stretching. Most importantly, a box contains a list l of content nodes inside the box.

```
⟨ hint types li ⟩ +≡ (160)
typedef struct
{ Dimen  $h$ ,  $d$ ,  $w$ ,  $a$ ; float32_t  $r$ ; int8_t  $s$ ,  $o$ ; List  $l$ ; } Box;
```

There are two types of boxes: horizontal boxes and vertical boxes. The difference between the two is simple: a horizontal box aligns the reference points of its content nodes horizontally, and a positive shift amount a shifts the box down; a vertical box aligns the reference points vertically, and a positive shift amount a shifts the box right.

Not all box parameters are used frequently. In the short format, we use the info bits to indicated which of the parameters are used. Where as the width of a horizontal box is most of the time (80%) nonzero, other parameters are most of the time zero, like the shift amount (99%) or the glue settings (99.8%). The depth is

zero in about 77%, the height in about 53%, and both together are zero in about 47%. The results for vertical boxes, which constitute about 20% of all boxes, are similar, except that the depth is zero in about 89%, but the height and width are almost never zero. For this reason we use bit *b001* to indicate a nonzero depth, bit *b010* for a nonzero shift amount, and *b100* for nonzero glue settings. Glue sign and glue order can be packed as two nibbles in a single byte.

Reading the long format:

— — — $\Rightarrow$

$\langle$ symbols 2ii $\rangle + \equiv$ (161)

%token HBOX "hbox"

%token VBOX "vbox"

%token SHIFTED "shifted"

%type < info > box box_dimen box_shift box_glue_set

$\langle$ scanning rules 3iii $\rangle + \equiv$ (162)

hbox **return** HBOX;

vbox **return** VBOX;

shifted **return** SHIFTED;

$\langle$ parsing rules 5v $\rangle + \equiv$ (163)

box_dimen: dimension dimension dimension

 { \$\$ = hput_box_dimen(\$1,\$2,\$3); };

box_shift: { \$\$ = b000; } | SHIFTED dimension { \$\$ = hput_box_shift(\$2); };

box_glue_set: { \$\$ = b000; }

 | PLUS stretch { \$\$ = hput_box_glue_set(+1,\$2.f,\$2.o); }

 | MINUS stretch { \$\$ = hput_box_glue_set(-1,\$2.f,\$2.o); };

box: box_dimen box_shift box_glue_set list { \$\$ = \$1 | \$2 | \$3; };

hbox_node: start HBOX box END { hput_tags(\$1,TAG(hbox_kind,\$3)); };

vbox_node: start VBOX box END { hput_tags(\$1,TAG(vbox_kind,\$3)); };

content_node: hbox_node | vbox_node;

Writing the long format:

$\Rightarrow$ — — —

$\langle$ write functions 21xxi $\rangle + \equiv$ (164)

void hwrite_box(**Box** *b)

{ hwrite_dimension(b→h);

 hwrite_dimension(b→d);

 hwrite_dimension(b→w);

if (b→a ≠ 0) { hwritef("_shifted"); hwrite_dimension(b→a); }

if (b→r ≠ 0.0 ∧ b→s ≠ 0)

 { **if** (b→s > 0) hwritef("_plus"); **else** hwritef("_minus");

 hwrite_float64(b→r,false); hwrite_order(b→o);

 }

 hwrite_list(&(b→l));

}

Reading the short format:

... $\Rightarrow$

$\langle$ cases to get content 20_{xx} $\rangle + \equiv$ (165)

```

case TAG(hbox_kind, b000):
    { Box b; HGET_BOX(b000, b); hwrite_box(&b); } break;
case TAG(hbox_kind, b001):
    { Box b; HGET_BOX(b001, b); hwrite_box(&b); } break;
case TAG(hbox_kind, b010):
    { Box b; HGET_BOX(b010, b); hwrite_box(&b); } break;
case TAG(hbox_kind, b011):
    { Box b; HGET_BOX(b011, b); hwrite_box(&b); } break;
case TAG(hbox_kind, b100):
    { Box b; HGET_BOX(b100, b); hwrite_box(&b); } break;
case TAG(hbox_kind, b101):
    { Box b; HGET_BOX(b101, b); hwrite_box(&b); } break;
case TAG(hbox_kind, b110):
    { Box b; HGET_BOX(b110, b); hwrite_box(&b); } break;
case TAG(hbox_kind, b111):
    { Box b; HGET_BOX(b111, b); hwrite_box(&b); } break;
case TAG(vbox_kind, b000):
    { Box b; HGET_BOX(b000, b); hwrite_box(&b); } break;
case TAG(vbox_kind, b001):
    { Box b; HGET_BOX(b001, b); hwrite_box(&b); } break;
case TAG(vbox_kind, b010):
    { Box b; HGET_BOX(b010, b); hwrite_box(&b); } break;
case TAG(vbox_kind, b011):
    { Box b; HGET_BOX(b011, b); hwrite_box(&b); } break;
case TAG(vbox_kind, b100):
    { Box b; HGET_BOX(b100, b); hwrite_box(&b); } break;
case TAG(vbox_kind, b101):
    { Box b; HGET_BOX(b101, b); hwrite_box(&b); } break;
case TAG(vbox_kind, b110):
    { Box b; HGET_BOX(b110, b); hwrite_box(&b); } break;
case TAG(vbox_kind, b111):
    { Box b; HGET_BOX(b111, b); hwrite_box(&b); } break;

```

$\langle$ get macros 19_{xix} $\rangle + \equiv$ (166)

```

#define HGET_BOX(I, B) HGET32 (B.h);
    if ((I) & b001) HGET32(B.d); else B.d = 0;
    HGET32(B.w);
    if ((I) & b010) HGET32(B.a); else B.a = 0;
    if ((I) & b100)
    { B.r = hget_float32(); B.s = HGET8; B.o = B.s & #F; B.s = B.s  $\gg$  4; }
    else { B.r = 0.0; B.o = B.s = 0; }
    hget_list(&(B.l));

```

(get functions 18xviii) $\vdash \equiv$ (167)
void *hget_hbox_node*(**void**)
 { **Box** *b*;
 (read the start byte *a* 16xvi)
 if (*KIND(a) $\neq$ hbox_kind*)
 QUIT("Hbox expected at 0x%x got %s", *node_pos*, *NAME(a)*);
 HGET_BOX(*INFO(a)*, *b*);
 (read and check the end byte *z* 17xvii)
 hwrite_start(); *hwritef*("hbox"); *hwrite_box*(&*b*); *hwrite_end*();
 }
void *hget_vbox_node*(**void**)
 { **Box** *b*;
 (read the start byte *a* 16xvi)
 if (*KIND(a) $\neq$ vbox_kind*)
 QUIT("Vbox expected at 0x%x got %s", *node_pos*, *NAME(a)*);
 HGET_BOX(*INFO(a)*, *b*);
 (read and check the end byte *z* 17xvii)
 hwrite_start(); *hwritef*("vbox"); *hwrite_box*(&*b*); *hwrite_end*();
 }

Writing the short format:

$\Rightarrow \dots$

(put functions 14xiv) $\vdash \equiv$ (168)
Info *hput_box_dimen*(**Dimen** *h*, **Dimen** *d*, **Dimen** *w*)
 { **Info** *i*; HPUT32(*h*);
 if (*d* $\neq$ 0) { HPUT32(*d*); *i* = *b001*; } **else** *i* = *b000*;
 HPUT32(*w*);
 return *i*;
 }
Info *hput_box_shift*(**Dimen** *a*)
 { **if** (*a* $\neq$ 0) { HPUT32(*a*); **return** *b010*; } **else** **return** *b000*;
 }
Info *hput_box_glue_set*(**int8_t** *s*, **float32_t** *r*, **Order** *o*)
 { **if** (*r* $\neq$ 0.0 $\wedge$ *s* $\neq$ 0) { *hput_float32*(*r*); HPUT8((*s* $\ll$ 4) | *o*); **return** *b100*; }
 else **return** *b000*;
 }

5.2 Extended Boxes

HiTeX produces two kinds of extended horizontal boxes, *hpack_kind* and *hset_kind*, and the same for vertical boxes using *vpack_kind* and *vset_kind*. Let us focus on horizontal boxes; the handling of vertical boxes is completely parallel.

The *hpack* procedure of HiTeX produces an extended box of *hset_kind* either if it is given an extended dimension as its width or if it discovers that the width of its content is an extended dimension. After the final width of the box has been

computed in the viewer, it just remains to set the glue; a very simple operation indeed.

If the *hpack* procedure of HiTeX can not determine the natural dimensions of the box content because it contains paragraphs or other extended boxes, it produces a box of *hpack_kind*. Now the viewer needs to traverse the list of content nodes to determine the natural dimensions. Even the amount of stretchability and shrinkability has to be determined in the viewer. For example, the final stretchability of a paragraph with some stretchability in the baseline skip will depend on the number of lines which, in turn, depends on *hsize*. It is not possible to merge these traversals of the box content with the traversal necessary when displaying the box. The latter needs to convert glue nodes into positioning instructions which requires a fixed glue ratio. The computation of the glue ratio, however, requires a complete traversal of the content.

In the short format of a box node of type *hset_kind*, *vset_kind*, *hpack_kind*, or *vpack_kind*, the info bit *b100* indicates, if set, a complete extended dimension, and if unset, a reference to a predefined extended dimension for the target size; the info bit *b010* indicates a nonzero shift amount. For a box of type *hset_kind* or *vset_kind*, the info bit *b001* indicates, if set, a nonzero depth. For a box of type *hpack_kind* or *vpack_kind*, the info bit *b001* indicates, if set, an additional target size, and if unset, an exact target size. For a box of type *vpack_kind* also the maximum depth is given. If in the long format the maximum depth is omitted, the value *MAX_DIMEN* is used.

The reference point of a vertical box is usually the reference point of the last box inside it and multiple vertical boxes are aligned along this common baseline. Occasionally, however, we want to align vertical boxes using the baselines of their first box. We indicate this alternative setting of the reference point using the keyword *top* in the long form. In the short form, we use the fact the the absolute value of any dimension is less or equal to *MAX_DIMEN* which is equal to *#3ffffff*. This means that the two most significant bits are always the same. So a *vtop* node can be marked by toggling the second of these bits.

Reading the long format:

— — — $\Rightarrow$

$\langle$ symbols 2ii $\rangle + \equiv$ (169)

%token HPACK "hpack"

%token HSET "hset"

%token VPack "vpack"

%token VSET "vset"

%token DEPTH "depth"

%token ADD "add"

%token TO "to"

%type < info > box_options box_goal hpack vpack vbox_dimen

%type < d > max_depth

$\langle$ scanning rules 3iii $\rangle + \equiv$ (170)

hpack **return** HPACK;

hset **return** HSET;

```

vpack          return VPACK;
vset           return VSET;
add            return ADD;
to             return TO;
depth         return DEPTH;

```

```

⟨parsing rules 5v⟩ +≡ (171)
  box_flex:  plus minus { hput_stretch(&($1)); hput_stretch(&($2)); };
  box_options: box_shift box_flex xdimen_ref list { $$ = $1; }
    | box_shift box_flex xdimen_node list { $$ = $1 | b100; };
  hbox_node: start HSET box_dimen box_options END {
    hput_tags($1, TAG(hset_kind, $3 | $4)); };
  vbox_dimen: box_dimen
    | TOP dimension dimension dimension
      { $$ = hput_box_dimen($2, $3 ⊕ #40000000, $4); };
  vbox_node: start VSET vbox_dimen box_options END {
    hput_tags($1, TAG(vset_kind, $3 | $4)); };
  box_goal: TO xdimen_ref { $$ = b000; }
    | ADD xdimen_ref { $$ = b001; }
    | TO xdimen_node { $$ = b100; }
    | ADD xdimen_node { $$ = b101; };
  hpack: box_shift box_goal list { $$ = $2; };
  hbox_node: start HPACK hpack END { hput_tags($1, TAG(hpack_kind, $3)); };
  max_depth: { $$ = MAX_DIMEN; }
    | MAX DEPTH dimension { $$ = $3; };
  vpack: max_depth { HPUT32($1); } box_shift box_goal list { $$ = $3 | $4; }
    | TOP max_depth { HPUT32($2 ⊕ #40000000); }
      box_shift box_goal list { $$ = $4 | $5; };
  vbox_node: start VPACK vpack END { hput_tags($1, TAG(vpack_kind, $3)); };
  content_node: vbox_node
    | hbox_node;

```

Reading the short format:

... $\Rightarrow$

(cases to get content 20_{xx}) +≡ (172)

```

case TAG(hset_kind, b000): HGET_SET(hset_kind, b000); break;
case TAG(hset_kind, b001): HGET_SET(hset_kind, b001); break;
case TAG(hset_kind, b010): HGET_SET(hset_kind, b010); break;
case TAG(hset_kind, b011): HGET_SET(hset_kind, b011); break;
case TAG(hset_kind, b100): HGET_SET(hset_kind, b100); break;
case TAG(hset_kind, b101): HGET_SET(hset_kind, b101); break;
case TAG(hset_kind, b110): HGET_SET(hset_kind, b110); break;
case TAG(hset_kind, b111): HGET_SET(hset_kind, b111); break;

case TAG(vset_kind, b000): HGET_SET(vset_kind, b000); break;
case TAG(vset_kind, b001): HGET_SET(vset_kind, b001); break;
case TAG(vset_kind, b010): HGET_SET(vset_kind, b010); break;
case TAG(vset_kind, b011): HGET_SET(vset_kind, b011); break;
case TAG(vset_kind, b100): HGET_SET(vset_kind, b100); break;
case TAG(vset_kind, b101): HGET_SET(vset_kind, b101); break;
case TAG(vset_kind, b110): HGET_SET(vset_kind, b110); break;
case TAG(vset_kind, b111): HGET_SET(vset_kind, b111); break;

case TAG(hpack_kind, b000): HGET_PACK(hpack_kind, b000); break;
case TAG(hpack_kind, b001): HGET_PACK(hpack_kind, b001); break;
case TAG(hpack_kind, b010): HGET_PACK(hpack_kind, b010); break;
case TAG(hpack_kind, b011): HGET_PACK(hpack_kind, b011); break;
case TAG(hpack_kind, b100): HGET_PACK(hpack_kind, b100); break;
case TAG(hpack_kind, b101): HGET_PACK(hpack_kind, b101); break;
case TAG(hpack_kind, b110): HGET_PACK(hpack_kind, b110); break;
case TAG(hpack_kind, b111): HGET_PACK(hpack_kind, b111); break;

case TAG(vpack_kind, b000): HGET_PACK(vpack_kind, b000); break;
case TAG(vpack_kind, b001): HGET_PACK(vpack_kind, b001); break;
case TAG(vpack_kind, b010): HGET_PACK(vpack_kind, b010); break;
case TAG(vpack_kind, b011): HGET_PACK(vpack_kind, b011); break;
case TAG(vpack_kind, b100): HGET_PACK(vpack_kind, b100); break;
case TAG(vpack_kind, b101): HGET_PACK(vpack_kind, b101); break;
case TAG(vpack_kind, b110): HGET_PACK(vpack_kind, b110); break;
case TAG(vpack_kind, b111): HGET_PACK(vpack_kind, b111); break;

```

(get macros 19_{xix}) +≡ (173)

```

#define HGET_SET(K, I)
{ Dimen h, d; HGET32(h);
  if ((I) & b001) HGET32(d); else d = 0;
  if (K ≡ vset_kind ∧ (d > MAX_DIMEN ∨ d < -MAX_DIMEN)) { hwritef("␣top");
    d ⊕= #40000000;
  }
  hwrite_dimension(h);
  hwrite_dimension(d); }

```

```

{ Dimen w; HGET32(w); hwrite_dimension(w); }
if ((I) & b010) { Dimen a; HGET32(a);
  hwritef("_shifted"); hwrite_dimension(a); }
{ Stretch p; HGET_STRETCH(p); hwrite_plus(&p); }
{ Stretch m; HGET_STRETCH(m); hwrite_minus(&m); }
if ((I) & b100) { Xdimen x; hget_xdimen_node(&x); hwrite_xdimen_node(&x);
}
else HGET_REF(xdimen_kind);
{ List l; hget_list(&l); hwrite_list(&l); }

#define HGET_PACK(K, I)
if (K  $\equiv$  vpack_kind) { Dimen d;
  HGET32(d);
  if (d > MAX_DIMEN  $\vee$  d < -MAX_DIMEN) { hwritef("_top");
    d  $\oplus$  = #40000000;
  }
  if (d  $\neq$  MAX_DIMEN) { hwritef("_max_depth"); hwrite_dimension(d);
  }
}
if ((I) & b010) { Dimen s;
  HGET32(s);
  hwritef("_shifted"); hwrite_dimension(s);
}
if ((I) & b001) hwritef("_add"); else hwritef("_to");
if ((I) & b100) { Xdimen x; hget_xdimen_node(&x); hwrite_xdimen_node(&x);
}
else HGET_REF(xdimen_kind);
{ List l; hget_list(&l); hwrite_list(&l); }

```

5.3 Leaders

Leaders are a special type of glue that is best explained by a few examples. Where as ordinary glue fills its designated space with whiteness, leaders fill their designated space with either a rule _____ or some sort of repeated content. In multiple leaders, the dots are usually aligned across lines, as in the last three lines. Unless you specify centered leaders or you specify expanded leaders. The former pack the repeated content tight and center the repeated content in the available space, the latter distributes the extra space between all the repeated instances.

In the short format, the two lowest info bits store the type of leaders: 1 for aligned, 2 for centered, and 3 for expanded. The *b100* info bit is usually set and only zero in the unlikely case that the glue is zero and therefore not present.

Reading the long format:

— — — $\Rightarrow$

```

⟨ symbols 2ii ⟩ +≡ (174)
%token LEADERS "leaders"
%token ALIGN "align"
%token CENTER "center"
%token EXPAND "expand"
%type < info > leaders
%type < info > ltype

```

```

⟨ scanning rules 3iii ⟩ +≡ (175)
leaders      return LEADERS;
align        return ALIGN;
center       return CENTER;
expand       return EXPAND;

```

```

⟨ parsing rules 5v ⟩ +≡ (176)
ltype: { $$ = 1; }
      | ALIGN { $$ = 1; } | CENTER { $$ = 2; } | EXPAND { $$ = 3; };
leaders: glue_node ltype rule_node { if ($1) $$ = $2 | b100; else $$ = $2; }
      | glue_node ltype hbox_node { if ($1) $$ = $2 | b100; else $$ = $2; }
      | glue_node ltype vbox_node { if ($1) $$ = $2 | b100; else $$ = $2; };
content_node: start LEADERS leaders END
              { hput_tags($1, TAG(leaders_kind, $3)); }

```

Writing the long format:

$\Rightarrow$ — — —

```

⟨ write functions 21xxi ⟩ +≡ (177)
void hwrite_leaders_type(int t)
{ if (t ≡ 2) hwritef("_center");
  else if (t ≡ 3) hwritef("_expand");
}

```

Reading the short format:

... $\Rightarrow$

```

⟨ cases to get content 20xx ⟩ +≡ (178)
case TAG(leaders_kind, 1): HGET_LEADERS(1); break;
case TAG(leaders_kind, 2): HGET_LEADERS(2); break;
case TAG(leaders_kind, 3): HGET_LEADERS(3); break;
case TAG(leaders_kind, b100 | 1): HGET_LEADERS(b100 | 1); break;
case TAG(leaders_kind, b100 | 2): HGET_LEADERS(b100 | 2); break;
case TAG(leaders_kind, b100 | 3): HGET_LEADERS(b100 | 3); break;

```

```

⟨ get macros 19xix ⟩ +≡ (179)
#define HGET_LEADERS(I)
  if ((I) & b100) hget_glue_node();
  hwrite_leaders_type((I) & b011);
  if (KIND(*hpos) ≡ rule_kind) hget_rule_node();
  else if (KIND(*hpos) ≡ hbox_kind) hget_hbox_node();
  else hget_vbox_node();

```

5.4 Baseline Skips

Baseline skips are small amounts of glue inserted between two consecutive lines of text. To get nice looking pages, the amount of glue inserted must take into account the depth of the line above the glue and the height of the line below the glue to achieve a constant distance of the baselines. For example, if we have the lines

“There is no
more gas
in the tank.”

TeX will insert 7.69446pt of baseline skip between the first and the second line and 3.11111pt of baseline skip between the second and the third line. This is due to the fact that the first line has no descenders, its depth is zero, the second line has no ascenders but the “g” descends below the baseline, and the third line has ascenders (“t”, “h”,...) so it is higher than the second line. TeX’s choice of baseline skips ensures that the baselines are exactly 12pt apart in both cases.

Things get more complicated if the text contains mathematical formulas because then a line can get so high or deep that it is impossible to keep the distance between baselines constant without two adjacent lines touching each other. In such cases, TeX will insert a small minimum line skip glue.

For the whole computation, TeX uses three parameters: **baselineskip**, **lineskiplimit**, and **lineskip**. **baselineskip** is a glue value; its size is the normal distance of two baselines. TeX adjusts the size of the **baselineskip** glue for the height and the depth of the two lines and then checks the result against **lineskiplimit**. If the result is smaller than **lineskiplimit** it will use the **lineskip** glue instead.

Because the depth and the height of lines depend on the outcome of the line breaking routine, baseline computations must be done in the viewer. The situation gets even more complicated because TeX can manipulate the insertion of baseline skips in various ways. Therefore HINT requires the insertion of baseline nodes wherever the viewer is supposed to perform a baseline skip computation.

In the short format of a baseline definition, we store only the nonzero components and use the info bits to mark them: *b100* implies *bs* ≠ 0, *b010* implies *ls* ≠ 0, and *b001* implies *lslimit* ≠ 0. If the baseline has only zero components, we put a reference to baseline number 0 in the output.

```

⟨ hint basic types 6vi ⟩ +≡ (180)
typedef struct { Glue bs, ls; Dimen lsl; } Baseline;

```


Reading the long format:

— — — $\Rightarrow$

$\langle$ symbols 2ii $\rangle + \equiv$ (181)

%token BASELINE "baseline"

%type < info > baseline

$\langle$ scanning rules 3iii $\rangle + \equiv$ (182)

baseline return BASELINE;

$\langle$ parsing rules 5v $\rangle + \equiv$ (183)

baseline: dimension {

if (\$1 $\neq$ 0) HPUT32(\$1);

} glue_node glue_node

{ \$\$ = b000;

if (\$1 $\neq$ 0) \$\$ |= b001;

if (\$3) \$\$ |= b100;

if (\$4) \$\$ |= b010; };

content_node: start BASELINE baseline END

{ if (\$3 $\equiv$ b000) HPUT8(0); hput_tags(\$1, TAG(baseline_kind, \$3)); };

Reading the short format:

... $\Rightarrow$

$\langle$ cases to get content 20xx $\rangle + \equiv$ (184)

case TAG(baseline_kind, b001):

{ Baseline b; HGET_BASELINE(b001, b); } break;

case TAG(baseline_kind, b010):

{ Baseline b; HGET_BASELINE(b010, b); } break;

case TAG(baseline_kind, b011):

{ Baseline b; HGET_BASELINE(b011, b); } break;

case TAG(baseline_kind, b100):

{ Baseline b; HGET_BASELINE(b100, b); } break;

case TAG(baseline_kind, b101):

{ Baseline b; HGET_BASELINE(b101, b); } break;

case TAG(baseline_kind, b110):

{ Baseline b; HGET_BASELINE(b110, b); } break;

case TAG(baseline_kind, b111):

{ Baseline b; HGET_BASELINE(b111, b); } break;

$\langle$ get macros 19xix $\rangle + \equiv$ (185)

#define HGET_BASELINE(I, B)

if ((I) & b001) HGET32((B).lsl); else B.lsl = 0;

hwrite_dimension(B.lsl);

if ((I) & b100) hget_glue_node();

else { B.bs.p.o = B.bs.m.o = B.bs.w.w = 0;

B.bs.w.h = B.bs.w.v = B.bs.p.f = B.bs.m.f = 0.0;

hwrite_glue_node(&(B.bs)); }

if ((I) & b010) hget_glue_node();

```

else { B.ls.p.o = B.ls.m.o = B.ls.w.w = 0;
      B.ls.w.h = B.ls.w.v = B.ls.p.f = B.ls.m.f = 0.0;
      hwrite_glue_node(&(B.ls)); }

```

Writing the short format:

$\Rightarrow \dots$

```

⟨put functions 14xiv⟩ +≡ (186)
Tag hput_baseline(Baseline *b)
{ Info info = b000;
  if (¬ZERO_GLUE(b→bs)) info |= b100;
  if (¬ZERO_GLUE(b→ls)) info |= b010;
  if (b→lsl ≠ 0) { HPUT32(b→lsl); info |= b001; }
  return TAG(baseline_kind, info);
}

```

5.5 Ligatures

Ligatures occur only in horizontal lists. They specify characters that combine the glyphs of several characters into one specialized glyph. For example in the word “difficult” the three letters “ffi” are combined into the ligature “ffi”. Hence, a ligature is very similar to a simple glyph node; the characters that got replaced are, however, retained in the ligature because they might be needed for example to support searching. Since ligatures are therefore only specialized list of characters and since we have a very efficient way to store such lists of characters, namely as a *text*, input and output of ligatures is quite simple.

The info value zero is reserved for references to a ligature. If the info value is between 1 and 6, it gives the number of bytes used to encode the characters in UTF8. Note that a ligature will always include a glyph byte, so the minimum size is 1. A typical ligature like “fi” will need 3 byte: the ligature character “fi”, and the replacement characters “f” and “i”. More byte might be required if the character codes exceed #7F since we use the UTF8 encoding scheme for larger character codes. If the info value is 7, a full text node follows the font byte. In the long format, we give the font, the character code, and then the replacement characters represented as a text.

```

⟨hint types 1i⟩ +≡ (187)
typedef struct { uint8_t f; List l; } Lig;

```

Reading the long format:

— — — $\Rightarrow$

$\langle$ symbols 2ii $\rangle + \equiv$ (188)

%**token** LIGATURE "ligature"

%**type** < u > lig_cc

%**type** < lg > ligature

%**type** < u > ref

$\langle$ scanning rules 3iii $\rangle + \equiv$ (189)

ligature **return** LIGATURE;

$\langle$ parsing rules 5v $\rangle + \equiv$ (190)

cc_list: | cc_list TXT_CC { hput_utf8(\$2); };

lig_cc: UNSIGNED { RNG("UTF-8_code", \$1, 0, #1FFFFFF); \$\$ = hpos - hstart;
hput_utf8(\$1); };

lig_cc: CHARCODE { \$\$ = hpos - hstart; hput_utf8(\$1); };

ref: REFERENCE { HPUT8(\$1); \$\$ = \$1; };

ligature: ref { REF(font_kind, \$1); } lig_cc TXT_START cc_list TXT_END
{ \$\$.f = \$1; \$\$.l.p = \$3; \$\$.l.s = (hpos - hstart) - \$3;
RNG("Ligature_size", \$\$.l.s, 0, 255); };

content_node: start LIGATURE ligature END {
hput_tags(\$1, hput_ligature(&(\$3))); };

Writing the long format:

$\Rightarrow$ — — —

$\langle$ write functions 21xxi $\rangle + \equiv$ (191)

void hwrite_ligature(**Lig** *l)

{ **uint32_t** pos = hpos - hstart;

hwrite_ref(l $\rightarrow$ f);

hpos = l $\rightarrow$ l.p + hstart;

hwrite_charcode(hget_utf8());

hwritef("_\");

while (hpos < hstart + l $\rightarrow$ l.p + l $\rightarrow$ l.s) hwrite_txt_cc(hget_utf8());

hwritec('');

hpos = hstart + pos;

}

Reading the short format:

... $\Rightarrow$

$\langle$ cases to get content 20xx $\rangle + \equiv$ (192)

```

case TAG(ligature_kind, 1): { Lig l; HGET_LIG(1, l); } break;
case TAG(ligature_kind, 2): { Lig l; HGET_LIG(2, l); } break;
case TAG(ligature_kind, 3): { Lig l; HGET_LIG(3, l); } break;
case TAG(ligature_kind, 4): { Lig l; HGET_LIG(4, l); } break;
case TAG(ligature_kind, 5): { Lig l; HGET_LIG(5, l); } break;
case TAG(ligature_kind, 6): { Lig l; HGET_LIG(6, l); } break;
case TAG(ligature_kind, 7): { Lig l; HGET_LIG(7, l); } break;

```

$\langle$ get macros 19xix $\rangle + \equiv$ (193)

```

#define HGET_LIG(I, L)
  (L).f = HGET8;
  REF(font_kind, (L).f);
  if ((I)  $\equiv$  7) hget_list(&((L).l));
  else { (L).l.s = (I);
        (L).l.p = hpos - hstart; hpos += (L).l.s;
      }
  hwrite_ligature(&(L));

```

Writing the short format:

$\Rightarrow$...

$\langle$ put functions 14xiv $\rangle + \equiv$ (194)

```

Tag hput_ligature(Lig *l)
{ if (l $\rightarrow$ l.s < 7) return TAG(ligature_kind, l $\rightarrow$ l.s);
  else
  { uint32_t pos = l $\rightarrow$ l.p;
    l $\rightarrow$ l.t = TAG(list_kind, b100);
    hput_tags(pos, hput_list(pos + 1, &(l $\rightarrow$ l)));
    return TAG(ligature_kind, 7);
  }
}

```

5.6 Discretionary breaks

HINT is capable to break lines into paragraphs. It does this primarily at interword spaces but it might also break a line in the middle of a word if it finds a discretionary line break there. These discretionary breaks are usually provided by an automatic hyphenation algorithm but they might be also explicitly inserted by the author of a document.

When a line break occurs at such a discretionary break, the line before the break ends with a *pre_break* list of nodes, the line after the break starts with a *post_break* list of nodes, and the next *replace_count* nodes after the discretionary break will be ignored. Both lists must consist entirely of glyphs, kerns, boxes, rules, or ligatures. For example, an ordinary discretionary break will have a *pre_break* list containing “.”, an empty *post_break* list, and a *replace_count* of zero.

The long format starts with an optional “!”, indicating an explicit discretionary break, followed by the replace-count. Then comes the pre-break list followed by the post-break list. The replace-count can be omitted if it is zero; an empty post-break list may be omitted as well. Both list may be omitted only if both are empty.

In the short format, the three components of a disc node are stored in this order: *replace_count*, *pre_break* list, and *post_break* list. The *b100* bit in the info value indicates the presence of a replace-count, the *b010* bit the presence of a *pre_break* list, and the *b001* bit the presence of a *post_break* list. Since the info value *b000* is reserved for references, at least one of these must be specified; so we represent a node with empty lists and a replace count of zero using the info value *b100* and a zero byte for the replace count.

Replace counts must be in the range 0 to 31; so the short format can set the high bit of the replace count to indicate an explicit break.

⟨ hint types 1i ⟩ +≡ (195)
typedef struct { **bool** *x*; **List** *p*, *q*; **uint8_t** *r*; } **Disc**;

Reading the long format:

— — — ⇒

⟨ symbols 2ii ⟩ +≡ (196)
 %token DISC "disc"
 %type < dc > disc
 %type < u > replace_count

⟨ scanning rules 3iii ⟩ +≡ (197)
disc **return** DISC;

⟨ parsing rules 5v ⟩ +≡ (198)
 replace_count: explicit { if (\$1) { \$\$ = #80; HPUT8(#80); } else \$\$ = #00; }
 explicit UNSIGNED { RNG("Replace_count", \$2, 0, 31);
 \$\$ = (\$2) | ((\$1) ? #80 : #00); if (\$\$ ≠ 0) HPUT8(\$\$); };
 disc: replace_count list list { \$\$.r = \$1; \$\$.p = \$2; \$\$.q = \$3;
 if (\$3.s ≡ 0) { hpos = hpos - 3; if (\$2.s ≡ 0) hpos = hpos - 3; } }
 replace_count list { \$\$.r = \$1; \$\$.p = \$2;
 if (\$2.s ≡ 0) hpos = hpos - 3; \$\$.q.s = 0; }
 replace_count { \$\$.r = \$1; \$\$.p.s = 0; \$\$.q.s = 0; };
 disc_node: start DISC disc END { hput_tags(\$1, hput_disc(&(\$3))); };
 content_node: disc_node;

Writing the long format:

⇒ - - -

```

⟨ write functions 21xxi ⟩ +≡ (199)
void hwrite_disc(Disc *h)
{ hwrite_explicit(h→x);
  if (h→r ≠ 0) hwritef("␣%d", h→r);
  if (h→p.s ≠ 0 ∨ h→q.s ≠ 0) hwrite_list(&(h→p));
  if (h→q.s ≠ 0) hwrite_list(&(h→q));
}
void hwrite_disc_node(Disc *h)
{ hwrite_start(); hwritef("disc"); hwrite_disc(h); hwrite_end();
}

```

Reading the short format:

... ⇒

```

⟨ cases to get content 20xx ⟩ +≡ (200)
case TAG(disc_kind, b001):
  { Disc h; HGET_DISC(b001, h); hwrite_disc(&h); } break;
case TAG(disc_kind, b010):
  { Disc h; HGET_DISC(b010, h); hwrite_disc(&h); } break;
case TAG(disc_kind, b011):
  { Disc h; HGET_DISC(b011, h); hwrite_disc(&h); } break;
case TAG(disc_kind, b100):
  { Disc h; HGET_DISC(b100, h); hwrite_disc(&h); } break;
case TAG(disc_kind, b101):
  { Disc h; HGET_DISC(b101, h); hwrite_disc(&h); } break;
case TAG(disc_kind, b110):
  { Disc h; HGET_DISC(b110, h); hwrite_disc(&h); } break;
case TAG(disc_kind, b111):
  { Disc h; HGET_DISC(b111, h); hwrite_disc(&h); } break;

```

```

⟨ get macros 19xix ⟩ +≡ (201)
#define HGET_DISC(I, Y)

```

```

  if ((I) & b100) { uint8_t r = HGET8;
    (Y).r = r & #7F; RNG("Replace␣count", (Y).r, 0, 31); (Y).x = (r & #80) ≠ 0;
  } else { (Y).r = 0; (Y).x = false; }
  if ((I) & b010) hget_list(&((Y).p));
  else { (Y).p.p = hpos - hstart; (Y).p.s = 0; (Y).p.t = TAG(list_kind, b000); }
  if ((I) & b001) hget_list(&((Y).q));
  else { (Y).q.p = hpos - hstart; (Y).q.s = 0; (Y).q.t = TAG(list_kind, b000); }

```

```

⟨ get functions 18xviii ⟩ +≡ (202)
void hget_disc_node(Disc *h)
{ ⟨ read the start byte a 16xvi ⟩
  if (KIND(a) ≠ disc_kind ∨ INFO(a) ≡ b000)
    QUIT("Hyphen␣expected␣at␣0x%x␣got␣%s,%d", node_pos, NAME(a),
        INFO(a));

```

```

HGET_DISC(INFO(a),*h);
⟨ read and check the end byte z 17xvii ⟩
}

```

When *hput_disc* is called, the node is already written to the output, but empty lists might have been deleted, and the info value needs to be determined. Because the info value *b000* is reserved for references, a zero reference count is written to avoid this case.

Writing the short format:

⇒ ...

```

⟨ put functions 14xiv ⟩ +≡ (203)
Tag hput_disc(Disc *h)
{ Info info = b000;
  if (h→r ≠ 0) info |= b100;
  if (h→q.s ≠ 0) info |= b011;
  else if (h→p.s ≠ 0) info |= b010;
  if (info ≡ b000) { info |= b100; HPUT8(0); }
  return TAG(disc_kind, info);
}

```

5.7 Paragraphs

The most important procedure that the **HINT** viewer inherits from **TEX** is the line breaking routine. If the horizontal size of the paragraph is not known, breaking the paragraph into lines must be postponed and this is done by creating a paragraph node. The paragraph node must contain all information that **TEX**'s line breaking algorithm needs to do its job.

Besides the horizontal list describing the content of the paragraph and the extended dimension describing the horizontal size, this is the set of parameters that guide the line breaking algorithm:

- Integer parameters:
 - pretolerance** (badness tolerance before hyphenation),
 - tolerance** (badness tolerance after hyphenation),
 - line_penalty** (added to the badness of every line, increase to get fewer lines),
 - hyphen_penalty** (penalty for break after hyphenation break),
 - ex_hyphen_penalty** (penalty for break after explicit break),
 - double_hyphen_demerits** (demerits for double hyphen break),
 - final_hyphen_demerits** (demerits for final hyphen break),
 - adj_demerits** (demerits for adjacent incompatible lines),
 - looseness** (make the paragraph that many lines longer than its optimal size),
 - inter_line_penalty** (additional penalty between lines),
 - club_penalty** (penalty for creating a club line),
 - widow_penalty** (penalty for creating a widow line),
 - display_widow_penalty** (ditto, just before a display),

broken_penalty (penalty for breaking a page at a broken line),
hang_after (start/end hanging indentation at this line).

- Dimension parameters:
line_skip_limit (threshold for **line_skip** instead of **baseline_skip**),
hang_indent (amount of hanging indentation),
emergency_stretch (stretchability added to every line in the final pass of line breaking).
- Glue parameters:
baseline_skip (desired glue between baselines),
line_skip (interline glue if **baseline_skip** is infeasible),
left_skip (glue at left of justified lines),
right_skip (glue at right of justified lines),
par_fill_skip (glue on last line of paragraph).

For a detailed explanation of these parameters and how they influence line breaking, you should consult the T_EXbook[8]; T_EX's **parshape** feature is currently not implemented. There are default values for all of these parameters (see section 11), and therefore it might not be necessary to specify any of them. Any local adjustments are contained in a list of parameters contained in the paragraph node.

A further complication arises from displayed formulas that interrupt a paragraph. Such displays are described in the next section.

To summarize, a paragraph node in the long format specifies an extended dimension, a parameter list, and a node list. The extended dimension is given either as an *xdimen* node (info bit *b100*) or as a reference; similarly the parameter list can be embedded in the node (info bit *b010*) or again it is given by a reference.

Reading the long format:

— — — $\Rightarrow$

```
<symbols 2ii > +≡ (204)
%token PAR "par"
%type < info > par
```

```
<scanning rules 3iii > +≡ (205)
par          return PAR;
```

The following parsing rules are slightly more complicated than I would like them to be, but it seems more important to achieve a regular layout of the short format nodes where all sub nodes are located at the end of a node. In this case, I want to put a *param_ref* before an *xdimen* node, but otherwise have the *xdimen_ref* before a *param_list*. The *par_dimen* rule is introduced only to avoid a reduce/reduce conflict in the parser.

```
<parsing rules 5v > +≡ (206)
par_dimen: xdimen { hput_xdimen_node(&($1)); };
par: xdimen_ref param_ref list { $$ = b000; }
    | xdimen_ref param_list list { $$ = b010; }
    | xdimen param_ref { hput_xdimen_node(&($1)); } list { $$ = b100; }
    | par_dimen param_list list { $$ = b110; };
```



```
content_node: start PAR par END { hput_tags($1,TAG(par_kind,$3)); };
```

Reading the short format:

... $\Rightarrow$

```
< cases to get content 20xx > +≡ (207)
```

```
case TAG(par_kind,b000): HGET_PAR(b000); break;
case TAG(par_kind,b010): HGET_PAR(b010); break;
case TAG(par_kind,b100): HGET_PAR(b100); break;
case TAG(par_kind,b110): HGET_PAR(b110); break;
```

```
< get macros 19xix > +≡ (208)
```

```
#define HGET_PAR(I)
{ uint8_t n;
  if ((I) == b100) { n = HGET8; REF(param_kind,n); }
  if ((I) & b100) { Xdimen x; hget_xdimen_node(&x); hwrite_xdimen(&x); }
  else HGET_REF(xdimen_kind);
  if ((I) & b010) { List l; hget_param_list(&l); hwrite_param_list(&l); }
  else if ((I) != b100) HGET_REF(param_kind)
  else hwrite_ref(n);
  { List l; hget_list(&l); hwrite_list(&l); }
}
```

5.8 Mathematics

Being able to handle mathematics nicely is one of the primary features of $\text{\TeX}$ and so you should expect the same from $\text{\HINT}$. We start here with the more complex case—displayed equations—and finish with the simpler case of mathematical formulas that are part of the normal flow of text.

Displayed equations occur inside a paragraph node. They interrupt normal processing of the paragraph and the paragraph processing is resumed after the display. Positioning of the display depends on several parameters, the shape of the paragraph, and the length of the last line preceding the display. Displayed formulas often feature an equation number which can be placed either left or right of the formula. Also the size of the equation number will influence the placement of the formula.

In a $\text{\HINT}$ file, the parameter list is followed by a list of content nodes, representing the formula, and an optional horizontal box containing the equation number.

In the short format, we use the info bit *b100* to indicate the presence of a parameter list (which might be empty—so it’s actually the absence of a reference to a parameter list); the info bit *b010* to indicate the presence of a left equation number; and the info bit *b001* for a right equation number.

In the long format, we use “eqno” or “left eqno” to indicate presence and placement of the equation number.

Reading the long format:

— — — $\Rightarrow$

$\langle$ symbols 2ii $\rangle + \equiv$ (209)

%token MATH "math"

%type < info > math

$\langle$ scanning rules 3iii $\rangle + \equiv$ (210)

math return MATH;

$\langle$ parsing rules 5v $\rangle + \equiv$ (211)

```
math: param_ref list { $$ = b000; }
    | param_ref list hbox_node { $$ = b001; }
    | param_ref hbox_node list { $$ = b010; }
    | param_list list { $$ = b100; }
    | param_list list hbox_node { $$ = b101; }
    | param_list hbox_node list { $$ = b110; };
```

```
content_node: start MATH math END
    { hput_tags($1, TAG(math_kind, $3)); };
```

Reading the short format:

$\dots \Rightarrow$

$\langle$ cases to get content 20xx $\rangle + \equiv$ (212)

```
case TAG(math_kind, b000): HGET_MATH(b000); break;
case TAG(math_kind, b001): HGET_MATH(b001); break;
case TAG(math_kind, b010): HGET_MATH(b010); break;
case TAG(math_kind, b100): HGET_MATH(b100); break;
case TAG(math_kind, b101): HGET_MATH(b101); break;
case TAG(math_kind, b110): HGET_MATH(b110); break;
```

$\langle$ get macros 19xix $\rangle + \equiv$ (213)

```
#define HGET_MATH(I)
if ((I) & b100) { List l; hget_param_list(&l); hwrite_param_list(&l); }
else HGET_REF(param_kind);
if ((I) & b010) hget_hbox_node();
{ List l; hget_list(&l); hwrite_list(&l); }
if ((I) & b001) hget_hbox_node();
```

Things are much simpler if mathematical formulas are embedded in regular text. Here it is just necessary to mark the beginning and the end of the formula because glue inside a formula is not a possible point for a line break. To break the line within a formula you can insert a penalty node.

In the long format, such a simple math node just consists of the keyword “on” or “off”. In the short format, there are two info values still unassigned: we use *b011* for “off” and *b111* for “on”.

Reading the long format:

— — — $\Rightarrow$

$\langle$ symbols 2ii $\rangle + \equiv$ (214)

%token ON "on"

%token OFF "off"

%type $\langle i \rangle$ on_off

$\langle$ scanning rules 3iii $\rangle + \equiv$ (215)

on return ON;

off return OFF;

$\langle$ parsing rules 5v $\rangle + \equiv$ (216)

on_off: ON { \$\$ = 1; }

| OFF { \$\$ = 0; };

math: on_off { \$\$ = b011 | (\$1 $\ll$ 2); };

Reading the short format:

$\dots \Rightarrow$

$\langle$ cases to get content 20xx $\rangle + \equiv$ (217)

case TAG(math_kind, b111): hwritef("_on"); break;

case TAG(math_kind, b011): hwritef("_off"); break;

Note that T_EX allows math nodes to specify a width using the current value of mathsurround. If this width is nonzero, it is equivalent to inserting a kern node before the math on node and after the math off node.

5.9 Adjustments

An adjustment occurs only in paragraphs. When the line breaking routine finds an adjustment, it inserts the vertical material contained in the adjustment node right after the current line. Adjustments simply contain a list node.

Reading the long format:

— — — $\Rightarrow$

Writing the short format:

$\Rightarrow \dots$

$\langle$ symbols 2ii $\rangle + \equiv$ (218)

%token ADJUST "adjust"

$\langle$ scanning rules 3iii $\rangle + \equiv$ (219)

adjust return ADJUST;

$\langle$ parsing rules 5v $\rangle + \equiv$ (220)

content_node: start ADJUST list END { hput_tags(\$1, TAG(adjust_kind, 1)); };

Reading the short format:

$\dots \Rightarrow$

Writing the long format:

$\Rightarrow$ — — —

$\langle$ cases to get content 20xx $\rangle + \equiv$ (221)

case TAG(adjust_kind, 1): { List l; hget_list(&l); hwrite_list(&l); } break;

5.10 Tables

As long as a table contains no dependencies on `hsize` and `vsize`, `HiTeX` can expand an alignment into a set of nested horizontal and vertical boxes and no special processing is required. As long as only the size of the table itself but neither the tabskip glues nor the table content depends on `hsize` or `vsize`, the table just needs an outer node of type *hset_kind* or *vset_kind*. If there is non aligned material inside the table that depends on `hsize` or `vsize`, a `vpack` or `hpack` node is still sufficient.

While it is reasonable to restrict the tabskip glues to be ordinary glue values without `hsize` or `vsize` dependencies, it might be desirable to have content in the table that does depend on `hsize` or `vsize`. For the latter case, we need a special kind of table node. Here is why:

As soon as the dimension of an item in the table is an extended dimension, it is no longer possible to compute the maximum natural width of a column, because it is not possible to compare extended dimensions without knowing `hsize` and `vsize`. Hence the computation of maximum widths needs to be done in the viewer. After knowing the width of the columns, the setting of tabskip glues is easy to compute.

To implement these extended tables, we will need a table node that specifies a direction, either horizontal or vertical; a list of tabskip glues, with the provision that the last tabskip glue in the list is repeated as long as necessary; and a list of table content. The table's content is stacked, either vertical or horizontal, orthogonal to the alignment direction of the table. The table's content consists of nonaligned content, for example extra glue or rules, and aligned content. Each element of aligned content is called an outer item and it consist of a list of inner items. For example in a horizontal alignment, each row is an outer item and each table entry in that row is an inner item. An inner item contains a box node (of kind *hbox_kind*, *vbox_kind*, *hset_kind*, *vset_kind*, *hpack_kind*, or *vpack_kind*) followed by an optional span count.

The glue of the boxes in the inner items will be reset so that all boxes in the same column reach the same maximum column width. The span counts will be replaced by the appropriate amount of empty boxes and tabskip glues. Finally the glue in the outer item will be set to obtain the desired size of the table.

The definitions below specify just a *list* for the list of tabskip glues and a list for the outer table items. This is just for convenience; the first list must contain glue nodes and the second list must contain nonaligned content and inner item nodes.

We reuse the `H` and `V` tokens, defined as part of the specification of extended dimensions, to indicate the alignment direction of the table. To tell a reference to an extended dimension from a reference to an ordinary dimension, we prefix the former with an `XDIMEN` token; for the latter, the `DIMEN` token is optional. The scanner will recognize not only “item” as an `ITEM` token but also “row” and “column”. This allows a more readable notation, for example by marking the outer items as rows and the inner items as columns.

In the short format, the *b010* bit is used to mark a vertical table and the *b101* bits indicate how the table size is specified; an outer item node has the info value *b000*, an inner item node with info value *b111* contains an extra byte for the span

count, otherwise the info value is equal to the span count.

Reading the long format:

— — — $\Rightarrow$

$\langle$ symbols 2ii $\rangle + \equiv$ (222)

%token TABLE "table"

%token ITEM "item"

%type < info > table span_count

$\langle$ scanning rules 3iii $\rangle + \equiv$ (223)

table **return** TABLE;

item **return** ITEM;

row **return** ITEM;

column **return** ITEM;

$\langle$ parsing rules 5v $\rangle + \equiv$ (224)

span_count: UNSIGNED { \$\$ = hput_span_count(\$1); };

content_node: start ITEM content_node END {
 hput_tags(\$1, TAG(item_kind, 1)); };

content_node: start ITEM span_count content_node END {
 hput_tags(\$1, TAG(item_kind, \$3)); };

content_node: start ITEM list END { hput_tags(\$1, TAG(item_kind, b000)); };

table: H box_goal list list { \$\$ = \$2; };

table: V box_goal list list { \$\$ = \$2 | b010; };

content_node: start TABLE table END { hput_tags(\$1, TAG(table_kind, \$3)); };

Reading the short format:

... $\Rightarrow$

$\langle$ cases to get content 20xx $\rangle + \equiv$ (225)

case TAG(table_kind, b000): HGET_TABLE(b000); break;

case TAG(table_kind, b001): HGET_TABLE(b001); break;

case TAG(table_kind, b010): HGET_TABLE(b010); break;

case TAG(table_kind, b011): HGET_TABLE(b011); break;

case TAG(table_kind, b100): HGET_TABLE(b100); break;

case TAG(table_kind, b101): HGET_TABLE(b101); break;

case TAG(table_kind, b110): HGET_TABLE(b110); break;

case TAG(table_kind, b111): HGET_TABLE(b111); break;

case TAG(item_kind, b000): { List l; hget_list(&l); hwrite_list(&l); } break;

case TAG(item_kind, b001): hget_content_node(); break;

case TAG(item_kind, b010): hwritef("\2"); hget_content_node(); break;

case TAG(item_kind, b011): hwritef("\3"); hget_content_node(); break;

case TAG(item_kind, b100): hwritef("\4"); hget_content_node(); break;

case TAG(item_kind, b101): hwritef("\5"); hget_content_node(); break;

case TAG(item_kind, b110): hwritef("\6"); hget_content_node(); break;

```

    case TAG(item_kind, b111): hwritef("_%u", HGET8); hget_content_node();
    break;

```

⟨ get macros 19xix ⟩ +≡ (226)

```

#define HGET_TABLE(I)
    if (I & b010) hwritef("_v"); else hwritef("_h");
    if ((I) & b001) hwritef("_add"); else hwritef("_to");
    if ((I) & b100) { Xdimen x;
        hget_xdimen_node(&x); hwrite_xdimen_node(&x); }
    else HGET_REF(xdimen_kind)
    { List l; hget_list(&l); hwrite_list(&l); } /* tabskip */
    { List l; hget_list(&l); hwrite_list(&l); } /* items */

```

Writing the short format: $\Rightarrow \dots$

⟨ put functions 14xiv ⟩ +≡ (227)

```

Info hput_span_count(uint32_t n)
{
    if (n ≡ 0) QUIT("Span_count_in_item_must_not_be_zero");
    else if (n < 7) return n;
    else if (n > #FF) QUIT("Span_count_d_must_be_less_than_255", n);
    else { HPUT8(n);
        return 7;
    }
}

```

6 Extensions

6.1 Images

In the first implementation attempt, images behaved pretty much like glue. They could stretch (or shrink) together with the surrounding glue to fill a horizontal or vertical box. While I thought this would be in line with T_EX's concepts, it proved to be a bad decision because images, as opposed to glue, would stretch or shrink horizontally *and* vertically at the same time. This would require a two pass algorithm to pack boxes: first to determine the glue setting and a second pass to determine the proper image dimensions. Otherwise incorrect width or height values would propagate all the way through a sequence of nested boxes. Even worse so, this two pass algorithm would be needed in the viewer if images were contained in boxes that had extended dimensions.

The new design described below allows images with extended dimensions. This covers the case of stretchable or shrinkable images inside of extended boxes. The given extended dimensions are considered maximum values. The stretching or shrinking of images will always preserve the aspect ratio = width/height.

For convenience, we allow missing values in the long format, for example the aspect ratio, if they can be determined from the image data. In the short format, the necessary information for a correct layout must be available without using the image data.

In the long format, the only required parts of an image node are: the number of the auxiliary section where the image data can be found and the descriptive text which is there to make the document more accessible. The section number is followed by the optional aspect ratio, width, and height of the image. If some of these values are missing, it must be possible to determine them from the image data. The node ends with the description.

The short format, starts with the section number of the image data and ends with the description. Missing values for aspect ratio, width, and height are only allowed if they can be recomputed from the image data. A missing width or height is represented by a reference to the zero extended dimension. If the *b100* bit is set, the aspect ratio is present as a 32 bit floating point value followed by extended dimensions for width and height. The info value *b100* indicates a width reference followed by a height reference; the value *b111* indicates a width node followed by a height node; the value *b110* indicates a height reference followed by a width node; and the value *b101* indicates a width reference followed by a height node. The last

two rules reflect the requirement that subnodes are always located at the end of a node.

The remaining info values are used as follows: The value *b000* is used for a reference to an image. The value *b011* indicates an immediate width and an immediate height. The value *b010* indicates an aspect ratio and an immediate width. The value *b001* indicates an aspect ratio and an immediate height.

The following data type stores image information. The width and height are either given as extended dimensions either directly in *w* and *h* or as references in *wr* and *hr*.

```
< hint types 1i > +≡ (228)
typedef struct { uint16_t n; float32_t a; Xdimen w, h; uint8_t wr, hr;
} Image;
```

Reading the long format:

— — — $\implies$

```
< symbols 2ii > +≡ (229)
%token IMAGE "image"
%token WIDTH "width"
%token HEIGHT "height"
%type < xd > image_width image_height
%type < f > image_aspect
%type < info > image_spec image
```

```
< scanning rules 3iii > +≡ (230)
image      return IMAGE;
width      return WIDTH;
height     return HEIGHT;
```

```
< parsing rules 5v > +≡ (231)
image_aspect: number { $$ = $1; }
| { $$ = 0.0; };
image_width: WIDTH xdimen { $$ = $2; }
| { $$ = xdimen_defaults[zero_xdimen_no]; };
image_height: HEIGHT xdimen { $$ = $2; }
| { $$ = xdimen_defaults[zero_xdimen_no]; };
image_spec: UNSIGNED image_aspect image_width image_height {
    $$ = hput_image_spec($1, $2, 0, &($3), 0, &($4)); }
| UNSIGNED image_aspect WIDTH REFERENCE image_height {
    $$ = hput_image_spec($1, $2, $4, NULL, 0, &($5)); }
| UNSIGNED image_aspect image_width HEIGHT REFERENCE {
    $$ = hput_image_spec($1, $2, 0, &($3), $5, NULL); }
| UNSIGNED image_aspect WIDTH REFERENCE HEIGHT REFERENCE {
    $$ = hput_image_spec($1, $2, $4, NULL, $6, NULL); };
image: image_spec list { $$ = $1; };
```



```
content_node: start IMAGE image END { hput_tags($1,TAG(image_kind,$3));
};
```

When a short format file is generated, the image width and height must be determined if necessary from the image file. The following function will write this information into the long format file. Editing the image file at a later time and converting the short format file back to a long format file will preserve the old information. This is not allways a desirable effect. It would be possible to eliminate information about the image size when writing the long format if that information can be derived from the image file. The latter solution might have the disadvantage, that infomation about a desired image size might get lost when editing an image file.

Writing the long format:

⇒ - - -

```
<write functions 21xxi> +≡ (232)
void hwrite_image(Image *x)
{ RNG("Section_number",x→n,3,max_section_no);
  hwritef("_%u",x→n);
  if (x→a ≠ 0.0) hwrite_float64(x→a,false);
  if (x→wr ≠ 0) hwritef("_width_%u",x→wr);
  else if (x→w.w ≠ 0 ∨ x→w.h ≠ 0.0 ∨ x→w.v ≠ 0.0) { hwritef("_width");
    hwrite_xdimen(&x→w);
  }
  if (x→hr ≠ 0) hwritef("_height_%u",x→hr);
  else if (x→h.w ≠ 0 ∨ x→h.h ≠ 0.0 ∨ x→h.v ≠ 0.0) { hwritef("_height");
    hwrite_xdimen(&x→h);
  }
}
```

Reading the short format:

... ⇒

```
<cases to get content 20xx> +≡ (233)
case TAG(image_kind,b001): HGET_IMAGE(b001); break;
case TAG(image_kind,b010): HGET_IMAGE(b010); break;
case TAG(image_kind,b011): HGET_IMAGE(b011); break;
case TAG(image_kind,b100): HGET_IMAGE(b100); break;
case TAG(image_kind,b101): HGET_IMAGE(b101); break;
case TAG(image_kind,b110): HGET_IMAGE(b110); break;
case TAG(image_kind,b111): HGET_IMAGE(b111); break;
```

```
<get macros 19xix> +≡ (234)
#define HGET_IMAGE(I)
{ Image x = {0};
  HGET16(x.n);
  if ((I) & b100) { x.a = hget_float32();
    if ((I) ≡ b111) { hget_xdimen_node(&x.w);
```

```

    hget_xdimen_node(&x.h);
}
else if ((I)  $\equiv$  b110) { x.hr = HGET8;
    hget_xdimen_node(&x.w);
}
else if ((I)  $\equiv$  b101) { x.wr = HGET8;
    hget_xdimen_node(&x.h);
}
else { x.wr = HGET8;
    x.hr = HGET8;
}
}
else if ((I)  $\equiv$  b011) { HGET32(x.w.w);
    HGET32(x.h.w);
}
else if ((I)  $\equiv$  b010) { x.a = hget_float32();
    HGET32(x.w.w);
}
else if ((I)  $\equiv$  b001) { x.a = hget_float32();
    HGET32(x.h.w);
}
hwrite_image(&x);
{ List d;
    hget_list(&d);
    hwrite_list(&d);
}
}

```

Writing the short format:

$\implies \dots$

$\langle$ put functions 14xiv $\rangle + \equiv$

(235)

$\langle$ image functions 236ccxxvi $\rangle$

Info *hput_image_spec*(**uint32_t** *n*, **float32_t** *a*, **uint32_t** *wr*, **Xdimen** **w*, **uint32_t** *hr*, **Xdimen** **h*)

```

{ HPUT16(n);
  if (w  $\neq$  NULL  $\wedge$  h  $\neq$  NULL) {
    if (w $\rightarrow$ h  $\equiv$  0.0  $\wedge$  w $\rightarrow$ v  $\equiv$  0.0  $\wedge$  h $\rightarrow$ h  $\equiv$  0.0  $\wedge$  h $\rightarrow$ v  $\equiv$  0.0) return
        hput_image_dims(n, a, w $\rightarrow$ w, h $\rightarrow$ w);
    else { hput_image_aspect(n, a);
        hput_xdimen_node(w);
        hput_xdimen_node(h);
        return b111;
    }
  }
}
else if (w  $\neq$  NULL  $\wedge$  h  $\equiv$  NULL) {

```

```

    if ( $w \rightarrow h \equiv 0.0 \wedge w \rightarrow v \equiv 0.0 \wedge hr \equiv \text{zero\_xdimen\_no}$ )
        return hput_image_dims(n, a,  $w \rightarrow w$ , 0);
    else { hput_image_aspect(n, a);
          HPUT8(hr);
          hput_xdimen_node(w);
          return b110;
        }
}
else if ( $w \equiv \text{NULL} \wedge h \neq \text{NULL}$ ) {
    if ( $wr \equiv \text{zero\_xdimen\_no} \wedge h \rightarrow h \equiv 0.0 \wedge h \rightarrow v \equiv 0.0$ )
        return hput_image_dims(n, a, 0,  $h \rightarrow w$ );
    else { hput_image_aspect(n, a);
          HPUT8(wr);
          hput_xdimen_node(h);
          return b101;
        }
}
else {
    if ( $wr \equiv \text{zero\_xdimen\_no} \wedge hr \equiv \text{zero\_xdimen\_no}$ )
        return hput_image_dims(n, a, 0, 0);
    else { hput_image_aspect(n, a);
          HPUT8(wr);
          HPUT8(hr);
          return b100;
        }
}
}

```

If extended dimensions are involved, the long format might very well specify different values than stored in the image. In this case the given dimensions are interpreted as maximum dimensions. If the aspect ratio is missing, we use *hextract_image_dims* to extract it from the image file.

(image functions 236ccxxvi) (236)

```

static void hput_image_aspect(int n, double a)
{
    if ( $a \equiv 0.0$ ) { Dimen w, h;
        hextract_image_dims(n, &a, &w, &h);
    }
    if ( $a \neq 0.0$ ) hput_float32(a);
    else QUIT("Unable to determine aspect ratio of image %s",
              dir[n].file_name);
}

```

Used in 235ccxxv.

If no extended dimensions are involved in an image specification, we use *hput_image_dimen*.
 Because the long format can omit part of the image specification, we use *hextract_image_dims*

to extract information from the image file and merge this information with the data supplied in the long format.

```

⟨ image functions 236ccxxxvi ⟩ +≡ (237)
⟨ auxiliar image functions 239ccxxxix ⟩
static Info hput_image_dimens(int n, float32_t a, Dimen w, Dimen h)
{ Dimen iw, ih;
  double ia;
  if (w > 0 ∧ h > 0) { HPUT32(w);
    HPUT32(h);
    return b011;
  }
  else if (a > 0 ∧ w > 0) { hput_float32((float32_t) a);
    HPUT32(w);
    return b010;
  }
  else if (a > 0 ∧ h > 0) { hput_float32((float32_t) a);
    HPUT32(h);
    return b001;
  }
  hextract_image_dimens(n, &ia, &iw, &ih);
  ⟨ merge stored image dimensions with dimensions given 238ccxxxviii ⟩
  if (iw > 0) { hput_float32((float32_t) ia);
    HPUT32(iw);
    return b010;
  }
  else if (ih > 0) { hput_float32((float32_t) ia);
    HPUT32(ih);
    return b001;
  }
  else { iw = -iw;
    ih = -h; /* we accept the default resolution */
    HPUT32(iw);
    HPUT32(ih);
    return b011;
  }
}

```

If the width, height or aspect ratio is stored in the image file, we can merge this information with the information given in the long format. It is considered an error, if the function *hextract_image_dimens* can not extract the aspect ratio. Absolute width and height values, however, might be missing. If the aspect ratio is computed from the number of horizontal and vertical pixels, *hextract_image_dimens* makes the reasonable assumption that the intended resolution is 72dpi and converts the image dimensions to scaled points. It negates these values to indicate that the resolution is just a guess. This allows other programs to used different default resolutions if desired.

⟨ merge stored image dimensions with dimensions given 238ccxxxviii ⟩ ≡ (238)

```

{
  if (ia ≡ 0.0) {
    if (a ≠ 0.0) ia = a;
    else if (w ≠ 0 ∧ h ≠ 0) ia = (double) w/(double) h;
    else QUIT("Unable_to_determine_aspect_ratio_of_image_%s",
              dir[n].file_name);
  }
  /* here the aspect ratio ia is known */
  if (w ≡ 0 ∧ h ≡ 0) /* neither width nor height specified */
  {
    if (ih > 0) iw = round(ih * ia);
    else if (iw > 0) ih = round(iw / ia);
  }
  else if (h ≡ 0) /* width specified */
  { iw = w; ih = round(w / ia); }
  else if (w ≡ 0) /* height specified */
  { ih = h; iw = round(h * ia); }
  else /* both specified */
  { ih = h; iw = w; }
}

```

Used in 237ccxxxvii.

Before we present the code to extract image dimensions from various types of image files, we define a few macros and variables for the reading these image files.

⟨ auxiliar image functions 239ccxxxix ⟩ ≡ (239)

```

#define IMG_BUF_MAX 54
#define IMG_HEAD_MAX 2
static unsigned char img_buf[IMG_BUF_MAX];
static size_t img_buf_size;
#define LittleEndian32(X) (img_buf[(X)] + (img_buf[(X) + 1] <<
                        8) + (img_buf[(X) + 2] << 16) + (img_buf[(X) + 3] << 24))
#define BigEndian16(X) (img_buf[(X) + 1] + (img_buf[(X)] << 8))
#define BigEndian32(X) (img_buf[(X) + 3] + (img_buf[(X) + 2] <<
                        8) + (img_buf[(X) + 1] << 16) + (img_buf[(X)] << 24))
#define Match2(X, A, B) ((img_buf[(X)] ≡ (A)) ∧ (img_buf[(X) + 1] ≡ (B)))
#define Match4(X, A, B, C, D) (Match2(X, A, B) ∧ Match2((X) + 2, C, D))
#define GET_IMG_BUF(X)
if (img_buf_size < X) {
  size_t i = fread(img_buf + img_buf_size, 1, (X) - img_buf_size, f);
  if (i < 0) QUIT("Unable_to_read_image_%s", fn);
  else if (i ≡ 0) QUIT("Unable_to_read_image_header_%s", fn);
  else img_buf_size += i;
}

```

Used in 237ccxxxvii.

Considering the different image formats, we start with Windows Bitmaps. A Windows bitmap file usually has the extension `.bmp` but the better way to check for a Windows bitmap file is to examine the first two byte of the file: the ASCII codes for ‘B’ and ‘M’. Once we have verified the file type, we find the width and height of the bitmap in pixels at offsets `#12` and `#16` stored as little-endian 32 bit integers. At offsets `#26` and `#2A`, we find the horizontal and vertical resolution in pixel per meter stored in the same format. This is sufficient to compute the true width and height of the image in scaled points.

The Windows Bitmap format is easy to process but not very efficient. So the support for this format in the HINT format is deprecated and will disappear. You should use one of the formats described next.

```
(auxiliar image functions 239ccxxxix ) +=
static bool get_BMP_info(FILE *f, char *fn, double *a, Dimen
    *w, Dimen *h)
{ double wpx, hpx;
  double xppm, yppm;
  GET_IMG_BUF(2);
  if (!Match2(0, 'B', 'M')) return false;
  GET_IMG_BUF(#2E);
  wpx = (double) LittleEndian32(#12);           /* width in pixel */
  hpx = (double) LittleEndian32(#16);           /* height in pixel */
  xppm = (double) LittleEndian32(#26);          /* horizontal pixel per meter */
  yppm = (double) LittleEndian32(#2A);          /* vertical pixel per meter */
  *w = floor(0.5 + ONE * (72.00 * 1000.0/25.4) * wpx/xppm);
  *h = floor(0.5 + ONE * (72.00 * 1000.0/25.4) * hpx/yppm);
  *a = (wpx/xppm)/(hpx/yppm);
  return true;
}
```

Now we repeat this process for image files using the Portable Network Graphics file format. This file format is well suited to simple graphics that do not use color gradients. These images usually have the extension `.png` and start with an eight byte signature: `#89` followed by the ASCII Codes ‘P’, ‘N’, ‘G’, followed by a carriage return (`#0D` and line feed (`#0A`), an DOS end-of-file character (`#1A`) and final line feed (`#0A`). After the signature follows a list of chunks. The first chunk is the image header chunk. Each chunk starts with the size of the chunk stored as big-endian 32 bit integer, followed by the chunk name stored as four ASCII codes followed by the chunk data and a CRC. The size, as stored in the chunk, does not include the size itself, nor the name, and neither the CRC. The first chunk is the IHDR chunk. The chunk data of the IHDR chunk starts with the width and the height of the image in pixels stored as 32 bit big-endian integers.

Finding the image resolution takes some more effort. The image resolution is stored in an optional chunk named “pHYs” for the physical pixel dimensions. All we know is that this chunk, if it exists, will appear after the IHDR chunk and before the (required) IDAT chunk. The pHYs chunk contains two 32 bit big-endian integers, giving the horizontal and vertical pixels per unit, and a one byte

unit specifier, which is either 0 for an undefined unit or 1 for the meter as unit. With an undefined unit, only the aspect ratio of the pixels and hence the aspect ratio of the image can be determined. It is not uncommon, however, that the resolution in such a case is given as dots per inch. So we decide to assume the latter.

If there is resolution can not be determined, we assume a resolution of 72dpi and negate width and height to inform the calling procedure of this arbitrary choice.

```
(auxiliar image functions 239ccxxxix) +≡ (241)
static bool get_PNG_info(FILE *f, char *fn, double *a, Dimen *w, Dimen
    *h)
{ int pos, size;
  double wpx, hpx; /* width and height in pixel */
  double xppu, yppu; /* pixel per unit in x and y direction */
  int unit;
  GET_IMG_BUF(24);
  if (¬Match4(0, #89, 'P', 'N', 'G') ∨ ¬Match4(4, #0D, #0A, #1A, #0A))
    return false;
  size = BigEndian32(8);
  if (¬Match4(12, 'I', 'H', 'D', 'R')) return false;
  wpx = (double) BigEndian32(16);
  hpx = (double) BigEndian32(20);
  pos = 20 + size;
  while (true) {
    if (fseek(f, pos, SEEK_SET) ≠ 0) return false;
    img_buf_size = 0;
    GET_IMG_BUF(17);
    size = BigEndian32(0);
    if (Match4(4, 'p', 'H', 'Y', 's')) /* must occur before IDAT chunk */
    { xppu = (double) BigEndian32(8);
      yppu = (double) BigEndian32(12);
      unit = img_buf[16];
      if (unit ≡ 0) /* assuming unit is inch */
      { *w = floor(0.5 + ONE * 72.27 * wpx / xppu);
        *h = floor(0.5 + ONE * 72.27 * hpx / yppu);
        *a = (wpx / xppu) / (hpx / yppu);
        return true;
      }
    }
    else if (unit ≡ 1) /* unit is meter */
    { *w = floor(0.5 + ONE * (72.27 / 0.0254) * wpx / xppu);
      *h = floor(0.5 + ONE * (72.27 / 0.0254) * hpx / yppu);
      *a = (wpx / xppu) / (hpx / yppu);
      return true;
    }
    else break;
  }
}
```

```

    else if (Match4(4, 'I', 'D', 'A', 'T')) break;
    else pos = pos + 12 + size;
} /* we assume 72dpi and negate the results */
*w = -floor(0.5 + ONE * 72.27 * wpx / 72.0);
*h = -floor(0.5 + ONE * 72.27 * hpx / 72.0);
*a = wpx / hpx;
return true;
}

```

For photographs, the JPEG File Interchange Format (JFIF) is more appropriate. JPEG files come with all sorts of file extensions like .jpg, .jpeg, or .jfif. We check the file signature: it starts with the the SOI (Start of Image) marker #FF, #D8.

Most likely it will be followed by the JIFI-Tag. The JIFI-Tag starts with the segment marker APP0 (#FF, #E0) followed by the 2 byte segment size, followed by the ASCII codes 'J', 'F', 'I', 'F' followed by a zero byte. Next is a two byte version number which we do not read. Before the resolution proper there is a resolution unit indicator byte (0 = no units, 1 = dots per inch, 2 = dots per cm) and then comes the horizontal and vertical resolution both as 16 Bit big-endian integers.

Instead of the JIFI-Tag, there might as well be a Exif-Tag which starts with the segment marker APP1 (#FF, #E1) followed by the 2 byte segment size. Currently this tag is not decoded.

To find the actual width and height, we have to search for a start of frame marker (#FF, #C0+n with $0 \leq n \leq 15$). Which is followed by the 2 byte segment size, the 1 byte sample precession, the 2 byte height and the 2 byte width.

If the resolution was given explicitly in the JIFI-Tag, we use it. If there was no such tag or the unit was undefined, we proceed as we did for the PNG file.

```

⟨auxiliar image functions 239ccxxxix ⟩ +≡ (242)
static bool get_JPG_info(FILE *f, char *fn, double *a, Dimen *w, Dimen
    *h)
{ int pos, size;
  double wpx, hpx;
  double xppu = 72.0, yppu = 72.0;
  int unit;
  GET_IMG_BUF(18);
  if (!Match2(0, #FF, #D8)) /* SOI Start of Image */
    return false;
  pos = 2;
  while (true) {
    if (fseek(f, pos, SEEK_SET) ≠ 0) return false;
    img_buf.size = 0;
    GET_IMG_BUF(16);
    if (img_buf[0] ≠ #FF) return false; /* Not the start of a segment */
    if (img_buf[1] ≡ #EO ∧ Match4(4, 'J', 'F', 'I', 'F')) /* APP0 JFIF Tag
        */
    { unit = img_buf[11];

```



```

    xppu = (double) BigEndian16(12);
    yppu = (double) BigEndian16(14);
    if (unit == 1) ; /* already in dpi */
    else if (unit == 2) { xppu = xppu * 2.54; /* convert dot per cm to dpi */
        yppu = yppu * 2.54;
    }
    else { yppu = 72.0 * yppu / xppu; /* assume 72dpi */
        xppu = 72.0;
    }
}
else if (img_buf[1] == #C0 || img_buf[1] == #C2) /* SOF Start of Frame */
{ hpx = (double) BigEndian16(5);
  wpx = (double) BigEndian16(7);
  *w = floor(0.5 + ONE * 72.27 * wpx / xppu);
  *h = floor(0.5 + ONE * 72.27 * hpx / yppu);
  *a = (wpx / xppu) / (hpx / yppu);
  return true;
}
else if (img_buf[1] == #D9) /* EOI End of Image */
  return false;
size = BigEndian16(2);
pos = pos + 2 + size;
}
return false;
}

```

There is still one image format missing: scalable vector graphics. In the moment, I tend not to include a further image format into the definition of the HINT file format but instead use the PostScript subset that is used for Type 1 fonts to encode vector graphics. Any HINT viewer must support Type 1 PostScript fonts and hence it has already the necessary interpreter. So it seems reasonable to put the burden of converting vector graphics into a Type 1 PostScript font on the generator of HINT files and keep the HINT viewer as small and simple as possible. An alternative which would impose only a slight burden on the HINT file viewer is the use of the rsvg library.

After having considered the various types of image files, we now determine width, height and aspect ratio based on such an image file.

We combine all the above functions into the *hextract_image_dimens* function.

```

⟨image functions 236ccxxvi⟩ += (243)
void hextract_image_dimens(int n, double *a, Dimen *w, Dimen *h)
{ char *fn;
  FILE *f;
  *a = 0.0;
  *w = *h = 0;
  fn = dir[n].file_name;
  f = fopen(fn, "rb");

```

```

if (f ≠ NULL) { img_buf_size = 0;
  if (¬get_BMP_info(f, fn, a, w,
    h) ∧ ¬get_PNG_info(f, fn, a, w, h) ∧ ¬get_JPG_info(f, fn, a, w, h))
    DBG(DBGDEF, "Unknown_image_type_%s", fn);
  fclose(f);
  DBG(DBGDEF, "image%d: width=%d fpt_height=%d fpt_aspect=%f\n", n,
    *w/(double) ONE, *h/(double) ONE, *a);
}
}

```

6.2 Positions, Outlines, Links, and Labels

A viewer can usually not display the entire content section of a HINT file. Instead it will display a page of content and will give its user various means to change the page. This might be as simple as a “page down” or “page up” button (or gesture) and as sophisticated as searching using regular expressions. More traditional ways to navigate the content include the use of a table of content or an index of keywords. All these methods of changing a page have in common that a part of the content that fits nicely in the screen area provided by the output device must be rendered given a position inside the content section.

Let’s assume that the viewer uses a HINT file in short format—after all that’s the format designed for precisely this use. A position inside the content section is then the position of the starting byte of a node. Such a position can be stored as a 32 bit number. Because even the smallest node contains two tag bytes, the position of any node is strictly smaller than the maximum 32 bit number which we can conveniently use as a “non position”.

```

⟨hint macros 13xiii⟩ +≡ (244)
#define HINT_NO_POS #FFFFFFFF

```

To render a page starting at a given position is not difficult: We just read content nodes, starting at the given position and feed them to T_EX’s page builder until the page is complete. To implement a “clickable” table of content this is good enough. We store with every entry in the table of content the position of the section header, and when the user clicks the entry, the viewer can display a new page starting exactly with that section header.

Things are slightly more complex if we want to implement a “page down” button. If we press this button, we want the next page to start exactly where the current page has ended. This is typically in the middle of a paragraph node, and it might even be in the middle of an hyphenated word in that paragraph. Fortunately, paragraph and table nodes are the only nodes that can be broken across page boundaries. But broken paragraph nodes are a common case non the less, and unless we want to search for the enclosing node, we need to augment in this case the primary 32 bit position inside the content section with a secondary position. Most of the time, 16 bit will suffice for this secondary position if we give it relative to the primary position. Further, if the list of nodes forming the paragraph is given as a text, we need to know the current font at the secondary position. Of course, the viewer can find it by scanning the initial part of the text, but when we think of

a page down button, the viewer might already know it from rendering the previous page.

Similar is the case of a “page up” button. Only here we need a page that ends precisely where our current page starts. Possibly even with the initial part of a hyphenated word. Here we need a reverse version of T_EX’s page builder that assembles a “good” page from the bottom up instead of from the top down. Sure the viewer can cache the start position of the previous page (or the rendering of the entire page) if the reader has reached the current page using the page down button. But this is not possible in all cases. The reader might have reached the current page using the table of content or even an index or a search form.

This is the most complex case to consider: a link from an index or a search form to the position of a keyword in the main text. Let’s assume someone looks up the word “München”. Should the viewer then generate a page that starts in the middle of a sentence with the word “München”? Probably not! We want a page that shows at least the whole sentence if not the whole paragraph. Of course the program that generates the link could specify the position of the start of the paragraph instead of the position of the word. But that will not solve the problem. Just imagine reading the groundbreaking masterpiece of a German philosopher on a small hand-held device: the paragraph will most likely be very long and perhaps only part of the first sentence will fit on the small screen. So the desired keyword might not be found on the page that starts with the beginning of the paragraph; it might not even be on the next or next to next page. Only the viewer can decide what is the best fragment of content to display around the position of the given keyword.

To summarize, we need three different ways to render a page for a given position:

- A page that starts exactly at the given position.
- A page that ends exactly at the given position.
- The “best” page that contains the given position somewhere in the middle.

A possible way to find the “best” page for the latter case could be the following:

- If the position is inside a paragraph, break the paragraph into lines. One line will contain the given position. Let’s call this the destination line.
- If the paragraph will not fit entirely on the page, start the page with the beginning of the paragraph if that will place the destination line on the page, otherwise start with a line in the paragraph that is about half a page before the destination line.
- Else traverse the content list backward for about 2/3 of the page height and forward for about 2/3 of the page height, searching for the smallest negative penalty node. Use the penalty node found as either the beginning or ending of the page.
- If there are several equally low negative penalty nodes. Prefer penalties preceding the destination line over penalty nodes following it. A good page start is more important than a good page end.
- If there are still several equally low negative penalty nodes, choose the one whose distance to the destination line is closest to 1/2 of the page height.

- If no negative penalty nodes could be found, start the page with the paragraph containing the destination line.
- Once the page start (or end) is found, use $\text{\TeX}$'s page builder (or its reverse variant) to complete the page.

We call content nodes that reference some position inside the content section “link” nodes. The position that is referenced is called the destination of the link. Link nodes occur always in pairs of an “start” link followed by a corresponding “end” link that both reference the same position and no other link nodes between them. The content between the two will constitute the visible part of the link.

To encode a position inside the content section that can be used as the destination of a link node, an other kind of node is needed which we call a “label”.

Links are not the only way to navigate inside a large document. The user interface can also present an “outline” of the document that can be used for navigation. An outline node implements an association between a name displayed by the user interface of the **HINT** viewer and the destination position in the **HINT** document.

It is possible though that outline nodes, link nodes, and label nodes can share the same kind-value and we have $outline_kind \equiv link_kind \equiv label_kind$. To distinguish an outline node from a label node—both occur in the short format definition section—the *b100* info bit is set in an outline node.

```

⟨get functions 18xviii⟩ +≡ (245)
void hget_outline_or_label_def(Info i, uint32_t node_pos)
{ if (i & b100) ⟨get and write an outline node 277cclxxvii⟩
  else ⟨get and store a label node 261cclxi⟩
}
```

The next thing we need to implement is a new maximum number for outline nodes. We store this number in the variable *max_outline* and limit it to a 16 bit value.

In the short format, the value of *max_outline* is stored with the other maximum values using the kind value $outline_kind \equiv label_kind$ and the info value *b100* for single byte and *b101* for a two byte value.

Reading the Short Format: ... $\Rightarrow$

```

⟨cases of getting special maximum values 246ccxlvi⟩ ≡ (246)
case TAG(outline_kind, b100): case TAG(outline_kind, b101): max_outline = n;
  DBG(DBGDEF | DBGLABEL, "max(outline) = %d\n", max_outline);
  break;
```

Used in 390ccxc.

Writing the Short Format:

⇒ ...

```

⟨ cases of putting special maximum values 247ccxlvii ⟩ ≡ (247)
    if (max_outline > -1) { uint32_t pos = hpos++ - hstart;
        DBG(DBGDEF | DBGLABEL, "max_outline=%d\n", max_outline);
        hput_tags(pos, TAG(outline_kind, b100 | (hput_n(max_outline) - 1)));
    }

```

Used in 391ccxcxi.

Writing the Long Format:

⇒ - - -

```

⟨ cases of writing special maximum values 248ccxlviii ⟩ ≡ (248)
    case label_kind:
        if (max_ref[label_kind] > -1)
        { hwrite_start();
          hwritef("label_%d", max_ref[label_kind]);
          hwrite_end(); }
        if (max_outline > -1)
        { hwrite_start();
          hwritef("outline_%d", max_outline);
          hwrite_end(); }
        break;

```

Used in 389ccclxxxix.

Reading the Long Format:

- - - ⇒

```

⟨ parsing rules 5v ⟩ +≡ (249)
    max_value: OUTLINE UNSIGNED { max_outline = $2;
        RNG("max_outline", max_outline, 0, #FFFF); DBG(DBGDEF | DBGLABEL,
            "Setting_max_outline_to_%d\n", max_outline); };

```

After having seen the maximum values, we now explain labels, then links, and finally outlines.

To store labels, we define a data type *Label* and an array *labels* indexed by the labels reference number.

```

⟨ hint basic types 6vi ⟩ +≡ (250)
    typedef struct { uint32_t pos; /* position */
        uint32_t pos0; /* secondary position */
        uint8_t where; /* where on the rendered page */
        bool used; /* label used in a link or an outline */
        int next; /* reference in a linked list */
        uint8_t f; /* font, currently not used */
    } Label;

```

The *where* field indicates where the label position should be on the rendered page: at the top, at the bottom, or somewhere in the middle. An undefined label has *where* equal to zero.

```

⟨ hint macros 13xiii ⟩ +≡ (251)
#define LABEL_UNDEF 0
#define LABEL_TOP 1
#define LABEL_BOT 2
#define LABEL_MID 3

```

```

⟨ common variables 252ccli ⟩ ≡ (252)
Label *labels = NULL;
int first_label = -1;

```

Used in 552dl*ii*, 554dl*iv*, 557dl*vii*, 558dl*viii*, and 560dl*x*.

The variable *first_label* will be used together with the *next* field of a label to construct a linked list of labels.

```

⟨ initialize definitions 253cclii ⟩ ≡ (253)
if (max_ref[label_kind] ≥ 0)
    ALLOCATE(labels, max_ref[label_kind] + 1, Label);

```

Used in 381ccclxx*xi* and 387ccclxxxv*ii*.

The implementation of labels has to solve the problem of forward links: a link node that references a label that is not yet defined. We solve this problem by keeping all labels in the definition section. So for every label at least a definition is available before we start with the content section and we can fill in the position when the label is found. If we restrict labels to the definition section and do not have an alternative representation, the number of possible references is a hard limit on the number of labels in a document. Therefore label references are allowed to use 16 bit reference numbers. In the short format, the *b001* bit indicates a two byte reference number if set, and a one byte reference number otherwise.

In the short format, the complete information about a label is in the definition section. In the long format, this is not possible because we do not have node positions. Therefore we will put label nodes at appropriate points in the content section and compute the label position when writing the short format.

Reading the long format: - - - ⇒

```

⟨ symbols 2ii ⟩ +≡ (254)
%token LABEL "label"
%token BOT "bot"
%token MID "mid"
%type < i > placement

```

```

⟨ scanning rules 3iii ⟩ +≡ (255)
label          return LABEL;
bot           return BOT;
mid          return MID;

```

A label node specifies the reference number and a placement.

```

⟨ parsing rules 5v ⟩ +≡ (256)
  placement: TOP { $$ = LABEL_TOP; }
    | BOT { $$ = LABEL_BOT; }
    | MID { $$ = LABEL_MID; }
    | { $$ = LABEL_MID; };
  content_node: START LABEL REFERENCE placement END
    { hset_label($3,$4); };

```

After parsing a label, the function *hset_label* is called.

```

⟨ put functions 14xiv ⟩ +≡ (257)
  void hset_label(int n, int w)
  { Label *t;
    REF_RNG(label_kind, n);
    t = labels + n;
    if (t→where ≠ LABEL_UNDEF)
      MESSAGE("Duplicate_definition_of_label_%d\n", n);
    t→where = w;
    t→pos = hpos - hstart;
    t→pos0 = hpos0 - hstart;
    t→next = first_label;
    first_label = n;
  }

```

The above function will simply store the data obtained in the *labels* array. The generation of the short format output is postponed until the entire content section has been parsed and the positions of all labels are known.

One more complication needs to be considered: The *hput_list* function is allowed to move lists in the output stream and if positions inside the list were recorded in a label, these labels need an adjustment. To find out quickly if any labels are affected, the *hset_label* function constructs a linked list of labels starting with the reference number of the most recent label in *first_label* and the reference number of the label preceding label *i* in *labels[i].next*. Because labels are recorded with increasing positions, the list will be sorted with positions decreasing.

```

⟨ adjust label positions after moving a list 258cclviii ⟩ ≡ (258)
  { int i;
    for (i = first_label; i ≥ 0 ∧ labels[i].pos ≥ l→p; i = labels[i].next) {
      DBG(DBGNODE | DBGLABEL, "Moving_label_%d_by_%d\n", i, d);
      labels[i].pos += d;
      if (labels[i].pos0 ≥ l→p) labels[i].pos0 += d;
    }
  }

```

Used in 148cxlvi.

The *hwrite_label* function is the reverse of the above parsing rule. Note that it is different from the usual *hwrite...* functions. And we will see shortly why that is so.

Writing the long format:

⇒ - - -

```

< write functions 21xxi > +≡ (259)
void hwrite_label(void) /* called in hwrite_end and at the start of a list */
{ while (first_label ≥ 0 ∧ (uint32_t)(hpos - hstart) ≥ labels[first_label].pos)
  { Label *t = labels + first_label;
    DBG(DBGLABEL, "Inserting_label_%d\n", first_label);
    hwrite_start();
    hwritef("label_%d", first_label);
    if (t→where ≡ LABEL_TOP) hwritef("_top");
    else if (t→where ≡ LABEL_BOT) hwritef("_bot");
    nesting--;
    hwritec('>'); /* avoid a recursive call to hwrite_end */
    first_label = labels[first_label].next;
  }
}

```

The short format specifies the label positions in the definition section. This is not possible in the long format because there are no “positions” in the long format. Therefore long format label nodes must be inserted in the content section just before those nodes that should come after the label. The function *hwrite_label* is called in *hwrite_end*. At that point *hpos* is the position of the next node and it can be compared with the positions of the labels taken from the definition section. Because *hpos* is strictly increasing while reading the content section, the comparison can be made efficient by sorting the labels. The sorting uses the *next* field in the array of *labels* to construct a linked list. After sorting, the value of *first_label* is the index of the label with the smallest position; and for each *i*, the value of *labels[i].next* is the index of the label with the next bigger position. If *labels[i].next* is negative, there is no next bigger position. Currently a simple insertion sort is used. The insertion sort will work well if the labels are already mostly in ascending order. If we expect lots of labels in random order, a more sophisticated sorting algorithm might be appropriate.

```

< write functions 21xxi > +≡ (260)
void hsort_labels(void)
{ int i;
  if (max_ref[label_kind] < 0) { first_label = -1;
    return; } /* empty list */
  first_label = max_ref[label_kind];
  while (first_label ≥ 0 ∧ labels[first_label].where ≡ LABEL_UNDEF)
    first_label--;
  if (first_label < 0) return; /* no defined labels */
  labels[first_label].next = -1;
  DBG(DBGLABEL, "Sorting_%d_labels\n", first_label + 1);
  for (i = first_label - 1; i ≥ 0; i--) /* insert label i */
    if (labels[i].where ≠ LABEL_UNDEF)
      { uint32_t pos = labels[i].pos;

```



```

    if (labels[first_label].pos ≥ pos)
    { labels[i].next = first_label;
      first_label = i; } /* new smallest */
  else
  { int j;
    for (j = first_label; labels[j].next ≥ 0 ∧ labels[labels[j].next].pos < pos;
         j = labels[j].next) continue;
    labels[i].next = labels[j].next;
    labels[j].next = i;
  }
}
}

```

The following code is used to get label information from the definition section and store it in the *labels* array. The *b010* bit indicates the presence of a secondary position for the label.

Reading the short format:

... $\implies$

```

⟨ get and store a label node 261cclxi ⟩ ≡ (261)
{ Label *t;
  int n;

  if (i & b001) HGET16(n); else n = HGET8;
  REF_RNG(label_kind, n);
  t = labels + n;
  if (t→where ≠ LABEL_UNDEF) DBG(DBGLABEL,
    "Duplicate_definition_of_label%d_at_0x%x\n", n, node_pos);
  HGET32(t→pos);
  t→where = HGET8;
  if (t→where ≡ LABEL_UNDEF ∨ t→where > LABEL_MID)
    DBG(DBGLABEL, "Label%d_where_value_invalid: %d_at_0x%x\n", n,
      t→where, node_pos);
  if (i & b010) /* secondary position */
  { HGET32(t→pos0);
    t→f = HGET8; }
  else t→pos0 = t→pos;
  DBG(DBGLABEL, "Defining_label%d_at_0x%x/0x%x\n", n, t→pos0, t→pos);
}

```

Used in 245ccxlv.

The function *hput_label* is simply the reverse of the above code.

Writing the short format:

$\Rightarrow \dots$

```

< put functions 14xiv > +≡ (262)
Tag hput_label(int n, Label *l)
{ Info i = b000;
  HPUTX(13);
  if (n > #FF) { i |= b001;
    HPUT16(n); } else HPUT8(n);
  HPUT32(l→pos);
  HPUT8(l→where);
  if (l→pos ≠ l→pos0) { i |= b010;
    HPUT32(l→pos0);
    HPUT8(l→f); }
  DBG(DBGLABEL, "Defining_label_%d_at_0x%x/0x%x\n", n, l→pos0, l→pos);
  return TAG(label_kind, i);
}

```

hput_label_defs is called by the parser after the entire content section has been processed; it appends the label definitions to the definition section. The outlines are stored after the labels because they reference the labels.

```

< put functions 14xiv > +≡ (263)
extern void hput_definitions_end(void);
extern Tag hput_outline(Outline *t);
void hput_label_defs(void)
{ int n;
  section_no = 1;
  hstart = dir[1].buffer;
  hend = hstart + dir[1].bsize;
  hpos = hstart + dir[1].size;
  < output the label definitions 264cclxiv >
  < output the outline definitions 284cclxxxiv >
  hput_definitions_end();
}

```

```

< output the label definitions 264cclxiv > ≡ (264)
for (n = 0; n ≤ max_ref[label_kind]; n++)
{ Label *l = labels + n;
  uint32_t pos;
  if (l→used)
  { pos = hpos++ - hstart;
    hput_tags(pos, hput_label(n, l));
    if (l→where ≡ LABEL_UNDEF)
      MESSAGE("WARNING: Label_%d_is_used_but_not_defined\n", n);
    else DBG(DBGDEF | DBGLABEL, "Label_%d_defined_0x%x\n", n, pos);
  }
else {

```

```

    if ( $l \rightarrow where \neq$  LABEL_UNDEF) {  $pos = hpos++ - hstart$ ;
         $hput\_tags(pos, hput.label(n, l))$ ;
        DBG(DBGDEF | DBGLABEL, "Label_*%d_defined_but_not_used_0x%x\n",
            n, pos);
    }
}
}

```

Used in 263cclxiii.

Links are simpler than labels. They are found only in the content section and resemble pretty much what we have seen for other content nodes. Let's look at them next. When reading a short format link node, we use again the *b001* info bit to indicate a 16 bit reference number to a label.

To help a reader tell a link from ordinary text, links should be visually different. This is supported in the HINT file format by associating a different color scheme to a link. In the short format, the *b100* bit indicates that a color set reference (see section 6.3) follows after the label reference. A color reference to 1 in the start node and to #FF in the end node is the default and is omitted.

Because color changes are local to the enclosing box or paragraph, a link is local as well. Without further mentioning, here and in the following, when we say “box” it also mean “paragraph”. A link starts with a “start” link and ends with either an “end” link or the end of the enclosing box. Links must not be nested. It is an error to have two start links in the same box without an end link between them. An application can choose to continue a link in the next box by inserting a copy of the start link node at the beginning of the new box. In short: “end” links are mandatory when separating two links but optional if they just precede the end of the box. The *b010* info bit indicates a “start” link; otherwise it is an “end” link.

Reading the short format:

... $\Rightarrow$

```

<get macros 19xix > +≡ (265)
#define HGET_LINK(I)

```

```

    { int n, c;
        if (I & b001) HGET16(n); else n = HGET8;
        if (I & b100) c = HGET8;
        else c = (I & b010) ? 1 : #FF;
         $hwrite\_link(n, c, I \& b010)$ ; }

```

```

<cases to get content 20xx > +≡ (266)

```

```

    case TAG(link_kind, b000): HGET_LINK(b000); break;
    case TAG(link_kind, b001): HGET_LINK(b001); break;
    case TAG(link_kind, b010): HGET_LINK(b010); break;
    case TAG(link_kind, b011): HGET_LINK(b011); break;
    case TAG(link_kind, b100): HGET_LINK(b100); break;
    case TAG(link_kind, b101): HGET_LINK(b101); break;
    case TAG(link_kind, b110): HGET_LINK(b110); break;
    case TAG(link_kind, b111): HGET_LINK(b111); break;

```

The function *hput_link* will insert the link in the output stream and return the appropriate tag.

Writing the short format:

$\Rightarrow \dots$

```

<put functions 14xiv> +≡ (267)
  Tag hput_link(int n, int c, int on)
  { Info i;
    REF_RNG(label_kind, n);
    labels[n].used = true;
    if (on) i = b010; else i = b000;
    if (n > #FF) { i |= b001;
      HPUT16(n); } else HPUT8(n);
    if ((on & c ≠ 1) ∨ (¬on & c ≠ #FF)) { i |= b100;
      HPUT8(c);
    }
    return TAG(link_kind, i);
  }

```

Reading the long format:

$--- \Rightarrow$

```

<symbols 2ii> +≡ (268)
%token LINK "link"

```

```

<scanning rules 3iii> +≡ (269)
link          return LINK;

```

```

<parsing rules 5v> +≡ (270)
content_node: start LINK REFERENCE on_off END {
  hput_tags($1, hput_link($3, $4 ? 1 : #FF, $4)); };
content_node: start LINK REFERENCE on_off REFERENCE END {
  hput_tags($1, hput_link($3, $5, $4)); };

```

Writing the long format:

$\Rightarrow ---$

```

<write functions 21xxi> +≡ (271)
void hwrite_link(int n, int c, uint8_t on)
{ REF_RNG(label_kind, n);
  if (labels[n].where ≡ LABEL_UNDEF)
    MESSAGE("WARNING: Link to an undefined label %d\n", n);
  hwrite_ref(n);
  if (on) hwritef("_on");
  else hwritef("_off");
  if ((on & c ≠ 1) ∨ (¬on & c ≠ #FF)) { REF_RNG(color_kind, c);
    hwrite_ref(c);
  }
}

```

Now we look at the outline nodes which are found only in the definition section. Every outline node is associated with a label node, giving the position in the document, and a unique title that should tell the user what to expect when navigating to this position. For example an item with the title “Table of Content” should navigate to the page that shows the table of content. The sequence of outline nodes found in the definition section gets a tree structure by assigning to each item a depth level.

```

< hint types 1i > +≡ (272)
    typedef struct { uint8_t *t;           /* title */
        int s;                          /* title size */
        int d;                          /* depth */
        uint16_t r;                      /* reference to a label */
    } Outline;

```

```

< shared put variables 273cclxxiii > ≡ (273)
    Outline *outlines;

```

Used in 554dliv, 557dlvii, 558dlviii, and 560dlx.

```

< initialize definitions 253ccliii > +≡ (274)
    if (max_outline ≥ 0)
        ALLOCATE(outlines, max_outline + 1, Outline);

```

Child items follow their parent item and have a bigger depth level. In the short format, the first item must be a root item, with a depth level of 0. Further, if any item has the depth d , then the item following it must have either the same depth d in which case it is a sibling, or the depth $d + 1$ in which case it is a child, or a depth d' with $0 \leq d' < d$ in which case it is a sibling of the latest ancestor with depth d' . Because the depth is stored in a single byte, the maximum depth is #FF.

In the long format, the depth assignments are more flexible. We allow any signed integer, but insist that the depth assignments can be compressed to depth levels for the short format using the following algorithm:

```

< compress long format depth levels 275cclxxv > ≡ (275)
    n = 0; while (n ≤ max_outline) n = hcompress_depth(n, 0);

```

Used in 284cclxxxiv.

Outline items must be listed in the order in which they should be displayed. The function $hcompress_depth(n, c)$ will compress the subtree starting at n with root level d to a new tree with the same structure and root level c . It returns the outline number of the following subtree.

```

< put functions 14xiv > +≡ (276)
    int hcompress_depth(int n, int c)
    { int d = outlines[n].d;
      if (c > #FF)
          QUIT("Outline_%d, depth_level_%d to %d out_of_range", n, d, c);
      while (n ≤ max_outline)
          if (outlines[n].d ≡ d) outlines[n++].d = c;
    }

```

```

    else if (outlines[n].d > d) n = hcompress_depth(n, c + 1);
    else break;
return n;
}

```

For an outline node, the *b001* bit indicates a two byte reference to a label. There is no reference number for an outline item itself: it is never referenced anywhere in an HINT file.

Reading the short format:

... $\Rightarrow$

Writing the long format:

$\Rightarrow$ - - -

$\langle$ get and write an outline node [277ccclxxvii](#) $\rangle \equiv$ (277)

```

{ int r, d;
  List l;
  static int outline_no = -1;

  hwrite_start(); hwritef("outline");
  ++outline_no;
  RNG("outline", outline_no, 0, max_outline);
  if (i & b001) HGET16(r); else r = HGET8;
  REF_RNG(link_kind, r);
  if (labels[r].where  $\equiv$  LABEL_UNDEF)
    MESSAGE("WARNING: Outline with undefined label %d at 0x%x\n",
            r, node_pos);
  hwritef("_*%d", r);
  d = HGET8;
  hwritef("_%d", d);
  hget_list(&l);
  hwrite_list(&l);
  hwrite_end();
}

```

Used in [245ccclxlv](#).

When parsing an outline definition in the long format, we parse the outline title as a *list* which will write the representation of the list to the output stream. Writing the outline definitions, however, must be postponed until the label have found their way into the definition section. So we save the list's representation in the outline node for later use and remove it again from the output stream.

Reading the long format:

— — — $\Rightarrow$

$\langle$ symbols 2ii $\rangle + \equiv$ (278)
`%token OUTLINE "outline"`

$\langle$ scanning rules 3iii $\rangle + \equiv$ (279)
`outline            return OUTLINE;`

$\langle$ parsing rules 5v $\rangle + \equiv$ (280)
`def_node: START OUTLINE REFERENCE integer position list END { static`
`int outline_no = -1;`
`$$k = outline_kind; $$n = $3;`
`if ($6.s  $\equiv$  0)`
`QUIT("Outline with empty title in line %d", yylineno);`
`outline_no++; hset_outline(outline_no, $3, $4, $5); };`

$\langle$ put functions 14xiv $\rangle + \equiv$ (281)
`void hset_outline(int m, int r, int d, uint32_t pos)`
`{ Outline *t;`
`RNG("Outline", m, 0, max_outline);`
`t = outlines + m;`
`REF_RNG(label_kind, r);`
`t→r = r;`
`t→d = d;`
`t→s = hpos - (hstart + pos);`
`hpos = (hstart + pos);`
`ALLOCATE(t→t, t→s, uint8_t);`
`memmove(t→t, hpos, t→s);`
`labels[r].used = true;`
`}`

To output the title, we need to move the list back to the output stream. Before doing so, we allocate space (and make sure there is room left for the end tag of the outline node), and after doing so, we release the memory used to save the title.

$\langle$ output the title of outline *t 282cclxxxii $\rangle \equiv$ (282)
`memmove(hpos, t→t, t→s);`
`hpos = hpos + t→s;`
`free(t→t);`

Used in 283cclxxxiii.

We output all outline definitions from 0 to *max_outline* and check that every one of them has a title. Thereby we make sure that in the short format *max_outline* matches the number of outline definitions.

Writing the short format:

$\Rightarrow \dots$

```

<put functions 14xiv> +≡ (283)
  Tag hput_outline(Outline *t)
  { Info i = b100;
    HPUTX(t→s + 4);
    if (t→r > #FF) { i |= b001; HPUT16(t→r); } else HPUT8(t→r);
    labels[t→r].used = true;
    HPUT8(t→d);
    <output the title of outline *t 282cclxxxii>
    return TAG(outline_kind, i);
  }

<output the outline definitions 284cclxxxiv> ≡ (284)
<compress long format depth levels 275cclxxv>
for (n = 0; n ≤ max_outline; n++) { Outline *t = outlines + n;
  uint32_t pos;
  pos = hpos++ - hstart;
  if (t→s ≡ 0 ∨ t→t ≡ NULL)
    QUIT("Definition of outline %d has an empty title", n);
  DBG(DBGDEF | DBGLABEL, "Outline *%d defined\n", n);
  hput_tags(pos, hput_outline(t));
}

```

Used in 263cclxiii.

6.3 Colors

This is the third draft of implementing color specifications in a HINT file.

According to the initial philosophy of a HINT file, a viewer must be capable of rendering a page given just any valid position in the content section without reading the entire file. This makes it impossible to use global information; only the information that is locally available can be used. Given a file position, the viewer will compute a representation of the page, insert it into a page template, and pass it to the renderer. Color will not effect the position of glyphs or rules and so it is sufficient to process the color information when rendering the page. The renderer will, however, render the page always from the top down and from left to right. As a consequence of the rendering order, it is very well possible to work with a color state within the top level boxes.

A separate issue is the specification of color changes on the top level. While a vertical list contains no character nodes, a color specification might still affect the background color and the foreground color of rules. Because we still want to avoid the search for color nodes on the top level, we restrict the scope of a color node on the top level. It will extend only to the next possible page break and applications like HiTeX must repeat a top level color node after every node that could be used as a page break.

The nesting of boxes on a page together with the transparency of colors leads to the problem of stacking several layers of color one on top of the other. Here

is an example: An outer box might specify blue as a background color and white as a foreground color while an inside box specifies a transparent grey background and a transparent black foreground. Then we expect text in the outer box to have white letters on blue background. Further we want to see the inner box casting a grey shadow on the blue background, resulting in a mix of blue and gray, with black letters on top of it that are not completely black but let the background shine through.

To limit the complexity, the `HINT` file format will allow this stacking of colors only when nesting boxes. But inside a box, there is at any position only one foreground and one background color; a color change inside a box will simply replace the current colors.

If an application like `HiTeX` wants to implement nesting colors inside a box, it has to implement its own color stack and compute the necessary color mixtures. There is only one exception to this concept: When a new box starts, the current colors will be those of the enclosing box. These colors can be restored after a color change by using the `< color off >` command.

The limited complexity is necessary to simplify the splitting of boxes, for example by the line breaking routine. Repeating the last color node before the split just after the split is sufficient.

Inside a horizontal list, a background color will extend from top to bottom; inside a vertical list a background color will extend from the left edge to the right edge. If the document does not want to change the background color, a completely transparent color should be used.

While the current implementation of searching does not use the background color, a color set will still specify background and foreground for all colors. This is simpler, easier to extend at a later time, and the overhead is small.

After these preliminaries let's turn our attention to the design of a suitable color concept.

Colors come in sets. A color set supports two modes: day and night mode. In future extensions it might be possible for an author to invent color sets for winter or summer, fall or spring, or any other resonable or unreasonable purpose. For each mode a color set specifies three color styles: one for normal text, one for marked text and one for in-focus text. The switching between different modes and different styles is left to the user interface.

We store a color set as an array of 12 words. The first 6 words are for day mode the next 6 byte are for night mode; For each mode we have three color pairs and each pair consists of a foreground and a background color each stored as an RGBA value.

```
< hint basic types 6vi > +≡
typedef uint32_t ColorSet[2 * 3 * 2];
```

(285)

To extract the various sub-arrays, we have the following macros:

```
< hint macros 13xiii > +≡
#define CURCOLOR (M, S, C) ((C) + 6 * (M) + 2 * (S))
#define DAY (C) CURCOLOR (0, 0, C)
#define NIGHT (C) CURCOLOR (1, 0, C)
```

(286)

```
#define HIGH (C) CURCOLOR (0, 1, C)
#define FOCUS (C) CURCOLOR (0, 2, C)
#define FG (C) ((C)[0])
#define BG (C) ((C)[1])
```

We will allow up to 255 color sets that are stored in the definition section and are referenced in the content section by a single byte. The definition of different color sets and the switching between them is left to the document author.

The color set with reference number zero specifies the default colors. At the root of a page template, the default color set is selected and the whole page is filled with the background color for normal text. For links, by default the color set with number one is used. Section 11.11 specifies default values for both color sets; the default colors can be overwritten. The color sets with reference numbers zero and one are not stored in the definition section of a short format file if they are the same as the default values. This makes files not using colors compatible with older versions of the HINT file format.

Now we are ready for the implementation.

Reading the long format: - - - $\implies$
Writing the short format: $\implies \dots$

```
<symbols 2ii > +≡ (287)
%token COLOR "color"
```

```
<scanning rules 3iii > +≡ (288)
color          return COLOR;
```

Colors can be specified as a single number, preferably in hexadecimal notation, giving the red, green, blue, and alpha value in a single number. For example 0xFF0000FF would be pure red, and 0x00FF0080 would be transparent green. Of course even decimal values can be used. A good example is the value 0 which is equivalent to but a bit shorter than 0x0 or 0x00000000 which describes a completely transparent black. It is invisible because the alpha value is zero.

Alternatively, colors can be given as a list of three or four numbers enclosed in pointed brackets < ... >. If only three numbers are given, the color is opaque with an alpha value equivalent to 0xFF. Using this format the same colors as before can be written <0xFF 0 0> (pure red), <0 0xFF 0 0x80> (transparent green) and <0 0 0 0> (transparent black).

The parser will put the color definition into *colors_n* using the index *colors_i*. As we will see later, the *colors_n* array is initialized with the colors in *colors_0* which in turn is initialized from *color_defaults*[0]. *colors_0* can be changed but only if that change occurs before any other color definition.

```
<common variables 252ccli > +≡ (289)
  ColorSet colors_0, colors_n;          /* default and current color set */
  int colors_i;                          /* current color */
```

```
<initialize definitions 253ccliii > +≡ (290)
{ int i;
```

```

    for (i = 0; i < sizeof(ColorSet)/4; i++) colors_0[i] = color_defaults[0][i];
}

⟨ parsing rules 5v ⟩ +≡ (291)
color:  START UNSIGNED UNSIGNED UNSIGNED UNSIGNED END
{  RNG("red", $2, 0, #FF);  RNG("green", $3, 0, #FF);
  RNG("blue", $4, 0, #FF);  RNG("alpha", $5, 0, #FF);
  colors_n[colors_i++] = ($2 << 24) | ($3 << 16) | ($4 << 8) | $5; }
|  START UNSIGNED UNSIGNED UNSIGNED UNSIGNED END {  RNG("red", $2, 0, #FF);
  RNG("green", $3, 0, #FF);  RNG("blue", $4, 0, #FF);
  colors_n[colors_i++] = ($2 << 24) | ($3 << 16) | ($4 << 8) | #FF; };

color:  UNSIGNED {  colors_n[colors_i++] = $1; };

```

Colors are always specified in pairs: a foreground color folowed by background color enclosed in pointed brackets `< ... >` as usual. For convenience, the background color can be omitted; in this case a completely transparent background is assumed.

$$\begin{aligned} & \langle \text{parsing rules } 5v \rangle + \equiv & (292) \\ & \text{color_pair: START color color END} \\ & \quad | \text{ START color END } \{ \text{colors_n[colors_i++] = 0; } \}; \\ & \text{color_unset:} \\ & \quad \{ \text{colors_i} += 2; \\ & \quad \} \\ & \quad ; \end{aligned}$$

A complete color set consists of six color pairs organized in two *color_trippls*: the first three pairs for normal, mark, and focus text in day mode are followed by the three pairs in night mode. The *color_tripple* for night mode is optional; and within a *color_tripple* all color pairs except the first one are optional. An omitted color is replaced by the corresponding color from the color set zero. To make the replacement process more predictable, the specification of color set zero—if present—must come first. If the default color set itself is redefined, an unspecified color will not change the default color.

To be open to future changes, color set definitions in the short format will contain after the reference number the number of color pairs that follow. Currently this value is always six.

```

⟨ parsing rules 5v ⟩ +≡ (293)
color_tripple:  START color_pair color_unset color_unset END
               |  START color_pair color_pair color_unset END
               |  START color_pair color_pair color_pair END;
color_set:  color_tripple color_tripple;
color_set:  color_tripple color_unset color_unset color_unset; def_node:  start
COLOR ref { HPUT8(6); color_init(); } color_set END {
DEF($$, color_kind, $3); hput_color_def($1, $3); };

```

(294)

```

⟨put functions 14xiv ⟩ +≡
    void color_init(void)
    { int i;
      for (i = 0; i < sizeof(ColorSet)/4; i++) colors_n[i] = colors_0[i];
      colors_i = 0;
    }

    static Tag hput_color_set(int n)
    { static bool first_color = true;
      int i;
      if (n ≡ 0) {
        if (first_color)
          for (i = 0; i < sizeof(ColorSet)/4; i++) colors_0[i] = colors_n[i];
        else QUIT("Redefinition of color_set must be the\
          first color definition");
      }
      first_color = false;
      HPUTX(sizeof(ColorSet) + 1);
      for (i = 0; i < sizeof(ColorSet)/4; i++) HPUT32(colors_n[i]);
      return TAG(color_kind, b000);
    }

```

The *hput_color_def* checks if color sets zero or one need to be written. If not, the function will reset *hpos* to undo the writing of the tag and the number of colors in the set.

(295)

```

⟨put functions 14xiv ⟩ +≡
    static bool colors_equal(ColorSet a, ColorSet b)
    { int i;
      for (i = 0; i < sizeof(ColorSet)/4; i++)
        if (a[i] ≠ b[i]) return false;
      return true;
    }

    void hput_color_def(uint32_t pos, int n)
    {
      if ((n ≡ 0 ∧ colors_equal(color_defaults[0],
        colors_n)) ∨ (n ≡ 1 ∧ colors_equal(color_defaults[1], colors_n))) {
        hpos = hstart + pos;
        return;
      }
      hput_tags(pos, hput_color_set(n));
    }

```

Compared to the definitions, the content nodes are pretty simple. The special color number *#FF* is reserved to indicate an ⟨color off⟩ node in the short format.

```

⟨ parsing rules 5v ⟩ +≡ (296)
  content_node: start COLOR ref END { REF_RNG(color_kind, $3);
    hput_tags($1, TAG(color_kind, b000)); };
  content_node: start COLOR OFF END { HPUT8(#FF);
    hput_tags($1, TAG(color_kind, b000)); };

```

Writing the long format:

⇒ - - -

Reading the short format:

... ⇒

We continue with the color content nodes:

```

⟨ cases to get content 20xx ⟩ +≡ (297)
  case TAG(color_kind, b000):
    { uint8_t n = HGET8; if (n ≡ #FF) hwritef("_off");
      else { REF(color_kind, n); hwrite_ref(n); }
    }
  break;

```

And now we turn to the color definitions:

```

⟨ get functions 18xviii ⟩ +≡ (298)
  void hwrite_color_pair(uint32_t f, uint32_t b)
  { hwritec('<');
    if (f ≡ 0) hwritec('0');
    else hwritef("0x%08X", f);
    if (b ≠ 0) hwritef("_0x%08X", b);
    hwritec('>');
  }

  void hget_color_set(uint32_t node_pos, ColorSet cs)
  { int i, m;
    for (i = 0; i < sizeof(ColorSet)/4; i++) HGET32(cs[i]);
    for (m = 0; m < 2; m++) { uint32_t *c, *d;
      bool diff_high, diff_focus;
      if (m ≡ 0) { c = cs;
        d = color_defaults[0];
      }
      else { c = NIGHT(cs);
        d = NIGHT(color_defaults[0]);
        if (memcmp(c, d, sizeof(ColorSet)/2) ≡ 0) return;
      }
      hwrite_start();
      diff_high = FG(HIGH(c)) ≠ FG(HIGH(d)) ∨ BG(HIGH(c)) ≠ BG(HIGH(d));
      diff_focus = FG(FOCUS(c)) ≠ FG(FOCUS(d)) ∨ BG(FOCUS(c)) ≠ BG(FOCUS(d));
      hwrite_color_pair(FG(c), BG(c));
      if (diff_high ∨ diff_focus) { hwritec('_');
        hwrite_color_pair(FG(HIGH(c)), BG(HIGH(c)));
      }
    }
  }

```

```

    if (diff_focus) { hwritec(' ');
        hwrite_color_pair(FG(FOCUS(c)), BG(FOCUS(c)));
    }
    hwrite_end();
}
}

```

(cases to get definitions for *color_kind* 299ccxcix) $\equiv$ (299)

case b000:

```

{ int k;
  ColorSet c;
  static bool first_color = true;

  k = HGET8;
  if (k < 6) QUIT("Definition of color set needs 6 color\
    r_pairs only d given\n", n, k);
  hget_color_set(node_pos, c);
  if (n  $\equiv$  0) {
    if ( $\neg$ first_color)
      QUIT("Definition of color set zero must be first");
    memcpy(&color_defaults[0], &c, sizeof(ColorSet));
  }
  first_color = false;
}
break;

```

Used in 398ccxcviii.

6.4 Rotation

When it comes to rotation, there is a big difference between printed books and computer displays. For example, if a book contains a table that is rotated to fill a page in landscape mode, the reader can rotate the book and read the table. If you are looking at the same page displayed on a big computer monitor, you will most likely not turn the whole monitor. Instead your viewing application will be able to perform the rotation for you before displaying the page. A smart phone, on the other hand, is easy to turn. But very likely, it will try to be smart and rerenders the content on the display to keep the same orientation.

Occasionally, however, rotation of text is a desirable feature. For example, if a table has lots of tall columns with lengthy column headers. It might be useful to rotate the column headers in order to keep the column width within reasonable limits.

A simple solution therefore would be optional parameters for boxes specifying center and angle for rotating the box.

6.5 Unknown Extensions

Starting with the inclusion in the T_EX Live 2022 distribution, the HINT file format became accessible to a wider audience which brought the constant rewrite and upgrade cycle to a sudden halt. Except for bug fixes, pretty much nothing happened for a about a year. When the T_EX Live 2023 distribution started to appear on the horizon, one extension that I had on my wish-list already for a long time—the support of T_EX’s *vtop* boxes—was definitely due for implementation. Adding new tag bytes to the specification of the short file format will, however, invalidate all HINT file viewers and requires everybody to upgrade the viewing application. Because the HINT file format is still in its infancy, more such additions are to be expected and the new version 2.0 file format needs a way to handle such yet unknown extensions gracefully. For this purpose the definition section now may specify additional entries for the *hnode_size* array. All HINT file viewers starting with version 2.0 will use these entries to skip unknown nodes and display the remaining content of HINT files.

Reading the long format:

— — — $\Rightarrow$

In the long format, unknown nodes, whether in the definition or the content section, start with the keyword *unknown*.

```
<symbols 2ii > +≡ (300)
%token UNKNOWN "unknown"
```

```
<scanning rules 3iii > +≡ (301)
unknown      return UNKNOWN;
```

In the definition section, the keyword is followed by the tag and the length of the initial part of the node (not counting the start byte), after which follows optionally the number of trailing nodes embedded in the unknown node. There is no need for a maximum value, because the information is stored directly in the *hnode_size* array.

```
<parsing rules 5v > +≡ (302)
def_node: start UNKNOWN UNSIGNED UNSIGNED END {
    hput_tags($1, hput_unknown_def($3, $4, 0)); }
| start UNKNOWN UNSIGNED UNSIGNED UNSIGNED END {
    hput_tags($1, hput_unknown_def($3, $4, $5)); };
```

In the content section, the keyword is followed by the tag value, the remaining byte values belonging to the initial part and the nodes belonging to the trailing part. The end byte, which is equal to the start byte, is omitted from the long format.

```
<symbols 2ii > +≡ (303)
%type < u > unknown_bytes
%type < u > unknown_nodes
```

```

⟨ parsing rules 5v ⟩ +≡ (304)
  content_node: start UNKNOWN UNSIGNED unknown_bytes unknown_nodes END
    { hput_tags($1, hput_unknown($1, $3, $4, $5)); }
  unknown_bytes:
    { $$ = 0;
    }
    | unknown_bytes UNSIGNED { RNG("byte", $2, 0, #FF); HPUT8($2);
      $$ = $1 + 1; };
  unknown_node: content_node
  | xdimen_node
  | list
  | named_param_list;
unknown_nodes:
  { $$ = 0;
  }
  | unknown_nodes unknown_node
  { RNG("unknown_subnodes", $1, 0, 3);
    $$ = $1 + 1;
  }
;

```

Writing the short format:

⇒ ...

In the short format, definitions for unknown nodes are marked with `TAG(unknown_kind, b100)`. This tag is not used elsewhere (see also page 53). We do not check for multiple definitions of the same tag. But only the first of them is considered valid. After the start byte follows the unknown tag and the corresponding entry in the `hnode_size` array.

```

⟨ put functions 14xiv ⟩ +≡ (305)
  uint32_t hput_unknown_def(uint32_t t, uint32_t b, uint32_t n)
  {
    if (n == 0) {
      RNG("unknown_tag", t, TAG(param_kind, 7) + 1, TAG(int_kind, 0) - 1);
      RNG("unknown_initial_bytes", b, 0, #7F - 2);
      HPUT8(t);
      HPUT8(b + 2); /* adding start and end byte */
      if (hnode_size[t] == 0) { hnode_size[t] = NODE_SIZE(b, 0);
        DBG(DBGTAGS,
          "Defining unknown_node_size %d, %d for tag 0x%x\n", b, n, t);
        }
      }
    }
    else { int i;
      RNG("unknown_tag", t, TAG(param_kind, 7) + 1, TAG(int_kind, 0) - 1);
      RNG("unknown_initial_bytes", b, 0, #1F - 1);
      RNG("unknown_trailing_nodes", n, 1, 4);
    }
  }

```



```

    HPUT8(t);
    i = NODE_SIZE(b, n);
    HPUT8(i);
    if (hnode_size[t] == 0) { hnode_size[t] = i;
        DBG(DBGTAGS,
            "Defining unknown node size %d, %d for tag 0x%x\n", b, n, t);
        }
    }
    return TAG(unknown_kind, b100);
}

```

In the content section, the unknown nodes are of course marked with their unknown tag.

(put functions 14xiv) +≡ (306)

```

Tag hput_unknown(uint32_t pos, uint32_t t, uint32_t b, uint32_t n)
{
    int s;
    RNG("unknown_tag", t, TAG(param_kind, 7) + 1, TAG(int_kind, 0) - 1);
    if (n == 0) { RNG("unknown_initial_bytes", b, 0, #7F - 2);
        s = NODE_SIZE(b, 0);
    }
    else { RNG("unknown_initial_bytes", b, 0, #1F - 2);
        RNG("unknown_trailing_nodes", n, 1, 4);
        s = NODE_SIZE(b, n);
    }
    DBG(DBGTAGS, "Adding unknown node size %d, %d tag 0x%x at 0x%x\n",
        b, n, t, pos);
    if (hnode_size[t] != s) QUIT("Size %d of unknown \
        node [%s, %d] at \"SIZE_F\" does not match %d\n", s, NAME(t),
        INFO(t), hpos - hstart, hnode_size[t]);
    return (Tag) t;
}

```

Reading the short format:

... ⇒

Writing the long format:

⇒ - - -

(get functions 18xviii) +≡ (307)

```

void hget_unknown_def(void)
{
    Tag t;
    signed char i, b = 0, n = 0;
    t = HGET8;
    i = HGET8;
    if (i == 0) QUIT("Zero not allowed for unknown node size at 0x%x\n",
        (uint32_t)(hpos - hstart - 2));
    hwrite_start(); hwritef("unknown");
    b = NODE_HEAD(i);
    n = NODE_TAIL(i);
}

```

```

    if (n == 0) hwritef("_0x%02X_", t, b);
    else hwritef("_0x%02X_", t, b, n);
    if (hnode_size[t] == 0) { hnode_size[t] = i;
        DBG(DBGTAGS, "Defining_node_size_%d,%d_for_tag_0x%x\n", b, n, t);
    }
    hwrite_end();
}

```

The *hget_unknown* function tries to process a unknown node with the help of an entry in the *hnode_size* array. The definition section can be used to provide this extra information. If successful the function returns 1 else 0.

```

⟨get functions 18xviii⟩ += (308)
int hget_unknown(Tag a)
{ int b, n;
  int8_t s;

  s = hnode_size[a];
  DBG(DBGTAGS, "Trying_unknown_tag_0x%x_at_0x%x\n", a,
      (uint32_t)(hpos - hstart - 1));
  if (s == 0) return 0;
  b = NODE_HEAD(s);
  n = NODE_TAIL(s);
  DBG(DBGTAGS, "Trying_unknown_node_size_%d_%d\n", b, n);
  hwritef("unknown_0x%02X", a);
  while (b > 0) { a = HGET8;
      hwritef("_0x%02X", a);
      b--;
  }
  while (n > 0) { a = *hpos;
      if (KIND(a) == xdimen_kind) { Xdimen x;
          hget_xdimen_node(&x); hwrite_xdimen_node(&x);
      }
      else if (KIND(a) == param_kind) { List l; hget_param_list(&l);
          hwrite_named_param_list(&l); }
      else if (KIND(a) ≤ list_kind) { List l; hget_list(&l); hwrite_list(&l); }
      else hget_content_node();
      n--;
  }
  return 1;
}

```

7 Replacing T_EX's Page Building Process

T_EX uses an output routine to finalize the page. It uses the accumulated material from the page builder, found in `box255`, attaches headers, footers, and floating material like figures, tables, and footnotes. The latter material is specified by insert nodes while headers and footers are often constructed using mark nodes. Running an output routine requires the full power of the T_EX engine and will not be part of the HINT viewer. Therefore, HINT replaces output routines by page templates. As T_EX can use different output routines for different parts of a book—for example the index might use a different output routine than the main body of text—HINT will allow multiple page templates. To support different output media, the page templates will be named and a suitable user interface may offer the user a selection of possible page layouts. In this way, the page layout remains in the hands of the book designer, and the user has still the opportunity to pick a layout that best fits the display device.

T_EX uses insertions to describe floating content that is not necessarily displayed where it is specified. Three examples may illustrate this:

- Footnotes* are specified in the middle of the text but are displayed at the bottom of the page. Several footnotes on the same page are collected and displayed together. The page layout may specify a short rule to separate footnotes from the main text, and if there are many short footnotes, it may use two columns to display them. In extreme cases, the page layout may demand a long footnote to be split and continued on the next page.
- Illustrations may be displayed exactly where specified if there is enough room on the page, but may move to the top of the page, the bottom of the page, the top of next page, or a separate page at the end of the chapter.
- Margin notes are displayed in the margin on the same page starting at the top of the margin.

HINT uses page templates and content streams to achieve similar effects. But before I describe the page building mechanisms of HINT, let me summarize T_EX's page builder.

T_EX's page builder ignores leading glue, kern, and penalty nodes until the first box or rule is encountered; `whatsit` nodes do not really contribute anything to a page; mark nodes are recorded for later use. Once the first box, rule, or insert arrives, T_EX makes copies of all parameters that influence the page building process

* Like this one.

and uses these copies. These parameters are the *page_goal* and the *page_max_depth*. Further, the variables *page_total*, *page_shrink*, *page_stretch*, *page_depth*, and *insert_penalties* are initialized to zero. The top skip adjustment is made when the first box or rule arrives—possibly after an insert.

Now the page builder accumulates material: normal material goes into `box255` and will change *page_total*, *page_shrink*, *page_stretch*, and *page_depth*. The latter is adjusted so that it does not exceed *page_max_depth*.

The handling of inserts is more complex. T_EX creates an insert class using `newinsert`. This reserves a number *n* and four registers: `boxn` for the inserted material, `countn` for the magnification factor *f*, `dimenn` for the maximum size per page *d*, and `skipn` for the extra space needed on a page if there are any insertions of class *n*.

For example plain T_EX allocates *n* = 254 for footnotes and sets `count254` to 1000, `dimen254` to 8in, and `skip254` to `\bigskipamount`.

An insertion node will specify the insertion class *n*, some vertical material, its natural height plus depth *x*, a *split_top_skip*, a *split_max_depth*, and a *floating_penalty*.

Now assume that an insert node with subtype 254 arrives at the page builder. If this is the first such insert, T_EX will decrease the *page_goal* by the width of `skip254` and adds its stretchability and shrinkability to the total stretchability and shrinkability of the page. Later, the output routine will add some space and the footnote rule to fill just that much space and add just that much shrinkability and stretchability to the page. Then T_EX will normally add the vertical material in the insert node to `box254` and decrease the *page_goal* by $x \times f/1000$.

Special processing is required if T_EX detects that there is not enough space on the current page to accommodate the complete insertion. If already a previous insert did not fit on the page, simply the *floating_penalty* as given in the insert node is added to the total *insert_penalties*. Otherwise T_EX will test that the total natural height plus depth of `box254` including *x* does not exceed the maximum size *d* and that the $page_total + page_depth + x \times f/1000 - page_shrink \leq page_goal$. If one of these tests fails, the current insertion is split in such a way as to make the size of the remaining insertions just pass the tests just stated.

Whenever a glue node, or penalty node, or a kern node that is followed by glue arrives at the page builder, it rates the current position as a possible end of the page based on the shrinkability of the page and the difference between *page_total* and *page_goal*. As the page fills, the page breaks tend to become better and better until the page starts to get overfull and the page breaks get worse and worse until they reach the point where they become *awful_bad*. At that point, the page builder returns to the best page break found so far and fires up the output routine.

Let's look next at the problems that show up when implementing a replacement mechanism for HINT.

1. An insertion node can not always specify its height *x* because insertions may contain paragraphs that need to be broken in lines and the height of a paragraph depends in some non obvious way on its width.
2. Before the viewer can compute the height *x*, it needs to know the width of the

insertion. Just imagine displaying footnotes in two columns or setting notes in the margin. Knowing the width, it can pack the vertical material and derive its height and depth.

3. T_EX's plain format provides an insert macro that checks whether there is still space on the current page, and if so, it creates a contribution to the main text body, otherwise it creates a topinsert. Such a decision needs to be postponed to the HINT viewer.
4. HINT has no output routines that would specify something like the space and the rule preceding the footnote.
5. T_EX's output routines have the ability to inspect the content of the boxes, split them, and distribute the content over the page. For example, the output routine for an index set in two column format might expect a box containing index entries up to a height of $2 \times \text{vsize}$. It will split this box in the middle and display the top part in the left column and the bottom part in the right column. With this approach, the last page will show two partly filled columns of about equal size.
6. HINT has no mark nodes that could be used to create page headers or footers. Marks, like output routines, contain token lists and need the full T_EX interpreter for processing them. Hence, HINT does not support mark nodes.

Here now is the solution I have chosen for HINT:

Instead of output routines, HINT will use page templates. Page templates are basically vertical boxes with placeholders marking the positions where the content of the box registers, filled by the page builder, should appear. To output the page, the viewer traverses the page template, replaces the placeholders by the appropriate box content, and sets the glue. Inside the page template, we can use insert nodes to act as placeholders.

It is only natural to treat the page's main body, the inserts, and the marks using the same mechanism. We call this mechanism a content stream. Content streams are identified by a stream number in the range 0 to 254; the number 255 is used to indicate an invalid stream number. The stream number 0 is reserved for the main content stream; it is always defined. Besides the main content stream, there are three types of streams:

- normal streams correspond to T_EX's inserts and accumulate content on the page,
- first streams correspond to T_EX's first marks and will contain only the first insertion of the page,
- last streams correspond to T_EX's bottom marks and will contain only the last insertion of the page, and
- top streams correspond to T_EX's top marks. Top streams are not yet implemented.

Nodes from the content section are considered contributions to stream 0 except for insert nodes which will specify the stream number explicitly. If the stream is not defined or is not used in the current page template, its content is simply ignored.

The page builder needs a mechanism to redirect contributions from one content stream to another content stream based on the availability of space. Hence a HINT

content stream can optionally specify a preferred stream number, where content should go if there is still space available, a next stream number, where content should go if the present stream has no more space available, and a split ratio if the content is to be split between these two streams before filling in the template.

Various stream parameters govern the treatment of contributions to the stream and the page building process.

- The magnification factor f : Inserting a box of height h to this stream will contribute $h \times f/1000$ to the height of the page under construction. For example, a stream that uses a two column format will have an f value of 500; a stream that specifies notes that will be displayed in the page margin will have an f value of zero.
- The height h : The extended dimension h gives the maximum height this stream is allowed to occupy on the current page. To continue the previous example, a stream that will be split into two columns will have $h = 2 \cdot \text{vsize}$, and a stream that specifies notes that will be displayed in the page margin will have $h = 1 \cdot \text{vsize}$. You can restrict the amount of space occupied by footnotes to the bottom quarter by setting the corresponding h value to $h = 0.25 \cdot \text{vsize}$.
- The depth d : The dimension d gives the maximum depth this stream is allowed to have after formatting.
- The width w : The extended dimension w gives the width of this stream when formatting its content. For example margin notes should have the width of the margin less some surrounding space.
- The “before” list b : If there are any contributions to this stream on the current page, the material in list b is inserted *before* the material from the stream itself. For example, the short line that separates the footnotes from the main page will go, together with some surrounding space, into the list b .
- The top skip glue g : This glue is inserted between the material from list b and the first box of the stream, reduced by the height of the first box. Hence it specifies the distance between the material in b and the first baseline of the stream content.
- The “after” list a : The list a is treated like list b but its material is placed *after* the material from the stream itself.
- The “preferred” stream number p : If $p \neq 255$, it is the number of the *preferred* stream. If stream p has still enough room to accommodate the current contribution, move the contribution to stream p , otherwise keep it. For example, you can move an illustration to the main content stream, provided there is still enough space for it on the current page, by setting $p = 0$.
- The “next” stream number n : If $n \neq 255$, it is the number of the *next* stream. If a contribution can not be accommodated in stream p nor in the current stream, treat it as an insertion to stream n . For example, you can move contributions to the next column after the first column is full, or move illustrations to a separate page at the end of the chapter.
- The split ratio r : If r is positive, both p and n must be valid stream numbers and contents is not immediately moved to stream p or n as described before.

Instead the content is kept in the stream itself until the current page is complete. Then, before inserting the streams into the page template, the content of this stream is formatted as a vertical box, the vertical box is split into a top fraction and a bottom fraction in the ratio $r/1000$ for the top and $(1000 - r)/1000$ for the bottom, and finally the top fraction is moved to stream p and the bottom fraction to stream n . You can use this feature for example to implement footnotes arranged in two columns of about equal size. By collecting all the footnotes in one stream and then splitting the footnotes with $r = 500$ before placing them on the page into a right and left column. Even three or more columns can be implemented by cascades of streams using this mechanism.

7.1 Stream Definitions

There are four types of streams: normal streams that work like T_EX's inserts; and first, last, and top streams that work like T_EX's marks. For the latter types, the long format uses a matching keyword and the short format the two least significant info bits. All stream definitions start with the stream number. In definitions of normal streams after the number follows in this order

- the maximum insertion height,
- the magnification factor, and
- information about splitting the stream. It consists of: a preferred stream, a next stream, and a split ratio. An asterisk indicates a missing stream reference, in the short format the stream number 255 serves the same purpose.

All stream definitions finish with

- the “before” list,
- an extended dimension node specifying the width of the inserted material,
- the top skip glue,
- the “after” list,
- and the total height, stretchability, and shrinkability of the material in the “before” and “after” list.

A special case is the stream definition for stream 0, the main content stream. None of the above information is necessary for it so it is omitted. Stream definitions, including the definition of stream 0, occur only inside page template definitions where they occur twice in two different roles: In the stream definition list, they define properties of the stream and in the template they mark the insertion point (see section 7.3). In the latter case, stream nodes just contain the stream number. Because a template looks like ordinary vertical material, we like to use the same functions for parsing it. But stream definitions are very different from stream content nodes. To solve the problem for the long format, the scanner will return two different tokens when it sees the keyword “**stream**”. In the definition section, it will return STREAMDEF and in the content section STREAM. The same problem is solved in the short format by using the *b100* bit to mark a definition.

Reading the long format:

Writing the short format:

-- -- $\Rightarrow$
 $\Rightarrow \dots$

$\langle$ symbols 2ii $\rangle + \equiv$ (309)

%token STREAM "stream"

%token STREAMDEF "stream_ (definition)"

%token FIRST "first"

%token LAST "last"

%token TOP "top"

%token NOREFERENCE "*"

%type < info > stream_type

%type < u > stream_ref

%type < rf > stream_def_node

$\langle$ scanning rules 3iii $\rangle + \equiv$ (310)

stream **if** (section_no $\equiv$ 1) **return** STREAMDEF;
 else return STREAM;

first **return** FIRST;

last **return** LAST;

top **return** TOP;

* **return** NOREFERENCE;

$\langle$ parsing rules 5v $\rangle + \equiv$ (311)

stream_link: ref { REF_RNG(stream_kind,\$1); }
 | NOREFERENCE { HPUT8(255); };

stream_split: stream_link stream_link UNSIGNED
 { RNG("split_ratio", \$3, 0, 1000); HPUT16(\$3); };

stream_info: xdimen_node UNSIGNED
 { RNG("magnification_factor", \$2, 0, 1000); HPUT16(\$2); } stream_split;

stream_type: stream_info { \$\$ = 0; }
 | FIRST { \$\$ = 1; } | LAST { \$\$ = 2; } | TOP { \$\$ = 3; };

stream_def_node: start STREAMDEF ref stream_type
 list xdimen_node glue_node list glue_node END
 { DEF(\$\$, stream_kind, \$3); hput_tags(\$1, TAG(stream_kind, \$4 | b100)); };

stream_ins_node: start STREAMDEF ref END
 { RNG("Stream_insertion", \$3, 0, max_ref[stream_kind]);
 hput_tags(\$1, TAG(stream_kind, b100)); };

content_node: stream_def_node | stream_ins_node;

Reading the short format:

... $\Rightarrow$

Writing the long format:

$\Rightarrow$ - - -

$\langle$ get stream information for normal streams 312cccxi $\rangle \equiv$ (312)

```
{ Xdimen x;
  uint16_t f, r;
  uint8_t n;
  DBG(DBGDEF, "Defining_stream_%d_at_%d", *hpos - 1,
    hpos - hstart - 2);
  hget_xdimen_node(&x); hwrite_xdimen_node(&x);
  HGET16(f); RNG("magnification_factor", f, 0, 1000); hwritef("%d", f);
  n = HGET8;
  if (n  $\equiv$  255) hwritef("_*");
  else { REF_RNG(stream_kind, n); hwrite_ref(n); }
  n = HGET8;
  if (n  $\equiv$  255) hwritef("_*");
  else { REF_RNG(stream_kind, n); hwrite_ref(n); }
  HGET16(r);
  RNG("split_ratio", r, 0, 1000);
  hwritef("%d", r);
}
```

Used in 313cccxi.

$\langle$ get functions 18xviii $\rangle + \equiv$ (313)

```
static bool hget_stream_def(void)
{ if (KIND(*hpos)  $\neq$  stream_kind  $\vee$   $\neg$ (INFO(*hpos) & b100)) return false;
  else { Ref df;
     $\langle$  read the start byte a 16xvi  $\rangle$ 
    DBG(DBGDEF, "Defining_stream_%d_at_%d", *hpos,
      hpos - hstart - 1);
    DEF(df, stream_kind, HGET8);
    hwrite_start(); hwritef("stream"); hwrite_ref(df.n);
    if (df.n > 0) { Xdimen x; List l;
      if (INFO(a)  $\equiv$  b100)  $\langle$  get stream information for normal streams
        312cccxi  $\rangle$ 
      else if (INFO(a)  $\equiv$  b101) hwritef("_first");
      else if (INFO(a)  $\equiv$  b110) hwritef("_last");
      else if (INFO(a)  $\equiv$  b111) hwritef("_top");
      hget_list(&l); hwrite_list(&l);
      hget_xdimen_node(&x); hwrite_xdimen_node(&x);
      hget_glue_node(); hget_list(&l); hwrite_list(&l); hget_glue_node();
    }
     $\langle$  read and check the end byte z 17xvii  $\rangle$ 
    hwrite_end();
    return true;
  }
```

```

    }
}

```

When stream definitions are part of the page template, we call them stream insertion points. They contain only the stream reference and are parsed by the usual content parsing functions.

```

⟨ cases to get content 20xx ⟩ +≡ (314)
  case TAG(stream_kind, b100):
    { uint8_t n = HGET8; REF_RNG(stream_kind, n); hwrite_ref(n); break; }

```

7.2 Stream Content

Stream nodes occur in the content section where they must not be inside other nodes except toplevel paragraph nodes. A normal stream node contains in this order: the stream reference number, the optional stream parameters, and the stream content. The content is either a vertical box or an extended vertical box. The stream parameters consists of the *floating_penalty*, the *split_max_depth*, and the *split_top_skip*. The parameterlist can be given explicitly or as a reference.

In the short format, the info bits *b010* indicate a normal stream content node with an explicit parameter list and the info bits *b000* a normal stream with a parameter list reference.

If the info bit *b001* is set, we have a content node of type top, first, or last. In this case, the short format has instead of the parameter list a single byte indicating the type. These types are currently not yet implemented.

Reading the long format: - - - ⇒
 Writing the short format: ⇒ ...

```

⟨ symbols 2ii ⟩ +≡ (315)
%type < info > stream

```

```

⟨ parsing rules 5v ⟩ +≡ (316)
stream: param_list list { $$ = b010; }
      | param_ref list { $$ = b000; };
content_node: start STREAM stream_ref stream END
              { hput_tags($1, TAG(stream_kind, $4)); };

```

Reading the short format: ... ⇒
 Writing the long format: ⇒ - - -

```

⟨ cases to get content 20xx ⟩ +≡ (317)
  case TAG(stream_kind, b000): HGET_STREAM(b000); break;
  case TAG(stream_kind, b010): HGET_STREAM(b010); break;

```

When we read stream numbers, we relax the define before use policy. We just check, that the stream number is in the correct range.

```

⟨get macros 19xix ⟩ +≡
#define HGET_STREAM(I)
{ uint8_t n = HGET8; REF_RNG(stream_kind, n); hwrite_ref(n); }
if ((I) & b010) { List l; hget_param_list(&l); hwrite_param_list(&l); }
else HGET_REF(param_kind);
{ List l; hget_list(&l); hwrite_list(&l); }

```

(318)

7.3 Page Template Definitions

A **HINT** file can define multiple page templates. Not only might an index demand a different page layout than the main body of text, also the front page or the chapter headings might use their own page templates. Further, the author of a **HINT** file might define a two column format as an alternative to a single column format to be used if the display area is wide enough.

To help in selecting the right page template, page template definitions start with a name and an optional priority; the default priority is 1. The names might appear in a menu from which the user can select a page layout that best fits her taste. Without user interaction, the system can pick the template with the highest priority. Of course, a user interface might provide means to alter priorities. Future versions might include sophisticated feature-vectors that identify templates that are good for large or small displays, landscape or portrait mode, etc ...

After the priority follows a glue node to specify the topskip glue and the dimension of the maximum page depth, an extended dimension to specify the page height and an extended dimension to specify the page width.

Then follows the main part of a page template definition: the template. The template consists of a list of vertical material. To construct the page, this list will be placed into a vertical box and the glue will be set. But of course before doing so, the viewer will scan the list and replace all stream insertion points by the appropriate content streams.

Let's call the vertical box obtained this way "the page". The page will fill the entire display area top to bottom and left to right. It defines not only the appearance of the main body of text but also the margins, the header, and the footer. Because the **vsize** and **hsize** variables of **T_EX** are used for the vertical and horizontal dimension of the main body of text—they do not include the margins—the page will usually be wider than **hsize** and taller than **vsize**. The dimensions of the page are part of the page template. The viewer, knowing the actual dimensions of the display area, can derive from them the actual values of **hsize** and **vsize**.

Stream definitions are listed after the template.

The page template with number 0 is always defined and has priority 0. It will display just the main content stream. It puts a small margin of **hsize**/8 – 4.5pt all around it. Given a letter size page, 8.5 inch wide, this formula yields a margin of 1 inch, matching **T_EX**'s plain format. The margin will be positive as long as the page is wider than 1/2 inch. For narrower pages, there will be no margin at all. In general, the **HINT** viewer will never set **hsize** larger than the width of the page and **vsize** larger than its height.

Reading the long format: - - - $\Rightarrow$
 Writing the short format: $\Rightarrow$...

$\langle$ symbols 2ii $\rangle + \equiv$ (319)
`%token PAGE "page"`

$\langle$ scanning rules 3iii $\rangle + \equiv$ (320)
`page                    return PAGE;`

$\langle$ parsing rules 5v $\rangle + \equiv$ (321)
`page_priority: { HPUT8(1); }
 | UNSIGNED { RNG("page_priority", $1, 0, 255); HPUT8($1); };
stream_def_list:
 | stream_def_list stream_def_node;
page: string { hput_string($1); } page_priority glue_node dimension {
 HPUT32($5); } xdimen_node xdimen_node list stream_def_list;`

Reading the short format: ... $\Rightarrow$
 Writing the long format: $\Rightarrow$ - - -

$\langle$ get functions 18xviii $\rangle + \equiv$ (322)
`void hget_page(void)
{ char *n;
 uint8_t p;
 Xdimen x;
 List l;
 HGET_STRING(n); hwrite_string(n);
 p = HGET8; if (p $\neq$ 1) hwritef("_%d", p);
 hget_glue_node();
 hget_dimen(TAG(dimen_kind, b001));
 hget_xdimen_node(&x); hwrite_xdimen_node(&x); /* page height */
 hget_xdimen_node(&x); hwrite_xdimen_node(&x); /* page width */
 hget_list(&l); hwrite_list(&l);
 while (hget_stream_def()) continue;
}`

7.4 Page Ranges

Not every template is necessarily valid for the entire content section. A page range specifies a start position a and an end position b in the content section and the page template is valid if the start position p of the page is within that range: $a \leq p < b$. If paging backward this definition might cause problems because the start position of the page is known only after the page has been build. In this case, the viewer might choose a page template based on the position at the bottom of the page. If it turns out that this “bottom template” is no longer valid when the page builder has found the start of the page, the viewer might display the page anyway with the

bottom template, it might just display the page with the new “top template”, or rerun the whole page building process using this time the “top template”. Neither of these alternatives is guaranteed to produce a perfect result because changing the page template might change the amount of material that fits on the page. A good page template design should take this into account.

The representation of page ranges differs significantly for the short format and the long format. The short format will include a list of page ranges in the definition section which consist of a page template number, a start position, and an end position. In the long format, the start and end position of a page range is marked with a page range node switching the availability of a page template on and off. Such a page range node must be a top level node. It is an error, to switch a page template off that was not switched on, or to switch a page template on that was already switched on. It is permissible to omit switching off a page template at the very end of the content section.

While we parse a long format **HINT** file, we store page ranges and generate the short format after reaching the end of the content section. While we parse a short format **HINT** file, we check at the end of each top level node whether we should insert a page range node into the output. For the **shrink** program, it is best to store the start and end positions of all page ranges in an array sorted by the position*. To check the restrictions on the switching of page templates, we maintain for every page template an index into the range array which identifies the position where the template was switched on. A zero value instead of an index will identify templates that are currently invalid. When switching a range off again, we link the two array entries using this index. These links are useful when producing the range nodes in short format.

A range node in short format contains the template number, the start position and the end position. A zero start position is not stored, the info bit *b100* indicates a nonzero start position. An end position equal to **HINT_NO_POS** is not stored, the info bit *b010* indicates a smaller end position. The info bit *b001* indicates that positions are stored using 2 byte otherwise 4 byte are used for the positions.

```
< hint types 1i > +≡ (323)
typedef struct { uint8_t pg; uint32_t pos; bool on; int link; } RangePos;
```

```
< common variables 252cclii > +≡ (324)
RangePos *range_pos;
int next_range = 1, max_range;
int *page_on;
```

```
< initialize definitions 253ccliii > +≡ (325)
ALLOCATE(page_on, max_ref[page_kind] + 1, int);
ALLOCATE(range_pos, 2 * (max_ref[range_kind] + 1), RangePos);
```

* For a **HINT** viewer, a data structure which allows fast retrieval of all valid page templates for a given position is needed.

⟨ hint macros 13xiii ⟩ +≡ (326)

```
#define ALLOCATE(R, S, T)
  ( (R) = ( T * ) calloc((S), sizeof (T)),
    (((R) ≡ NULL) ? QUIT("Out_of_memory_for_" #R) : (void) 0) )
#define REALLOCATE(R, S, T)
  ( (R) = ( T * ) realloc((R), (S) * sizeof (T)),
    (((R) ≡ NULL) ? QUIT("Out_of_memory_for_" #R) : (void) 0) )
```

Reading the long format: — — — ⇒

⟨ symbols 2ii ⟩ +≡ (327)

```
%token RANGE "range"
```

⟨ scanning rules 3iii ⟩ +≡ (328)

```
range          return RANGE;
```

⟨ parsing rules 5v ⟩ +≡ (329)

```
content_node: START RANGE REFERENCE ON END
  { REF(page_kind, $3); hput_range($3, true); }
| START RANGE REFERENCE OFF END
  { REF(page_kind, $3); hput_range($3, false); };
```

Writing the long format: ⇒ — — —

⟨ write functions 21xxi ⟩ +≡ (330)

```
void hwrite_range(void) /* called in hwrite_end */
{ uint32_t p = hpos - hstart;
  DBG(DBG_RANGE, "Range_check_at_pos_0x%x_next_at_0x%x\n", p,
    range_pos[next_range].pos);
  while (next_range < max_range ∧ range_pos[next_range].pos ≤ p) {
    hwrite_start();
    hwritef("range_%d", range_pos[next_range].pg);
    if (range_pos[next_range].on) hwritef("on");
    else hwritef("off");
    nesting--; hwritec('>'); /* avoid a recursive call to hwrite_end */
    next_range++;
  }
}
```

Reading the short format:

... $\Rightarrow$

```

⟨get functions 18xviii⟩ +≡ (331)
void hget_range(Info info, uint8_t pg)
{ uint32_t from, to;
  REF(page_kind, pg);
  REF(range_kind, (next_range - 1)/2);
  if (info & b100) { if (info & b001) HGET32(from); else HGET16(from); }
  else from = 0;
  if (info & b010) { if (info & b001) HGET32(to); else HGET16(to); }
  else to = HINT_NO_POS;
  range_pos[next_range].pg = pg;
  range_pos[next_range].on = true;
  range_pos[next_range].pos = from;
  DBG(DBG_RANGE, "Range_%d_from_0x%x\n", pg, from);
  DBG(DBG_RANGE, "Range_%d_to_0x%x\n", pg, to);
  next_range++;
  if (to ≠ HINT_NO_POS)
  { range_pos[next_range].pg = pg;
    range_pos[next_range].on = false;
    range_pos[next_range].pos = to;
    next_range++;
  }
}

⟨write functions 21xxi⟩ +≡ (332)
void hsort_ranges(void) /* simple insert sort by position */
{ int i;
  DBG(DBG_RANGE, "Range_sorting_%d_positions\n", next_range - 1);
  for (i = 3; i < next_range; i++)
  { int j = i - 1;
    if (range_pos[i].pos < range_pos[j].pos)
    { RangePos t;
      t = range_pos[i];
      do { range_pos[j + 1] = range_pos[j];
          j--;
        } while (range_pos[i].pos < range_pos[j].pos);
      range_pos[j + 1] = t;
    }
  }
  max_range = next_range; next_range = 1; /* prepare for hwrite_range */
}

```

Writing the short format:

⇒ ...

```

⟨ put functions 14xiv ⟩ +≡ (333)
void hput_range(uint8_t pg, bool on)
{
    if (((next_range - 1)/2) > max_ref[range_kind])
        QUIT("Page_range_d_>d", (next_range - 1)/2, max_ref[range_kind]);
    if (on & page_on[pg] ≠ 0)
        QUIT("Template_d_is_switched_on_at_0x%x_and_SIZE_F",
            pg, range_pos[page_on[pg]].pos, hpos - hstart);
    else if (¬on & page_on[pg] ≡ 0)
        QUIT("Template_d_is_switched_off_at_SIZE_F" _but_was_not_on",
            pg, hpos - hstart);
    DBG(DBG_RANGE, "Range_*d_s_at_SIZE_F"\n", pg, on ? "on" : "off",
        hpos - hstart);
    range_pos[next_range].pg = pg;
    range_pos[next_range].pos = hpos - hstart;
    range_pos[next_range].on = on;
    if (on) page_on[pg] = next_range;
    else
    {
        range_pos[next_range].link = page_on[pg];
        range_pos[page_on[pg]].link = next_range;
        page_on[pg] = 0;
    }
    next_range++;
}

void hput_range_defs(void)
{
    int i;
    section_no = 1;
    hstart = dir[1].buffer;
    hend = hstart + dir[1].bsize;
    hpos = hstart + dir[1].size;
    for (i = 1; i < next_range; i++)
        if (range_pos[i].on)
        {
            Info info = b000;
            uint32_t p = hpos++ - hstart;
            uint32_t from, to;
            HPUT8(range_pos[i].pg);
            from = range_pos[i].pos;
            if (range_pos[i].link ≠ 0) to = range_pos[range_pos[i].link].pos;
            else to = HINT_NO_POS;
            if (from ≠ 0)
            {
                info = info | b100; if (from > #FFFF) info = info | b001; }
            if (to ≠ HINT_NO_POS)
            {
                info = info | b010; if (to > #FFFF) info = info | b001; }
        }
}

```



```
    if (info & b100)
    { if (info & b001) HPUT32(from); else HPUT16(from); }
    if (info & b010)
    { if (info & b001) HPUT32(to); else HPUT16(to); }
    DBG(DBGGRANGE, "Range_%d_from_0x%x_to_0x%x\n",
        range_pos[i].pg, from, to);
    hput_tags(p, TAG(range_kind, info));
}
hput_definitions_end();
}
```


8 File Structure

All HINT files start with a banner as described below. After that, they contain three mandatory sections: the directory section, the definition section, and the content section. Usually, further optional sections follow. In short format files, these contain auxiliary files (fonts, images, ...) necessary for rendering the content. In long format files, the directory section will simply list the file names of the auxiliary files.

8.1 Banner

All HINT files start with a banner. The banner contains only printable ASCII characters and spaces; its end is marked with a newline character. The first four byte are the “magic” number by which you recognize a HINT file. It consists of the four ASCII codes ‘H’, ‘I’, ‘N’, and ‘T’ in the long format and ‘h’, ‘i’, ‘n’, and ‘t’ in the short format. Then follows a space, then the version number, a dot, the sub-version number, and another space. Both numbers are encoded as decimal ASCII strings. The remainder of the banner is simply ignored but may be used to contain other useful information about the file. The maximum size of the banner is 256 byte.

```
< hint macros 13xiii > += (334)
#define MAX_BANNER 256
```

To check the banner, we have the function *hcheck_banner*; it returns *true* if successful.

```
< common variables 252ccli > += (335)
char hbanner[MAX_BANNER + 1];
int hbanner_size = 0;
```

```
< function to check the banner 336cccxvvi > ≡ (336)
bool hcheck_banner(char *magic)
{ int v, s;
  char *t;
  t = hbanner;
  if (strncmp(magic, hbanner, 4) ≠ 0) {
    MESSAGE("This is not a %s file\n", magic);
    return false;
  }
  else t += 4;
```

```

    if (hbanner[hbanner_size - 1] != '\n') {
        MESSAGE("Banner exceeds maximum size=0x%x\n", MAX_BANNER);
        return false;
    }
    if (*t != '_') {
        MESSAGE("Space expected in banner after %s\n", magic);
        return false;
    }
    else t++;
    v = strtol(t, &t, 10);
    if (*t != '.') {
        MESSAGE("Dot expected in banner after HINT version number\n");
        return false;
    }
    else t++;
    s = strtol(t, &t, 10);
    if (v != HINT_VERSION) {
        MESSAGE("Wrong HINT version: got %d.%d, expected %d.%d\n", v, s,
            HINT_VERSION, HINT_MINOR_VERSION);
        return false;
    }
    #if 0 /* minor versions should be downward compatible */
    if (s < HINT_MINOR_VERSION)
        { MESSAGE("Outdated HINT minor version: got %d.%d, \
            expected %d.%d\n", v, s, HINT_VERSION, HINT_MINOR_VERSION);
        }
    else
    #endif
        if (s > HINT_MINOR_VERSION)
            { MESSAGE("More recent HINT minor version: got %d.\
                %d, expected %d.%d, update your application\n", v, s,
                    HINT_VERSION, HINT_MINOR_VERSION);
            }
    if (*t != '_' ^ *t != '\n') {
        MESSAGE("Space expected in banner after HINT minor version\n");
        return false;
    }
    LOG("%s file version "HINT_VERSION_STRING":%s", magic, t);
    DBG(DBGDIR, "banner size=0x%x\n", hbanner_size);
    return true;
}

```

Used in [552dl ii](#), [557dl vii](#), [558dl viii](#), and [560dl x](#).

To read a short format file, we use the macro HGET8. It returns a single byte. We read the banner knowing that it ends with a newline character and is at most MAX_BANNER byte long. Because this is the first access to a yet unknown file, we

are very careful and make sure we do not read past the end of the file. Checking the banner is a separate step.

Reading the short format:

... $\Rightarrow$

```

⟨get file functions 337cccxxxvii⟩ ≡ (337)
void hget_banner(void)
{
    hbanner_size = 0;
    while (hbanner_size < MAX_BANNER ∧ hpos < hend) {
        uint8_t c = HGET8;
        hbanner[hbanner_size++] = c;
        if (c ≡ '\n') break;
    }
    hbanner[hbanner_size] = 0;
}

```

Used in 552dlii, 558dlviii, and 560dlx.

To read a long format file, we use the function *fgetc*.

Reading the long format:

-- -- $\Rightarrow$

```

⟨read the banner 338cccxxxviii⟩ ≡ (338)
{
    hbanner_size = 0;
    while (hbanner_size < MAX_BANNER) {
        int c = fgetc(hin);
        if (c ≡ EOF) break;
        hbanner[hbanner_size++] = c;
        if (c ≡ '\n') break;
    }
    hbanner[hbanner_size] = 0;
}

```

Used in 557dlvii.

Writing the banner to a short format file is accomplished by calling *hput_banner* with the “magic” string “hint” as a first argument and a (short) comment as the second argument.

Writing the short format:

$\Rightarrow$...

```

⟨function to write the banner 339cccxxxix⟩ ≡ (339)
static size_t hput_banner(char *magic, char *str)
{
    size_t s = fprintf(hout, "%s" HINT_VERSION_STRING "%s\n", magic, str);
    if (s > MAX_BANNER) QUIT("Banner too big");
    return s;
}

```

Used in 554dliv, 557dlvii, and 558dlviii.

Writing the long format:

$\Rightarrow$ -- --

Writing the banner of a long format file is essentially the same as for a short format file calling *hput_banner* with “HINT” as a first argument.

8.2 Long Format Files

After reading and checking the banner, reading a long format file is simply done by calling *yyparse*. The following rule gives the big picture:

Reading the long format: — — — $\Rightarrow$

$\langle$ parsing rules 5v $\rangle + \equiv$ (340)

hint: directory_section definition_section content_section;

8.3 Short Format Files

A short format file starts with the banner and continues with a list of sections. Each section has a maximum size of 2^{32} byte or 4GByte. This restriction ensures that positions inside a section can be stored as 32 bit integers, a feature that we will need only for the so called “content” section, but it is also nice for implementers to know in advance what sizes to expect. The big picture is captured by the *put_hint* function:

$\langle$ put functions 14xiv $\rangle + \equiv$ (341)

```
static size_t hput_root(void);
static size_t hput_section(uint16_t n);
static size_t hput_optional_section(int i);
size_t hput_hint(char *str)
{ size_t s;
  int i;

  DBG(DBGBASIC, "Writing_hint_output_%s\n", str);
  s = hput_banner("hint", str);
  DBG(DBGDIR, "Root_entry_at_SIZE_F"\n", s);
  s += hput_root();
  DBG(DBGDIR, "Directory_section_at_SIZE_F"\n", s);
  s += hput_section(0);
  DBG(DBGDIR, "Definition_section_at_SIZE_F"\n", s);
  s += hput_section(1);
  DBG(DBGDIR, "Content_section_at_SIZE_F"\n", s);
  s += hput_section(2);
  DBG(DBGDIR, "Auxiliary_sections_at_SIZE_F"\n", s);
  for (i = 3; i ≤ max_section_no; i++)
    s += hput_optional_section(i);
  DBG(DBGDIR, "Total_number_of_bytes_written_SIZE_F"\n", s);
  return s;
}
```

When we work on a section, we will have the entire section in memory and use three variables to access it: *hstart* points to the first byte of the section, *hend* points to the byte after the last byte of the section, and *hpos* points to the current position inside the section. The auxiliary variable *hpos0* contains the *hpos* value of the last content node on nesting level zero.

⟨ common variables 252ccli ⟩ +≡ (342)
uint8_t *hpos = NULL, *hstart = NULL, *hend = NULL, *hpos0 = NULL;

There are two sets of macros that read or write binary data at the current position and advance the stream position accordingly.

Reading the short format: ... ⇒

⟨ shared get macros 38xxxviii ⟩ +≡ (343)
#define HGET_ERROR
 QUIT ("HGET_␣overrun_␣in_␣section_␣%d_␣at_␣"SIZE_F"\n",
 section_no, hpos - hstart)
#define HEND ((hpos ≤ hend) ? 0 : (HGET_ERROR, 0))
#define HGET8 ((hpos < hend) ? *(hpos++) : (HGET_ERROR, 0))
#define HGET16(X) ((X) = (hpos[0] ≪ 8) + hpos[1], hpos += 2, HEND)
#define HGET24(X)
 ((X) = (hpos[0] ≪ 16) + (hpos[1] ≪ 8) + hpos[2], hpos += 3, HEND)
#define HGET32(X)
 ((X) = (hpos[0] ≪ 24) + (hpos[1] ≪ 16) + (hpos[2] ≪ 8) + hpos[3], hpos += 4,
 HEND)
#define HGETTAG(A) A = HGET8, DBGTAG(A, hpos - 1)

Writing the short format: ⇒ ...

⟨ put functions 14xiv ⟩ +≡ (344)
void hput_error(**void**)
 { **if** (hpos < hend) **return**;
 QUIT("HPUT_␣overrun_␣section_␣%d_␣pos="SIZE_F"\n",
 section_no, hpos - hstart);
 }

⟨ put macros 345cccxlv ⟩ ≡ (345)
extern void hput_error(**void**);
#define HPUT8(X) (hput_error(), *(hpos++) = (X))
#define HPUT16(X) (HPUT8(((X) ≫ 8) & #FF), HPUT8((X) & #FF))
#define HPUT24(X)
 (HPUT8(((X) ≫ 16) & #FF), HPUT8(((X) ≫ 8) & #FF), HPUT8((X) & #FF))
#define HPUT32(X) (HPUT8(((X) ≫ 24) & #FF), HPUT8(((X) ≫ 16) & #FF),
 HPUT8(((X) ≫ 8) & #FF), HPUT8((X) & #FF))

Used in 553dlviii and 557dlvii.

The above macros test for buffer overruns; allocating sufficient buffer space is done separately.

Before writing a node, we will insert a test and increase the buffer if necessary.

⟨ put macros 345cccxlv ⟩ +≡ (346)
void hput_increase_buffer(**uint32_t** n);
#define HPUTX(N) (((hend - hpos) < (N)) ? hput_increase_buffer(N) : (**void**) 0)
#define HPUTNODE HPUTX(MAX_TAG_DISTANCE)

```
#define HPUTTAG(K, I)
    (HPUTNODE, DBGTAG(TAG(K, I), hpos), HPUT8(TAG(K, I)))
```

Fortunately the only data types that have an unbounded size are strings and texts. For these we insert specific tests. For all other cases a relatively small upper bound on the maximum distance between two tags can be determined. Currently the maximum distance between tags is 26 byte as can be determined from the *hnode_size* array described in appendix A. The definition below uses a slightly larger value leaving some room for future changes in the design of the short file format.

```
< hint macros 13xiii > += (347)
#define MAX_TAG_DISTANCE 32
```

8.4 Mapping a Short Format File to Memory

In the following, we implement two alternatives to map a file into memory. The first implementation, opens the file, gets its size, allocates memory, and reads the file. The second implementation uses a call to *mmap*.

Since modern computers with 64bit hardware have a huge address space, using *mmap* to map the entire file into virtual memory is the most efficient way to access a large file. “Mapping” is not the same as “reading” and it is not the same as allocating precious memory, all that is done by the operating system when needed. Mapping just reserves addresses. There is one disadvantage of mapping: it typically locks the underlying file and will not allow a separate process to modify it. This prevents using this method for previewing a HINT file while editing and recompiling it. In this case, the first implementation, which has a copy of the file in memory, is the better choice. To select the second implementation, define the macro *USE_MMAP*.

The following functions map and unmap a short format input file setting *hin_addr* to its address and *hin_size* to its size. The value *hin_addr* $\equiv$ NULL indicates, that no file is open. The variable *hin_time* is set to the time when the file was last modified. It can be used to detect modifications of the file and reload it.

```
< common variables 252cclii > += (348)
char *hin_name = NULL;
uint64_t hin_size = 0;
uint8_t *hin_addr = NULL;
uint64_t hin_time = 0;
```

```
< map functions 349cccxlxi > ≡ (349)
#ifndef USE_MMAP
void hget_unmap(void)
{ if (hin_addr  $\neq$  NULL) free(hin_addr);
  hin_addr = NULL;
  hin_size = 0;
}
bool hget_map(void)
{ FILE *f;
```



```

struct stat st;
size_t s, t;
uint64_t u;

f = fopen(hin_name, "rb");
if (f == NULL)
{ MESSAGE("Unable to open file: %s\n", hin_name); return false; }
if (stat(hin_name, &st) < 0) {
    MESSAGE("Unable to obtain file size: %s\n", hin_name);
    fclose(f);
    return false;
}
if (st.st_size == 0) { MESSAGE("File %s is empty\n", hin_name);
    fclose(f);
    return false;
}
u = st.st_size;
if (hin_addr != NULL) hget_unmap();
hin_addr = malloc(u);
if (hin_addr == NULL) {
    MESSAGE("Unable to allocate 0x%" PRIx64 " byte for File %s\n", u,
            hin_name);
    fclose(f);
    return 0;
}
t = 0;
do { s = fread(hin_addr + t, 1, u, f);
    if (s ≤ 0) { MESSAGE("Unable to read file %s\n", hin_name);
        fclose(f);
        free(hin_addr);
        hin_addr = NULL;
        return false;
    }
    t = t + s; u = u - s;
} while (u > 0);
hin_size = st.st_size;
hin_time = st.st_mtime;
return true;
}
#else
#include <sys/mman.h>
void hget_unmap(void)
{ munmap(hin_addr, hin_size);
  hin_addr = NULL;
  hin_size = 0;
}

```

```

bool hget_map(void)
{ struct stat st;
  int fd;

  fd = open(hin_name, O_RDONLY, 0);
  if (fd < 0)
  { MESSAGE("Unable to open file %s\n", hin_name); return false; }
  if (fstat(fd, &st) < 0) { MESSAGE("Unable to get file size\n");
    close(fd);
    return false;
  }
  if (st.st_size == 0) { MESSAGE("File %s is empty\n", hin_name);
    close(fd);
    return false;
  }
  if (hin_addr != NULL) hget_unmap();
  hin_size = st.st_size;
  hin_time = st.st_mtime;
  hin_addr = mmap(NULL, hin_size, PROT_READ, MAP_PRIVATE, fd, 0);
  if (hin_addr == MAP_FAILED) { close(fd);
    hin_addr = NULL;
    hin_size = 0;
    MESSAGE("Unable to map file into memory\n");
    return 0;
  }
  close(fd);
  return hin_size;
}
#endif

```

Used in 552dlii, 558dlviii, and 560dlx.

8.5 Compression

The short file format offers the possibility to store sections in compressed form. We use the **zlib** compression library[2][1] to deflate and inflate individual sections. When one of the following functions is called, we can get the section buffer, the buffer size and the size actually used from the directory entry. If a section needs to be inflated, its size after decompression is found in the *xsize* field; if a section needs to be deflated, its size after compression will be known after deflating it.

```

⟨ get file functions 337cccxxvii ⟩ +≡ (350)
static void hdecompress(uint16_t n)
{ z_stream z; /* decompression stream */
  uint8_t *buffer;
  int i;

```

```

DBG(DBGCOMPRESS,
    "Decompressing_section_d_from_0x%x_to_0x%x_byte\n",
    n, dir[n].size, dir[n].xsize);
z.zalloc = (alloc_func)0; z.zfree = (free_func)0; z.opaque = (voidpf)0;
z.next_in = hstart;
z.avail_in = hend - hstart;
if (inflateInit(&z) ≠ Z_OK)
    QUIT("Unable_to_initialize_decompression:_%s", z.msg);
ALLOCATE(buffer, dir[n].xsize + MAX_TAG_DISTANCE, uint8_t);
DBG(DBGBUFFER,
    "Allocating_output_buffer_size=0x%x,margin=0x%x\n",
    dir[n].xsize, MAX_TAG_DISTANCE);
z.next_out = buffer;
z.avail_out = dir[n].xsize + MAX_TAG_DISTANCE;
i = inflate(&z, Z_FINISH);
DBG(DBGCOMPRESS, "in:_avail/total=0x%x/0x%lx"
    "out:_avail/total=0x%x/0x%lx,_return_%d;\n",
    z.avail_in, z.total_in, z.avail_out, z.total_out, i);
if (i ≠ Z_STREAM_END)
    QUIT("Unable_to_complete_decompression:_%s", z.msg);
if (z.avail_in ≠ 0) QUIT("Decompression_missed_input_data");
if (z.total_out ≠ dir[n].xsize)
    QUIT("Decompression_output_size_mismatch_0x%lx!=_0x%x",
        z.total_out, dir[n].xsize);
if (inflateEnd(&z) ≠ Z_OK)
    QUIT("Unable_to_finalize_decompression:_%s", z.msg);
dir[n].buffer = buffer;
dir[n].bsize = dir[n].xsize;
hpos0 = hpos = hstart = buffer;
hend = hstart + dir[n].xsize;
}

⟨ put functions 14xiv ⟩ +≡ (351)
static void hcompress(uint16_t n)
{
    z_stream z; /* compression stream */
    uint8_t *buffer;
    int i;
    if (dir[n].size ≡ 0) { dir[n].xsize = 0; return; }
    DBG(DBGCOMPRESS, "Compressing_section_d_of_size_0x%x\n", n,
        dir[n].size);
    z.zalloc = (alloc_func)0; z.zfree = (free_func)0; z.opaque = (voidpf)0;
    if (deflateInit(&z, Z_DEFAULT_COMPRESSION) ≠ Z_OK)
        QUIT("Unable_to_initialize_compression:_%s", z.msg);
    ALLOCATE(buffer, dir[n].size + MAX_TAG_DISTANCE, uint8_t);
    z.next_out = buffer;
    z.avail_out = dir[n].size + MAX_TAG_DISTANCE;

```

```

    z.next_in = dir[n].buffer;
    z.avail_in = dir[n].size;
    i = deflate(&z, Z_FINISH);
    DBG(DBGCOMPRESS, "deflate_in: avail/total=0x%x/0x%lx out:\n",
        avail/total=0x%x/0x%lx, return%d;\n",
        z.avail_in, z.total_in, z.avail_out, z.total_out, i);
    if (z.avail_in != 0) QUIT("Compression_missed_input_data");
    if (i != Z_STREAM_END) QUIT("Compression_incomplete:%s", z.msg);
    if (deflateEnd(&z) != Z_OK)
        QUIT("Unable_to_finalize_compression:%s", z.msg);
    DBG(DBGCOMPRESS, "Compressed 0x%lx byte to 0x%lx byte\n",
        z.total_in, z.total_out);
    free(dir[n].buffer);
    dir[n].buffer = buffer;
    dir[n].bsize = dir[n].size + MAX_TAG_DISTANCE;
    dir[n].xsize = dir[n].size;
    dir[n].size = z.total_out;
}

```

8.6 Reading Short Format Sections

After mapping the file at address *hin_addr* access to sections of the file is provided by decompressing them if necessary and setting the three pointers *hpos*, *hstart*, and *hend*.

To read sections of a short format input file, we use the function *hget_section*.

Reading the short format:

... $\Rightarrow$

```

⟨ get file functions 337cccxvii ⟩ +≡ (352)
void hget_section(uint16_t n)
{
    DBG(DBGDIR, "Reading_section%d\n", n);
    RNG("Section_number", n, 0, max_section_no);
    if (dir[n].buffer != NULL ^ dir[n].xsize > 0) {
        hpos0 = hpos = hstart = dir[n].buffer;
        hend = hstart + dir[n].xsize;
    }
    else { hpos0 = hpos = hstart = hin_addr + dir[n].pos;
        hend = hstart + dir[n].size;
        if (dir[n].xsize > 0) hdecompress(n);
    }
}

```

8.7 Writing Short Format Sections

To write a short format file, we allocate for each of the first three sections a suitable buffer, then fill these buffers, and finally write them out in sequential order.

```

⟨put functions 14xiv ⟩ +≡ (353)
#define BUFFER_SIZE #400
void new_output_buffers(void)
{
    dir[0].bsize = dir[1].bsize = dir[2].bsize = BUFFER_SIZE;
    DBG(DBGBUFFER,
        "Allocating_output_buffer_size=0x%x,margin=0x%x\n",
        BUFFER_SIZE, MAX_TAG_DISTANCE);
    ALLOCATE(dir[0].buffer, dir[0].bsize + MAX_TAG_DISTANCE, uint8_t);
    ALLOCATE(dir[1].buffer, dir[1].bsize + MAX_TAG_DISTANCE, uint8_t);
    ALLOCATE(dir[2].buffer, dir[2].bsize + MAX_TAG_DISTANCE, uint8_t);
}

void hput_increase_buffer(uint32_t n)
{
    size_t bsize;
    uint32_t pos, pos0;
    const double buffer_factor = 1.4142136; /*  $\sqrt{2}$  */

    pos = hpos - hstart;
    pos0 = hpos0 - hstart;
    bsize = dir[section_no].bsize * buffer_factor + 0.5;
    if (bsize < pos + n) bsize = pos + n;
    if (bsize ≥ HINT_NO_POS) bsize = HINT_NO_POS;
    if (bsize < pos + n)
        QUIT("Unable_to_increase_buffer_size" SIZE_F "\by 0x%x byte",
            hpos - hstart, n);
    DBG(DBGBUFFER, "Reallocating_output_buffer"
        "for_section_%d from 0x%x to " SIZE_F " byte\n", section_no,
        dir[section_no].bsize, bsize);
    REALLOCATE(dir[section_no].buffer, bsize, uint8_t);
    dir[section_no].bsize = (uint32_t) bsize;
    hstart = dir[section_no].buffer;
    hend = hstart + bsize;
    hpos0 = hstart + pos0;
    hpos = hstart + pos;
}

static size_t hput_data(uint16_t n, uint8_t *buffer, uint32_t size)
{
    size_t s;
    s = fwrite(buffer, 1, size, hout);
    if (s ≠ size)
        QUIT("short_write" SIZE_F "< %d in section %d", s, size, n);
    return s;
}

```

```
static size_t hput_section(uint16_t n)
{ return hput_data(n, dir[n].buffer, dir[n].size);
}
```

9 Directory Section

A HINT file is subdivided in sections and each section can be identified by its section number. The first three sections, numbered 0, 1, and 2, are mandatory: directory section, definition section, and content section. The directory section, which we explain now, lists all sections that make up a HINT file.

A document will often contain not only plain text but also other media for example illustrations. Illustrations are produced with specialized tools and stored in specialized files. Because a HINT file in short format should be self contained, these special files are embedded in the HINT file as optional sections. Because a HINT file in long format should be readable, these special files are written to disk and only the file names are retained in the directory. Writing special files to disk has also the advantage that you can modify them individually before embedding them in a short format file.

9.1 Directories in Long Format

The directory section of a long format HINT file starts with the “**directory**” keyword; then follows the maximum section number used and a list of directory entries, one for each optional section numbered 3 and above. Each entry consists of the keyword “**section**” followed by the section number, followed by the file name. The section numbers must be unique and fit into 16 bit. The directory entries must be ordered with strictly increasing section numbers. Keeping section numbers consecutive is recommended because it reduces the memory footprint if directories are stored as arrays indexed by the section number as we will do below.

Reading the long format:

— — — $\Rightarrow$

$\langle$ symbols 2ii $\rangle + \equiv$ (354)

%token DIRECTORY "directory"

%token SECTION "entry"

$\langle$ scanning rules 3iii $\rangle + \equiv$ (355)

directory **return** DIRECTORY;

section **return** SECTION;

$\langle$ parsing rules 5v $\rangle + \equiv$ (356)

directory_section: START DIRECTORY UNSIGNED

 { *new_directory*(\$3 + 1); *new_output_buffers*(); } *entry_list* END;

entry_list: | *entry_list* *entry*;

```

entry: START SECTION UNSIGNED string END
    { RNG("Section_number", $3, 3, max_section_no);
      hset_entry(&(dir[$3]), $3, 0, 0, $4); };

```

We use a dynamically allocated array of directory entries to store the directory.

⟨ directory entry type 357ccclvii ⟩ ≡ (357)

```

typedef struct {
    uint64_t pos;
    uint32_t size, xsize;
    uint16_t section_no;
    char *file_name;
    uint8_t *buffer;
    uint32_t bsize;
} Entry;

```

Used in 551dli, 553dliii, and 560dlx.

The function *new_directory* allocates the directory.

⟨ directory functions 358ccclviii ⟩ ≡ (358)

```

Entry *dir = NULL;
uint16_t section_no, max_section_no;
void new_directory(uint32_t entries)
{ DBG(DBGDIR, "Creating_directory_with_%d_entries\n", entries);
  RNG("Directory_entries", entries, 3, #10000);
  max_section_no = entries - 1; ALLOCATE(dir, entries, Entry);
  dir[0].section_no = 0; dir[1].section_no = 1; dir[2].section_no = 2;
}

```

Used in 552dlii, 554dliv, 557dlvii, 558dlviii, and 560dlx.

The function *hset_entry* fills in the appropriate entry.

⟨ directory functions 358ccclviii ⟩ +≡ (359)

```

void hset_entry(Entry *e, uint16_t i, uint32_t size, uint32_t xsize,
               char *file_name)
{ e→section_no = i;
  e→size = size; e→xsize = xsize;
  if (file_name ≡ NULL ∨ *file_name ≡ 0) e→file_name = NULL;
  else e→file_name = strdup(file_name);
  DBG(DBGDIR, "Creating_entry%d:_%s\_%size=0x%x_xsize=0x%x\n",
       i, file_name, size, xsize);
}

```

Writing the auxiliary files depends on the *-a*, *-g* and *-f* options.

⟨ without *-f* skip writing an existing file 360ccclx ⟩ ≡ (360)

```

if (¬option_force ∧ access(aux_name, F_OK) ≡ 0) {
    MESSAGE("File_%s' exists.\n"
           "To_rewrite_the_file_use_the_-f_option.\n", aux_name);
    continue;
}

```



```
}
```

Used in 366ccclxvi.

The above code uses the *access* function, and we need to make sure it is defined:

```
<make sure access is defined 361ccclxi > ≡ (361)
```

```
#ifndef WIN32
#include <io.h>
#define access(N, M) _access(N, M)
#define F_OK 0
#else
#include <unistd.h>
#endif
```

Used in 366ccclxvi.

With the *-g* option, filenames are considered global, and files are written to the filesystem possibly overwriting the existing files. For example a font embedded in a *HINT* file might replace a font of the same name in some operating systems font folder. If the *HINT* file is *shrunk* on one system and *stretched* on another system, this is usually not the desired behavior. Without the *-g* option, the files will be written in two local directories. The names of these directories are derived from the output file name, replacing the extension “*.hint*” with “*.abs*” if the original filename contained an absolute path, and replacing it with “*.rel*” if the original filename contained a relative path. Inside these directories, the path as given in the filename is retained. When *shrinking* a *HINT* file without the *-g* option, the original filenames can be reconstructed.

```
<compute a local aux_name 362ccclxii > ≡ (362)
```

```
{ char *path = dir[i].file_name;
  int path_length = (int) strlen(path);

  <determine whether path is absolute or relative 363ccclxiii >
  <replace links to the parent directory 364ccclxiv >
  DBG(DBGDIR, "Replacing_auxiliary_file_name:\n\t%s\n->\t%s\n", path,
      aux_name);
}
```

Used in 366ccclxvi and 372ccclxxii.

```
<determine whether path is absolute or relative 363ccclxiii > ≡ (363)
```

```
int aux_length;
enum {
    absolute = 0, relative = 1
} name_type;
char *aux_ext[2] = {".abs/", ".rel/"};
int ext_length = 5;

aux_length = stem_length + ext_length + path_length;
ALLOCATE(aux_name, aux_length + 1, char);
strcpy(aux_name, stem_name);
if (path[0] ≡ '/') { name_type = absolute;
```

```

    strcpy(aux_name + stem_length, aux_ext[name_type]);
    strcpy(aux_name + stem_length + ext_length, path + 1);
}
else if (path_length > 3 ∧ isalpha(path[0]) ∧ path[1] ≡ ':' ∧ path[2] ≡ '/') {
    name_type = absolute;
    strcpy(aux_name + stem_length, aux_ext[name_type]);
    strcpy(aux_name + stem_length + ext_length, path);
    aux_name[stem_length + ext_length + 1] = '_';
}
else name_type = relative;

```

Used in 362ccclxii.

When the `-g` is not given, auxiliar files are written into special subdirectories. To prevent them from escaping into the global file system, we replace links to the parent direcorly “`../`” by “`_../`”.

```

⟨replace links to the parent directory 364ccclxiv⟩ ≡
{ int k;
  for (k = stem_length + ext_length; k < aux_length - 3; k++)
    if (aux_name[k] ≡ '.' ∧ aux_name[k + 1] ≡ '.' ∧ aux_name[k + 2] ≡ '/')
      { aux_name[k] = aux_name[k + 1] = '_';
        k = k + 2;
      }
}

```

Used in 362ccclxii.

It remains to create the directories along the path we might have constructed.

```

⟨make sure the path in aux_name exists 365ccclxv⟩ ≡
{ char *path_end;
  path_end = aux_name + 1;
  while (*path_end ≠ 0) {
    if (*path_end ≡ '/') { struct stat s;
      *path_end = 0;
      if (stat(aux_name, &s) ≡ -1) {
#ifdef WIN32
        if (mkdir(aux_name) ≠ 0)
#else
        if (mkdir(aux_name, 0777) ≠ 0)
#endif
        QUIT("Unable to create directory %s", aux_name);
        DBG(DBGDIR, "Creating directory %s\n", aux_name);
      }
      else if (¬(S_IFDIR & (s.st_mode)))
        QUIT("Unable to create directory %s, file exists", aux_name);
      *path_end = '/';
    }
  }
}

```

```

    path_end++;
}
}

```

Used in [366ccclxvi](#) and [456cdlvi](#).

Writing the long format:

⇒ - - -

```

⟨ write functions 21xxi ⟩ +≡ (366)
  ⟨ make sure access is defined 361ccclxi ⟩
  extern char *stem_name;
  extern int stem_length;
  void hget_section(uint16_t n);
  void hwrite_aux_files(void)
  { int i;
    if (¬option_aux) return;
    DBG(DBGBASIC | DBGDIR, "Writing_d_aux_files\n", max_section_no - 2);
    for (i = 3; i ≤ max_section_no; i++) { FILE *f;
      char *aux_name = NULL;
      if (option_global) aux_name = strdup(dir[i].file_name);
      else ⟨ compute a local aux_name 362ccclxii ⟩
      ⟨ without -f skip writing an existing file 360ccclx ⟩
      ⟨ make sure the path in aux_name exists 365ccclxv ⟩
      f = fopen(aux_name, "wb");
      if (f ≡ NULL)
        QUIT("Unable_to_open_file_'s'_for_writing", aux_name);
      else { size_t s;
        hget_section(i);
        DBG(DBGDIR, "Writing_file_s\n", aux_name);
        s = fwrite(hstart, 1, dir[i].size, f);
        if (s ≠ dir[i].size) QUIT("writing_file_s", aux_name);
        fclose(f);
      }
      free(aux_name);
    }
  }
}

```

We write the directory, and the directory entries in long format using the following functions.

```

⟨ write functions 21xxi ⟩ +≡ (367)
  static void hwrite_entry(int i)
  { hwrite_start();
    hwritef("section_u", dir[i].section_no); hwrite_string(dir[i].file_name);
    hwrite_end();
  }
}

```

```

void hwrite_directory(void)
{ int i;
  if (dir  $\equiv$  NULL) QUIT("Directory_not_allocated");
  section_no = 0;
  hwritef("<directory_%"u", max_section_no);
  for (i = 3; i  $\leq$  max_section_no; i++) hwrite_entry(i);
  hwritef("\n>\n");
}

```

9.2 Directories in Short Format

The directory section of a short format file contains entries for all sections including the directory section itself. After reading the directory section, enough information—position and size—is available to access any section directly. As usual, a directory entry starts and ends with a tag byte. The kind part of an entry's tag is not used; it is always zero. The value s of the two least significant bits of the info part indicate that sizes are stored using $s + 1$ byte. The most significant bit of the info part is 1 if the section is stored in compressed form. In this case the size of the section is followed by the size of the section after decompressing it. After the tag byte follows the section number. In the short format file, section numbers must be strictly increasing and consecutive. This is redundant but helps with checking. Then follows the size—or the sizes—of the section. After the size follows the file name terminated by a zero byte. The file name might be an empty string in which case there is just the zero byte. After the zero byte follows a copy of the tag byte.

Here is the macro and function to read a directory entry:

Reading the short format:

... $\implies$

```

< shared get macros 38xxxviii > +=
#define HGET_SIZE(I) (368)
if ((I) & b100) {
  if (((I) & b011)  $\equiv$  0) s = HGET8, xs = HGET8;
  else if (((I) & b011)  $\equiv$  1) HGET16(s), HGET16(xs);
  else if (((I) & b011)  $\equiv$  2) HGET24(s), HGET24(xs);
  else if (((I) & b011)  $\equiv$  3) HGET32(s), HGET32(xs);
}
else {
  if (((I) & b011)  $\equiv$  0) s = HGET8;
  else if (((I) & b011)  $\equiv$  1) HGET16(s);
  else if (((I) & b011)  $\equiv$  2) HGET24(s);
  else if (((I) & b011)  $\equiv$  3) HGET32(s);
}
#define HGET_ENTRY(I, E)
{ uint16_t i;
  uint32_t s = 0, xs = 0;
  char *file_name;

```

```

    HGET16(i); HGET_SIZE(I); HGET_STRING(file_name);
    hset_entry(&(E), i, s, xs, file_name);
}

⟨get file functions 337cccxxvii ⟩ +≡ (369)
void hget_entry(Entry *e)
{ ⟨read the start byte a 16xvi ⟩
  DBG(DBGDIR, "Reading_directory_entry\n");
  switch (a) {
  case TAG(0, b000 + 0): HGET_ENTRY(b000 + 0, *e); break;
  case TAG(0, b000 + 1): HGET_ENTRY(b000 + 1, *e); break;
  case TAG(0, b000 + 2): HGET_ENTRY(b000 + 2, *e); break;
  case TAG(0, b000 + 3): HGET_ENTRY(b000 + 3, *e); break;
  case TAG(0, b100 + 0): HGET_ENTRY(b100 + 0, *e); break;
  case TAG(0, b100 + 1): HGET_ENTRY(b100 + 1, *e); break;
  case TAG(0, b100 + 2): HGET_ENTRY(b100 + 2, *e); break;
  case TAG(0, b100 + 3): HGET_ENTRY(b100 + 3, *e); break;
  default: TAGERR(a); break;
  }
  ⟨read and check the end byte z 17xvii ⟩
}

```

Because the first entry in the directory section describes the directory section itself, we can not check its info bits in advance to determine whether it is compressed or not. Therefore the directory section starts with a root entry, which is always uncompressed. It describes the remainder of the directory which follows. There are two differences between the root entry and a normal entry: it starts with the maximum section number instead of the section number zero, and we set its position to the position of the entry for section 1 (which might already be compressed). The name of the directory section must be the empty string.

Reading the short format: ... ⇒

```

⟨get file functions 337cccxxvii ⟩ +≡ (370)
static void hget_root(Entry *root)
{ DBG(DBGDIR, "Root_entry_at_SIZE_F\n", hpos - hstart);
  hget_entry(root);
  root→pos = hpos - hstart;
  max_section_no = root→section_no;
  root→section_no = 0;
  if (max_section_no < 2) QUIT("Sections_0,1,and_2_are_mandatory");
}

void hget_directory(void)
{ int i;
  Entry root = {0};

```

```

    hget_root(&root);
    DBG(DBGDIR, "Directory\n");
    new_directory(max_section_no + 1);
    dir[0] = root;
    DBG(DBGDIR, "Directory_entry_1_at_0x%" PRIx64 "\n", dir[0].pos);
    hget_section(0);
    for (i = 1; i ≤ max_section_no; i++)
    { hget_entry(&(dir[i])); dir[i].pos = dir[i - 1].pos + dir[i - 1].size;
      DBG(DBGDIR, "Section_%d_at_0x%" PRIx64 "\n", i, dir[i].pos);
    }
}

void hclear_dir(void)
{ int i;
  if (dir == NULL) return;
  for (i = 0; i < 3; i++) /* currently the only compressed sections */
    if (dir[i].xsize > 0 ∧ dir[i].buffer ≠ NULL) free(dir[i].buffer);
  free(dir);
  dir = NULL;
}

```

Armed with these preparations, we can put the directory into the HINT file.

Writing the short format:

⇒ ...

⟨put functions 14xiv ⟩ +≡

(371)

```

static void hput_entry(Entry *e)
{ Info b;
  if (e→size < #100 ∧ e→xsize < #100) b = b000;
  else if (e→size < #10000 ∧ e→xsize < #10000) b = b001;
  else if (e→size < #1000000 ∧ e→xsize < #1000000) b = b010;
  else b = b011;
  if (e→xsize ≠ 0) b = b | b100;
  DBG(DBGTAGS, "Directory_entry_no=%d_size=0x%x_xsize=0x%x\n",
    e→section_no, e→size, e→xsize);
  HPUTTAG(0, b);
  HPUT16(e→section_no);
  switch (b) {
  case b000: HPUT8(e→size); break;
  case b001: HPUT16(e→size); break;
  case b010: HPUT24(e→size); break;
  case b011: HPUT32(e→size); break;
  case b100: HPUT8(e→size); HPUT8(e→xsize); break;
  case b101: HPUT16(e→size); HPUT16(e→xsize); break;
  case b110: HPUT24(e→size); HPUT24(e→xsize); break;
  case b111: HPUT32(e→size); HPUT32(e→xsize); break;
  default: QUIT("Can't happen"); break;
  }
}

```

```

    }
    hput_string(e→file_name);
    DBGTAG(TAG(0, b), hpos); HPUT8(TAG(0, b));
}

static void hput_directory_start(void)
{ DBG(DBGDIR, "Directory_Section\n");
  section_no = 0;
  hpos = hstart = dir[0].buffer;
  hend = hstart + dir[0].bsize;
}

static void hput_directory_end(void)
{ dir[0].size = hpos - hstart;
  DBG(DBGDIR, "End_Directory_Section_size=0x%x\n", dir[0].size);
}

static size_t hput_root(void)
{ uint8_t buffer[MAX_TAG_DISTANCE];
  size_t s;
  hpos = hstart = buffer;
  hend = hstart + MAX_TAG_DISTANCE;
  dir[0].section_no = max_section_no;
  hput_entry(&dir[0]);
  s = hput_data(0, hstart, hpos - hstart);
  DBG(DBGDIR, "Writing_root_size=SIZE_F"\n", s);
  return s;
}

extern int option_compress;
static char **aux_names;

void hput_directory(void)
{ int i;
  ⟨ update the file sizes of optional sections 372ccclxxii ⟩
  if (option_compress) { hcompress(1); hcompress(2); }
  hput_directory_start();
  for (i = 1; i ≤ max_section_no; i++) {
    dir[i].pos = dir[i - 1].pos + dir[i - 1].size;
    DBG(DBGDIR, "writing_entry_u_at_0x%" PRIx64 "\n", i, dir[i].pos);
    hput_entry(&dir[i]);
  }
  hput_directory_end();
  if (option_compress) hcompress(0);
}

```

Now let us look at the optional sections described in the directory entries 3 and above. Where these files are found depends on the `-g` and `-a` options.

With the `-g` option given, only the file names as given in the directory entries are used. With the `-a` option given, the file names are translated to filenames in

the *hin_name.abs* and *hin_name.rel* directories, as described in section 9.1. If neither the **-a** nor the **-g** option is given, **shrink** first tries the translated filename and then the global filename before it gives up.

When the **shrink** program writes the directory section in the short format, it needs to know the sizes of all the sections—including the optional sections. These sizes are not provided in the long format because it is safer and more convenient to let the machine figure out the file sizes. But before we can determine the size, we need to determine the file.

```

⟨ update the file sizes of optional sections 372ccclxxii ⟩ ≡
{
  int i;
  ALLOCATE(aux_names, max_section_no + 1, char *);
  for (i = 3; i ≤ max_section_no; i++)
    if (dir[i].size ≡ 0) {
      struct stat s;
      if (¬option_global) {
        char *aux_name = NULL;
        ⟨ compute a local aux_name 362ccclxii ⟩
        if (stat(aux_name, &s) ≡ 0) aux_names[i] = aux_name;
        else {
          if (option_aux) QUIT("Unable to find file '%s'", aux_name);
          free(aux_name);
          aux_name = NULL;
        }
      }
      if ((aux_names[i] ≡ NULL ∧ ¬option_aux) ∨ option_global) {
        if (stat(dir[i].file_name, &s) ≠ 0)
          QUIT("Unable to find file '%s'", dir[i].file_name);
      }
      dir[i].size = s.st_size;
      dir[i].xsize = 0;
      DBG(DBGDIR, "section%i: found file %s size %u\n", i,
          aux_names[i] ? aux_names[i] : dir[i].file_name, dir[i].size);
    }
}

```

Used in 371ccclxxi.

```

⟨ rewrite the file names of optional sections 373ccclxxiii ⟩ ≡
{
  int i;
  for (i = 3; i ≤ max_section_no; i++)
    if (aux_names[i] ≠ NULL) {
      free(dir[i].file_name);
      dir[i].file_name = aux_names[i];
      aux_names[i] = NULL;
    }
}

```

Used in 557dlvii.

The computation of the sizes of the mandatory sections will be explained later.

To conclude this section, here is the function that adds the files that are described in the directory entries 3 and above to a `HINT` file in short format.

Writing the short format:

⇒ ...

```

⟨ put functions 14xiv ⟩ +≡ (374)
static size_t hput_optional_section(int i)
{ size_t fsize;
  if (dir[i].xsize ≠ 0)
    DBG(DBGDIR,
        "Compressing_of_auxiliary_files_currently_not_supported");
  if (dir[i].buffer ≡ NULL) ⟨ append section i from file 375ccclxxv ⟩
  else ⟨ append section i from buffer 376ccclxxvi ⟩
  if (fsize ≠ dir[i].size)
    QUIT("Size_written_SIZE_F_does_not_match_section_size",
        fsize, i, dir[i].size);
  return fsize;
}

```

If the content of the auxiliary section i was not loaded into $dir[i].buffer$ earlier, the file i loaded from disk and appended to the output.

```

⟨ append section i from file 375ccclxxv ⟩ ≡ (375)
{ FILE *f;
  char *file_name = dir[i].file_name;
  DBG(DBGDIR, "Adding_file_d:s\n", dir[i].section_no, file_name);
  f = fopen(file_name, "rb");
  if (f ≡ NULL) QUIT("Unable_to_read_section_d,file_s",
                    dir[i].section_no, file_name);
  fsize = 0;
  while (!feof(f))
  { size_t s, t;
    char buffer[1 << 13]; /* 8kByte */
    s = fread(buffer, 1, 1 << 13, f);
    t = fwrite(buffer, 1, s, hout);
    if (s ≠ t) QUIT("Unable_to_write_file_s", file_name);
    fsize = fsize + t;
  }
  fclose(f);
}

```

Used in 374ccclxxiv.

In the simpler case, that the file was loaded already into the buffer., the buffer is appended to the output.

```

⟨ append section i from buffer 376ccclxxvi ⟩ ≡ (376)
{ fsize = fwrite(dir[i].buffer, 1, dir[i].size, hout);

```

}

Used in [374ccclxxiv](#).

10 Definition Section

In a typical HINT file, there are many things that are used over and over again. For example the interword glue of a specific font or the indentation of the first line of a paragraph. The definition section contains this information so that it can be referenced in the content section by a simple reference number. In addition there are a few parameters that guide the routines of T_EX. An example is the “above display skip”, which controls the amount of white space inserted above a displayed equation, or the “hyphen penalty” that tells T_EX the “aesthetic cost” of ending a line with a hyphenated word. These parameters also get their values in the definition section as explained in section 11.

The most simple way to store these definitions is to store them in an array indexed by the reference numbers. To simplify the dynamic allocation of these arrays, the list of definitions will always start with the list of maximum values: a list that contains for each node type the maximum reference number used.

In the long format, the definition section starts with the keyword **definitions**, followed by the list of maximum values, followed by the definitions proper.

When writing the short format, we start by positioning the output stream at the beginning of the definition buffer and we end with recording the size of the definition section in the directory.

Reading the long format:

— — — $\implies$

```
<symbols 2ii > +≡ (377)
%token DEFINITIONS "definitions"
```

```
<scanning rules 3iii > +≡ (378)
definitions      return DEFINITIONS;
```

```
<parsing rules 5v > +≡ (379)
  definition_section: START DEFINITIONS { hput_definitions_start(); }
                    max_definitions definition_list
                    END { hput_definitions_end(); };
  definition_list: | definition_list def_node;
```

Writing the long format:

⇒ - - -

```

⟨ write functions 21xxi ⟩ +≡ (380)
    void hwrite_definitions_start(void)
    { section_no = 1; hwritef("<definitions");
    }

    void hwrite_definitions_end(void)
    { hwritef("\n>\n");
    }

```

```

⟨ get functions 18xviii ⟩ +≡ (381)
    void hget_definition_section(void)
    { DBG(DBGBASIC | DBGDEF, "Definitions\n");
      hget_section(1);
      hwrite_definitions_start();
      DBG(DBGDEF, "List_of_maximum_values\n");
      hget_max_definitions();
      ⟨ initialize definitions 253ccliii ⟩
      hwrite_max_definitions();
      DBG(DBGDEF, "List_of_definitions\n");
      while (hpos < hend) hget_def_node();
      hwrite_definitions_end();
    }

```

Writing the short format:

⇒ ...

```

⟨ put functions 14xiv ⟩ +≡ (382)
    void hput_definitions_start(void)
    { DBG(DBGDEF, "Definition_Section\n");
      section_no = 1;
      hpos = hstart = dir[1].buffer;
      hend = hstart + dir[1].bsize;
    }

    void hput_definitions_end(void)
    { dir[1].size = hpos - hstart;
      DBG(DBGDEF, "End_Definition_Section_size=0x%x\n", dir[1].size);
    }

```

10.1 Maximum Values

To help implementations allocating the right amount of memory for the definitions, the definition section starts with a list of maximum values. For each kind of node, we store the maximum valid reference number in the array *max_ref* which is indexed by the kind-values. For a reference number *n* and kind-value *k* we have $0 \leq n \leq \text{max_ref}[k]$. To make sure that a hint file without any definitions will work, some definitions have default values. The initialization of default and maximum values is described in section 11. The maximum reference number that has a default value is stored in the array *max_default*. We have $-1 \leq \text{max_default}[k] \leq \text{max_ref}[k] < 2^{16}$, and for most *k* even $\text{max_ref}[k] < 2^8$. Specifying maximum values that are lower than the default values is not allowed in the short format; in the long format, lower values are silently ignored. Some default values are permanently fixed; for example the zero glue with reference number *zero_skip_no* must never change. The array *max_fixed* stores the maximum reference number that has a fixed value for a given kind. Definitions with reference numbers less or equal than the corresponding *max_fixed*[*k*] number are disallowed. Usually we have $-1 \leq \text{max_fixed}[k] \leq \text{max_default}[k]$, but if for a kind-value *k* no definitions, and hence no maximum values are allowed, we set $\text{max_fixed}[k] = \#10000 > \text{max_default}[k]$.

We use the *max_ref* array whenever we find a reference number in the input to check if it is within the proper range.

```
< debug macros 383ccclxxxiii > ≡ (383)
#define REF_RNG(K, N) if ((int)(N) > max_ref[K])
    QUIT("Reference_␣d_␣to_␣s_␣out_␣of_␣range_␣[0_␣-_␣d] ", (N),
    definition_name[K], max_ref[K])
```

Used in 548dxlviii.

In the long format file, the list of maximum values starts with “<max ”, then follow pairs of keywords and numbers like “<glue 57>”, and it ends with “>”. In the short format, we start the list of maximums with a *list_kind* tag and end it with a *list_kind* tag. Each maximum value is preceded and followed by a tag byte with the appropriate kind-value. The info value has its *b001* bit cleared if the maximum value is in the range 0 to #FF and fits into a single byte; the info value has its *b001* bit set if it fits into two byte. Currently only the *label_kind* may need to use two byte.

```
< debug macros 383ccclxxxiii > +≡ (384)
#define MAX_REF (K) ((K) ≡ label_kind ? #FFFF : #FF)
```

Other info values are reserved for future extensions. After reading the maximum values, we initialize the data structures for the definitions.

Reading the long format:

— — — $\Rightarrow$

$\langle$ symbols 2ii $\rangle + \equiv$ (385)

%token MAX "max"

$\langle$ scanning rules 3iii $\rangle + \equiv$ (386)

max return MAX;

$\langle$ parsing rules 5v $\rangle + \equiv$ (387)

max_definitions: START MAX max_list END

{ $\langle$ initialize definitions 253ccliii $\rangle$ hput_max_definitions(); };

max_list: | max_list START max_value END;

max_value: FONT UNSIGNED { hset_max(font_kind,\$2); }
 | INTEGER UNSIGNED { hset_max(int_kind,\$2); }
 | DIMEN UNSIGNED { hset_max(dimen_kind,\$2); }
 | LIGATURE UNSIGNED { hset_max(ligature_kind,\$2); }
 | DISC UNSIGNED { hset_max(disc_kind,\$2); }
 | GLUE UNSIGNED { hset_max(glue_kind,\$2); }
 | LANGUAGE UNSIGNED { hset_max(language_kind,\$2); }
 | RULE UNSIGNED { hset_max(rule_kind,\$2); }
 | IMAGE UNSIGNED { hset_max(image_kind,\$2); }
 | LEADERS UNSIGNED { hset_max(leaders_kind,\$2); }
 | BASELINE UNSIGNED { hset_max(baseline_kind,\$2); }
 | XDIMEN UNSIGNED { hset_max(xdimen_kind,\$2); }
 | PARAM UNSIGNED { hset_max(param_kind,\$2); }
 | STREAMDEF UNSIGNED { hset_max(stream_kind,\$2); }
 | PAGE UNSIGNED { hset_max(page_kind,\$2); }
 | RANGE UNSIGNED { hset_max(range_kind,\$2); }
 | LABEL UNSIGNED { hset_max(label_kind,\$2); }
 | COLOR UNSIGNED { hset_max(color_kind,\$2); };

$\langle$ parsing functions 388ccclxxxviii $\rangle \equiv$ (388)

void hset_max(Kind k, int n)

{ DBG(DBGDEF, "Setting_max_%s_to_%d\n", definition_name[k], n);

RNG("Maximum", n, max_fixed[k] + 1, MAX_REF(k));

if (n > max_ref[k]) max_ref[k] = n;

}

Used in 556dlvi.

Writing the long format:

⇒ - - -

```

⟨ write functions 21xxi ⟩ +≡ (389)
void hwrite_max_definitions(void)
{ Kind k;
  hwrite_start(); hwritef("max");
  for (k = 0; k < 32; k++)
    if (max_ref[k] > max_default[k])
      { switch (k) { ⟨ cases of writing special maximum values 248ccxlvi ⟩
        default: hwrite_start();
                  hwritef("%s%d", definition_name[k], max_ref[k]);
                  hwrite_end();
                  break;
              }
      }
  hwrite_end();
}

```

Reading the short format:

... ⇒

```

⟨ get file functions 337ccxxxvii ⟩ +≡ (390)
void hget_max_definitions(void)
{ Kind k;
  ⟨ read the start byte a 16xvi ⟩
  if (a ≠ TAG(list_kind, 0)) QUIT("Start_of_maximum_list_expected");
  for (k = 0; k < 32; k++) max_ref[k] = max_default[k];
  max_outline = -1;
  while (true)
  { int n;
    if (hpos ≥ hend) QUIT("Unexpected_end_of_maximum_list");
    node_pos = hpos - hstart;
    HGETTAG(a); k = KIND(a); if (k ≡ list_kind) break;
    if (INFO(a) & b001) HGET16(n); else n = HGET8;
    switch (a) { ⟨ cases of getting special maximum values 246ccxli ⟩
    default:
      if (max_fixed[k] > max_default[k])
        MESSAGE("Maximum_value_for_kind%s_not_supported\n",
                definition_name[k]);
      else { RNG("Maximum_number", n, max_default[k], MAX_REF(k));
              max_ref[k] = n;
              DBG(DBGDEF, "max(%s)=%d\n", definition_name[k], max_ref[k]);
            }
      break;
    }
  }
  ⟨ read and check the end byte z 17xvii ⟩
}

```

```

    if (INFO(a) ≠ 0) QUIT("End_of_maximum_list_with_info%d", INFO(a));
    DBG(DBGDEF, "Getting_Max_Definitions_END\n");
}

```

Writing the short format:

⇒ ...

```

⟨ put functions 14xiv ⟩ +≡ (391)
void hput_max_definitions(void)
{ Kind k;
  DBG(DBGDEF, "Writing_Max_Definitions\n");
  HPUTTAG(list_kind, 0);
  for (k = 0; k < 32; k++)
    if (max_ref[k] > max_default[k]) { uint32_t pos = hpos++ - hstart;
      DBG(DBGDEF, "max(%s)=%d\n", definition_name[k], max_ref[k]);
      hput_tags(pos, TAG(k, hput_n(max_ref[k]) - 1));
    }
  ⟨ cases of putting special maximum values 247ccxlvii ⟩
  HPUTTAG(list_kind, 0);
  DBG(DBGDEF, "Writing_Max_Definitions_End\n");
}

```

10.2 Definitions

A definition associates a reference number with a content node. Here is an example: A glue definition associates a glue number, for example 71, with a glue specification. In the long format this might look like “<glue *71 4pt plus 5pt minus 0.5pt>” which makes glue number 71 refer to a 4pt glue with a stretchability of 5pt and a shrinkability of 0.5pt. Such a glue definition differs from a normal glue node just by an extra byte value immediately following the keyword respectively start byte.

Whenever we need this glue in the content section, we can say “<glue *71>”. Because we restrict the number of glue definitions to at most 256, a single byte is sufficient to store the reference number. The `shrink` and `stretch` programs will, however, not bother to store glue definitions. Instead they will write them in the new format immediately to the output.

The parser will handle definitions in any order, but the order is relevant if a definition references another definition, and of course, it never does any harm to present definitions in a systematic way.

As a rule, the definition of a reference must always precede the use of that reference. While this is always the case for references in the content section, it restricts the use of references inside the definition section.

The definitions for integers, dimensions, extended dimensions, languages, rules, ligatures, and images are “simple”. They never contain references and so it is always possible to list them first. The definition of glues may contain extended dimensions, the definitions of baselines may reference glue nodes, and the definitions of parameter lists contain definitions of integers, dimensions, and glues. So these definitions should follow in this order.

The definitions of leaders and discretionary breaks allow boxes. While these boxes are usually quite simple, they may contain arbitrary references—including again references to leaders and discretionary breaks. So, at least in principle, they might impose complex (or even unsatisfiable) restrictions on the order of those definitions.

The definitions of fonts contain not only “simple” definitions but also the definitions of interword glues and hyphens introducing additional ordering restrictions. The definition of hyphens regularly contain glyphs which in turn reference a font—typically the font that just gets defined. Therefore we relax the define before use policy for glyphs: Glyphs may reference a font before the font is defined.

The definitions of page templates contain lists of arbitrary content nodes, and while the boxes inside leaders or discretionary breaks tend to be simple, the content of page templates is often quite complex. Page templates are probably the source of most ordering restrictions. Placing page templates towards the end of the list of definitions might be a good idea. A special case are stream definitions. These occur only as part of the corresponding page template definition and are listed at its end. So references to them will occur in the page template always before their definition. Finally, the definitions of page ranges always reference a page template and they should come after the page template definitions. For technical reasons explained in section 6.2, definitions of labels and outlines come last.

To avoid complex dependencies, an application can always choose not to use references in the definition section. There are only three types of nodes where references can not be avoided: fonts are referenced in glyph nodes, labels are referenced in outlines, and languages are referenced in boxes or page templates. Possible ordering restrictions can be satisfied if languages are defined early. To check the define before use policy, we use an array of bitvectors, but we limit checking to the first 256 references. We have for every reference number $N < 256$ and every kind K a single bit which is set if and only if the corresponding reference is defined.

```

⟨ definition checks 392cccxcii ⟩ ≡
  uint32_t definition_bits[#100/32][32] = {{0}};
#define SET_DBIT(N, K)
  ((N) > #FF ? 1 : (definition_bits[N/32][K] |= (1 << ((N) & (32 - 1)))))
#define GET_DBIT(N, K)
  ((N) > #FF ? 1 : ((definition_bits[N/32][K] >> ((N) & (32 - 1))) & 1))
#define DEF(D, K, N) (D).k = K; (D).n = (N); SET_DBIT((D).n, (D).k);
  DBG(DBGDEF, "Defining %s %d\n", definition_name[(D).k], (D).n);
  RNG("Definition", (D).n, max_fixed[(D).k] + 1, max_ref[(D).k]);
#define REF(K, N) REF_RNG (K, N); if (¬GET_DBIT(N, K))
  QUIT("Reference %d to %s before definition", (N),
    definition_name[K])

```

Used in 556dlvi, 558dlviii, and 560dlx.

```

⟨ initialize definitions 253ccliii ⟩ +≡
  definition_bits[0][list_kind] = (1 << (MAX_LIST_DEFAULT + 1)) - 1;
  definition_bits[0][param_kind] = (1 << (MAX_LIST_DEFAULT + 1)) - 1;

```

```

definition_bits[0][int_kind] = (1 << (MAX_INT_DEFAULT + 1)) - 1;
definition_bits[0][dimen_kind] = (1 << (MAX_DIMEN_DEFAULT + 1)) - 1;
definition_bits[0][xdimen_kind] = (1 << (MAX_XDIMEN_DEFAULT + 1)) - 1;
definition_bits[0][glue_kind] = (1 << (MAX_GLUE_DEFAULT + 1)) - 1;
definition_bits[0][baseline_kind] = (1 << (MAX_BASELINE_DEFAULT + 1)) - 1;
definition_bits[0][page_kind] = (1 << (MAX_PAGE_DEFAULT + 1)) - 1;
definition_bits[0][stream_kind] = (1 << (MAX_STREAM_DEFAULT + 1)) - 1;
definition_bits[0][range_kind] = (1 << (MAX_RANGE_DEFAULT + 1)) - 1;
definition_bits[0][color_kind] = (1 << (MAX_COLOR_DEFAULT + 1)) - 1;

```

Reading the long format:

— — — $\Rightarrow$

Writing the short format:

$\Rightarrow \dots$

$\langle \text{symbols } 2\text{ii} \rangle + \equiv$ (394)

%**type** < rf > def_node

$\langle \text{parsing rules } 5\text{v} \rangle + \equiv$ (395)

```

def_node: start FONT ref font END
    { DEF($$, font_kind, $3); hput_tags($1, $4); }
| start INTEGER ref integer END
    { DEF($$, int_kind, $3); hput_tags($1, hput_int($4)); }
| start DIMEN ref dimension END
    { DEF($$, dimen_kind, $3); hput_tags($1, hput_dimen($4)); }
| start LANGUAGE ref string END
    { DEF($$, language_kind, $3); hput_string($4);
      hput_tags($1, TAG(language_kind, 0)); }
| start GLUE ref glue END
    { DEF($$, glue_kind, $3); hput_tags($1, hput_glue(&($4))); }
| start XDIMEN ref xdimen END
    { DEF($$, xdimen_kind, $3); hput_tags($1, hput_xdimen(&($4))); }
| start RULE ref rule END
    { DEF($$, rule_kind, $3); hput_tags($1, hput_rule(&($4))); }
| start LEADERS ref leaders END
    { DEF($$, leaders_kind, $3); hput_tags($1, TAG(leaders_kind, $4)); }
| start BASELINE ref baseline END
    { DEF($$, baseline_kind, $3); hput_tags($1, TAG(baseline_kind, $4)); }
| start LIGATURE ref ligature END
    { DEF($$, ligature_kind, $3); hput_tags($1, hput_ligature(&($4))); }
| start DISC ref disc END
    { DEF($$, disc_kind, $3); hput_tags($1, hput_disc(&($4))); }
| start IMAGE ref image END
    { DEF($$, image_kind, $3); hput_tags($1, TAG(image_kind, $4)); }
| start PARAM ref parameters END
    { DEF($$, param_kind, $3); hput_tags($1, hput_list($1 + 2, &($4))); }
| start PAGE ref page END
    { DEF($$, page_kind, $3); hput_tags($1, TAG(page_kind, 0)); };

```

There are a few cases where one wants to define a reference by a reference. For example, a HINT file may want to set the `parfillskip` glue to zero. While there are multiple ways to define the zero glue, the canonical way is a reference using the `zero_glue.no`. All these cases have in common that the reference to be defined is one of the default references and the defining reference is one of the fixed references. We add a few parsing rules and a testing macro for those cases where the number of default definitions is greater than the number of fixed definitions.

(definition checks 392cccxcii) +≡ (396)

```
#define DEF_REF(D,K,M,N) DEF (D,K,M);
  if ((int)(M) > max_default[K])
    QUIT("Defining_non_default_reference_d_for_s", M,
         definition_name[K]);
  if ((int)(N) > max_fixed[K])
    QUIT("Defining_reference_d_for_s_by_non_fixed_reference_d", M,
         definition_name[K], N);
```

(parsing rules 5v) +≡ (397)

```
def_node: start INTEGER ref ref END
  { DEF_REF($$,int_kind,$3,$4); hput_tags($1,TAG(int_kind,0)); }
| start DIMEN ref ref END
  { DEF_REF($$,dimen_kind,$3,$4); hput_tags($1,TAG(dimen_kind,0)); }
| start GLUE ref ref END
  { DEF_REF($$,glue_kind,$3,$4); hput_tags($1,TAG(glue_kind,0)); };
```

Reading the short format:

... ⇒

Writing the long format:

⇒ - - -

(get functions 18xviii) +≡ (398)

```
void hget_definition(int n, Tag a, uint32_t node_pos)
{ switch (KIND(a)) {
  case font_kind: hget_font_def(INFO(a),n); break;
  case param_kind:
    { List l;
      l.t = a; HGET_LIST(INFO(a),l); hwrite_parameters(&l); break; }
  case page_kind: hget_page(); break;
  case dimen_kind: hget_dimen(a); break;
  case xdimen_kind:
    { Xdimen x; hget_xdimen(a,&x); hwrite_xdimen(&x); break; }
  case language_kind:
    if (INFO(a) ≠ b000)
      QUIT("Info_value_of_language_definition_must_be_zero");
    else { char *n;
      HGET_STRING(n); hwrite_string(n);
    }
  break;
```

```

case color_kind:
  switch (INFO(a)) { < cases to get definitions for color_kind 299ccxcix >
  default:
    QUIT("Undefined_tag_d_for_color_kind_definition_at_0x%x",
        INFO(a), node_pos);
  }
  break;
default: hget_content(a); break;
}
}

void hget_def_node()
{ Kind k;
  < read the start byte a 16xvi >
  k = KIND(a);
  if (k  $\equiv$  unknown_kind  $\wedge$  INFO(a)  $\equiv$  b100) hget_unknown_def();
  else if (k  $\equiv$  label_kind) hget_outline_or_label_def(INFO(a), node_pos);
  else { int n;
    n = HGET8;
    if (k  $\neq$  range_kind) REF_RNG(k, n);
    SET_DBIT(n, k);
    if (k  $\equiv$  range_kind) hget_range(INFO(a), n);
    else { hwrite_start(); hwritef("%s*%d", definition_name[k], n);
      hget_definition(n, a, node_pos);
      hwrite_end();
    }
    if (n > max_ref[k]  $\vee$  n  $\leq$  max_fixed[k])
      QUIT("Definition_d_for_s_out_of_range[%d-_%d]",
          n, definition_name[k], max_fixed[k] + 1, max_ref[k]);
    if (max_fixed[k] > max_default[k])
      QUIT("Definitions_for_kind%s_not_supported",
          definition_name[k]);
  }
  < read and check the end byte z 17xvii >
}

```

10.3 Parameter Lists

Because the content section is a “stateless” list of nodes, the definitions we see in the definition section can never change. It is however necessary to make occasionally local modifications of some of these definitions, because some definitions are parameters of the algorithms borrowed from T_EX. Nodes that need such modifications, for example the paragraph nodes that are passed to T_EX’s line breaking algorithm, contain a list of local definitions called parameters. Typically sets of related parameters are needed. To facilitate a simple reference to such a set of parameters, we allow predefined parameter lists that can be referenced by a single number. The parameters of T_EX’s routines are quite basic—integers, dimensions,

and glues—and all of them have default values. Therefore we restrict the definitions in parameter lists to such basic definitions.

```

⟨ parsing functions 388ccclxxxviii ⟩ +≡ (399)
  void check_param_def(Ref * df)
  {
    if (df → k ≠ int_kind ∧ df → k ≠ dimen_kind ∧
        df → k ≠ glue_kind)
      QUIT("Kind_%s_not_allowed_in_parameter_list",
          definition_name[df → k]);
    if (df → n ≤ max_fixed[df → k] ∨ max_default[df → k] < df → n)
      QUIT("Parameter_%d_for_%s_not_allowed_in_parameter_list", df → n,
          definition_name[df → k]);
  }

```

The definitions below repeat the definitions we have seen for lists in section 4.1 with small modifications. For example we use the kind-value *param_kind*. An empty parameter list is omitted in the long format as well as in the short format.

Reading the long format:

— — — ⇒

Writing the short format:

⇒ ...

```

⟨ symbols 2ii ⟩ +≡ (400)
%token PARAM "param"
%type < u > def_list
%type < l > parameters

```

```

⟨ scanning rules 3iii ⟩ +≡ (401)
param          return PARAM;

```

```

⟨ parsing rules 5v ⟩ +≡ (402)
def_list: position | def_list def_node { check_param_def(&($2)); };
parameters: estimate def_list { $$p = $2; $$t = TAG(param_kind, b001);
    $$s = (hpos - hstart) - $2; };

```

Using a parsing rule like “*param_list*: start PARAM *parameters* END”, an empty parameter list will be written as “<param>”. This looks ugly and seems like unnecessary syntax because the parser knows anyway that a parameter list will come next. Therefore the keyword can be omitted except in definitions and in unknown nodes.

```

⟨ parsing rules 5v ⟩ +≡ (403)
named_param_list: start PARAM parameters END
    { hput_tags($1, hput_list($1 + 1, &($3))); };
param_list: named_param_list
    | start parameters END
    { hput_tags($1, hput_list($1 + 1, &($2))); };

```

Writing the long format:

⇒ - - -

```

⟨ write functions 21xxi ⟩ +≡ (404)
void hwrite_parameters(List *l)
{ uint32_t h = hpos - hstart, e = hend - hstart; /* save hpos and hend */
  hpos = l→p + hstart; hend = hpos + l→s;
  if (l→s > #FF) hwritef("□%d", l→s);
  while (hpos < hend) hget_def_node();
  hpos = hstart + h; hend = hstart + e; /* restore hpos and hend */
}

void hwrite_param_list(List *l)
{ hwrite_start(); hwrite_parameters(l);
  hwrite_end();
}

void hwrite_named_param_list(List *l)
{ hwrite_start(); hwritef("param");
  hwrite_parameters(l);
  hwrite_end();
}

```

Reading the short format:

... ⇒

```

⟨ get functions 18xviii ⟩ +≡ (405)
void hget_param_list(List *l)
{ if (KIND(*hpos) ≠ param_kind)
  QUIT("Parameter□list□expected□at□0x%x", (uint32_t)(hpos - hstart));
  else hget_list(l);
}

```

10.4 Fonts

Another definition that has no corresponding content node is the font definition. Fonts by themselves do not constitute content, instead they are used in glyph nodes. Further, fonts are never directly embedded in a content node; in a content node, a font is always specified by its font number. This limits the number of fonts that can be used in a HINT file to at most 256.

A long format font definition starts with the keyword “font” and is followed by the font number, as usual prefixed by an asterisk. Then comes the font specification with the font name, the font size, the section number of the T_EX font metric file, and the section number of the file containing the glyphs for the font. The HINT format supports .pk files, the traditional font format for T_EX, and the more modern PostScript Type 1 fonts, TrueType fonts, and OpenType fonts.

Starting with version 2.2, there is new support for TrueType and OpenType fonts while .pk fonts are considered deprecated. The previously mandatory .tfm file is no longer required for TrueType and OpenType fonts. Instead, the hint viewer is required to extract the necessary font metrics directly from the font files.

In a HINT file, text is represented as a sequence of numbers called character codes. HINT files use the UTF-8 character encoding scheme (CES) to map these numbers to their representation as byte sequences. For example the number “#E4” is encoded as the byte sequence “#C3 #A4”. The same number #E4 now can represent different characters depending on the coded character set (CCS). For example in the common ISO-8859-1 (Latin 1) encoding the number #E4 is the umlaut “ä” where as in the ISO-8859-7 (Latin/Greek) it is the Greek letter “δ” and in the EBCDIC encoding, used on IBM mainframes, it is the upper case letter “U”.

The character encoding is irrelevant for rendering a HINT file as long as the character codes in the glyph nodes are consistent with the character codes used in the font file, but the character encoding is necessary for all programs that need to “understand” the content of the HINT file. For example programs that want to translate a HINT document to a different language, or for text-to-speech conversion.

For FreeType and OpenType fonts, the HINT viewer is required to support only two character encodings: FT_ENCODING_ADOBE_CUSTOM, used for the traditional T_EX encoding schema of fonts; and FT_ENCODING_UNICODE used for TrueType and OpenType fonts that do not have a .tfm file along with them. To be precise: a font that has no .tfm file along with it must be encoded in Unicode. That is a glyph node will specify the unicode value of the desired character, which is then translated to the glyph number using FT_ENCODING_UNICODE. For ligature nodes, the node will specify the glyph number directly without a need to translate it further, but the replacement list of the ligature will contain the unicode values of the replacement characters.

In the short format we use the info value *b000* for a font with .tfm file and the info value *b001* for a font without .tfm file.

The Internet Engineering Task Force IETF has established a character set registry[14] that defines an enumeration of all registered coded character sets[3]. The coded character set numbers are in the range 1–2999. This encoding number, as given in [4], might be one possibility for specifying the font encoding as part of a font definition. But none such addition is planned at the moment.

Currently, it is only required that a font specifies an interword glue and a default discretionary break. After that comes a list of up to 12 font specific parameters.

The font size specifies the desired “at size” which might be different from the “design size” of the font as stored in the .tfm file.

In the short format, the font specification is given in the same order as in the long format.

Our internal representation of a font just stores the font name because in the long format we add the font name as a comment to glyph nodes.

```
< common variables 252cclii > +≡ (406)
  char **hfont_name; /* dynamically allocated array of font names */
```

```
< hint basic types 6vi > +≡ (407)
#define MAX_FONT_PARAMS 11
```

```
< initialize definitions 253ccliii > +≡ (408)
```

```
ALLOCATE(hfont_name, max_ref[font_kind] + 1, char *);
```

Reading the long format:

— — — $\Rightarrow$

$\langle$ symbols 2ii $\rangle + \equiv$ (409)

```
%token FONT "font"
```

```
%type < info > font font_head
```

$\langle$ scanning rules 3iii $\rangle + \equiv$ (410)

```
font          return FONT;
```

Note that we set the definition bit early because the definition of font f might involve glyphs that reference font f (or other fonts).

$\langle$ parsing rules 5v $\rangle + \equiv$ (411)

```
font: font_head font_param_list;
```

```
font_head: string dimension UNSIGNED UNSIGNED
```

```
{ uint8_t f = $ < u > 0;
```

```
  SET_DBIT(f, font_kind); hfont_name[f] = strdup($1);
```

```
  $$ = hput_font_head(f, hfont_name[f], $2, $3, $4); };
```

```
font_head: string dimension UNSIGNED
```

```
{ uint8_t f = $ < u > 0;
```

```
  SET_DBIT(f, font_kind); hfont_name[f] = strdup($1);
```

```
  $$ = hput_font_head(f, hfont_name[f], $2, -1, $3); };
```

```
font_param_list: glue_node disc_node | font_param_list font_param;
```

```
font_param:
```

```
  start PENALTY fref penalty END { hput_tags($1, hput_int($4)); }
```

```
| start KERN fref kern END { hput_tags($1, hput_kern(&($4))); }
```

```
| start LIGATURE fref ligature END { hput_tags($1, hput_ligature(&($4))); }
```

```
| start DISC fref disc END { hput_tags($1, hput_disc(&($4))); }
```

```
| start GLUE fref glue END { hput_tags($1, hput_glue(&($4))); }
```

```
| start LANGUAGE fref string END { hput_string($4);
```

```
  hput_tags($1, TAG(language_kind, 0)); }
```

```
| start RULE fref rule END { hput_tags($1, hput_rule(&($4))); }
```

```
| start IMAGE fref image END { hput_tags($1, TAG(image_kind, $4)); };
```

```
fref: ref
```

```
{ RNG("Font_parameter", $1, 0, MAX_FONT_PARAMS); };
```

Reading the short format:

$\dots \Rightarrow$

Writing the long format:

$\Rightarrow$ — — —

$\langle$ get functions 18xviii $\rangle + \equiv$ (412)

```
static void hget_font_params(void)
```

```
{ Disc h;
```



```

    hget_glue_node();
    hget_disc_node(&(h)); hwrite_disc_node(&h);
    DBG(DBGDEF, "Start_font_parameters\n");
    while (KIND(*hpos) ≠ font_kind)
    { Ref df;
      ⟨ read the start byte a 16xvi ⟩
      df.k = KIND(a);
      df.n = HGET8;
      DBG(DBGDEF, "Reading_font_parameter_d: s\n", df.n,
          definition_name[df.k]);
      if (df.k ≠ penalty_kind ∧ df.k ≠ kern_kind ∧ df.k ≠ ligature_kind ∧
          df.k ≠ disc_kind ∧ df.k ≠ glue_kind ∧ df.k ≠ language_kind ∧
          df.k ≠ rule_kind ∧ df.k ≠ image_kind)
          QUIT("Font_parameter_d_has_invalid_type_s", df.n,
              content_name[df.n]);
      RNG("Font_parameter", df.n, 0, MAX_FONT_PARAMS);
      hwrite_start(); hwritef("%s%d", content_name[KIND(a)], df.n);
      hget_definition(df.n, a, node_pos);
      hwrite_end();
      ⟨ read and check the end byte z 17xvii ⟩
    }
    DBG(DBGDEF, "End_font_parameters\n");
}

void hget_font_def(Info i, uint8_t f)
{ char *n; Dimen s = 0; uint16_t m = 0, y;
  HGET_STRING(n); hwrite_string(n); hfont_name[f] = strdup(n);
  HGET32(s); hwrite_dimension(s);
  DBG(DBGDEF, "Font_s_size_0x%x\n", n, s);
  if (i ≡ b000) { HGET16(m); RNG("Font_metrics", m, 3, max_section_no);
  }
  HGET16(y); RNG("Font_glyphs", y, 3, max_section_no);
  if (i ≡ b000) hwritef("%d", m);
  hwritef("%d", y);
  hget_font_params();
  DBG(DBGDEF, "End_font_definition\n");
}

```

Writing the short format:

$\Rightarrow \dots$

(413)

```

<put functions 14xiv> +≡
  Tag hput_font_head(uint8_t f, char *n, Dimen s,
    int m, uint16_t y)
  { Info i;
    DBG(DBGDEF, "Defining_font_d_(%s)_size_0x%x\n", f, n, s);
    hput_string(n);
    HPUT32(s); if (m ≥ 0) { i = b000;
      HPUT16(m);
    }
    else i = b001;
    HPUT16(y);
    return TAG(font_kind, i);
  }

```

10.5 References

We have seen how to make definitions, now let's see how to reference them. In the long form, we can simply write the reference number, after the keyword like this: "<glue *17>". The asterisk is necessary to keep apart, for example, a penalty with value 50, written "<penalty 50>", from a penalty referencing the integer definition number 50, written "<penalty *50>".

(414)

```

<hint types 1i> +≡
  typedef struct { Kind k; int n; } Ref;

```

Reading the long format:

$--- \Rightarrow$

Writing the short format:

$\Rightarrow \dots$

(415)

```

<parsing rules 5v> +≡
  xdimen_ref: ref { REF(xdimen_kind, $1); };
  param_ref:  ref { REF(param_kind, $1); };
  stream_ref: ref { REF_RNG(stream_kind, $1); };
  content_node: start PENALTY ref END
    { REF(penalty_kind, $3); hput_tags($1, TAG(penalty_kind, 0)); }
  | start KERN explicit ref END
    { REF(dimen_kind, $4); hput_tags($1, TAG(kern_kind, ($3) ? b100 : b000)); }
  | start KERN explicit XDIMEN ref END
    { REF(xdimen_kind, $5);
      hput_tags($1, TAG(kern_kind, ($3) ? b101 : b001)); }
  | start GLUE ref END
    { REF(glue_kind, $3); hput_tags($1, TAG(glue_kind, 0)); }
  | start LIGATURE ref END
    { REF(ligature_kind, $3); hput_tags($1, TAG(ligature_kind, 0)); }
  | start DISC ref END

```

```

    { REF(disc_kind, $3); hput_tags($1, TAG(disc_kind, 0)); }
| start RULE ref END
    { REF(rule_kind, $3); hput_tags($1, TAG(rule_kind, 0)); }
| start IMAGE ref END
    { REF(image_kind, $3); hput_tags($1, TAG(image_kind, 0)); }
| start LEADERS ref END
    { REF(leaders_kind, $3); hput_tags($1, TAG(leaders_kind, 0)); }
| start BASELINE ref END
    { REF(baseline_kind, $3); hput_tags($1, TAG(baseline_kind, 0)); }
| start LANGUAGE REFERENCE END
    { REF(language_kind, $3); hput_tags($1, hput_language($3)); };
glue_node: start GLUE ref END
    { REF(glue_kind, $3);
      if ($3 == zero_skip_no) { hpos = hpos - 2; $$ = false; }
      else { hput_tags($1, TAG(glue_kind, 0)); $$ = true; }
    };

```

Reading the short format:

... $\Rightarrow$

```

⟨ cases to get content 20xx ⟩ +≡ (416)
  case TAG(penalty_kind, 0): HGET_REF(penalty_kind); break;
  case TAG(kern_kind, b000): HGET_REF(dimen_kind); break;
  case TAG(kern_kind, b100): hwritef("_!"); HGET_REF(dimen_kind); break;
  case TAG(kern_kind, b001):
    hwritef("_xdimen"); HGET_REF(xdimen_kind); break;
  case TAG(kern_kind, b101):
    hwritef("_!_xdimen"); HGET_REF(xdimen_kind); break;
  case TAG(ligature_kind, 0): HGET_REF(ligature_kind); break;
  case TAG(disc_kind, 0): HGET_REF(disc_kind); break;
  case TAG(glue_kind, 0): HGET_REF(glue_kind); break;
  case TAG(language_kind, b000): HGET_REF(language_kind); break;
  case TAG(rule_kind, 0): HGET_REF(rule_kind); break;
  case TAG(image_kind, 0): HGET_REF(image_kind); break;
  case TAG(leaders_kind, 0): HGET_REF(leaders_kind); break;
  case TAG(baseline_kind, 0): HGET_REF(baseline_kind); break;

```

```

⟨ get macros 19xix ⟩ +≡ (417)
#define HGET_REF(K)
  { uint8_t n = HGET8; REF(K, n); hwrite_ref(n); }

```

Writing the long format:

$\Rightarrow$ — — —

⟨ write functions 21_{xxi} ⟩ +≡

(418)

```
void hwrite_ref(int n)
{ hwritef("_*%d", n); }
```

```
void hwrite_ref_node(Kind k, uint8_t n)
{ hwrite_start(); hwritef("%s", content_name[k]); hwrite_ref(n); hwrite_end();
}
```

11 Defaults

Several of the predefined values found in the definition section are used as parameters for the routines borrowed from T_EX to display the content of a HINT file. These values must be defined, but it is inconvenient if the same standard definitions need to be placed in each and every HINT file. Therefore we specify in this chapter reasonable default values. As a consequence, even a HINT file without any definitions should produce sensible results when displayed.

The definitions that have default values are integers, dimensions, extended dimensions, glues, baselines, labels, page templates, streams, and page ranges. Each of these defaults has its own subsection below. Actually the defaults for extended dimensions, baselines, and labels are not needed by T_EX's routines, but it is nice to have default values for the extended dimensions that represent **hsize**, **vsize**, a zero baseline skip, and a label for the table of content.

The array *max_default* contains for each kind-value the maximum number of the default values. The function *hset_max* is used to initialize them.

The programs **shrink** and **stretch** actually do not use the defaults, but it would be possible to suppress definitions if the defined value is the same as the default value. We start by setting *max_default*[*k*] $\equiv$ -1 , meaning no defaults, and *max_fixed*[*k*] $\equiv$ $\#10000$, meaning no definitions. The following subsections will then overwrite these values for all kinds of definitions that have defaults. It remains to reset *max_fixed* to -1 for all those kinds that have no defaults but allow definitions.

(take care of variables without defaults 419cdxix) $\equiv$ (419)
for (*k* = 0; *k* < 32; *k*++) *max_default*[*k*] = -1 , *max_fixed*[*k*] = $\#10000$;
max_fixed[*font_kind*] = *max_fixed*[*ligature_kind*] = *max_fixed*[*disc_kind*]
 = *max_fixed*[*language_kind*] = *max_fixed*[*rule_kind*] = *max_fixed*[*image_kind*]
 = *max_fixed*[*leaders_kind*] = *max_fixed*[*param_kind*] = *max_fixed*[*label_kind*]
 = -1 ;

Used in 549dxlix.

11.1 Integers

Integers are very simple objects, and it might be tempting not to use predefined integers at all. But the T_EX typesetting engine, which is used by HINT, uses many integer parameters to fine tune its operations. As we will see, all these integer parameters have a predefined integer number that refers to an integer definition.

Integers and penalties share the same kind-value. So a penalty node that references one of the predefined penalties, simply contains the integer number as a reference number.

The following integer numbers are predefined. The zero integer is fixed with integer number zero. The default values are taken from `plain.tex`.

`< default names 420cdxx > ≡` (420)

```
typedef enum {
    zero_int_no = 0, pretolerance_no = 1, tolerance_no = 2, line_penalty_no = 3,
    hyphen_penalty_no = 4, ex_hyphen_penalty_no = 5, club_penalty_no = 6,
    widow_penalty_no = 7, display_widow_penalty_no = 8, broken_penalty_no = 9,
    pre_display_penalty_no = 10, post_display_penalty_no = 11,
    inter_line_penalty_no = 12, double_hyphen_demerits_no = 13,
    final_hyphen_demerits_no = 14, adj_demerits_no = 15, looseness_no = 16,
    time_no = 17, day_no = 18, month_no = 19, year_no = 20,
    hang_after_no = 21, floating_penalty_no = 22
```

```
} Int_no;
```

```
#define MAX_INT_DEFAULT floating_penalty_no
```

Used in 548dxlviii.

`< define int_defaults 421cdxxi > ≡` (421)

```
max_default[int_kind] = MAX_INT_DEFAULT;
max_fixed[int_kind] = zero_int_no;
int_defaults[zero_int_no] = 0;
int_defaults[pretolerance_no] = 100;
int_defaults[tolerance_no] = 200;
int_defaults[line_penalty_no] = 10;
int_defaults[hyphen_penalty_no] = 50;
int_defaults[ex_hyphen_penalty_no] = 50;
int_defaults[club_penalty_no] = 150;
int_defaults[widow_penalty_no] = 150;
int_defaults[display_widow_penalty_no] = 50;
int_defaults[broken_penalty_no] = 100;
int_defaults[pre_display_penalty_no] = 10000;
int_defaults[post_display_penalty_no] = 0;
int_defaults[inter_line_penalty_no] = 0;
int_defaults[double_hyphen_demerits_no] = 10000;
int_defaults[final_hyphen_demerits_no] = 5000;
int_defaults[adj_demerits_no] = 10000;
int_defaults[looseness_no] = 0;
int_defaults[time_no] = 720;
int_defaults[day_no] = 4;
int_defaults[month_no] = 7;
int_defaults[year_no] = 1776;
int_defaults[hang_after_no] = 1;
int_defaults[floating_penalty_no] = 20000;
```

```
printf("int32_t int_defaults[MAX_INT_DEFAULT+1]={");
for (i = 0; i ≤ max_default[int_kind]; i++)
{ printf("%d", int_defaults[i]); if (i < max_default[int_kind]) printf(", "); }
printf("};\n\n");
```

Used in 549dxlix.

11.2 Dimensions

Notice that there are default values for the two dimensions `hsize` and `vsize`. These are the “design sizes” for the hint file. While it might not be possible to display the HINT file using these values of `hsize` and `vsize`, these are the author’s recommendation for the best “viewing experience”.

(default names 420cdxx) ≡ (422)

```
typedef enum {
    zero_dimen_no = 0, hsize_dimen_no = 1, vsize_dimen_no = 2,
    line_skip_limit_no = 3, max_depth_no = 4, split_max_depth_no = 5,
    hang_indent_no = 6, emergency_stretch_no = 7, quad_no = 8,
    math_quad_no = 9
} Dimen_no;
#define MAX_DIMEN_DEFAULT math_quad_no
```

(define dimen_defaults 423cdxxiii) ≡ (423)

```
max_default[dimen_kind] = MAX_DIMEN_DEFAULT;
max_fixed[dimen_kind] = zero_dimen_no;

dimen_defaults[zero_dimen_no] = 0;
dimen_defaults[hsize_dimen_no] = (Dimen)(6.5 * 72.27 * ONE);
dimen_defaults[vsize_dimen_no] = (Dimen)(8.9 * 72.27 * ONE);
dimen_defaults[line_skip_limit_no] = 0;
dimen_defaults[split_max_depth_no] = (Dimen)(3.5 * ONE);
dimen_defaults[hang_indent_no] = 0;
dimen_defaults[emergency_stretch_no] = 0;
dimen_defaults[quad_no] = 10 * ONE;
dimen_defaults[math_quad_no] = 10 * ONE;

printf("Dimen dimen_defaults[MAX_DIMEN_DEFAULT+1]={");
for (i = 0; i ≤ max_default[dimen_kind]; i++) {
    printf("0x%x", dimen_defaults[i]);
    if (i < max_default[dimen_kind]) printf(", ");
}
printf("};\n\n");
```

Used in 549dxlix.

11.3 Extended Dimensions

Extended dimensions can be used in a variety of nodes for example kern and box nodes. We define three fixed extended dimensions: `zero`, `hsize`, and `vsize`. In contrast to the `hsize` and `vsize` dimensions defined in the previous section, the extended dimensions defined here are linear functions that always evaluate to the current horizontal and vertical size in the viewer.

```
< default names 420cdxx > +≡ (424)
```

```
  \bgroup\def\enum{
    \def\zero_xdimen_no{0},\def\hsize_xdimen_no{1},\def\vsize_xdimen_no{2}
  } \Xdimen_no;
```

```
#define MAX_XDIMEN_DEFAULT \vsize_xdimen_no
```

```
< define xdimen_defaults 425cdxxv > ≡ (425)
```

```
  \max_default[xdimen_kind] = MAX_XDIMEN_DEFAULT;
  \max_fixed[xdimen_kind] = \vsize_xdimen_no;
  \print{"Xdimen_\xdimen_defaults[MAX_XDIMEN_DEFAULT+1]={"}
  "{0x0,\_0.0,\_0.0},\_0x0,\_1.0,\_0.0},\_0x0,\_0.0,\_1.0}"
  "};\n\n";
```

Used in 549dxlix.

11.4 Glue

There are predefined glue numbers that correspond to the skip parameters of T_EX. The default values are taken from `plain.tex`.

```
< default names 420cdxx > +≡ (426)
```

```
  \bgroup\def\enum{
    \def\zero_skip_no{0},\def\fil_skip_no{1},\def\fill_skip_no{2},\def\line_skip_no{3},
    \def\baseline_skip_no{4},\def\above_display_skip_no{5},\def\below_display_skip_no{6},
    \def\above_display_short_skip_no{7},\def\below_display_short_skip_no{8},
    \def\left_skip_no{9},\def\right_skip_no{10},\def\top_skip_no{11},\def\split_top_skip_no{12},
    \def\tab_skip_no{13},\def\par_fill_skip_no{14}
  } \Glue_no;
```

```
#define MAX_GLUE_DEFAULT \par_fill_skip_no
```

```
< define glue_defaults 427cdxxvii > ≡ (427)
```

```
  \max_default[glue_kind] = MAX_GLUE_DEFAULT;
  \max_fixed[glue_kind] = \fill_skip_no;
  \glue_defaults[\fil_skip_no].p.f = 1.0;
  \glue_defaults[\fil_skip_no].p.o = \fil_o;
  \glue_defaults[\fill_skip_no].p.f = 1.0;
  \glue_defaults[\fill_skip_no].p.o = \fill_o;
  \glue_defaults[\line_skip_no].w.w = 1 * ONE;
  \glue_defaults[\baseline_skip_no].w.w = 12 * ONE;
  \glue_defaults[\above_display_skip_no].w.w = 12 * ONE;
  \glue_defaults[\above_display_skip_no].p.f = 3.0;
  \glue_defaults[\above_display_skip_no].p.o = \normal_o;
```



```

glue_defaults[above_display_skip_no].m.f = 9.0;
glue_defaults[above_display_skip_no].m.o = normal_o;
glue_defaults[below_display_skip_no].w.w = 12 * ONE;
glue_defaults[below_display_skip_no].p.f = 3.0;
glue_defaults[below_display_skip_no].p.o = normal_o;
glue_defaults[below_display_skip_no].m.f = 9.0;
glue_defaults[below_display_skip_no].m.o = normal_o;
glue_defaults[above_display_short_skip_no].p.f = 3.0;
glue_defaults[above_display_short_skip_no].p.o = normal_o;
glue_defaults[below_display_short_skip_no].w.w = 7 * ONE;
glue_defaults[below_display_short_skip_no].p.f = 3.0;
glue_defaults[below_display_short_skip_no].p.o = normal_o;
glue_defaults[below_display_short_skip_no].m.f = 4.0;
glue_defaults[below_display_short_skip_no].m.o = normal_o;
glue_defaults[top_skip_no].w.w = 10 * ONE;
glue_defaults[split_top_skip_no].w.w = (Dimen) 8.5 * ONE;
glue_defaults[par_fill_skip_no].p.f = 1.0;
glue_defaults[par_fill_skip_no].p.o = fil_o;
#define PRINT_GLUE(G) printf("{0x%x, %f, %f}, {f, %d}, {f, %d}",
    G.w.w, G.w.h, G.w.v, G.p.f, G.p.o, G.m.f, G.m.o)

printf("Glue glue_defaults[MAX_GLUE_DEFAULT+1]={\n");
for (i = 0; i ≤ max_default[glue_kind]; i++)
{ PRINT_GLUE(glue_defaults[i]); if (i < max_default[int_kind]) printf(", \n");
}
printf("}; \n\n");

```

Used in 549dxlix.

We fix the glue definition with number zero to be the “zero glue”: a glue with width zero and zero stretchability and shrinkability. Here is the reason: In the short format, the info bits of a glue node indicate which components of a glue are nonzero. Therefore the zero glue should have an info value of zero—which on the other hand is reserved for a reference to a glue definition. Hence, the best way to represent a zero glue is as a predefined glue.

11.5 Baseline Skips

The zero baseline which inserts no baseline skip is predefined.

⟨ default names 420cdxx ⟩ +≡ (428)

```

typedef enum { zero_baseline_no = 0 } Baseline_no;
#define MAX_BASELINE_DEFAULT zero_baseline_no

```

⟨ define baseline_defaults 429cdxxix ⟩ ≡ (429)

```

max_default[baseline_kind] = MAX_BASELINE_DEFAULT;
max_fixed[baseline_kind] = zero_baseline_no;
{ Baseline z = {{0}};

```

```

printf("Baseline\baseline_defaults[MAX_BASELINE_DEFAULT+1]={"});
PRINT_GLUE(z.bs); printf(","); PRINT_GLUE(z.ls);
printf(",\0x%x}");\n\n", z.lsl);
}

```

Used in 549dxlix.

11.6 Labels

The zero label is predefined. It should point to the “home” position of the document which should be the position where a user can start reading or navigating the document. For a short document this is usually the start of the document, and hence, the default is the first position of the content section. For a larger document, the home position could point to the table of content where a reader will find links to other parts of the document.

```

⟨ default names 420cdxx ⟩ +≡ (430)

```

```

typedef enum { zero_label_no = 0 } Label_no;
#define MAX_LABEL_DEFAULT zero_label_no

```

```

⟨ define label_defaults 431cdxxxi ⟩ ≡ (431)

```

```

max_default[label_kind] = MAX_LABEL_DEFAULT;
printf("Label\label_defaults[MAX_LABEL_DEFAULT+1]="
      "{0,0,LABEL_TOP,true,0,0}");\n\n");

```

Used in 549dxlix.

11.7 Streams

The zero stream is predefined for the main content.

```

⟨ default names 420cdxx ⟩ +≡ (432)

```

```

typedef enum { zero_stream_no = 0 } Stream_no;
#define MAX_STREAM_DEFAULT zero_stream_no

```

```

⟨ define stream_defaults 433cdxxxiii ⟩ ≡ (433)

```

```

max_default[stream_kind] = MAX_STREAM_DEFAULT;
max_fixed[stream_kind] = zero_stream_no;

```

Used in 549dxlix.

11.8 Page Templates

The zero page template is a predefined, built-in page template.

```

⟨ default names 420cdxx ⟩ +≡ (434)

```

```

typedef enum { zero_page_no = 0 } Page_no;
#define MAX_PAGE_DEFAULT zero_page_no

```

```

⟨ define page_defaults 435cdxxxv ⟩ ≡ (435)

```

```

max_default[page_kind] = MAX_PAGE_DEFAULT;
max_fixed[page_kind] = zero_page_no;

```

Used in 549dxlix.


```
"\x00\xEE0000FF,\x00FFFFFF00,\n"          /* dark red on white */
"\x00\x00EE00FF,\x00FFFFFF00,\n"          /* dark green on white */
"\x00\x1111FFFF,\x0000000000,\n"          /* light blue on black */
"\x00\xFF1111FF,\x0000000000,\n"          /* light red on black */
"\x00\x11FF11FF,\x0000000000\n"          /* light green on black */
"}\};\n\n");
```

Used in [549](#)dxlix.

12 Content Section

The content section is just a list of nodes. Within the **shrink** program, reading a node in long format will trigger writing the node in short format. Similarly within the **stretch** program, reading a node in short form will cause writing it in long format. As a consequence, the main task of writing the content section in long format is accomplished by calling *get_content* and writing it in the short format is accomplished by parsing the *content_list*.

Reading the Long Format:

— — — $\Rightarrow$

$\langle$ symbols 2ii $\rangle + \equiv$ (442)
`%token CONTENT "content"`

$\langle$ scanning rules 3iii $\rangle + \equiv$ (443)
`content            return CONTENT;`

$\langle$ parsing rules 5v $\rangle + \equiv$ (444)
`content_section: START CONTENT { hput_content_start(); }
 content_list END
 { hput_content_end(); hput_range_defs(); hput_label_defs(); };`

Writing the Long Format:

$\Rightarrow$ — — —

$\langle$ write functions 21xxi $\rangle + \equiv$ (445)
`void hwrite_content_section(void)
{ section_no = 2;
 hwritef("<content");
 hsort_ranges();
 hsort_labels();
 hget_content_section();
 hwritef("\n>\n");
}`

Reading the Short Format:

... $\Rightarrow$

```

⟨ get functions 18xviii ⟩ +≡ (446)
void hget_content_section()
{
    DBG(DBGBASIC | DBGDIR, "Content\n");
    hget_section(2);
    hwrite_range();
    hwrite_label();
    while (hpos < hend) hget_content_node();
}

```

Writing the Short Format:

$\Rightarrow$...

```

⟨ put functions 14xiv ⟩ +≡ (447)
void hput_content_start(void)
{
    DBG(DBGDIR, "Content_Section\n");
    section_no = 2;
    hpos0 = hpos = hstart = dir[2].buffer;
    hend = hstart + dir[2].bsize;
}

void hput_content_end(void)
{
    dir[2].size = hpos - hstart; /* Updating the directory entry */
    DBG(DBGDIR, "End_Content_Section, size=0x%x\n", dir[2].size);
}

```

13 Processing the Command Line

The following code explains the command line parameters and options. It tells us what to expect in the rest of this section.

```
(explain usage 448cdxlviii) ≡ (448)
    fprintf(stdout, "Usage: %s [OPTION]... FILENAME%s\n", prog_name, in_ext);
    fprintf(stdout, DESCRIPTION);
    fprintf(stdout, "\nOptions:\n"
        "\t --help \t display this message\n"
        "\t --version\t display the HINT version\n"
        "\t -l \t redirect stderr to a log file\n"
    #if defined (STRETCH) ∨ defined (SHRINK)
        "\t -o FILE\t specify an output file name\n"
    #endif
    #if defined (STRETCH)
        "\t -a \t write auxiliary files\n"
        "\t -g \t do not use localized names (implies -a)\n"
        "\t -f \t force overwriting existing auxiliary files\n"
        "\t -u \t enable writing utf8 character codes\n"
        "\t -x \t enable writing hexadecimal character codes\n"
    #elif defined (SHRINK)
        "\t -a \t use only localized names\n"
        "\t -g \t do not use localized names\n"
        "\t -c \t enable compression\n"
    #endif
    );
    #ifdef DEBUG
        fprintf(stdout, "\t -d XXXX \t set debug flag to hexadec\
            imal value XXXX.\n" "\t\t\t OR together these values:\n");
        fprintf(stdout, "\t\t\t XX=%03X \t basic debugging\n", DBGBASIC);
        fprintf(stdout, "\t\t\t XX=%03X \t tag debugging\n", DBGTAGS);
        fprintf(stdout, "\t\t\t XX=%03X \t node debugging\n", DBGNODE);
        fprintf(stdout, "\t\t\t XX=%03X \t definition debugging\n", DBGDEF);
        fprintf(stdout, "\t\t\t XX=%03X \t directory debugging\n", DBGDIR);
        fprintf(stdout, "\t\t\t XX=%03X \t range debugging\n", DBGRANGE);
        fprintf(stdout, "\t\t\t XX=%03X \t float debugging\n", DBGFLOAT);
        fprintf(stdout, "\t\t\t XX=%03X \t compression debugging\n",
            DBGCOMPRESS);
```

```

    fprintf(stdout, "\t\t\t XX=%03X    buffer debugging\n", DBGBUFFER);
    fprintf(stdout, "\t\t\t XX=%03X    flex debugging\n", DBGFLEX);
    fprintf(stdout, "\t\t\t XX=%03X    bison debugging\n", DBGBISON);
    fprintf(stdout, "\t\t\t XX=%03X    TeX debugging\n", DBGTEX);
    fprintf(stdout, "\t\t\t XX=%03X    Page debugging\n", DBGPAGE);
    fprintf(stdout, "\t\t\t XX=%03X    Font debugging\n", DBGFONT);
    fprintf(stdout, "\t\t\t XX=%03X    Render debugging\n", DBGRENDER);
    fprintf(stdout, "\t\t\t XX=%03X    Label debugging\n", DBGLABEL);
#endif

```

Used in [452cdlii](#).

We define constants for different debug flags.

```

< debug constants 449cdxlix > ≡ (449)
#define DBGNONE #0
#define DBGBASIC #1
#define DBGTAGS #2
#define DBGNODE #4
#define DBGDEF #8
#define DBGDIR #10
#define DBGRANGE #20
#define DBGFLOAT #40
#define DBGCOMPRESS #80
#define DBGBUFFER #100
#define DBGFLEX #200
#define DBGBISON #400
#define DBGTEX #800
#define DBGPAGE #1000
#define DBGFONT #2000
#define DBGRENDER #4000
#define DBGLABEL #8000

```

Used in [548dxlviii](#).

Next we define common variables that are needed in all three programs defined here.

```

< common variables 252ccli > +≡ (450)
    unsigned int debugflags = DBGNONE;
    int option_utf8 = false;
    int option_hex = false;
    int option_force = false;
    int option_global = false;
    int option_aux = false;
    int option_compress = false;
    char *stem_name = NULL;
    int stem_length = 0;

```

The variable *stem_name* contains the name of the input file not including the extension. The space allocated for it is large enough to append an extension with

up to five characters. It can be used with the extension `.log` for the log file, with `.hint` or `.hnt` for the output file, and with `.abs` or `.rel` when writing or reading the auxiliary sections. The `stretch` program will overwrite the `stem_name` using the name of the output file if it is set with the `-o` option.

Next are the variables that are local in the *main* program.

(local variables in *main* 451cdli) $\equiv$ (451)

```
char *prog_name;
char *in_ext;
char *out_ext;
int option_log = false;
#ifdef SKIP
char *file_name = NULL;
int file_name_length = 0;
#endif
```

Used in 557dlvii, 558dlviii, and 560dlx.

Processing the command line looks for options and then sets the input file name. For compatibility with GNU standards, the long options `--help` and `--version` are supported in addition to the short options.

(process the command line 452cdlii) $\equiv$ (452)

```
debugflags = DBG_BASIC;
prog_name = argv[0];
if (argc < 2)
{ fprintf(stderr, "%s: no input file given\n" "Try '%s --help' for\n"
  more information\n", prog_name, prog_name);
  exit(1);
}
argv++; /* skip the program name */
while (*argv != NULL) {
  if ((*argv)[0] == '-') { char option = (*argv)[1];
    switch (option) {
      case '-':
        if (strcmp(*argv, "--version") == 0) { fprintf(stderr,
          "%s version %s\n", prog_name, HINT_VERSION_STRING);
          exit(0);
        }
        else if (strcmp(*argv, "--help") == 0) { explain usage 448cdxlviii }
        fprintf(stdout, "\nFor further information and reporting\n"
          bugs see https://hint.userweb.mwn.de/\n");
        exit(0);
      }
      case 'l': option_log = true; break;
    }
  }
  if defined (STRETCH) || defined (SHRINK)
    case 'o': argv++;
      file_name_length = (int) strlen(*argv);
```

```

        ALLOCATE(file_name, file_name.length + 6, char);    /* plus extension */
        strcpy(file_name, *argv); break;
    case 'g': option_global = option_aux = true; break;
    case 'a': option_aux = true; break;
#endif
#if defined (STRETCH)
    case 'u': option_utf8 = true; break;
    case 'x': option_hex = true; break;
    case 'f': option_force = true; break;
#elif defined (SHRINK)
    case 'c': option_compress = true; break;
#endif
case 'd':
    argv++;
    if (*argv == NULL) { fprintf(stderr, "%s: option_d expects\
        ts an argument\n" "Try '%s --help' for\
        more information\n", prog_name, prog_name);
        exit(1);
    }
    debugflags = strtol(*argv, NULL, 16);
    break;
default:
    { fprintf(stderr,
        "%s: unrecognized option '%s'\n" "Try '%s --help' for\
        more information\n", prog_name, *argv, prog_name);
        exit(1);
    }
}
}
else /* the input file name */
{ int path_length = (int) strlen(*argv);
  int ext_length = (int) strlen(in_ext);

  ALLOCATE(hin_name, path_length + ext_length + 1, char);
  strcpy(hin_name, *argv);
  if (path_length < ext_length ∨ strcmp(hin_name + path_length - ext_length,
      in_ext, ext_length) ≠ 0) { strcat(hin_name, in_ext);
      path_length += ext_length;
  }
  stem_length = path_length - ext_length;
  ALLOCATE(stem_name, stem_length + 6, char);
  strncpy(stem_name, hin_name, stem_length);
  stem_name[stem_length] = 0;
  if (*(argv + 1) ≠ NULL)
      { fprintf(stderr, "%s: extra argument after input file nam\

```

```

        e:_%s'\n'"Try_%s'--help' for more information\n",
        prog_name, *(argv + 1), prog_name);
    exit(1);
}
}
    argv++;
}
if (hin_name == NULL) { fprintf(stderr, "%s: missing input file\
    ile_name\n'"Try_%s'--help' for more information\n",
    prog_name, prog_name);
    exit(1);
}

```

Used in 557dlvii, 558dlviii, and 560dlx.

After the command line has been processed, three file streams need to be opened: The input file *hin* and the output file *hout*. Further we need a log file *hlog* if debugging is enabled. For technical reasons, the scanner generated by *flex* needs an input file *yyin* which is set to *hin* and an output file *yyout* (which is not used).

⟨ common variables 252cclii ⟩ += (453)
FILE *hin = NULL, *hout = NULL, *hlog = NULL;

The log file is opened first because this is the place where error messages should go while the other files are opened. It inherits its name from the input file name.

⟨ open the log file 454cdliv ⟩ = (454)
 if (option_log) { strcat(stem_name, ".log");
 hlog = freopen(stem_name, "w", stderr);
 if (hlog == NULL) {
 fprintf(stderr, "Unable to open logfile %s", stem_name);
 hlog = stderr;
 }
 stem_name[stem_length] = 0;
 }
 else hlog = stderr;

Used in 557dlvii, 558dlviii, and 560dlx.

Once we have established logging, we can try to open the other files.

⟨ open the input file 455cdlv ⟩ = (455)
 hin = fopen(hin_name, "rb");
 if (hin == NULL) QUIT("Unable to open input file %s", hin_name);

Used in 557dlvii.

⟨ open the output file 456cdlvi ⟩ = (456)
 if (file_name != NULL) { int ext_length = (int) strlen(out_ext);
 if (file_name.length ≤ ext_length ∨ strncmp(file_name + file_name.length -
 ext_length, out_ext, ext_length) ≠ 0) { strcat(file_name, out_ext);
 file_name.length += ext_length;
 }

```

}
else { file_name_length = stem_length + (int) strlen(out_ext);
    ALLOCATE(file_name, file_name_length + 1, char);
    strcpy(file_name, stem_name); strcpy(file_name + stem_length, out_ext);
}
{ char *aux_name = file_name;
  ⟨make sure the path in aux_name exists 365cdclxv⟩
  aux_name = NULL;
}
hout = fopen(file_name, "wb");
if (hout == NULL) QUIT("Unable to open output file %s", file_name);

```

Used in 557dlvii and 558dlviii.

The **stretch** program will replace the *stem_name* using the stem of the output file.

```

⟨determine the stem_name from the output file_name 457cdlvii⟩ ≡ (457)
  stem_length = file_name_length - (int) strlen(out_ext);
  ALLOCATE(stem_name, stem_length + 6, char);
  strncpy(stem_name, file_name, stem_length);
  stem_name[stem_length] = 0;

```

Used in 558dlviii.

At the very end, we will close the files again.

```

⟨close the input file 458cdlviii⟩ ≡ (458)
  if (hin_name != NULL) free(hin_name);
  if (hin != NULL) fclose(hin);

```

Used in 557dlvii.

```

⟨close the output file 459cdlix⟩ ≡ (459)
  if (file_name != NULL) free(file_name);
  if (hout != NULL) fclose(hout);

```

Used in 557dlvii and 558dlviii.

```

⟨close the log file 460cdlx⟩ ≡ (460)
  if (hlog != NULL) fclose(hlog);
  if (stem_name != NULL) free(stem_name);

```

Used in 557dlvii, 558dlviii, and 560dlx.

14 Error Handling and Debugging

There is no good program without good error handling. To print messages or indicate errors, I define the following macros:

```

<error.h 461cdlxi > ≡ (461)
#ifndef _ERROR_H
#define _ERROR_H
#include <stdlib.h>
#include <stdio.h>
extern FILE *hlog;
extern uint8_t *hpos, *hstart;
#ifndef LOG_PREFIX
#define LOG_PREFIX "HINT_"
#endif
#define LOG(...) (fprintf(hlog, LOG_PREFIX__VA_ARGS__, fflush(hlog))
#define MESSAGE(...) (fprintf(hlog, LOG_PREFIX__VA_ARGS__, fflush(hlog))
#define QUIT(...)
(MESSAGE("ERROR:_"__VA_ARGS__), fprintf(hlog, "\n"), exit(1))
#endif

```

The amount of debugging depends on the debugging flags. For portability, we first define the output specifier for expressions of type **size_t**.

```

<debug macros 383ccclxxxiii > +≡ (462)
#ifndef WIN32
#define SIZE_F "0x%x"
#else
#define SIZE_F "0x%tx"
#endif
#ifndef DEBUG
#define DBG(FLAGS, ...) ((debugflags & (FLAGS)) ? LOG(__VA_ARGS__) : 0)
#else
#define DBG(FLAGS, ...)(void) 0
#endif
#define DBGTAG(A, P) DBG(DBGTAGS, "tag_[%s,%d]_at_"SIZE_F"\n",
NAME(A), INFO(A), (P) - hstart)
#define RNG(S, N, A, Z)
if ((int)(N) < (int)(A) ∨ (int)(N) > (int)(Z))
QUIT(S " "d_out_of_range_"d-"d", N, A, Z)

```

```
#define TAGERR(A) QUIT("Unknown_tag[%s,%d]_at_"SIZE_F"\n",NAME(A),
    INFO(A),hpos - hstart)
```

The `bison` generated parser will need a function `yyerror` for error reporting. We can define it now:

```
<parsing functions 388ccclxxxviii > += (463)
extern int yylineno;
int yyerror(const char *msg)
{ QUIT("_in_line_%d_s",yylineno,msg);
  return 0;
}
```

To enable the generation of debugging code `bison` needs also the following:

```
<enable bison debugging 464cdlxiv > ≡ (464)
#ifdef DEBUG
#define YYDEBUG 1
extern int yydebug;
#else
#define YYDEBUG 0
#endif
```

Used in 555dlv, 556dlvi, and 557dlvii.

Appendix

A Traversing Short Format Files

For applications like searching or repositioning a file after reloading a possibly changed version of a file, it is useful to have a fast way of getting from one content node to the next. For quite some nodes, it is possible to know the size of the node from the tag. So the fastest way to get to the next node is looking up the node size in a table.

Other important nodes, for example hbox, vbox, or par nodes, end with a list node and it is possible to know the size of the node up to the final list. With that knowledge it is possible to skip the initial part of the node, then skip the list, and finally skip the tag byte. The size of the initial part can be stored in the same node size table using negated values. What works for lists, of course, will work for other kinds of nodes as well. So we use the lowest two bits of the values in the size table to store the number of embedded nodes that follow after the initial part. To combine the number of leading bytes and the number of trailing nodes into a single number that encodes both values according to this formula we use the macro `NODE_SIZE`. We can get back both values using the macros `NODE_HEAD` and `NODE_TAIL`.

```
< hint macros 13xiii > +≡ (465)
#define NODE_SIZE ( H , T) ((T) ≡ 0 ? (H) + 2 : -4 * ((H) + 1) + ((T) - 1))
#define NODE_HEAD(N) ((N) > 0 ? (N) - 2 : -((N) ≫ 2) - 1)
#define NODE_TAIL(N) ((N) < 0 ? ((N) & #3) + 1 : 0)
```

For list nodes neither of these methods works and these nodes can be marked with a zero entry in the node size table.

This leads to the following code for a “fast forward” function for *hpos*:

```
< shared skip functions 466cdlxvi > ≡ (466)
uint32_t hff_list_pos = 0, hff_list_size = 0;
Tag hff_tag;
```

```

void hff_hpos(void)
{ signed char i, b, n;
  hff_tag = *hpos; DBGTAG(hff_tag, hpos);
  i = hnode_size[hff_tag];
  if (i > 0) { hpos = hpos + NODE_HEAD(i) + 2; return; }
  else if (i < 0) { n = NODE_TAIL(i); b = NODE_HEAD(i);
    hpos = hpos + 1 + b; /* skip initial part */
    while (n > 0) { hff_hpos(); n--; } /* skip trailing nodes */
    hpos++; /* skip end byte */
    return;
  }
  else if (hff_tag ≤ TAG(param_kind, 7)) { advance hpos over a list 469cdlxix }
  TAGERR(hff_tag);
}

```

Used in 552dlII and 560dlx.

We will put the *hnode_size* variable into the `tables.c` file using the following function. We add some comments and split negative values into their components, to make the result more readable.

```

⟨print the hnode_size variable 467cdlxvii⟩ ≡ (467)
printf("signed_char_hnode_size[0x100]=\n");
for (i = 0; i ≤ #ff; i++)
{ signed char s = hnode_size[i];
  if (s ≥ 0) printf("%d", s);
  else printf("-4*d+%d", -(s >> 2), s & 3);
  if (i < #ff) printf(",");
  else printf("};");
  if ((i & #7) ≡ #7) printf("\/*%s*/\n", content_name[KIND(i)]);
}
printf("\n\n");

```

Used in 549dxlix.

When dealing with unknown content nodes, it is convenient to know which nodes are known and which are not. For this purpose the *content_known* array contains one byte for each kind value and each such bytes will indicate using the seven least significant bits for which info values the corresponding nodes are known.

```

⟨print the content_known variable 468cdlxviii⟩ ≡ (468)
for (k = 0; k < 32; k++)
  for (i = 0; i < 8; i++)
    if (hnode_size[TAG(k, i)] ≠ 0) content_known[k] |= (1 << i);
printf("uint8_t_content_known[32]=\n");
for (k = 0; k < 32; k++)
{ printf("0x%02X", content_known[k]);
  if (k < 31) printf(",");
  else printf("};");
  printf("\/*%s*/\n", content_name[k]);
}

```



```

}
printf("\n");

```

Used in 549dxlix.

A.1 Lists

List don't follow the usual schema of nodes. They have a variable size that is stored in the node. We keep position and size in global variables so that the list that ends a node can be conveniently located.

```

⟨ advance hpos over a list 469cdlxix ⟩ ≡ (469)
    switch (INFO(hff_tag) & #3) {
    case 0: hff_list_pos = hpos - hstart + 1;
            hff_list_size = 0;
            hpos = hpos + 3; return;
    case 1: hpos++; hff_list_size = HGET8; hff_list_pos = hpos - hstart + 1;
            hpos = hpos + 1 + hff_list_size + 1 + 1 + 1; return;
    case 2: hpos++; HGET16(hff_list_size); hff_list_pos = hpos - hstart + 1;
            hpos = hpos + 1 + hff_list_size + 1 + 2 + 1; return;
    case 3: hpos++; HGET32(hff_list_size); hff_list_pos = hpos - hstart + 1;
            hpos = hpos + 1 + hff_list_size + 1 + 4 + 1; return;
    default: QUIT("List with unknown info [%s,%d] at SIZE_F\n",
                 NAME(hff_tag), INFO(hff_tag), hpos - hstart);
    }

```

Used in 466cdlxvi.

Actually list nodes never occur as content nodes in their own right but only as subnodes of content nodes.

Now let's consider the different kinds of nodes.

A.2 Glyphs

We start with the glyph nodes. All glyph nodes have a start and an end tag, one byte for the font, and depending on the info from 1 to 4 bytes for the character code.

```

⟨ initialize the hnode_size array 470cdlxx ⟩ ≡ (470)
    hnode_size[TAG(glyph_kind, 1)] = NODE_SIZE(1 + 1, 0);
    hnode_size[TAG(glyph_kind, 2)] = NODE_SIZE(1 + 2, 0);
    hnode_size[TAG(glyph_kind, 3)] = NODE_SIZE(1 + 3, 0);
    hnode_size[TAG(glyph_kind, 4)] = NODE_SIZE(1 + 4, 0);

```

Used in 549dxlix.

A.3 Penalties

Penalty nodes either contain a one byte reference, a one byte number, or a two byte number.

```
( initialize the hnode_size array 470cdlxx ) +≡ (471)
  hnode_size[TAG(penalty_kind, 0)] = NODE_SIZE(1, 0);
  hnode_size[TAG(penalty_kind, 1)] = NODE_SIZE(1, 0);
  hnode_size[TAG(penalty_kind, 2)] = NODE_SIZE(2, 0);
  hnode_size[TAG(penalty_kind, 3)] = NODE_SIZE(4, 0);
```

A.4 Kerns

Kern nodes can contain a reference (either to a dimension or an extended dimension) followed by either a dimension or an extended dimension node.

```
( initialize the hnode_size array 470cdlxx ) +≡ (472)
  hnode_size[TAG(kern_kind, b000)] = NODE_SIZE(1, 0);
  hnode_size[TAG(kern_kind, b001)] = NODE_SIZE(1, 0);
  hnode_size[TAG(kern_kind, b010)] = NODE_SIZE(4, 0);
  hnode_size[TAG(kern_kind, b011)] = NODE_SIZE(0, 1);
  hnode_size[TAG(kern_kind, b100)] = NODE_SIZE(1, 0);
  hnode_size[TAG(kern_kind, b101)] = NODE_SIZE(1, 0);
  hnode_size[TAG(kern_kind, b110)] = NODE_SIZE(4, 0);
  hnode_size[TAG(kern_kind, b111)] = NODE_SIZE(0, 1);
```

A.5 Extended Dimensions

Extended dimensions contain either one two or three 4 byte values depending on the info bits.

```
( initialize the hnode_size array 470cdlxx ) +≡ (473)
  hnode_size[TAG(xdimen_kind, b100)] = NODE_SIZE(4, 0);
  hnode_size[TAG(xdimen_kind, b010)] = NODE_SIZE(4, 0);
  hnode_size[TAG(xdimen_kind, b001)] = NODE_SIZE(4, 0);
  hnode_size[TAG(xdimen_kind, b110)] = NODE_SIZE(4 + 4, 0);
  hnode_size[TAG(xdimen_kind, b101)] = NODE_SIZE(4 + 4, 0);
  hnode_size[TAG(xdimen_kind, b011)] = NODE_SIZE(4 + 4, 0);
  hnode_size[TAG(xdimen_kind, b111)] = NODE_SIZE(4 + 4 + 4, 0);
```

A.6 Language

Language nodes either code the language in the info value or they contain a reference byte.

```
( initialize the hnode_size array 470cdlxx ) +≡ (474)
  hnode_size[TAG(language_kind, b000)] = NODE_SIZE(1, 0);
  hnode_size[TAG(language_kind, 1)] = NODE_SIZE(0, 0);
  hnode_size[TAG(language_kind, 2)] = NODE_SIZE(0, 0);
  hnode_size[TAG(language_kind, 3)] = NODE_SIZE(0, 0);
  hnode_size[TAG(language_kind, 4)] = NODE_SIZE(0, 0);
```

```

hnode_size[TAG(language_kind, 5)] = NODE_SIZE(0, 0);
hnode_size[TAG(language_kind, 6)] = NODE_SIZE(0, 0);
hnode_size[TAG(language_kind, 7)] = NODE_SIZE(0, 0);

```

A.7 Rules

Rules usually contain a reference, otherwise they contain either one, two, or three 4 byte values depending on the info bits.

```

⟨ initialize the hnode_size array 470cdlxx ⟩ +≡ (475)
hnode_size[TAG(rule_kind, b000)] = NODE_SIZE(1, 0);
hnode_size[TAG(rule_kind, b100)] = NODE_SIZE(4, 0);
hnode_size[TAG(rule_kind, b010)] = NODE_SIZE(4, 0);
hnode_size[TAG(rule_kind, b001)] = NODE_SIZE(4, 0);
hnode_size[TAG(rule_kind, b110)] = NODE_SIZE(4 + 4, 0);
hnode_size[TAG(rule_kind, b101)] = NODE_SIZE(4 + 4, 0);
hnode_size[TAG(rule_kind, b011)] = NODE_SIZE(4 + 4, 0);
hnode_size[TAG(rule_kind, b111)] = NODE_SIZE(4 + 4 + 4, 0);

```

A.8 Glue

Glues usually contain a reference or they contain either one two or three 4 byte values depending on the info bits, and possibly even an extended dimension node followed by two 4 byte values.

```

⟨ initialize the hnode_size array 470cdlxx ⟩ +≡ (476)
hnode_size[TAG(glue_kind, b000)] = NODE_SIZE(1, 0);
hnode_size[TAG(glue_kind, b100)] = NODE_SIZE(4, 0);
hnode_size[TAG(glue_kind, b010)] = NODE_SIZE(4, 0);
hnode_size[TAG(glue_kind, b001)] = NODE_SIZE(4, 0);
hnode_size[TAG(glue_kind, b110)] = NODE_SIZE(4 + 4, 0);
hnode_size[TAG(glue_kind, b101)] = NODE_SIZE(4 + 4, 0);
hnode_size[TAG(glue_kind, b011)] = NODE_SIZE(4 + 4, 0);
hnode_size[TAG(glue_kind, b111)] = NODE_SIZE(4 + 4, 1);

```

A.9 Boxes

The layout of boxes is quite complex and explained in section 5.1. All boxes contain height and width, some contain a depth, some a shift amount, and some a glue setting together with glue sign and glue order. The last item in a box is a node list.

```

⟨ initialize the hnode_size array 470cdlxx ⟩ +≡ (477)
hnode_size[TAG(hbox_kind, b000)] = NODE_SIZE(4 + 4, 1); /* tag, height,
width */
hnode_size[TAG(hbox_kind, b001)] = NODE_SIZE(4 + 4 + 4, 1); /* and depth */
hnode_size[TAG(hbox_kind, b010)] = NODE_SIZE(4 + 4 + 4, 1); /* or shift */
hnode_size[TAG(hbox_kind, b011)] = NODE_SIZE(4 + 4 + 4 + 4, 1); /* or both */
hnode_size[TAG(hbox_kind, b100)] = NODE_SIZE(4 + 4 + 5, 1); /* and glue
setting */

```

```

hnode_size[TAG(hbox_kind, b101)] = NODE_SIZE(4 + 4 + 4 + 5, 1); /* and depth
*/
hnode_size[TAG(hbox_kind, b110)] = NODE_SIZE(4 + 4 + 4 + 5, 1); /* or shift */
hnode_size[TAG(hbox_kind, b111)] = NODE_SIZE(4 + 4 + 4 + 4 + 5, 1); /* or both
*/
hnode_size[TAG(vbox_kind, b000)] = NODE_SIZE(4 + 4, 1); /* same for vbox */
hnode_size[TAG(vbox_kind, b001)] = NODE_SIZE(4 + 4 + 4, 1);
hnode_size[TAG(vbox_kind, b010)] = NODE_SIZE(4 + 4 + 4, 1);
hnode_size[TAG(vbox_kind, b011)] = NODE_SIZE(4 + 4 + 4 + 4, 1);
hnode_size[TAG(vbox_kind, b100)] = NODE_SIZE(4 + 4 + 5, 1);
hnode_size[TAG(vbox_kind, b101)] = NODE_SIZE(4 + 4 + 4 + 5, 1);
hnode_size[TAG(vbox_kind, b110)] = NODE_SIZE(4 + 4 + 4 + 5, 1);
hnode_size[TAG(vbox_kind, b111)] = NODE_SIZE(4 + 4 + 4 + 4 + 5, 1);

```

A.10 Extended Boxes

Extended boxes start with height, width, depth, stretch, or shrink components. Then follows an extended dimension either as a reference or a node. The node ends with a list.

```

( initialize the hnode_size array 470cdlxx ) +=≡ (478)
hnode_size[TAG(hset_kind, b000)] = NODE_SIZE(4 + 4 + 4 + 4 + 1, 1);
hnode_size[TAG(hset_kind, b001)] = NODE_SIZE(4 + 4 + 4 + 4 + 4 + 1, 1);
hnode_size[TAG(hset_kind, b010)] = NODE_SIZE(4 + 4 + 4 + 4 + 4 + 1, 1);
hnode_size[TAG(hset_kind, b011)] = NODE_SIZE(4 + 4 + 4 + 4 + 4 + 4 + 1, 1);
hnode_size[TAG(vset_kind, b000)] = NODE_SIZE(4 + 4 + 4 + 4 + 1, 1);
hnode_size[TAG(vset_kind, b001)] = NODE_SIZE(4 + 4 + 4 + 4 + 4 + 1, 1);
hnode_size[TAG(vset_kind, b010)] = NODE_SIZE(4 + 4 + 4 + 4 + 4 + 1, 1);
hnode_size[TAG(vset_kind, b011)] = NODE_SIZE(4 + 4 + 4 + 4 + 4 + 4 + 1, 1);
hnode_size[TAG(hset_kind, b100)] = NODE_SIZE(4 + 4 + 4 + 4, 2);
hnode_size[TAG(hset_kind, b101)] = NODE_SIZE(4 + 4 + 4 + 4 + 4, 2);
hnode_size[TAG(hset_kind, b110)] = NODE_SIZE(4 + 4 + 4 + 4 + 4, 2);
hnode_size[TAG(hset_kind, b111)] = NODE_SIZE(4 + 4 + 4 + 4 + 4 + 4, 2);
hnode_size[TAG(vset_kind, b100)] = NODE_SIZE(4 + 4 + 4 + 4, 2);
hnode_size[TAG(vset_kind, b101)] = NODE_SIZE(4 + 4 + 4 + 4 + 4, 2);
hnode_size[TAG(vset_kind, b110)] = NODE_SIZE(4 + 4 + 4 + 4 + 4, 2);
hnode_size[TAG(vset_kind, b111)] = NODE_SIZE(4 + 4 + 4 + 4 + 4 + 4, 2);

```

The hpack and vpack nodes start with a shift amount and in case of vpack a depth. Then again an extended dimension and a list.

```

( initialize the hnode_size array 470cdlxx ) +=≡ (479)
hnode_size[TAG(hpack_kind, b000)] = NODE_SIZE(1, 1);
hnode_size[TAG(hpack_kind, b001)] = NODE_SIZE(1, 1);
hnode_size[TAG(hpack_kind, b010)] = NODE_SIZE(4 + 1, 1);
hnode_size[TAG(hpack_kind, b011)] = NODE_SIZE(4 + 1, 1);
hnode_size[TAG(vpack_kind, b000)] = NODE_SIZE(4 + 1, 1);
hnode_size[TAG(vpack_kind, b001)] = NODE_SIZE(4 + 1, 1);

```

```

hnode_size[TAG(vpack_kind, b010)] = NODE_SIZE(4 + 4 + 1, 1);
hnode_size[TAG(vpack_kind, b011)] = NODE_SIZE(4 + 4 + 1, 1);
hnode_size[TAG(hpack_kind, b100)] = NODE_SIZE(0, 2);
hnode_size[TAG(hpack_kind, b101)] = NODE_SIZE(0, 2);
hnode_size[TAG(hpack_kind, b110)] = NODE_SIZE(4, 2);
hnode_size[TAG(hpack_kind, b111)] = NODE_SIZE(4, 2);
hnode_size[TAG(vpack_kind, b100)] = NODE_SIZE(4, 2);
hnode_size[TAG(vpack_kind, b101)] = NODE_SIZE(4, 2);
hnode_size[TAG(vpack_kind, b110)] = NODE_SIZE(4 + 4, 2);
hnode_size[TAG(vpack_kind, b111)] = NODE_SIZE(4 + 4, 2);

```

A.11 Leaders

Most leader nodes will use a reference. Otherwise they contain a glue node followed by a box or rule node.

```

{ initialize the hnode_size array 470cdlxx } +≡ (480)
hnode_size[TAG(leaders_kind, b000)] = NODE_SIZE(1, 0);
hnode_size[TAG(leaders_kind, 1)] = NODE_SIZE(0, 1);
hnode_size[TAG(leaders_kind, 2)] = NODE_SIZE(0, 1);
hnode_size[TAG(leaders_kind, 3)] = NODE_SIZE(0, 1);
hnode_size[TAG(leaders_kind, b100 | 1)] = NODE_SIZE(0, 2);
hnode_size[TAG(leaders_kind, b100 | 2)] = NODE_SIZE(0, 2);
hnode_size[TAG(leaders_kind, b100 | 3)] = NODE_SIZE(0, 2);

```

A.12 Baseline Skips

Here we expect either a reference or two optional glue nodes followed by an optional dimension.

```

{ initialize the hnode_size array 470cdlxx } +≡ (481)
hnode_size[TAG(baseline_kind, b000)] = NODE_SIZE(1, 0);
hnode_size[TAG(baseline_kind, b001)] = NODE_SIZE(4, 0);
hnode_size[TAG(baseline_kind, b010)] = NODE_SIZE(0, 1);
hnode_size[TAG(baseline_kind, b100)] = NODE_SIZE(0, 1);
hnode_size[TAG(baseline_kind, b110)] = NODE_SIZE(0, 2);
hnode_size[TAG(baseline_kind, b011)] = NODE_SIZE(4, 1);
hnode_size[TAG(baseline_kind, b101)] = NODE_SIZE(4, 1);
hnode_size[TAG(baseline_kind, b111)] = NODE_SIZE(4, 2);

```

A.13 Ligatures

As usual a reference is possible, otherwise the font is followed by character bytes as given by the info. Only if the info value is 7, the number of character bytes is stored separately.

```

{ initialize the hnode_size array 470cdlxx } +≡ (482)
hnode_size[TAG(ligature_kind, b000)] = NODE_SIZE(1, 0);
hnode_size[TAG(ligature_kind, 1)] = NODE_SIZE(1 + 1, 0);

```

```

hnode_size[TAG(ligature_kind, 2)] = NODE_SIZE(1 + 2, 0);
hnode_size[TAG(ligature_kind, 3)] = NODE_SIZE(1 + 3, 0);
hnode_size[TAG(ligature_kind, 4)] = NODE_SIZE(1 + 4, 0);
hnode_size[TAG(ligature_kind, 5)] = NODE_SIZE(1 + 5, 0);
hnode_size[TAG(ligature_kind, 6)] = NODE_SIZE(1 + 6, 0);
hnode_size[TAG(ligature_kind, 7)] = NODE_SIZE(1, 1);

```

A.14 Discretionary breaks

The simple cases here are references, discretionary breaks with empty pre- and post-list, or with a zero line skip limit. Otherwise one or two lists are followed by an optional replace count.

```

⟨ initialize the hnode_size array 470cdlxx ⟩ +≡ (483)
hnode_size[TAG(disc_kind, b000)] = NODE_SIZE(1, 0);
hnode_size[TAG(disc_kind, b010)] = NODE_SIZE(0, 1);
hnode_size[TAG(disc_kind, b011)] = NODE_SIZE(0, 2);
hnode_size[TAG(disc_kind, b100)] = NODE_SIZE(1, 0);
hnode_size[TAG(disc_kind, b110)] = NODE_SIZE(1, 1);
hnode_size[TAG(disc_kind, b111)] = NODE_SIZE(1, 2);

```

A.15 Paragraphs

Paragraph nodes contain an extended dimension, an parameter list and a list. The first two can be given as a reference.

```

⟨ initialize the hnode_size array 470cdlxx ⟩ +≡ (484)
hnode_size[TAG(par_kind, b000)] = NODE_SIZE(1 + 1, 1);
hnode_size[TAG(par_kind, b010)] = NODE_SIZE(1, 2);
hnode_size[TAG(par_kind, b110)] = NODE_SIZE(0, 3);
hnode_size[TAG(par_kind, b100)] = NODE_SIZE(1, 2);

```

A.16 Mathematics

Displayed math needs a parameter list, either as list or as reference followed by an optional left or right equation number and a list. Text math is simpler: the only information is in the info value.

```

⟨ initialize the hnode_size array 470cdlxx ⟩ +≡ (485)
hnode_size[TAG(math_kind, b000)] = NODE_SIZE(1, 1);
hnode_size[TAG(math_kind, b001)] = NODE_SIZE(1, 2);
hnode_size[TAG(math_kind, b010)] = NODE_SIZE(1, 2);
hnode_size[TAG(math_kind, b100)] = NODE_SIZE(0, 2);
hnode_size[TAG(math_kind, b101)] = NODE_SIZE(0, 3);
hnode_size[TAG(math_kind, b110)] = NODE_SIZE(0, 3);
hnode_size[TAG(math_kind, b111)] = NODE_SIZE(0, 0);
hnode_size[TAG(math_kind, b011)] = NODE_SIZE(0, 0);

```

A.17 Adjustments

```
(initialize the hnode_size array 470cdlxx ) +≡ (486)
    hnode_size[TAG(adjust_kind, 1)] = NODE_SIZE(0, 1);
```

A.18 Tables

Tables have an extended dimension either as a node or as a reference followed by two lists.

```
(initialize the hnode_size array 470cdlxx ) +≡ (487)
    hnode_size[TAG(table_kind, b000)] = NODE_SIZE(1, 2);
    hnode_size[TAG(table_kind, b001)] = NODE_SIZE(1, 2);
    hnode_size[TAG(table_kind, b010)] = NODE_SIZE(1, 2);
    hnode_size[TAG(table_kind, b011)] = NODE_SIZE(1, 2);
    hnode_size[TAG(table_kind, b100)] = NODE_SIZE(0, 3);
    hnode_size[TAG(table_kind, b101)] = NODE_SIZE(0, 3);
    hnode_size[TAG(table_kind, b110)] = NODE_SIZE(0, 3);
    hnode_size[TAG(table_kind, b111)] = NODE_SIZE(0, 3);
```

Outer item nodes are lists of inner item nodes, inner item nodes are box nodes followed by an optional span count.

```
(initialize the hnode_size array 470cdlxx ) +≡ (488)
    hnode_size[TAG(item_kind, b000)] = NODE_SIZE(0, 1); /* outer */
    hnode_size[TAG(item_kind, 1)] = NODE_SIZE(0, 1); /* inner */
    hnode_size[TAG(item_kind, 2)] = NODE_SIZE(0, 1);
    hnode_size[TAG(item_kind, 3)] = NODE_SIZE(0, 1);
    hnode_size[TAG(item_kind, 4)] = NODE_SIZE(0, 1);
    hnode_size[TAG(item_kind, 5)] = NODE_SIZE(0, 1);
    hnode_size[TAG(item_kind, 6)] = NODE_SIZE(0, 1);
    hnode_size[TAG(item_kind, 7)] = NODE_SIZE(1, 1);
```

A.19 Images

If not given by a reference, images contain a section reference and optional dimensions and a descriptive list.

```
(initialize the hnode_size array 470cdlxx ) +≡ (489)
    hnode_size[TAG(image_kind, b000)] = NODE_SIZE(1, 0);
    hnode_size[TAG(image_kind, b001)] = NODE_SIZE(2 + 4 + 4, 1);
    hnode_size[TAG(image_kind, b010)] = NODE_SIZE(2 + 4 + 4, 1);
    hnode_size[TAG(image_kind, b011)] = NODE_SIZE(2 + 4 + 4, 1);
    hnode_size[TAG(image_kind, b100)] = NODE_SIZE(2 + 4 + 1 + 1, 1);
    hnode_size[TAG(image_kind, b101)] = NODE_SIZE(2 + 4 + 1, 2);
    hnode_size[TAG(image_kind, b110)] = NODE_SIZE(2 + 4 + 1, 2);
    hnode_size[TAG(image_kind, b111)] = NODE_SIZE(2 + 4, 3);
```

A.20 Links

Links contain either a 2 byte or a 1 byte reference and possibly a color reference.

```

⟨ initialize the hnode_size array 470cdlxx ⟩ +≡ (490)
  hnode_size[TAG(link_kind, b000)] = NODE_SIZE(1, 0);
  hnode_size[TAG(link_kind, b001)] = NODE_SIZE(2, 0);
  hnode_size[TAG(link_kind, b010)] = NODE_SIZE(1, 0);
  hnode_size[TAG(link_kind, b011)] = NODE_SIZE(2, 0);
  hnode_size[TAG(link_kind, b100)] = NODE_SIZE(2, 0);
  hnode_size[TAG(link_kind, b101)] = NODE_SIZE(3, 0);
  hnode_size[TAG(link_kind, b110)] = NODE_SIZE(2, 0);
  hnode_size[TAG(link_kind, b111)] = NODE_SIZE(3, 0);

```

A.21 Streams

After the stream reference follows a parameter list, either as reference or as a list, and then a content list.

```

⟨ initialize the hnode_size array 470cdlxx ⟩ +≡ (491)
  hnode_size[TAG(stream_kind, b000)] = NODE_SIZE(1 + 1, 1);
  hnode_size[TAG(stream_kind, b010)] = NODE_SIZE(1, 2);
  hnode_size[TAG(stream_kind, b100)] = NODE_SIZE(1, 0);

```

A.22 Colors

```

⟨ initialize the hnode_size array 470cdlxx ⟩ +≡ (492)
  hnode_size[TAG(color_kind, b000)] = NODE_SIZE(1, 0);

```


B Reading Short Format Files Backwards

This section is not really part of the file format definition, but it illustrates an important property of the content section in short format files: it can be read in both directions. This is important because we want to be able to start at an arbitrary point in the content and from there move pagewise backward.

The program `skip` described in this section does just that. As we see in appendix C.12, its *main* program is almost the same as the *main* program of the program `stretch` in appendix C.11. The major difference is the removal of an output file and the replacement of the call to `hwrite_content_section` by a call to `hteg_content_section`.

```

⟨ skip functions 493cdxciii ⟩ ≡ (493)
    static void hteg_content_section(void)
    { hget_section(2);
      hpos = hend;
      while (hpos > hstart) hteg_content_node();
    }

```

Used in 560dlx.

The functions `hteg_content_section` and `hteg_content_node` above are reverse versions of the functions `hget_content_section` and `hget_content_node`. Many such “reverse functions” will follow now and we will consistently use the same naming scheme: replacing “*get*” by “*teg*” or “*GET*” by “*TEG*”. The `skip` program does not do much input checking; it will just extract enough information from a content node to skip a node and “advance” or better “retreat” to the previous node.

```

⟨ skip functions 493cdxciii ⟩ +≡ (494)
    static void hteg_content_node(void)
    { ⟨ skip the end byte z 495cdxcv ⟩
      hteg_content(z);
      ⟨ skip and check the start byte a 496cdxcvi ⟩
    }

    static void hteg_content(Tag z)
    { switch (z)
      { ⟨ cases to skip content 503diii ⟩
        default:
          if (¬hteg_unknown(z)) TAGERR(z);
          break;
      }
    }

```

```
}
```

The code to skip the end byte z and to check the start byte a is used repeatedly.

```
< skip the end byte  $z$  495cdxcv >  $\equiv$  (495)
```

```
    Tag  $a$ ,  $z$ ; /* the start and the end byte */
    uint32_t node_pos = hpos - hstart;
    if (hpos  $\leq$  hstart) return;
    HTEGTAG( $z$ );
```

Used in 494cdxciv, 500d, 511dxi, 514dxiv, 517dxvii, and 538dxxxviii.

```
< skip and check the start byte  $a$  496cdxcvi >  $\equiv$  (496)
```

```
    HTEGTAG( $a$ );
    if ( $a \neq z$ )
        QUIT("Tag_mismatch_ [%s,%d] != [%s,%d]_at_ \"SIZE_F\"_to_0x%x\n",
            NAME( $a$ ), INFO( $a$ ), NAME( $z$ ), INFO( $z$ ),
            hpos - hstart, node_pos - 1);
```

Used in 494cdxciv, 500d, 511dxi, 514dxiv, 517dxvii, and 538dxxxviii.

We replace the “GET” macros by the following “TEG” macros:

```
< shared get macros 38xxxviii >  $\equiv$  (497)
```

```
#define HBACK( $X$ )
    ((hpos - ( $X$ ) < hstart) ? (QUIT("HTEG_underflow\n"), NULL) : (hpos -= ( $X$ )))
#define HTEG8 (HBACK(1), hpos[0])
#define HTEG16( $X$ ) (HBACK(2), ( $X$ ) = (hpos[0]  $\ll$  8) + hpos[1])
#define HTEG24( $X$ ) (HBACK(3), ( $X$ ) = (hpos[0]  $\ll$  16) + (hpos[1]  $\ll$  8) + hpos[2])
#define HTEG32( $X$ )
    (HBACK(4), ( $X$ ) = (hpos[0]  $\ll$  24) + (hpos[1]  $\ll$  16) + (hpos[2]  $\ll$  8) + hpos[3])
#define HTEGTAG( $X$ )  $X$  = HTEG8, DBGTAG( $X$ , hpos)
```

Now we review step by step the different kinds of nodes.

B.1 Floating Point Numbers

```
< shared skip functions 466cdlxvi >  $\equiv$  (498)
```

```
float32_t hteg_float32(void)
{ union { float32_t  $d$ ; uint32_t bits; }  $u$ ;
    HTEG32( $u$ .bits);
    return  $u$ . $d$ ;
}
```

B.2 Extended Dimensions

⟨ skip macros 499cdxcix ⟩ ≡ (499)

```
#define HTEG_XDIMEN(I, X)
    if ((I) & b001) HTEG32((X).v);
    if ((I) & b010) HTEG32((X).h);
    if ((I) & b100) HTEG32((X).w);
```

Used in 549dxlix and 560dlx.

⟨ skip functions 493cdxciii ⟩ +≡ (500)

```
static void hteg_xdimen_node(Xdimen *x)
{ ⟨ skip the end byte z 495cdxcv ⟩
  switch (z) {
# if 0 /* currently the info value 0 is not supported */
    case TAG(xdimen_kind, b000): /* see section 10.5 */
      { uint8_t n; n = HTEG8;
        } break;
# endif
    case TAG(xdimen_kind, b001): HTEG_XDIMEN(b001, *x); break;
    case TAG(xdimen_kind, b010): HTEG_XDIMEN(b010, *x); break;
    case TAG(xdimen_kind, b011): HTEG_XDIMEN(b011, *x); break;
    case TAG(xdimen_kind, b100): HTEG_XDIMEN(b100, *x); break;
    case TAG(xdimen_kind, b101): HTEG_XDIMEN(b101, *x); break;
    case TAG(xdimen_kind, b110): HTEG_XDIMEN(b110, *x); break;
    case TAG(xdimen_kind, b111): HTEG_XDIMEN(b111, *x); break;
    default: QUIT("Extent expected at 0x%x got %s", node_pos, NAME(z));
             break;
  }
  ⟨ skip and check the start byte a 496cdxcvi ⟩
}
```

B.3 Stretch and Shrink

⟨ skip macros 499cdxcix ⟩ +≡ (501)

```
#define HTEG_STRETCH(S)
    { Stch st; HTEG32(st.u); S.o = st.u & 3; st.u &= ~3; S.f = st.f; }
```

B.4 Glyphs

⟨ skip macros 499cdxcix ⟩ +≡ (502)

```
#define HTEG_GLYPH(I, G) (G).f = HTEG8;
    if (I ≡ 1) (G).c = HTEG8;
    else if (I ≡ 2) HTEG16((G).c);
    else if (I ≡ 3) HTEG24((G).c);
    else if (I ≡ 4) HTEG32((G).c);
```

⟨ cases to skip content 503diii ⟩ ≡ (503)

```

case TAG(glyph_kind, 1): { Glyph g; HTEG_GLYPH(1, g); } break;
case TAG(glyph_kind, 2): { Glyph g; HTEG_GLYPH(2, g); } break;
case TAG(glyph_kind, 3): { Glyph g; HTEG_GLYPH(3, g); } break;
case TAG(glyph_kind, 4): { Glyph g; HTEG_GLYPH(4, g); } break;

```

Used in 494cdxciv.

B.5 Penalties

⟨ skip macros 499cdxcix ⟩ +≡ (504)

```

#define HTEG_PENALTY(I, P)
  if (I ≡ 1) { int8_t n; n = HTEG8; P = n; }
  else if (I ≡ 2) { int16_t n; HTEG16(n); P = n; }
  else if (I ≡ 3) { int32_t n; HTEG32(n); P = n; }

```

⟨ cases to skip content 503diii ⟩ +≡ (505)

```

case TAG(penalty_kind, 1): { int32_t p; HTEG_PENALTY(1, p); } break;
case TAG(penalty_kind, 2): { int32_t p; HTEG_PENALTY(2, p); } break;
case TAG(penalty_kind, 3): { int32_t p; HTEG_PENALTY(2, p); } break;

```

B.6 Kerns

⟨ skip macros 499cdxcix ⟩ +≡ (506)

```

#define HTEG_KERN(I, X)
  if (((I) & b011) ≡ 2) HTEG32(X.w);
  else if (((I) & b011) ≡ 3) hteg_xdimen_node(&(X))

```

⟨ cases to skip content 503diii ⟩ +≡ (507)

```

case TAG(kern_kind, b010): { Xdimen x; HTEG_KERN(b010, x); } break;
case TAG(kern_kind, b011): { Xdimen x; HTEG_KERN(b011, x); } break;
case TAG(kern_kind, b110): { Xdimen x; HTEG_KERN(b110, x); } break;
case TAG(kern_kind, b111): { Xdimen x; HTEG_KERN(b111, x); } break;

```

B.7 Language

⟨ cases to skip content 503diii ⟩ +≡ (508)

```

case TAG(language_kind, 1): case TAG(language_kind, 2):
case TAG(language_kind, 3): case TAG(language_kind, 4):
case TAG(language_kind, 5): case TAG(language_kind, 6):
case TAG(language_kind, 7): break;

```

B.8 Rules

```
< skip macros 499cdxcix > +≡ (509)
```

```
#define HTEG_RULE(I, R)
  if ((I) & b001) HTEG32((R).w); else (R).w = RUNNING_DIMEN;
  if ((I) & b010) HTEG32((R).d); else (R).d = RUNNING_DIMEN;
  if ((I) & b100) HTEG32((R).h); else (R).h = RUNNING_DIMEN;
```

```
< cases to skip content 503dihi > +≡ (510)
```

```
case TAG(rule_kind, b011): { Rule r; HTEG_RULE(b011, r); } break;
case TAG(rule_kind, b101): { Rule r; HTEG_RULE(b101, r); } break;
case TAG(rule_kind, b001): { Rule r; HTEG_RULE(b001, r); } break;
case TAG(rule_kind, b110): { Rule r; HTEG_RULE(b110, r); } break;
case TAG(rule_kind, b111): { Rule r; HTEG_RULE(b111, r); } break;
```

```
< skip functions 493cdxciii > +≡ (511)
```

```
static void hteg_rule_node(void)
{ < skip the end byte z 495cdxcv >
  if (KIND(z) ≡ rule_kind) { Rule r; HTEG_RULE(INFO(z), r); }
  else QUIT("Rule expected at 0x%x got %s", node_pos, NAME(z));
  < skip and check the start byte a 496cdxcvi >
}
```

B.9 Glue

```
< skip macros 499cdxcix > +≡ (512)
```

```
#define HTEG_GLUE(I, G)
  if (I ≡ b111) hteg_xdimen_node(&((G).w));
  else (G).w.h = (G).w.v = 0.0;
  if ((I) & b001) HTEG_STRETCH((G).m) else (G).m.f = 0.0, (G).m.o = 0;
  if ((I) & b010) HTEG_STRETCH((G).p) else (G).p.f = 0.0, (G).p.o = 0;
  if ((I) ≠ b111) { if ((I) & b100) HTEG32((G).w.w); else (G).w.w = 0; }
```

```
< cases to skip content 503dihi > +≡ (513)
```

```
case TAG(glue_kind, b001): { Glue g; HTEG_GLUE(b001, g); } break;
case TAG(glue_kind, b010): { Glue g; HTEG_GLUE(b010, g); } break;
case TAG(glue_kind, b011): { Glue g; HTEG_GLUE(b011, g); } break;
case TAG(glue_kind, b100): { Glue g; HTEG_GLUE(b100, g); } break;
case TAG(glue_kind, b101): { Glue g; HTEG_GLUE(b101, g); } break;
case TAG(glue_kind, b110): { Glue g; HTEG_GLUE(b110, g); } break;
case TAG(glue_kind, b111): { Glue g; HTEG_GLUE(b111, g); } break;
```

```
< skip functions 493cdxciii > +≡ (514)
```

```
static void hteg_glue_node(void)
{ < skip the end byte z 495cdxcv >
  if (INFO(z) ≡ b000) HTEG_REF(glue_kind);
  else { Glue g; HTEG_GLUE(INFO(z), g); }
```

```

    < skip and check the start byte a 496cdxcvi >
}

```

B.10 Boxes

```

< skip macros 499cdxcix > +≡ (515)

```

```

#define HTEG_BOX(I, B) hteg_list (&(B.l));
    if ((I) & b100)
    { B.s = HTEG8; B.r = hteg_float32(); B.o = B.s & #F; B.s = B.s >> 4; }
    else { B.r = 0.0; B.o = B.s = 0; }
    if ((I) & b010) HTEG32(B.a); else B.a = 0;
    HTEG32(B.w);
    if ((I) & b001) HTEG32(B.d); else B.d = 0;
    HTEG32(B.h);

```

```

< cases to skip content 503diii > +≡ (516)

```

```

    case TAG(hbox_kind, b000): { Box b; HTEG_BOX(b000, b); } break;
    case TAG(hbox_kind, b001): { Box b; HTEG_BOX(b001, b); } break;
    case TAG(hbox_kind, b010): { Box b; HTEG_BOX(b010, b); } break;
    case TAG(hbox_kind, b011): { Box b; HTEG_BOX(b011, b); } break;
    case TAG(hbox_kind, b100): { Box b; HTEG_BOX(b100, b); } break;
    case TAG(hbox_kind, b101): { Box b; HTEG_BOX(b101, b); } break;
    case TAG(hbox_kind, b110): { Box b; HTEG_BOX(b110, b); } break;
    case TAG(hbox_kind, b111): { Box b; HTEG_BOX(b111, b); } break;
    case TAG(vbox_kind, b000): { Box b; HTEG_BOX(b000, b); } break;
    case TAG(vbox_kind, b001): { Box b; HTEG_BOX(b001, b); } break;
    case TAG(vbox_kind, b010): { Box b; HTEG_BOX(b010, b); } break;
    case TAG(vbox_kind, b011): { Box b; HTEG_BOX(b011, b); } break;
    case TAG(vbox_kind, b100): { Box b; HTEG_BOX(b100, b); } break;
    case TAG(vbox_kind, b101): { Box b; HTEG_BOX(b101, b); } break;
    case TAG(vbox_kind, b110): { Box b; HTEG_BOX(b110, b); } break;
    case TAG(vbox_kind, b111): { Box b; HTEG_BOX(b111, b); } break;

```

```

< skip functions 493cdxciii > +≡ (517)

```

```

static void hteg_hbox_node(void)
{ Box b;
    < skip the end byte z 495cdxcv >
    if (KIND(z) ≠ hbox_kind)
        QUIT("Hbox expected at 0x%x got %s", node_pos, NAME(z));
    HTEG_BOX(INFO(z), b);
    < skip and check the start byte a 496cdxcvi >
}

static void hteg_vbox_node(void)
{ Box b;

```

```

    < skip the end byte z 495cdxcv >
    if (KIND(z) ≠ vbox_kind)
        QUIT("Vbox expected at 0x%x got %s", node_pos, NAME(z));
    HTEG_BOX(INFO(z), b);
    < skip and check the start byte a 496cdxcvi >
}

```

B.11 Extended Boxes

< skip macros 499cdxcix > +≡ (518)

```

#define HTEG_SET(I)
{ List l; hteg_list(&l); }

if ((I) & b100) { Xdimen x; hteg_xdimen_node(&x); }
else HTEG_REF(xdimen_kind);
{ Stretch m; HTEG_STRETCH(m); }
{ Stretch p; HTEG_STRETCH(p); }
if ((I) & b010) { Dimen a; HTEG32(a); }
{ Dimen w; HTEG32(w); }
{ Dimen d; if ((I) & b001) HTEG32(d); else d = 0; }
{ Dimen h; HTEG32(h); }

#define HTEG_PACK(K, I)
{ List l; hteg_list(&l); }

if ((I) & b100) { Xdimen x;
    hteg_xdimen_node(&x); } else HTEG_REF(xdimen_kind);
if ((I) & b010) { Dimen d; HTEG32(d); }
if (K ≡ vpack_kind) { Dimen d; HTEG32(d); }

```

< cases to skip content 503diii > +≡ (519)

```

case TAG(hset_kind, b000): HTEG_SET(b000); break;
case TAG(hset_kind, b001): HTEG_SET(b001); break;
case TAG(hset_kind, b010): HTEG_SET(b010); break;
case TAG(hset_kind, b011): HTEG_SET(b011); break;
case TAG(hset_kind, b100): HTEG_SET(b100); break;
case TAG(hset_kind, b101): HTEG_SET(b101); break;
case TAG(hset_kind, b110): HTEG_SET(b110); break;
case TAG(hset_kind, b111): HTEG_SET(b111); break;
case TAG(vset_kind, b000): HTEG_SET(b000); break;
case TAG(vset_kind, b001): HTEG_SET(b001); break;
case TAG(vset_kind, b010): HTEG_SET(b010); break;
case TAG(vset_kind, b011): HTEG_SET(b011); break;
case TAG(vset_kind, b100): HTEG_SET(b100); break;
case TAG(vset_kind, b101): HTEG_SET(b101); break;
case TAG(vset_kind, b110): HTEG_SET(b110); break;
case TAG(vset_kind, b111): HTEG_SET(b111); break;

```

```

case TAG(hpack_kind, b000): HTEG_PACK(hpack_kind, b000); break;
case TAG(hpack_kind, b001): HTEG_PACK(hpack_kind, b001); break;
case TAG(hpack_kind, b010): HTEG_PACK(hpack_kind, b010); break;
case TAG(hpack_kind, b011): HTEG_PACK(hpack_kind, b011); break;
case TAG(hpack_kind, b100): HTEG_PACK(hpack_kind, b100); break;
case TAG(hpack_kind, b101): HTEG_PACK(hpack_kind, b101); break;
case TAG(hpack_kind, b110): HTEG_PACK(hpack_kind, b110); break;
case TAG(hpack_kind, b111): HTEG_PACK(hpack_kind, b111); break;

case TAG(vpack_kind, b000): HTEG_PACK(vpack_kind, b000); break;
case TAG(vpack_kind, b001): HTEG_PACK(vpack_kind, b001); break;
case TAG(vpack_kind, b010): HTEG_PACK(vpack_kind, b010); break;
case TAG(vpack_kind, b011): HTEG_PACK(vpack_kind, b011); break;
case TAG(vpack_kind, b100): HTEG_PACK(vpack_kind, b100); break;
case TAG(vpack_kind, b101): HTEG_PACK(vpack_kind, b101); break;
case TAG(vpack_kind, b110): HTEG_PACK(vpack_kind, b110); break;
case TAG(vpack_kind, b111): HTEG_PACK(vpack_kind, b111); break;

```

B.12 Leaders

⟨ skip macros 499cdxcix ⟩ +≡ (520)

```

#define HTEG_LEADERS(I)
  if (KIND(hpos[-1]) ≡ rule_kind) hteg_rule_node();
  else if (KIND(hpos[-1]) ≡ hbox_kind) hteg_hbox_node();
  else hteg_vbox_node();
  if ((I) & b100) hteg_glue_node();

```

⟨ cases to skip content 503diii ⟩ +≡ (521)

```

case TAG(leaders_kind, 1): HTEG_LEADERS(1); break;
case TAG(leaders_kind, 2): HTEG_LEADERS(2); break;
case TAG(leaders_kind, 3): HTEG_LEADERS(3); break;
case TAG(leaders_kind, b100 | 1): HTEG_LEADERS(b100 | 1); break;
case TAG(leaders_kind, b100 | 2): HTEG_LEADERS(b100 | 2); break;
case TAG(leaders_kind, b100 | 3): HTEG_LEADERS(b100 | 3); break;

```

B.13 Baseline Skips

⟨ skip macros 499cdxcix ⟩ +≡ (522)

```

#define HTEG_BASELINE(I, B)
  if ((I) & b010) hteg_glue_node();
  else { B.ls.p.o = B.ls.m.o = B.ls.w.w = 0;
        B.ls.w.h = B.ls.w.v = B.ls.p.f = B.ls.m.f = 0.0; }
  if ((I) & b100) hteg_glue_node();
  else { B.bs.p.o = B.bs.m.o = B.bs.w.w = 0;
        B.bs.w.h = B.bs.w.v = B.bs.p.f = B.bs.m.f = 0.0; }
  if ((I) & b001) HTEG32(B).lsl; else B.lsl = 0;

```



```

⟨ cases to skip content 503diii ⟩ +≡ (523)
  case TAG(baseline_kind, b001): { Baseline b; HTEG_BASELINE(b001, b); }
    break;
  case TAG(baseline_kind, b010): { Baseline b; HTEG_BASELINE(b010, b); }
    break;
  case TAG(baseline_kind, b011): { Baseline b; HTEG_BASELINE(b011, b); }
    break;
  case TAG(baseline_kind, b100): { Baseline b; HTEG_BASELINE(b100, b); }
    break;
  case TAG(baseline_kind, b101): { Baseline b; HTEG_BASELINE(b101, b); }
    break;
  case TAG(baseline_kind, b110): { Baseline b; HTEG_BASELINE(b110, b); }
    break;
  case TAG(baseline_kind, b111): { Baseline b; HTEG_BASELINE(b111, b); }
    break;

```

B.14 Ligatures

```

⟨ skip macros 499cdxcix ⟩ +≡ (524)
#define HTEG_LIG(I, L)
  if ((I) ≡ 7) hteg_list(&((L).l));
  else { (L).l.s = (I); hpos -= (L).l.s; (L).l.p = hpos - hstart; }
  (L).f = HTEG8;

```

```

⟨ cases to skip content 503diii ⟩ +≡ (525)
  case TAG(ligature_kind, 1): { Lig l; HTEG_LIG(1, l); } break;
  case TAG(ligature_kind, 2): { Lig l; HTEG_LIG(2, l); } break;
  case TAG(ligature_kind, 3): { Lig l; HTEG_LIG(3, l); } break;
  case TAG(ligature_kind, 4): { Lig l; HTEG_LIG(4, l); } break;
  case TAG(ligature_kind, 5): { Lig l; HTEG_LIG(5, l); } break;
  case TAG(ligature_kind, 6): { Lig l; HTEG_LIG(6, l); } break;
  case TAG(ligature_kind, 7): { Lig l; HTEG_LIG(7, l); } break;

```

B.15 Discretionary breaks

```

⟨ skip macros 499cdxcix ⟩ +≡ (526)
#define HTEG_DISC(I, H)
  if ((I) & b001) hteg_list(&((H).q));
  else { (H).q.p = hpos - hstart; (H).q.s = 0; (H).q.t = TAG(list_kind, b000); }
  if ((I) & b010) hteg_list(&((H).p));
  else { (H).p.p = hpos - hstart; (H).p.s = 0; (H).p.t = TAG(list_kind, b000); }
  if ((I) & b100) (H).r = HTEG8; else (H).r = 0;

```

```

⟨ cases to skip content 503diii ⟩ +≡ (527)
  case TAG(disc_kind, b001): { Disc h; HTEG_DISC(b001, h); } break;
  case TAG(disc_kind, b010): { Disc h; HTEG_DISC(b010, h); } break;
  case TAG(disc_kind, b011): { Disc h; HTEG_DISC(b011, h); } break;
  case TAG(disc_kind, b100): { Disc h; HTEG_DISC(b100, h); } break;
  case TAG(disc_kind, b101): { Disc h; HTEG_DISC(b101, h); } break;
  case TAG(disc_kind, b110): { Disc h; HTEG_DISC(b110, h); } break;
  case TAG(disc_kind, b111): { Disc h; HTEG_DISC(b111, h); } break;

```

B.16 Paragraphs

```

⟨ skip macros 499cdxcix ⟩ +≡ (528)
#define HTEG_PAR(I)
  { List l; hteg_list(&l); }
  if ((I) & b010) { List l; hteg_param_list(&l); }
  else if ((I) ≠ b100) HTEG_REF(param_kind);
  if ((I) & b100) { Xdimen x; hteg_xdimen_node(&x); }
  else HTEG_REF(xdimen_kind);
  if ((I) ≡ b100) HTEG_REF(param_kind);

```

```

⟨ cases to skip content 503diii ⟩ +≡ (529)
  case TAG(par_kind, b000): HTEG_PAR(b000); break;
  case TAG(par_kind, b010): HTEG_PAR(b010); break;
  case TAG(par_kind, b100): HTEG_PAR(b100); break;
  case TAG(par_kind, b110): HTEG_PAR(b110); break;

```

B.17 Mathematics

```

⟨ skip macros 499cdxcix ⟩ +≡ (530)
#define HTEG_MATH(I)
  if ((I) & b001) hteg_hbox_node();
  { List l; hteg_list(&l); }
  if ((I) & b010) hteg_hbox_node();
  if ((I) & b100) { List l; hteg_param_list(&l); } else HTEG_REF(param_kind);

```

```

⟨ cases to skip content 503diii ⟩ +≡ (531)
  case TAG(math_kind, b000): HTEG_MATH(b000); break;
  case TAG(math_kind, b001): HTEG_MATH(b001); break;
  case TAG(math_kind, b010): HTEG_MATH(b010); break;
  case TAG(math_kind, b100): HTEG_MATH(b100); break;
  case TAG(math_kind, b101): HTEG_MATH(b101); break;
  case TAG(math_kind, b110): HTEG_MATH(b110); break;
  case TAG(math_kind, b011): case TAG(math_kind, b111): break;

```

B.18 Images

```

⟨ skip macros 499cdxcix ⟩ +≡ (532)
#define HTEG_IMAGE(I)
{ Image x = {0};
  List d;
  hteg_list(&d);
  if ((I) & b100) {
    if ((I) ≡ b111) { hteg_xdimen_node(&x.h);
      hteg_xdimen_node(&x.w);
    }
    else if ((I) ≡ b110) { hteg_xdimen_node(&x.w);
      x.hr = HTEG8;
    }
    else if ((I) ≡ b101) { hteg_xdimen_node(&x.h);
      x.wr = HTEG8;
    }
    else { x.hr = HTEG8;
      x.wr = HTEG8;
    }
    x.a = hteg_float32();
  }
  else if ((I) ≡ b011) { HTEG32(x.h.w);
    HTEG32(x.w.w);
  }
  else if ((I) ≡ b010) { HTEG32(x.w.w);
    x.a = hteg_float32();
  }
  else if ((I) ≡ b001) { HTEG32(x.h.w);
    x.a = hteg_float32();
  }
  HTEG16(x.n);
}

```

```

⟨ cases to skip content 503diii ⟩ +≡ (533)
case TAG(image_kind, b001): HTEG_IMAGE(b001); break;
case TAG(image_kind, b010): HTEG_IMAGE(b010); break;
case TAG(image_kind, b011): HTEG_IMAGE(b011); break;
case TAG(image_kind, b100): HTEG_IMAGE(b100); break;
case TAG(image_kind, b101): HTEG_IMAGE(b101); break;
case TAG(image_kind, b110): HTEG_IMAGE(b110); break;
case TAG(image_kind, b111): HTEG_IMAGE(b111); break;

```

B.19 Links and Labels

```

⟨ skip macros 499cdxcix ⟩ +≡ (534)
#define HTEG_LINK(I)
{ uint16_t n;
  if (I & b100) n = HTEG8;
  if (I & b001) HTEG16(n); else n = HTEG8; }

⟨ cases to skip content 503dihi ⟩ +≡ (535)
case TAG(link_kind, b000): HTEG_LINK(b000); break;
case TAG(link_kind, b001): HTEG_LINK(b001); break;
case TAG(link_kind, b010): HTEG_LINK(b010); break;
case TAG(link_kind, b011): HTEG_LINK(b011); break;
case TAG(link_kind, b100): HTEG_LINK(b100); break;
case TAG(link_kind, b101): HTEG_LINK(b101); break;
case TAG(link_kind, b110): HTEG_LINK(b110); break;
case TAG(link_kind, b111): HTEG_LINK(b111); break;

```

B.20 Colors

```

⟨ cases to skip content 503dihi ⟩ +≡ (536)
case TAG(color_kind, b000): (void) HTEG8; break;

```

B.21 Plain Lists, Texts, and Parameter Lists

```

⟨ shared skip functions 466cdlxvi ⟩ +≡ (537)
uint32_t hteg_list_size(Info info)
{ uint32_t n = 0;
  info = info & #3;
  if (info ≡ 0) return 0;
  else if (info ≡ 1) n = HTEG8;
  else if (info ≡ 2) HTEG16(n);
  else if (info ≡ 3) HTEG32(n);
  else QUIT("List_info%d must be 0, 1, 2, or 3", info);
  return n;
}

⟨ skip functions 493cdxciii ⟩ +≡ (538)
static void hteg_size_boundary(Info info)
{ uint32_t n;
  info = info & #3;
  if (info ≡ 0) return;
  n = HTEG8;
  if (n ≠ #100 - info) QUIT("List_size_boundary_byte 0x%x does not match info value %d at %SIZE_F, n, info, hpos - hstart);
}

```

```

static void hteg_list(List *l){ < skip the end byte z 495cdxcv >
    if (KIND(z) ≠ list_kind ∧ KIND(z) ≠ param_kind)
        QUIT("List_expected_at_0x%x", (uint32_t)(hpos - hstart));
    else if ((INFO(z) & #3) ≡ 0) { HBACK(1);
        l→s = 0; }
    else { uint32_t s;
        l→t = z;
        l→s = hteg_list_size(INFO(z));
        hteg_size_boundary(INFO(z));
        hpos = hpos - l→s;
        l→p = hpos - hstart;
        hteg_size_boundary(INFO(z));
        s = hteg_list_size(INFO(z));
        if (s ≠ l→s) QUIT("List_sizes_at_SIZE_F_and_0x%x_do_not_ma\
            tch_0x%x!=_0x%x", hpos - hstart, node_pos - 1, s, l→s);
    }
    < skip and check the start byte a 496cdxcvi >
}

static void hteg_param_list(List *l)
{ if (KIND(*(hpos - 1)) ≠ param_kind) return;
  hteg_list(l);
}

```

B.22 Adjustments

```

< cases to skip content 503diii > +≡ (539)
    case TAG(adjust_kind, b001): { List l; hteg_list(&l); } break;

```

B.23 Tables

```

< skip macros 499cdxcix > +≡ (540)
#define HTEG_TABLE(I)

```

```

    { List l; hteg_list(&l); }
    { List l; hteg_list(&l); }
    if ((I) & b100) { Xdimen x; hteg_xdimen_node(&x); }
    else HTEG_REF(xdimen_kind)

```

```

< cases to skip content 503diii > +≡ (541)

```

```

    case TAG(table_kind, b000): HTEG_TABLE(b000); break;
    case TAG(table_kind, b001): HTEG_TABLE(b001); break;
    case TAG(table_kind, b010): HTEG_TABLE(b010); break;
    case TAG(table_kind, b011): HTEG_TABLE(b011); break;
    case TAG(table_kind, b100): HTEG_TABLE(b100); break;
    case TAG(table_kind, b101): HTEG_TABLE(b101); break;
    case TAG(table_kind, b110): HTEG_TABLE(b110); break;
    case TAG(table_kind, b111): HTEG_TABLE(b111); break;

```

```

case TAG(item_kind, b000): { List l; hteg_list(&l); } break;
case TAG(item_kind, b001): hteg_content_node(); break;
case TAG(item_kind, b010): hteg_content_node(); break;
case TAG(item_kind, b011): hteg_content_node(); break;
case TAG(item_kind, b100): hteg_content_node(); break;
case TAG(item_kind, b101): hteg_content_node(); break;
case TAG(item_kind, b110): hteg_content_node(); break;
case TAG(item_kind, b111): hteg_content_node(); { uint8_t n; n = HTEG8; }
break;

```

B.24 Stream Nodes

⟨ skip macros 499cdxcix ⟩ +≡ (542)

```

#define HTEG_STREAM(I)
{ List l; hteg_list(&l); }
if ((I) & b010) { List l; hteg_param_list(&l); } else HTEG_REF(param_kind);
HTEG_REF(stream_kind);

```

⟨ cases to skip content 503diii ⟩ +≡ (543)

```

case TAG(stream_kind, b000): HTEG_STREAM(b000); break;
case TAG(stream_kind, b010): HTEG_STREAM(b010); break;

```

B.25 References

⟨ skip macros 499cdxcix ⟩ +≡ (544)

```

#define HTEG_REF(K) do { uint8_t n; n = HTEG8; } while (false)

```

⟨ cases to skip content 503diii ⟩ +≡ (545)

```

case TAG(penalty_kind, 0): HTEG_REF(penalty_kind); break;
case TAG(kern_kind, b000): HTEG_REF(dimen_kind); break;
case TAG(kern_kind, b100): HTEG_REF(dimen_kind); break;
case TAG(kern_kind, b001): HTEG_REF(xdimen_kind); break;
case TAG(kern_kind, b101): HTEG_REF(xdimen_kind); break;
case TAG(ligature_kind, 0): HTEG_REF(ligature_kind); break;
case TAG(disc_kind, 0): HTEG_REF(disc_kind); break;
case TAG(glue_kind, 0): HTEG_REF(glue_kind); break;
case TAG(language_kind, 0): HTEG_REF(language_kind); break;
case TAG(rule_kind, 0): HTEG_REF(rule_kind); break;
case TAG(image_kind, 0): HTEG_REF(image_kind); break;
case TAG(leaders_kind, 0): HTEG_REF(leaders_kind); break;
case TAG(baseline_kind, 0): HTEG_REF(baseline_kind); break;

```

B.26 Unknown nodes

(skip functions 493cdxciii) +≡ (546)

```

static int hteg_unknown(Tag z)
{
    int b, n;
    int8_t s;

    s = hnode_size[z];
    DBG(DBGTAGS, "Trying_unknown_tag_0x%x_at_0x%x\n", z,
        (uint32_t)(hpos - hstart - 1));
    if (s == 0) return 0;
    b = NODE_HEAD(s);
    n = NODE_TAIL(s);
    DBG(DBGTAGS, "Trying_unknown_node_size_%d_%d\n", b, n);
    while (n > 0) {
        z = *(hpos - 1);
        if (KIND(z) == xdimen_kind) {
            Xdimen x;
            hteg_xdimen_node(&x);
        }
        else if (KIND(z) == param_kind) {
            List l;
            hteg_param_list(&l);
        }
        else if (KIND(z) == list_kind) {
            List l;
            hteg_list(&l);
        }
        else {
            hteg_content_node();
        }
        n--;
    }
    while (b > 0) {
        z = HTEG8;
        b--;
    }
    return 1;
}

```


C Code and Header Files

C.1 basetypes.h

To define basic types in a portable way, we create an include file. The macro `_MSC_VER` (Microsoft Visual C Version) is defined only if using the respective compiler.

```

⟨basetypes.h 547dxlvii ⟩ ≡ (547)
#ifndef __BASCTYPES_H__
#define __BASCTYPES_H__
#include <stdlib.h>
#include <stdio.h>
#ifndef _STDLIB_H
#define _STDLIB_H
#endif
#ifdef _MSC_VER
#include <windows.h>
#define uint8_t  UINT8
#define uint16_t  UINT16
#define uint32_t  UINT32
#define uint64_t  UINT64
#define int8_t    INT8
#define int16_t   INT16
#define int32_t   INT32
#define bool      BOOL
#define true      (0 ≡ 0)
#define false     (¬true)
#define __SIZEOF_FLOAT__  4
#define __SIZEOF_DOUBLE__ 8
#define PRIx64  "I64x"
#pragma warning(disable:4244 4996 4127)
#else
#include <stdint.h>
#include <stdbool.h>
#include <inttypes.h>
#include <unistd.h>
#endif WIN32

```

```

#include <io.h>
#endif
#endif
    typedef float float32_t;
    typedef double float64_t;
#if __SIZEOF_FLOAT__ != 4
#error float32_type_must_have_size_4
#endif
#if __SIZEOF_DOUBLE__ != 8
#error float64_type_must_have_size_8
#endif
#define HINT_VERSION 2
#define HINT_MINOR_VERSION 2
#define AS_STR (X)#X
#define VERSION_AS_STR (X,Y) AS_STR (X) "." AS_STR (Y)
#define HINT_VERSION_STRING VERSION_AS_STR
    (HINT_VERSION,HINT_MINOR_VERSION)
#endif

```

C.2 format.h

The `format.h` file contains definitions of types, macros, variables and functions that are needed in other compilation units.

```

<format.h 548dxlviii > ≡ (548)
#ifndef _HFORMAT_H_
#define _HFORMAT_H_
    <debug macros 383ccclxxxiii >
    <debug constants 449cdxlix >
    <hint macros 13xiii >
    <hint basic types 6vi >
    <default names 420cdxx >

    extern const char *content_name[32];
    extern const char *definition_name[32];
    extern unsigned int debugflags;
    extern FILE *hlog;
    extern int max_fixed[32], max_default[32], max_ref[32], max_outline;
    extern int32_t int_defaults[MAX_INT_DEFAULT + 1];
    extern Dimen dimen_defaults[MAX_DIMEN_DEFAULT + 1];
    extern Xdimen xdimen_defaults[MAX_XDIMEN_DEFAULT + 1];
    extern Glue glue_defaults[MAX_GLUE_DEFAULT + 1];
    extern Baseline baseline_defaults[MAX_BASELINE_DEFAULT + 1];
    extern Label label_defaults[MAX_LABEL_DEFAULT + 1];
    extern ColorSet color_defaults[MAX_COLOR_DEFAULT + 1];
    extern signed char hnode_size[#100];
    extern uint8_t content_known[32];
#endif

```

C.3 tables.c

For maximum flexibility and efficiency, the file `tables.c` is generated by a C program. Here is the *main* program of `mktables`:

```

⟨mktables.c 549dxlix⟩ ≡ (549)
#include "basetypes.h"
#include "format.h"
⟨skip macros 499cdxcix⟩
int max_fixed[32], max_default[32];
int32_t int_defaults[MAX_INT_DEFAULT + 1] = {0};
Dimen dimen_defaults[MAX_DIMEN_DEFAULT + 1] = {0};
Xdimen xdimen_defaults[MAX_XDIMEN_DEFAULT + 1] = {{{0}}};
Glue glue_defaults[MAX_GLUE_DEFAULT + 1] = {{{{0}}}};
Baseline baseline_defaults[MAX_BASELINE_DEFAULT + 1] = {{{{{0}}}}};
signed char hnode_size[#100] = {0};
uint8_t content_known[32] = {0};
⟨define content_name and definition_name 7vii⟩
int main(void)
{
    Kind k;
    int i;

    printf("#include_\\"basetypes.h\\"\\n"
           "#include_\\"format.h\\"\\n\\n");
    ⟨print content_name and definition_name 8viii⟩
    printf("int_max_outline=-1;\\n\\n");
    ⟨take care of variables without defaults 419cdxix⟩
    ⟨define int_defaults 421cdxxi⟩
    ⟨define dimen_defaults 423cdxxiii⟩
    ⟨define glue_defaults 427cdxxvii⟩
    ⟨define xdimen_defaults 425cdxxv⟩
    ⟨define baseline_defaults 429cdxxix⟩
    ⟨define page defaults 435cdxxxv⟩
    ⟨define stream defaults 433cdxxxiii⟩
    ⟨define range defaults 437cdxxxvii⟩
    ⟨define label_defaults 431cdxxxi⟩
    ⟨define color_defaults 441cdxli⟩
    ⟨print defaults 550dl⟩
    ⟨initialize the hnode_size array 470cdlxx⟩
    ⟨print the hnode_size variable 467cdlxvii⟩
    ⟨print the content_known variable 468cdlxviii⟩
    return 0;
}

```

The following code prints the arrays containing the default values.

```

⟨print defaults 550dl ⟩ ≡ (550)
    printf("int_max_fixed[32]=\n");
    for (k = 0; k < 32; k++)
    { printf("%d", max_fixed[k]); if (k < 31) printf(", "); }
    printf("};\n\n");
    printf("int_max_default[32]=\n");
    for (k = 0; k < 32; k++)
    { printf("%d", max_default[k]); if (k < 31) printf(", "); }
    printf("};\n\n");
    printf("int_max_ref[32]=\n");
    for (k = 0; k < 32; k++)
    { printf("%d", max_default[k]); if (k < 31) printf(", "); }
    printf("};\n\n");

```

Used in 549dxlix.

C.4 get.h

The `get.h` file contains function prototypes for all the functions that read the short format.

```

⟨get.h 551dli ⟩ ≡ (551)
    ⟨hint types 1i ⟩
    ⟨directory entry type 357ccclvii ⟩
    ⟨shared get macros 38xxxviii ⟩

    extern Entry *dir;
    extern uint16_t section_no, max_section_no;
    extern uint8_t *hpos, *hstart, *hend, *hpos0;
    extern uint64_t hin_size, hin_time;
    extern uint8_t *hin_addr;
    extern Label *labels;
    extern char *hin_name;
    extern bool hget_map(void);
    extern void hget_unmap(void);
    extern void new_directory(uint32_t entries);
    extern void hset_entry(Entry *e, uint16_t i,
        uint32_t size, uint32_t xsize, char *file_name);
    extern void hget_banner(void);
    extern void hget_section(uint16_t n);
    extern void hget_entry(Entry *e);
    extern void hget_directory(void);
    extern void hclear_dir(void);
    extern bool hcheck_banner(char *magic);
    extern int max_range;
    extern void hget_max_definitions(void);
    extern uint32_t hget_utf8(void);
    extern void hget_size_boundary(Info info);

```

```

extern uint32_t hget_list_size(Info info);
extern void hget_list(List *l);
extern float32_t hget_float32(void);
extern void hff_hpos(void);
extern uint32_t hff_list_pos, hff_list_size;
extern Tag hff_tag;
extern float32_t hteg_float32(void);
extern uint32_t hteg_list_size(Info info);    /* seems like these are declared
static */
#if 0
extern void hteg_list(List *l);
extern void hteg_size_boundary(Info info);
#endif

```

C.5 get.c

```

⟨get.c 552dlii⟩ ≡ (552)
#include "basetypes.h"
#include <string.h>
#include <math.h>
#include <zlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include "error.h"
#include "format.h"
#include "get.h"
  ⟨common variables 252cclii⟩
  ⟨map functions 349cccxlx⟩
  ⟨function to check the banner 336cccxxxvi⟩
  ⟨directory functions 358ccclviii⟩
  ⟨get file functions 337cccxxxvii⟩
  ⟨shared get functions 53liii⟩
  ⟨shared skip functions 466cdlxvi⟩

```

C.6 put.h

The put.h file contains function prototypes for all the functions that write the short format.

```

⟨put.h 553dliii⟩ ≡ (553)
  ⟨put macros 345cccxlx⟩
  ⟨hint macros 13xiii⟩
  ⟨hint types 1i⟩
  ⟨directory entry type 357ccclvii⟩

```

```

extern Entry *dir;
extern uint16_t section_no, max_section_no;
extern uint8_t *hpos, *hstart, *hend, *hpos0;
extern int next_range;
extern RangePos *range_pos;
extern int next_range, max_range;
extern int *page_on;
extern Label *labels;
extern int first_label;
extern int max_outline;
extern Outline *outlines;
extern FILE *hout;
extern void new_directory(uint32_t entries);
extern void new_output_buffers(void); /* declarations for the parser */
extern void hput_definitions_start(void);
extern void hput_definitions_end(void);
extern void hput_content_start(void);
extern void hput_content_end(void);
extern void hset_label(int n, int w);
extern Tag hput_link(int n, int c, int on);
extern void hset_outline(int m, int r, int d, uint32_t p);
extern void hput_label_defs(void);
extern void hput_tags(uint32_t pos, Tag tag);
extern Tag hput_glyph(Glyph *g);
extern Tag hput_xdimen(Xdimen *x);
extern Tag hput_int(int32_t p);
extern Tag hput_language(uint8_t n);
extern Tag hput_rule(Rule *r);
extern Tag hput_glue(Glue *g);
extern Tag hput_list(uint32_t size_pos, List *y);
extern uint8_t hsize_bytes(uint32_t n);
extern void hput_txt_cc(uint32_t c);
extern void hput_txt_font(uint8_t f);
extern void hput_txt_global(Ref *d);
extern void hput_txt_local(uint8_t n);
extern Info hput_box_dimen(Dimen h, Dimen d, Dimen w);
extern Info hput_box_shift(Dimen a);
extern Info hput_box_glue_set(int8_t s, float32_t r, Order o);
extern void hput_stretch(Stretch *s);
extern Tag hput_kern(Kern *k);
extern void hput_utf8(uint32_t c);
extern Tag hput_ligature(Lig *l);
extern Tag hput_disc(Disc *h);
extern Info hput_span_count(uint32_t n);
extern void hextract_image_dims(int n, double *a, Dimen *w, Dimen
    *h);

```

```

extern Info hput_image_spec(uint32_t n, float32_t a, uint32_t wr, Xdimen
    *w, uint32_t hr, Xdimen *h);
extern int colors_i;
extern ColorSet colors_0, colors_n;
extern void color_init(void);
extern void hput_color_def(uint32_t pos, int n);
extern void hput_string(char *str);
extern void hput_range(uint8_t pg, bool on);
extern void hput_max_definitions(void);
extern Tag hput_dimen(Dimen d);
extern Tag hput_font_head(uint8_t f, char *n, Dimen s,
    int m, uint16_t y);
extern void hput_range_defs(void);
extern void hput_xdimen_node(Xdimen *x);
extern void hput_directory(void);
extern size_t hput_hint(char *str);
extern void hput_list_size(uint32_t n, int i);
extern uint32_t hput_unknown_def(uint32_t t, uint32_t b, uint32_t n);
extern Tag hput_unknown(uint32_t pos, uint32_t t, uint32_t b, uint32_t
    n);
extern int hcompress_depth(int n, int c);

```

C.7 put.c

```

<put.c 554dliv > ≡ (554)
#include "basetypes.h"
#include <string.h>
#include <ctype.h>
#include <math.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <zlib.h>
#include "error.h"
#include "format.h"
#include "put.h"
    <common variables 252cclii >
    <shared put variables 273cclxxiii >
    <directory functions 358ccclviii >
    <function to write the banner 339cccxix >
    <put functions 14xiv >

```

C.8 `lexer.l`

The definitions for `lex` are collected in the file `lexer.l`

```
<lexer.l 555dlv> ≡ (555)
%{
#include "basetypes.h"
#include "error.h"
#include "format.h"
#include "put.h"
    <enable bison debugging 464cdlxiv>
#include "parser.h"
    <scanning macros 23xxiii> <scanning functions 62lxii>
    int yywrap(void) { return 1; }
#ifdef _MSC_VER
#pragma warning (disable: 4267)
#endif
}%
%option yylineno stack batch never - interactive
%option debug
%option nounistd nounput noinput noyy_top_state
    <scanning definitions 24xxiv>
%%
    <scanning rules 3iii>
[a-z]+      QUIT("Unexpected keyword '%s' in line %d",
               yytext, yylineno);
.           QUIT("Unexpected character '%c' (0x%02X) in line %d",
               yytext[0] > ' ' ? yytext[0] : ' ', yytext[0], yylineno);
%%
```

C.9 `parser.y`

The grammar rules for `bison` are collected in the file `parser.y`.

```
<parser.y 556dlvi> ≡ (556)
%{
#include "basetypes.h"
#include <string.h>
#include <math.h>
#include "error.h"
#include "format.h"
#include "put.h"
    extern char **hfont_name; /* in common variables */
    <definition checks 392ccxcxii>
    extern void hset_entry(Entry *e, uint16_t i,
                          uint32_t size, uint32_t xsize, char *file_name);
    <enable bison debugging 464cdlxiv>
```



```

extern int yylex(void);
  < parsing functions 388ccclxxxviii >
%}

%union { uint32_t u; int32_t i; char *s; float64_t f; Glyph c; Dimen d;
  Stretch st; Xdimen xd; Kern kt; Rule r; Glue g; Image x; List l; Box
    h; Disc dc; Lig lg; Ref rf; Info info; Order o;
  bool b; }
%define parse.error verbose
%start hint
  < symbols 2ii >
%%
  < parsing rules 5v >
%%

```

C.10 shrink.c

shrink is a C program translating a HINT file in long format into a HINT file in short format.

```

< shrink.c 557dlvii > ≡ (557)
#include "basetypes.h"
#include <math.h>
#include <string.h>
#include <ctype.h>
#include <sys/types.h>
#include <sys/stat.h>
#ifdef WIN32
#include <direct.h>
#endif
#include <zlib.h>
#include "error.h"
#include "format.h"
#include "put.h"
  < enable bison debugging 464cdlxiv >
#include "parser.h"

extern void yyset_debug(int lex_debug);
extern int yylineno;
extern FILE *yyin, *yyout;
extern int yyparse(void);

  < put macros 345cccxlvi >
  < common variables 252cccli >
  < shared put variables 273ccclxxiii >
  < function to check the banner 336cccxxxvi >
  < directory functions 358ccclviii >
  < function to write the banner 339cccxxxix >

```

```

    <put functions 14xiv >
#define SHRINK
#define DESCRIPTION "\nConvert a 'long' ASCII HINT file into\
    a 'short' binary HINT file.\n"

int main(int argc, char *argv[])
{ <local variables in main 451cdli >
    in_ext = ".hint";
    out_ext = ".hnt";
    <process the command line 452cdlii >
    if (debugflags & DBGFLEX) yyset_debug(1);
    else yyset_debug(0);
#if YYDEBUG
    if (debugflags & DBGBISON) yydebug = 1;
    else yydebug = 0;
#endif
    <open the log file 454cdliv >
    <open the input file 455cdlv >
    <open the output file 456cdlvi >
    yyin = hin;
    yyout = hlog;
    <read the banner 338cccxxviii >
    if (!hcheck_banner("HINT")) QUIT("Invalid banner");
    yylineno++;
    DBG(DBGBISON | DBGFLEX, "Parsing Input\n");
    yyparse();
    hput_directory();
    <rewrite the file names of optional sections 373ccclxxiii >
    hput_hint("created by shrink");
    <close the output file 459cdlix >
    <close the input file 458cdlviii >
    <close the log file 460cdlx >
    return 0;
}

```

C.11 stretch.c

stretch is a C program translating a HINT file in short format into a HINT file in long format.

<stretch.c 558dlviii > ≡ (558)

```

#include "basetypes.h"
#include <math.h>
#include <string.h>
#include <ctype.h>
#include <zlib.h>
#include <sys/types.h>
#include <sys/stat.h>

```

```

#ifdef WIN32
#include <direct.h>
#endif
#include <fcntl.h>
#include "error.h"
#include "format.h"
#include "get.h"
    <get macros 19xix >
    <write macros 22xxii >
    <common variables 252cclii >
    <shared put variables 273cclxxiii >
    <map functions 349ccclix >
    <function to check the banner 336cccxvvi >
    <function to write the banner 339cccxvix >
    <directory functions 358ccclviii >
    <definition checks 392ccxcii >
    <get function declarations 559dlxi >
    <write functions 21xxi >
    <get file functions 337cccxvii >
    <shared get functions 53liii >
    <get functions 18xviii >
#define STRETCH
#define DESCRIPTION "\nConvert_a_‘short’_binary_HINT_file_in\
    to_a_‘long’_ASCII_HINT_file.\n"
int main(int argc, char *argv[])
{ <local variables in main 451cdli >
    in_ext = ".hnt";
    out_ext = ".hint";
    <process the command line 452cdlii >
    <open the log file 454cdliv >
    <open the output file 456cdlvi >
    <determine the stem_name from the output file_name 457cdlvii >
    if (!hget_map()) QUIT("Unable_to_map_the_input_file");
    hpos = hstart = hin_addr;
    hend = hstart + hin_size;
    hget_banner();
    if (!hcheck_banner("hint")) QUIT("Invalid_banner");
    hput_banner("HINT", "created_by_stretch");
    hget_directory();
    hwrite_directory();
    hget_definition_section();
    hwrite_content_section();
    hwrite_aux_files();
    hget_unmap();
    <close the output file 459cdlix >

```

```

    DBG(DBGBASIC, "End_of_Program\n");
    <close the log file 460cdlx >
    return 0;
}

```

In the above program, the get functions call the write functions and the write functions call some get functions. This requires function declarations to satisfy the define before use requirement of C. Some of the necessary function declarations are already contained in `get.h`. The remaining declarations are these:

```

<get function declarations 559dlx > ≡ (559)
extern void hget_xdimen_node(Xdimen *x);
extern void hget_def_node(void);
extern void hget_font_def(Info i, uint8_t f);
extern void hget_content_section(void);
extern Tag hget_content_node(void);
extern void hget_glue_node(void);
extern void hget_rule_node(void);
extern void hget_hbox_node(void);
extern void hget_vbox_node(void);
extern void hget_param_list(List *l);
extern int hget_txt(void);
extern int hget_unknown(Tag a);

```

Used in 558dlviii and 560dlx.

C.12 skip.c

`skip` is a C program reading the content section of a HINT file in short format backwards.

```

<skip.c 560dlx > ≡ (560)
#include "basetypes.h"
#include <math.h>
#include <string.h>
#include <ctype.h>
#include <zlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include "error.h"
#include "format.h"
#if 1
#include "get.h"
#else
    <hint types 1i >
    <directory entry type 357ccclvii >
    <shared get macros 38xxxviii >
#endif

```

```

    <get macros 19xix >
    <write macros 22xxii >
    <common variables 252cclii >
    <shared put variables 273cclxxiii >
    <map functions 349cccxlxi >
    <function to check the banner 336cccxxxvi >
    <directory functions 358ccclviii >
    <shared get macros 38xxxviii >
    <get file functions 337cccxxxvii >
    <skip macros 499cdxcix >
    <skip function declarations 561dlxi >
    <shared skip functions 466cdlxvi >
    <skip functions 493cdxciii >
    <definition checks 392cccxcii >
    <get function declarations 559dlxi >
    <write functions 21xxi >
    <shared get functions 53liii >
    <get functions 18xviii >
#define SKIP
#define DESCRIPTION "\nThis program tests parsing a binary\
    HINT file in reverse direction.\n"

int main(int argc, char *argv[])
{
    <local variables in main 451cdli >
    in_ext = ".hnt";
    out_ext = ".bak";
    <process the command line 452cdlii >
    <open the log file 454cdliv >
    hout = NULL;
    if (!hget_map()) QUIT("Unable to map the input file");
    hpos = hstart = hin_addr;
    hend = hstart + hin_size;
    hget_banner();
    if (!hcheck_banner("hint")) QUIT("Invalid banner");
    hget_directory();
    hget_definition_section();
    DBG(DBGBASIC, "Skipping Content Section\n");
    hteg_content_section();
    DBG(DBGBASIC, "Fast forward Content Section\n");
    hpos = hstart;
    while (hpos < hend) {
        hff_hpos();
        if (KIND(*(hpos - 1)) == par_kind ^ KIND(hff_tag) == list_kind ^ hff_list_size >
            0 ^ !(INFO(hff_tag) & b100)) {
            uint8_t *p = hpos, *q;
            DBG(DBGTAGS, "Fast forward list at 0x%x, size %d\n", hff_list_pos,
                hff_list_size);
            hpos = hstart + hff_list_pos;
        }
    }
}

```

```

    q = hpos + hff_list_size;
    while (hpos < q) hff_hpos();
    DBG(DBGTAGS, "Fast_forward_list_end_at_0x%x\n",
        (uint32_t)(hpos - hstart));
    hpos = p;
    DBG(DBGTAGS, "Continue_at_0x%x\n", (uint32_t)(hpos - hstart - 1));
}
}
hget_unmap();
⟨close the log file 460cdlx⟩
return 0;
}

```

As we have seen already in the `stretch` program, a few function declarations are necessary to satisfy the define before use requirement of C.

```

⟨skip function declarations 561dlxi⟩ ≡ (561)
static void hteg_content_node(void);
static void hteg_content(Tag z);
static void hteg_xdimen_node(Xdimen *x);
static void hteg_list(List *l);
static void hteg_param_list(List *l);
static void hteg_rule_node(void);
static void hteg_hbox_node(void);
static void hteg_vbox_node(void);
static void hteg_glue_node(void);
static int hteg_unknown(Tag z);

```

Used in 560dlx.

D Format Definitions

D.1 Reading the Long Format

Data Types

Integers	13
Strings	15
Character Codes	16
Floating Point Numbers	20
Dimensions	27
Extended Dimensions	29
Stretch and Shrink	32

Simple Nodes

Glyphs	2
Penalties	35
Languages	37
Rules	38
Kerns	41
Glue	43

Lists

Plain Lists	49
Texts	55

Composite Nodes

Boxes	62
Extended Boxes	65
Leaders	69
Baseline Skips	71
Ligatures	73
Discretionary breaks	75
Paragraphs	78
Displayed Math	80
Adjustments	81
Text Math	81
Tables	83

Extensions

Images	86
Labels	100
Links	106
Outlines	109
Colors	112
Unknown Extensions	117
Stream Definitions	126
Stream Content	128
Page Template Definitions	130
Page Ranges	132

File Structure

Banner	139
Banner	140
Directory Section	149
Definition Section	161
Content Section	187

Definitions

Special Maximum Values	99
Maximum Values	164
Definitions	168
Parameter Lists	171
Fonts	174
References	176

D.2 Writing the Long Format

Data Types

Integers	14
Strings	15
Character Codes	18
Floating Point Numbers	24
Fixed Point Numbers	26
Dimensions	28
Extended Dimensions	29
Stretch and Shrink	33

Simple Nodes

Glyphs	10
Languages	37
Rules	39
Kerns	41
Glue	44

Lists

Plain Lists	50
Texts	57

Composite Nodes	
Boxes	62
Leaders	69
Ligatures	73
Discretionary breaks	76
Adjustments	81
Extensions	
Images	87
Labels	102
Links	106
Outlines	108
Colors	115
Unknown Extensions	119
Stream Definitions	127
Stream Content	128
Page Template Definitions	130
Page Ranges	132
File Structure	
Banner	139
Directory Section	153
Definition Section	162
Content Section	187
Definitions	
Special Maximum Values	99
Maximum Values	165
Definitions	169
Parameter Lists	172
Fonts	175
References	178

D.3 Reading the Short Format

Data Types	
Strings	16
Character Codes	19
Dimensions	28
Extended Dimensions	30
Stretch and Shrink	32
Simple Nodes	
Content Nodes	9
Glyphs	9
Penalties	36
Languages	37
Rules	39
Kerns	41
Glue	44

Lists

Plain Lists	50
Texts	58

Composite Nodes

Boxes	63
Extended Boxes	67
Leaders	69
Baseline Skips	71
Ligatures	74
Discretionary breaks	76
Paragraphs	79
Displayed Math	80
Adjustments	81
Text Math	81
Tables	83

Extensions

Images	87
Labels	103
Links	105
Outlines	108
Colors	115
Unknown Extensions	119
Stream Definitions	127
Stream Content	128
Page Template Definitions	130
Page Ranges	133

File Structure

Banner	139
Primitives	141
Sections	146
Directory Entries	154
Directory Section	155
Content Section	188

Definitions

Special Maximum Values	98
Maximum Values	165
Definitions	169
Parameter Lists	172
Fonts	175
References	177

D.4 Writing the Short Format

Data Types

Strings	16
Character Codes	20
Dimensions	28
Extended Dimensions	30
Stretch and Shrink	31

Simple Nodes

Glyphs	7
Penalties	36
Languages	37
Rules	40
Kerns	42
Glue	45

Lists

Plain Lists	51
Texts	59

Composite Nodes

Boxes	64
Baseline Skips	72
Ligatures	74
Discretionary breaks	77
Adjustments	81
Tables	84

Extensions

Images	88
Labels	104
Links	106
Outlines	110
Colors	112
Unknown Extensions	118
Stream Definitions	126
Stream Content	128
Page Template Definitions	130
Page Ranges	134

File Structure

Banner	139
Primitives	141
Directory Section	156
Optional Sections	159
Definition Section	162
Content Section	188

Definitions	
Special Maximum Values	99
Maximum Values	166
Definitions	168
Parameter Lists	171
Fonts	176
References	176

Crossreference of Code

- ⟨adjust label positions after moving a list⟩ Defined in 258cclviii Used in 148cxlvi
- ⟨advance *hpos* over a list⟩ Defined in 469cdlxix Used in 466cdlxvi
- ⟨alternative kind names⟩ Defined in 10x Used in 6vi
- ⟨append section *i* from file⟩ Defined in 375ccclxxv Used in 374ccclxxiv
- ⟨append section *i* from *buffer*⟩ Defined in 376ccclxxvi Used in 374ccclxxiv
- ⟨auxiliar image functions⟩ Defined in 239ccxxxix, 240ccxl, 241ccxli, and 242ccxlii
Used in 237ccxxxvii
- ⟨**basetypes.h**⟩ Defined in 547dxlvii
- ⟨cases of getting special maximum values⟩ Defined in 246ccxlvi Used in 390ccxc
- ⟨cases of putting special maximum values⟩ Defined in 247ccxlvi Used in 391ccxc
- ⟨cases of writing special maximum values⟩ Defined in 248ccxlvi
Used in 389ccclxxxix
- ⟨cases to get content⟩ Defined in 20xx, 106cvi, 111cxi, 119cxix, 128cxxxviii, 137cxxxvii, 165clxv, 172clxxii, 178clxxviii, 184clxxxiv, 192cxii, 200cc, 207ccvii, 212ccxii, 217ccxvii, 221ccxxi, 225ccxxv, 233ccxxxiii, 266cclxvi, 297ccxvii, 314cccxiv, 317cccxvii, and 416cdxvi
Used in 18xviii
- ⟨cases to get definitions for *color_kind*⟩ Defined in 299ccxcix Used in 398cccxviii
- ⟨cases to skip content⟩ Defined in 503diii, 505dv, 507dvii, 508dviii, 510dx, 513dxiii, 516dxvi, 519dxix, 521dxxi, 523dxxiii, 525dxxv, 527dxxvii, 529dxxix, 531dxxxi, 533dxxxiii, 535dxxxv, 536dxxxvi, 539dxxxix, 541dxli, 543dxliii, and 545dxlv
Used in 494cdxciv
- ⟨close the input file⟩ Defined in 458cdlviii Used in 557dlvii
- ⟨close the log file⟩ Defined in 460cdlx Used in 557dlvii, 558dlviii, and 560dlx
- ⟨close the output file⟩ Defined in 459cdlix Used in 557dlvii and 558dlviii
- ⟨common variables⟩ Defined in 252ccli, 289ccclxxxix, 324cccxiv, 335cccxv, 342cccxli, 348cccxlviii, 406cdvi, 450cdl, and 453cdliii
Used in 552dlii, 554dliv, 557dlvii, 558dlviii, and 560dlx
- ⟨compress long format depth levels⟩ Defined in 275ccclxxv Used in 284ccclxxxiv
- ⟨compute a local *aux_name*⟩ Defined in 362ccclxii Used in 366ccclxvi and 372ccclxxii
- ⟨debug constants⟩ Defined in 449cdlxix Used in 548dxlviii
- ⟨debug macros⟩ Defined in 383ccclxxxiii, 384ccclxxxiv, and 462cdlxii
Used in 548dxlviii
- ⟨default names⟩ Defined in 420cdxx, 422cdxxii, 424cdxxiv, 426cdxxvi, 428cdxxviii, 430cdxxx, 432cdxxxii, 434cdxxxiv, 436cdxxxvi, 438cdxxxviii, and 440cdxl
Used in 548dxlviii
- ⟨define page defaults⟩ Defined in 435cdxxxv Used in 549dxlix

- ⟨define range defaults⟩ Defined in 437cdxxxvii and 439cdxxxix Used in 549dxlix
- ⟨define stream defaults⟩ Defined in 433cdxxxiii Used in 549dxlix
- ⟨define *baseline_defaults*⟩ Defined in 429cdxxix Used in 549dxlix
- ⟨define *color_defaults*⟩ Defined in 441cdxli Used in 549dxlix
- ⟨define *content_name* and *definition_name*⟩ Defined in 7vii Used in 549dxlix
- ⟨define *dimen_defaults*⟩ Defined in 423cdxxxiii Used in 549dxlix
- ⟨define *glue_defaults*⟩ Defined in 427cdxxxvii Used in 549dxlix
- ⟨define *int_defaults*⟩ Defined in 421cdxxi Used in 549dxlix
- ⟨define *label_defaults*⟩ Defined in 431cdxxxix Used in 549dxlix
- ⟨define *xdimen_defaults*⟩ Defined in 425cdxxv Used in 549dxlix
- ⟨definition checks⟩ Defined in 392cccxcii and 396cccxcvi
 - Used in 556dlvi, 558dlviii, and 560dlx
- ⟨determine the *stem_name* from the output *file_name*⟩ Defined in 457cdlvii
 - Used in 558dlviii
- ⟨determine whether *path* is absolute or relative⟩ Defined in 363ccclxiii
 - Used in 362ccclxii
- ⟨directory entry type⟩ Defined in 357ccclvii Used in 551dli, 553dlviii, and 560dlx
- ⟨directory functions⟩ Defined in 358ccclviii and 359ccclix
 - Used in 552dlii, 554dliv, 557dlvii, 558dlviii, and 560dlx
- ⟨enable bison debugging⟩ Defined in 464cdlxiv Used in 555dlv, 556dlvi, and 557dlvii
- ⟨**error.h**⟩ Defined in 461cdlxi
- ⟨explain usage⟩ Defined in 448cdxlvi Used in 452cdlii
- ⟨extract mantissa and exponent⟩ Defined in 69lxix, 70lxx, and 71lxxi
 - Used in 68lxviii
- ⟨**format.h**⟩ Defined in 548cdxlvi
- ⟨function to check the banner⟩ Defined in 336cccxxxvi
 - Used in 552dlii, 557dlvii, 558dlviii, and 560dlx
- ⟨function to write the banner⟩ Defined in 339cccxxxix
 - Used in 554dliv, 557dlvii, and 558dlviii
- ⟨get and store a label node⟩ Defined in 261ccclxi Used in 245ccxlv
- ⟨get and write an outline node⟩ Defined in 277cclxxvii Used in 245ccxlv
- ⟨get file functions⟩ Defined in 337cccxxxvii, 350cccl, 352ccclii, 369ccclxix, 370ccclxx, and 390cccxc Used in 552dlii, 558dlviii, and 560dlx
- ⟨get function declarations⟩ Defined in 559dlx Used in 558dlviii and 560dlx
- ⟨get functions⟩ Defined in 18xviii, 85lxxxv, 93xciii, 94xciv, 121cxxi, 139cxxxix, 158clviii, 167clxvii, 202ccii, 245ccxlv, 298ccxcviii, 307cccvi, 308cccvi, 313cccxi, 322cccxxii, 331cccxxxi, 381ccclxxxi, 398cccxcviii, 405cdv, 412cdxii, and 446cdxli
 - Used in 558dlviii and 560dlx
- ⟨get macros⟩ Defined in 19xix, 92xcii, 98xcviii, 107cvii, 120cxx, 129cxxxix, 138cxxxviii, 157clvii, 166clxvi, 173clxxiii, 179clxxix, 185clxxxv, 193cxciii, 201cci, 208ccviii, 213cccxi, 226ccxxvi, 234ccxxxiv, 265cclxv, 318cccxcviii, and 417cdxvii
 - Used in 558dlviii and 560dlx
- ⟨get stream information for normal streams⟩ Defined in 312cccxi
 - Used in 313cccxi
- ⟨**get.c**⟩ Defined in 552dlii
- ⟨**get.h**⟩ Defined in 551dli

- ⟨hint basic types⟩ Defined in 6vi, 11xi, 12xii, 57lvii, 77lxxvii, 82lxxxii, 87lxxxvii, 96xcvi, 131cxxxi, 180clxxx, 250ccl, 285cclxxxv, and 407cdvii Used in 548dxlviii
- ⟨hint macros⟩ Defined in 13xiii, 78lxxviii, 113cxiii, 132cxxxii, 244ccxliv, 251ccli, 286cclxxxvi, 326cccxxvi, 334cccxxxiv, 347cccxlvi, and 465cdlxv Used in 548dxlviii and 553dliai
- ⟨hint types⟩ Defined in 1i, 114cxiv, 123cxxiii, 141cxli, 149cxlx, 160clx, 187clxxxvii, 195cxcv, 228ccxxviii, 272cclxxii, 323cccxxiii, and 414cdxv
Used in 551dli, 553dliai, and 560dlx
- ⟨image functions⟩ Defined in 236ccxxxvi, 237ccxxxvii, and 243ccxlii
Used in 235ccxxxv
- ⟨initialize definitions⟩ Defined in 253ccliii, 274cclxxiv, 290ccxc, 325cccxv, 393cccxciii, and 408cdviii Used in 381ccclxxxi and 387ccclxxxvii
- ⟨initialize the *hnode_size* array⟩ Defined in 470cdlxx, 471cdlxxi, 472cdlxxii, 473cdlxxiii, 474cdlxxiv, 475cdlxxv, 476cdlxxvi, 477cdlxxvii, 478cdlxxviii, 479cdlxxix, 480cdlxxx, 481cdlxxxi, 482cdlxxxii, 483cdlxxxiii, 484cdlxxxiv, 485cdlxxxv, 486cdlxxxvi, 487cdlxxxvii, 488cdlxxxviii, 489cdlxxxix, 490cdxc, 491cdxci, and 492cdxcii Used in 549dxlix
- ⟨kinds⟩ Defined in 9ix Used in 6vi and 7vii
- ⟨lexer.1⟩ Defined in 555dlv
- ⟨local variables in *main*⟩ Defined in 451cdli Used in 557dlvii, 558dlviii, and 560dlx
- ⟨make sure the path in *aux_name* exists⟩ Defined in 365ccclxv
Used in 366ccclxvi and 456cdlvi
- ⟨make sure *access* is defined⟩ Defined in 361ccclxi Used in 366ccclxvi
- ⟨map functions⟩ Defined in 349cccxlx Used in 552dlai, 558dlviii, and 560dlx
- ⟨merge stored image dimensions with dimensions given⟩ Defined in 238ccxxxvii
Used in 237ccxxxvii
- ⟨mktables.c⟩ Defined in 549dxlix
- ⟨normalize the mantissa⟩ Defined in 65lxv Used in 62lxii
- ⟨open the input file⟩ Defined in 455cdlv Used in 557dlvii
- ⟨open the log file⟩ Defined in 454cdliv Used in 557dlvii, 558dlviii, and 560dlx
- ⟨open the output file⟩ Defined in 456cdlvi Used in 557dlvii and 558dlviii
- ⟨output the label definitions⟩ Defined in 264ccclxiv Used in 263ccclxiii
- ⟨output the outline definitions⟩ Defined in 284ccclxxxiv Used in 263ccclxiii
- ⟨output the title of outline **t*⟩ Defined in 282ccclxxxii Used in 283ccclxxxiii
- ⟨parser.y⟩ Defined in 556dlvi
- ⟨parsing functions⟩ Defined in 388ccclxxxviii, 399cccxcix, and 463cdlxxii
Used in 556dlvi
- ⟨parsing rules⟩ Defined in 5v, 30xxx, 39xxxix, 51li, 59lix, 83lxxxiii, 90xc, 101ci, 105cv, 117cxvii, 126cxxvi, 135cxxxv, 143cxlai, 154cliv, 163clxiii, 171clxxi, 176clxxxvi, 183clxxxiii, 190cxc, 198cxcviii, 206ccvi, 211ccxi, 216ccxvi, 220ccxx, 224ccxxxiv, 231ccxxxi, 249ccxlx, 256cclvi, 270cclxx, 280cclxxx, 291ccxc, 292ccxcii, 293cccclii, 296ccxcvi, 302ccci, 304ccciv, 311cccx, 316cccvi, 321cccxii, 329cccxxxix, 340ccccl, 356ccclvi, 379ccclxxxix, 387ccclxxxvii, 395cccxcv, 397cccxcvii, 402cdii, 403cdiii, 411cdxi, 415cdxv, and 444cdxlv Used in 556dlvi
- ⟨print defaults⟩ Defined in 550dl Used in 549dxlix
- ⟨print the *content_known* variable⟩ Defined in 468cdlxviii Used in 549dxlix
- ⟨print the *hnode_size* variable⟩ Defined in 467cdlxvii Used in 549dxlix

- ⟨*print content_name* and *definition_name*⟩ Defined in 8viii Used in 549dxlix
- ⟨process the command line⟩ Defined in 452cdlii
 - Used in 557dlvii, 558dlviii, and 560dlx
- ⟨put functions⟩ Defined in 14xiv, 15xv, 37xxxvii, 54liv, 75lxxv, 86lxxxvi, 95xcv, 97xcvii, 108cviii, 112cxii, 122cxxii, 130cxxx, 140cxl, 148cxlvi, 159clix, 168clxviii, 186clxxxvi, 194xciv, 203cciii, 227ccxxvii, 235ccxxxv, 257cclvii, 262cclxii, 263cclxiii, 267cclxvii, 276cclxxvi, 281cclxxi, 283cclxxxiii, 294ccxciv, 295ccxcv, 305cccv, 306cccevi, 333cccxiii, 341cccxli, 344cccxliv, 351cccli, 353cccliii, 371ccclxxi, 374ccclxxiv, 382ccclxxxii, 391cccxci, 413cdxiii, and 447cdxlvii Used in 554dliv and 557dlvii
- ⟨put macros⟩ Defined in 345cccxlv and 346cccxlvi Used in 553dliai and 557dlvii
- ⟨*put.c*⟩ Defined in 554dliv
- ⟨*put.h*⟩ Defined in 553dliai
- ⟨read and check the end byte *z*⟩ Defined in 17xvii
 - Used in 18xviii, 94xciv, 121cxxi, 139ccxxxix, 146cxlvi, 158clviii, 167clxvii, 202ccii, 313cccxiii, 369ccclxix, 390ccxc, 398cccxviii, and 412cdxii
- ⟨read the banner⟩ Defined in 338cccxviii Used in 557dlvii
- ⟨read the mantissa⟩ Defined in 64lxiv Used in 62lxii
- ⟨read the optional exponent⟩ Defined in 66lxvi Used in 62lxii
- ⟨read the optional sign⟩ Defined in 63lxiii Used in 62lxii
- ⟨read the start byte *a*⟩ Defined in 16xvi Used in 18xviii, 94xciv, 121cxxi, 139ccxxxix, 146cxlvi, 158clviii, 167clxvii, 202ccii, 313cccxiii, 369ccclxix, 390ccxc, 398cccxviii, and 412cdxii
- ⟨replace links to the parent directory⟩ Defined in 364ccclxiv Used in 362ccclxii
- ⟨return the binary representation⟩ Defined in 67lxvii Used in 62lxii
- ⟨rewrite the file names of optional sections⟩ Defined in 373ccclxxiii Used in 557dlvii
- ⟨scanning definitions⟩ Defined in 24xxiv, 33xxxiii, 40xl, 42xlii, 44xliv, 46xlvi, and 150cl
 - Used in 555dlv
- ⟨scanning functions⟩ Defined in 62lxii Used in 555dlv
- ⟨scanning macros⟩ Defined in 23xxiii, 26xxvi, 29xxix, 32xxxii, 41xli, 43xliai, 45xlvi, 47xlvi, 58lviii, 61lxi, and 153cliai Used in 555dlv
- ⟨scanning rules⟩ Defined in 3iii, 25xxv, 28xxviii, 35xxxv, 49xlvi, 56lvi, 60lx, 81lxxxi, 89lxxxix, 100c, 104civ, 110cx, 116cxvi, 125cxxv, 134ccxxxiv, 152clii, 162clxii, 170clxx, 175clxxv, 182clxxxii, 189clxxxix, 197ccxcvii, 205ccv, 210ccx, 215ccxcv, 219ccxcix, 223cccxiii, 230ccxxx, 255cclv, 269cclxix, 279cclxxix, 288cclxxxviii, 301ccci, 310cccx, 320cccx, 328cccxviii, 355ccclv, 378ccclxxviii, 386ccclxxxvi, 401cdi, 410cdx, and 443cdxlii Used in 555dlv
- ⟨shared get functions⟩ Defined in 53lii, 76lxxvi, and 146cxlvi
 - Used in 552dlai, 558dlviii, and 560dlx
- ⟨shared get macros⟩ Defined in 38xxxviii, 147cxlvi, 343ccclxiii, 368ccclxviii, and 497cdxcvii
 - Used in 551dlai and 560dlx
- ⟨shared put variables⟩ Defined in 273cclxxiii
 - Used in 554dlv, 557dlvii, 558dlviii, and 560dlx
- ⟨shared skip functions⟩ Defined in 466cdlxvi, 498cdxcviii, and 537dxxxvii
 - Used in 552dlai and 560dlx
- ⟨*shrink.c*⟩ Defined in 557dlvii

- ⟨ skip and check the start byte *a* ⟩ Defined in 496cdxcvi
Used in 494cdxciv, 500d, 511dxi, 514dxiv, 517dxvii, and 538dxxxviii
- ⟨ skip function declarations ⟩ Defined in 561dlxi Used in 560dlx
- ⟨ skip functions ⟩ Defined in 493cdxciii, 494cdxciv, 500d, 511dxi, 514dxiv, 517dxvii, 538dxxxviii, and 546dxlvi Used in 560dlx
- ⟨ skip macros ⟩ Defined in 499cdxcix, 501di, 502dii, 504div, 506dvi, 509dix, 512dxii, 515dxv, 518dxviii, 520dxx, 522dxxii, 524dxxiv, 526dxxvi, 528dxxviii, 530dxxx, 532dxxxii, 534dxxxiv, 540dxl, 542dxlii, and 544dxliv Used in 549dxlix and 560dlx
- ⟨ skip the end byte *z* ⟩ Defined in 495cdxcv
Used in 494cdxciv, 500d, 511dxi, 514dxiv, 517dxvii, and 538dxxxviii
- ⟨ **skip.c** ⟩ Defined in 560dlx
- ⟨ **stretch.c** ⟩ Defined in 558dlviii
- ⟨ symbols ⟩ Defined in 2ii, 4iv, 27xxvii, 34xxxiv, 48xlvi, 50l, 55lv, 80lxxx, 88lxxxviii, 99xcix, 103ciii, 109cix, 115cxv, 124cxxiv, 133cxxxiii, 142cxl, 151cli, 161clxi, 169clxiv, 174clxxiv, 181clxxxi, 188clxxxviii, 196cxcvi, 204cciv, 209ccix, 214ccxiv, 218ccxviii, 222ccxxii, 229ccxxix, 254ccliv, 268cclxviii, 278cclxxviii, 287cclxxxvii, 300ccc, 303ccci, 309cccix, 315cccxcv, 319cccxcix, 327cccxcvii, 354cccliv, 377ccclxxvii, 385ccclxxxv, 394cccxciv, 400cd, 409cdix, and 442cdxlii Used in 556dlvi
- ⟨ take care of variables without defaults ⟩ Defined in 419cdxix Used in 549dxlix
- ⟨ update the file sizes of optional sections ⟩ Defined in 372ccclxxii Used in 371ccclxxi
- ⟨ without **-f** skip writing an existing file ⟩ Defined in 360ccclx Used in 366ccclxvi
- ⟨ write a list ⟩ Defined in 145cxlv Used in 144cxliv
- ⟨ write a text ⟩ Defined in 155clv Used in 144cxliv
- ⟨ write functions ⟩ Defined in 21xxi, 31xxxi, 36xxxvi, 52lii, 68lxviii, 79lxxix, 84lxxxiv, 91xci, 102cii, 118cxviii, 127cxxvii, 136cxxxvi, 144cxliv, 156clvi, 164clxiv, 177clxxvii, 191xcxi, 199cxcix, 232ccxxxii, 259cclix, 260cclx, 271cclxxi, 330cccxxx, 332cccxxxii, 366ccclxvi, 367ccclxvii, 380ccclxxx, 389ccclxxxix, 404cdiv, 418cdxviii, and 445cdxlv
Used in 558dlviii and 560dlx
- ⟨ write large numbers ⟩ Defined in 72lxxii Used in 68lxviii
- ⟨ write macros ⟩ Defined in 22xxii Used in 558dlviii and 560dlx
- ⟨ write medium numbers ⟩ Defined in 73lxxiii Used in 68lxviii
- ⟨ write small numbers ⟩ Defined in 74lxxiv Used in 68lxviii

References

- [1] Peter Deutsch and Jaen-Loup Gailly. *RFC1950, ZLIB Compressed Data Format Specification Version 3.3*. RFC Editor, United States, 1996.
- [2] Jaen-loup Gailly and Mark Adler. zlib.
<http://zlib.net/>.
- [3] IANA Internet Assigned Numbers Authority, Los Angeles, CA. *IANA Charset MIB*, May 2014.
- [4] IANA Internet Assigned Numbers Authority, Los Angeles, CA. *Character Sets Registry*, December 2018.
- [5] IANA Internet Assigned Numbers Authority, Los Angeles, CA. *Language Tags*, April 2020.
- [6] IEEE Computer Society Standards Committee. Working group of the Microprocessor Standards Subcommittee and American National Standards Institute. *IEEE standard for binary floating-point arithmetic*. IEEE Computer Society Press, 1985.
- [7] IEEE Computer Society Standards Committee. Working group of the Microprocessor Standards Subcommittee and American National Standards Institute. IEEE Standard 754-2008. Technical report, August 2008.
- [8] Donald E. Knuth. *The T_EX book*. Computers & Typesetting, Volume A. Addison-Wesley Publishing Company, 1984.
- [9] Donald E. Knuth. *T_EX: The Program*. Computers & Typesetting, Volume B. Addison-Wesley, 1986.
- [10] Donald E. Knuth. *Literate Programming*. CSLI Lecture Notes Number 27. Center for the Study of Language and Information, Stanford, CA, 1992.
- [11] Donald E. Knuth and Silvio Levy. *The CWEB System of Structured Documentation*. Addison Wesley, 1994. <https://ctan.org/pkg/cweb>.
- [12] John R. Levine. *flex & bison*. O'Reilly Media, 2009. ISBN 978-0-596-15597-1.
- [13] John R. Levine, Tony Mason, and Doug Brown. *lex & yacc*. O'Reilly Media, 2012.
- [14] Ira McDonald. *IANA Charset MIB*. IETF Internet Engineering Task Force, rfc 3808 edition, June 2004.

-
- [15] Addison Phillips and Mark Davis. Tags for Identifying Languages. RFC 5646, September 2009.
 - [16] Martin Ruckert. Computer Modern Roman fonts for ebooks. *TUGboat*, 37(3):277–280, 2016.
 - [17] Martin Ruckert. Converting T_EX from WEB to cweb. *TUGboat*, 38(3):353–358, 2017.
 - [18] Martin Ruckert. `web2w` package on CTAN. <https://ctan.org/pkg/web2w>, August 2017.
 - [19] Martin Ruckert. HINT: Reflowing T_EX output. *TUGboat*, 39(3):217–223, 2018.
 - [20] Martin Ruckert. *WEB to cweb*. 2 edition, 2021. ISBN 979-8-54989510-2. <https://www.amazon.de/dp/B09BY85KG9>.

Index

Symbols

' 17
'g' 16
'a' 1
- 53–55
/ 14
< 54
<max 163
> 54, 163
□ 53–55
\\ 14, 53
\\- 54
\\< 54
\\> 54
\\@ 55
_ 54
_ 54
\\a 54
\\b 54
\\c 54
\\j 54
\\k 54
\\l 54

__BASETYPES_H__ 221
__SIZEOF_DOUBLE__ 221
__SIZEOF_FLOAT__ 221
_access 151
_HFORMAT_H_ 222
_MSC_VER 221, 228

A

above_display_short_skip_no 182
above_display_skip_no 182
absolute 151
access 150
ADD 65
adj_demerits_no 180

ADJUST 81
adjust_kind 5, 53, 81, 205, 219
adjustment 81, 205, 219
ALIGN 69
alignment 61, 68, 82, 205
alloc_func 145
ALLOCATE 100, 107, 109, 131, 145, 147,
150, 158, 174, 192, 194
argc 191, 230, 233
argv 191–193, 230, 233
AS_STR 222
asterisk 176
atof 20
atoi 56
aux_ext 151
aux_length 151
aux_name 150–153, 158, 194
aux_names 157
auxiliary file 137
avail_in 145
avail_out 145
awful_bad 122

B

b000 6
b001 6
b010 6
b011 6
b100 6
b101 6
b110 6
b111 6
backslash 53
banner 137
BASELINE 71, 164, 168, 177
baseline 71, 168
baseline_defaults 222
baseline_kind 5, 71, 164, 168, 177, 183,
203, 214, 220

baseline skip 65, 70, 183, 203, 214
baseline_skip_no 182
below_display_short_skip_no 182
below_display_skip_no 182
BG 112, 115
BigEndian16 91, 95
BigEndian32 91, 93
bison 3
bits 23–26, 208
BOOL 221
bool 221
BOT 100
 box 61, 74, 121, 182, 201, 211
box 62
 box 255 122
box_dimen 62, 66
box_flex 66
box_glue_set 62
box_goal 65, 83
box_options 65
box_shift 62, 66
broken_penalty_no 180
bsize 104, 134, 145–147, 150, 157, 162, 188
 buffer 147
buffer 104, 134, 144–148, 150, 156, 159, 162, 188
buffer_factor 147
 buffer overrun 141
BUFFER_SIZE 147
build_page v

C

calloc 132
 carriage return character 54
cc_list 73
ceil 26
CENTER 69
 centered 68
 character code 16, 52
CHARCODE 16, 18, 73
check_param_def 171
close 144
club_penalty_no 180
 code file 221
COLOR 112, 115, 164
 color 110, 206
 color 113
color_defaults 112–116, 222

color_init 113, 227
color_kind 106, 113–115, 164, 168, 170, 185, 206, 217
color_pair 113
color_set 113
color_tripple 113
color_unset 113
colors_0 112–114, 227
colors_equal 114
colors_i 112–114, 227
colors_n 112–114, 227
 command line 189
 comment 3
 compression 144, 154
CONTENT 187
content_known 9, 198, 222
content_list 49, 187
content_name 5, 8, 50, 59, 175, 178, 198, 222
content_node 3, 35, 39, 41, 44, 49, 57, 62, 66, 69, 71, 73, 75, 79–81, 83, 87, 101, 106, 115, 118, 126, 128, 132, 176
 content section 137, 187
content_section 140, 187
 control code 52
cs 115
CURCOLOR 111
 current font 52

D

DAY 111
day_no 180
DBG 195
DBGBASIC 140, 153, 162, 188–191, 232
DBGBISON 190, 230
DBGBUFFER 145, 147, 190
DBGCOMPRESS 145, 189
DBGDEF 96, 98, 104, 110, 127, 162, 164–167, 175, 189
DBGDIR 138, 140, 146, 150–153, 155–159, 188–190
DBGFLEX 190, 230
DBGFLOAT 21–25, 31, 189
DBGFONT 190
DBGLABEL 98, 101–105, 110, 190
DBGNODE 50–52, 101, 189
DBGNONE 190
DBGPAGE 190

DBGRANGE 132–135, 189
 DBGRENDER 190
 DBGTAG 195
 DBGTAGS 118–120, 156, 189, 195, 220, 233
 DBGTEX 190
 DBL_E_BITS 20, 24
 DBL_EXCESS 20, 23
 DBL_M_BITS 20, 22–25
dc 75, 229
 DEBUG 189, 195
debug 228
debugflags 190–192, 195, 222, 230
 debugging 189, 195
 decimal number 13
 DEF 113, 126, 167–169
 DEF_KIND 4–6
def_list 171
def_node 109, 113, 117, 161, 168, 171
 DEF_REF 169
 default value 163, 179
definition_bits 167
definition_list 161
definition_name 5, 163–167, 169–171, 175, 222
 definition section 137, 161, 166
definition_section 140, 161
 DEFINITIONS 161
 deflate 144
deflate 146
deflateEnd 146
deflateInit 145
 DEPTH 65
 DESCRIPTION 189, 230, 233
df 127, 171, 175
diff_focus 115
diff_high 115
digits 21–25
 DIMEN 27, 82, 164, 168
dimen_defaults 181, 222
dimen_kind 5, 28, 130, 164, 168, 171, 176, 181, 220
 dimension 27, 170
dimension 27, 29, 38, 62, 66, 71, 130, 168, 174
dir 89, 91, 95, 104, 134, 145–148, 150, 153, 156–159, 162, 188, 224, 226
 DIRECTORY 149
 directory entry 154
 directory section 137, 149, 154

directory_section 140, 149
disable 221, 228
 DISC 75, 164, 168, 174, 176
disc 75, 168, 174
disc_kind 5, 58, 60, 76, 164, 168, 175, 177, 179, 204, 215, 220
disc_node 75, 174
 discretionary break 74, 204
 discretionary breaks 215
 discretionary hyphen 54
display_widow_penalty_no 180
 displayed formula 78, 204, 216
double 20
double_hyphen_demerits_no 180
 double quote 53

E

emergency_stretch_no 181
 empty list 49
empty_list_no 185
 END 2–4, 8, 29, 35, 39, 41, 44, 49, 62, 66, 69, 71, 73, 75, 79–81, 83, 87, 101, 106, 109, 113, 115, 117, 126, 128, 132, 149, 161, 164, 168, 171, 174, 176, 187
 end byte 6, 8, 208
entries 150, 224, 226
 entry 149
entry_list 149
 EOF 139
 equation number 79
 error message 193, 195
 estimate 48, 53
estimate 49, 171
ex_hyphen_penalty_no 180
exit 191–193, 195
exp 21–26
 EXPAND 69
 expanded 68
 EXPLICIT 41
 explicit 74, 77
 explicit kern 40
 exponent 20
ext_length 151, 192
 extended box 64, 202, 212
 extended dimension 28, 64, 182, 200, 208

F

- F_OK** 150
 - false* 221
 - fclose* 96, 143, 153, 159, 194
 - fd* 144
 - feof* 159
 - fflush* 195
 - FG** 112, 115
 - fgetc* 139
 - FIL** 32
 - fil_o* 31–33, 182
 - fil_skip_no* 182
 - file* 137
 - file name* 14, 191
 - file_name* 89, 91, 95, 150, 153–155, 157–159, 191–194, 224, 228
 - file_name_length* 191–194
 - file size* 158
 - FILL** 32
 - fill_o* 31–33, 182
 - fill_skip_no* 182
 - FILLL** 32
 - filll_o* 31–33
 - final_hyphen_demerits_no* 180
 - FIRST** 126
 - first_color* 114, 116
 - first_label* 100–103, 226
 - first stream* 123
 - fixed point number* 26
 - FLAGS** 195
 - flex* 2
 - float** 20
 - float32_t** 20
 - float32_t** 31
 - float64_t** 20
 - float64_t** 26, 31
 - floating_penalty* 122, 128
 - floating_penalty_no* 180
 - floating point number* 20, 208
 - floor* 24, 26, 92–95
 - FLT_E_BITS** 20, 31
 - FLT_EXCESS** 20, 32
 - FLT_M_BITS** 20, 31
 - fn* 91–96
 - FOCUS** 112, 115
 - FONT** 164, 168, 174
 - font* 52, 137, 172
 - font** 172
 - font* 168, 174
 - font at size* 173
 - font design size* 173
 - font_head* 174
 - font_kind* 3, 5, 9, 16, 57, 60, 73, 164, 168, 174–176, 179
 - font_param* 174
 - font_param_list* 174
 - font parameter* 54
 - footnote* 121
 - fopen* 95, 143, 153, 159, 193
 - format.h** 222
 - FPNUM** 20
 - fprintf* 11, 139, 189–193, 195
 - fread* 91, 143, 159
 - free* 109, 142, 146, 153, 156, 158, 194
 - free_func* 145
 - fref* 174
 - freopen* 193
 - from* 133–135
 - fseek* 93
 - fsize* 159
 - fstat* 144
 - FT_ENCODING_ADOBE_CUSTOM** 173
 - FT_ENCODING_UNICODE** 173
 - fwrite* 147, 153, 159
- G**
- get_BMP_info* 92, 96
 - get_content* 187
 - GET_DBIT** 167
 - GET_IMG_BUF** 91–94
 - get_JPG_info* 94, 96
 - get_PNG_info* 93, 96
 - get.c** 225
 - get.h** 224
 - GLUE** 43, 164, 168, 174, 176
 - glue* 31, 42, 54, 70, 85, 121, 171, 182, 201, 211
 - glue* 43, 168, 174
 - glue_defaults* 182, 222
 - glue_kind* 5, 44, 58, 60, 164, 168, 171, 175–177, 182, 201, 211, 220
 - glue_node* 43, 69, 71, 126, 130, 174, 177
 - glue ratio* 61, 65
 - GLYPH** 2–4
 - glyph* 1, 74, 172, 199, 209
 - glyph** 2
 - glyph* 3, 16
 - glyph_kind* 5–7, 10, 199, 209

grammar 3, 61

H

hang_after_no 180

hang_indent_no 181

HBACK 208, 218

hbanner 137–139

hbanner_size 137–139

HBOX 62

hbox_kind 5, 62–64, 70, 82, 201, 212, 214

hbox_node 62, 69, 80

hcheck_banner 137, 224, 230, 233

hclear_dir 156, 224

hcompress 145, 157

hcompress_depth 107, 227

hdecompress 144, 146

header file 221

HEIGHT 86

HEND 141

hend 8, 16, 50, 57, 104, 134, 139–141,
145–147, 157, 162, 165, 172,
188, 207, 224, 226, 231, 233

hexadecimal 13, 21

hextract_image_dims 89, 95, 226

hff_hpos 198, 225, 233

hff_list_pos 197, 199, 225, 233

hff_list_size 197, 199, 225, 233

hff_tag 197–199, 225, 233

hfont_name 11, 173–175, 228

hget_banner 139, 224, 231, 233

HGET_BASELINE 71

HGET_BOX 63

hget_color_set 115

hget_content 8, 59, 170

hget_content_node 8, 49, 83, 120, 188,
207, 232

hget_content_section 187, 207, 232

hget_def_node 162, 170, 172, 232

hget_definition 169, 175

hget_definition_section 162, 231, 233

hget_dimen 28, 130, 169

hget_directory 155, 224, 231, 233

HGET_DISC 76

hget_disc_node 76, 175

HGET_ENTRY 154

hget_entry 155, 224

HGET_ERROR 141

hget_float32 26, 30, 63, 87, 225

hget_font_def 169, 175, 232

hget_font_params 175

HGET_GLUE 44

hget_glue_node 45, 70, 127, 130, 175, 232

HGET_GLYPH 9

HGET_GREF 58

hget_hbox_node 64, 70, 80, 232

HGET_IMAGE 87

HGET_KERN 41

HGET_LEADERS 69

HGET_LIG 74

HGET_LINK 105

HGET_LIST 51, 169

hget_list 51, 63, 68, 74, 76, 79–81, 83,
88, 108, 120, 127, 129, 172, 225

hget_list_size 50, 225

hget_map 142, 144, 224, 231, 233

HGET_MATH 80

hget_max_definitions 162, 165, 224

HGET_N 9

hget_outline_or_label_def 98, 170

HGET_PACK 67

hget_page 130, 169

HGET_PAR 79

hget_param_list 79, 120, 129, 172, 232

HGET_PENALTY 36

hget_range 133, 170

HGET_REF 68, 79, 84, 129, 177

hget_root 155

HGET_RULE 39

hget_rule_node 40, 70, 232

hget_section 146, 153, 156, 162, 188,
207, 224

HGET_SET 67

HGET_SIZE 154

hget_size_boundary 50, 224

HGET_STREAM 128

hget_stream_def 127, 130

HGET_STRETCH 32, 45, 68

HGET_STRING 15, 130, 155, 169, 175

HGET_TABLE 83

hget_txt 57, 232

hget_unknown 9, 120, 232

hget_unknown_def 119, 170

hget_unmap 142–144, 224, 231, 234

hget_utf8 19, 58, 73, 224

HGET_UTF8C 19

hget_vbox_node 64, 70, 232

HGET_XDIMEN 30

hget_xdimen 30, 169

- hget_xdimen_node* 30, 41, 45, 68, 79, 84, 87, 120, 127, 130, 232
- HGETTAG 8, 141, 165
- HIGH 112, 115
- hin* 139, 193, 230
- hin_addr* 142–144, 146, 224, 231, 233
- hin_name* 142–144, 158, 192–194, 224
- hin_size* 142–144, 224, 231, 233
- hin_time* 142–144, 224
- hint* 140, 229
- HINT_MINOR_VERSION 138, 222
- HINT_NO_POS 96, 131, 133, 147
- HINT_VERSION 138, 222
- HINT_VERSION_STRING 138, 191, 222
- hlog* 193–195, 222, 230
- hnode_size* 117–120, 142, 198–206, 220, 222
- horizontal box 61
- horizontal list 38
- hout* 11, 139, 147, 159, 193, 226, 233
- HPACK 65
- hpack* 64
- hpack* 65
- hpack_kind* 5, 64–67, 82, 202, 213
- hpos0* 56, 101, 140, 145–147, 188, 224, 226
- hput_banner* 139, 231
- hput_baseline* 72
- hput_box_dimen* 62, 64, 66, 226
- hput_box_glue_set* 62, 64, 226
- hput_box_shift* 62, 64, 226
- hput_color_def* 113, 227
- hput_color_set* 114
- hput_content_end* 187, 226
- hput_content_start* 187, 226
- hput_data* 147, 157
- hput_definitions_end* 104, 135, 161, 226
- hput_definitions_start* 161, 226
- hput_dimen* 28, 168, 227
- hput_directory* 157, 227, 230
- hput_directory_end* 157
- hput_directory_start* 157
- hput_disc* 75, 77, 168, 174, 226
- hput_entry* 156
- hput_error* 141
- hput_float32* 26, 30, 64, 89
- hput_font_head* 174, 176, 227
- hput_glue* 44, 168, 174, 226
- hput_glyph* 3, 6, 9, 226
- hput_hint* 140, 227, 230
- hput_image_aspect* 88
- hput_image_dimen* 89
- hput_image_dimens* 88–90
- hput_image_spec* 86, 88, 227
- hput_increase_buffer* 141, 147
- hput_int* 35, 168, 174, 226
- hput_kern* 41, 174, 226
- hput_label* 103–105
- hput_label_defs* 104, 187, 226
- hput_language* 37, 177, 226
- hput_ligature* 73, 168, 174, 226
- hput_link* 106, 226
- hput_list* 49, 52, 56, 74, 101, 168, 171, 226
- hput_list_size* 49, 51, 227
- hput_max_definitions* 164, 166, 227
- hput_n* 7, 99, 166
- hput_optional_section* 140, 159
- hput_outline* 104, 110
- hput_range* 132, 134, 227
- hput_range_defs* 134, 187, 227
- hput_root* 140, 157
- hput_rule* 39, 168, 174, 226
- hput_section* 140, 148
- hput_span_count* 83, 226
- hput_stretch* 31, 45, 66, 226
- hput_string* 15, 130, 157, 168, 174, 176, 227
- hput_txt_cc* 57, 59, 226
- hput_txt_font* 57, 59, 226
- hput_txt_global* 57, 60, 226
- hput_txt_local* 57, 60, 226
- hput_unknown* 118, 227
- hput_unknown_def* 117, 227
- hput_utf8* 20, 59, 73, 226
- hput_xdimen* 29–31, 168, 226
- hput_xdimen_node* 31, 42, 45, 78, 88, 227
- HPUTNODE 3, 15, 141
- HPUTTAG 142, 156, 166
- HPUTX 7, 16, 20, 52, 57, 59, 104, 110, 114, 141
- hpx* 92–95
- hr* 86–89, 216, 227
- HSET 65
- hset_entry* 150, 155, 224, 228
- hset_kind* 5, 64–67, 82, 202, 213
- hset_label* 101, 226
- hset_max* 164, 179
- hset_outline* 109, 226
- hsize* 28

- hsize_bytes* 49, 51, 226
- hsize_dimen_no* 181
- hsize_xdimen_no* 182
- hsort_labels* 102, 187
- hsort_ranges* 133, 187
- hstart* 3, 7, 19, 31, 49–52, 56, 73, 76, 99, 101, 104, 109, 114, 119, 127, 132, 134, 140, 145–147, 153, 155, 157, 162, 165, 171, 188, 195, 199, 207, 215, 218, 220, 224, 226, 231, 233
- HTEG_BASELINE 214
- HTEG_BOX 211
- hteg_content* 207, 234
- hteg_content_node* 207, 219, 234
- hteg_content_section* 207, 233
- HTEG_DISC 215
- hteg_float32* 208, 211, 216, 225
- HTEG_GLUE 211
- hteg_glue_node* 211, 214, 234
- HTEG_GLYPH 209
- hteg_hbox_node* 212, 214, 216, 234
- HTEG_IMAGE 216
- HTEG_KERN 210
- HTEG_LEADERS 214
- HTEG_LIG 215
- HTEG_LINK 217
- hteg_list* 211–213, 215, 218–220, 225, 234
- hteg_list_size* 218, 225
- HTEG_MATH 216
- HTEG_PACK 213
- HTEG_PAR 215
- hteg_param_list* 215, 218–220, 234
- HTEG_PENALTY 210
- HTEG_REF 211–213, 215, 219
- HTEG_RULE 210
- hteg_rule_node* 211, 214, 234
- HTEG_SET 212
- hteg_size_boundary* 218, 225
- HTEG_STREAM 219
- HTEG_STRETCH 209, 211
- HTEG_TABLE 219
- hteg_unknown* 207, 220, 234
- hteg_vbox_node* 212, 214, 234
- HTEG_XDIMEN 208
- hteg_xdimen_node* 209–213, 215, 219, 234
- HTEG16 208–210, 217
- HTEG24 208
- HTEG32 208–211, 213, 217
- HTEG8 208–211, 215–220
- HTEGTAG 208
- hwrite_* 101
- hwrite_aux_files* 153, 231
- hwrite_box* 62–64
- hwrite_charcode* 10, 18, 73
- hwrite_color_pair* 115
- hwrite_comment* 11
- hwrite_content_section* 187, 207, 231
- hwrite_definitions_end* 162
- hwrite_definitions_start* 162
- hwrite_dimension* 28, 39, 62, 67, 71, 175
- hwrite_directory* 154, 231
- hwrite_disc* 76
- hwrite_disc_node* 76, 175
- hwrite_end* 8–10, 29, 40, 44, 50, 64, 76, 99, 102, 108, 116, 120, 127, 153, 165, 170, 172, 175, 178
- hwrite_entry* 153
- hwrite_explicit* 41, 76
- hwrite_float64* 23, 26, 29, 33, 62, 87
- hwrite_glue* 44
- hwrite_glue_node* 44, 71
- hwrite_glyph* 9–11
- hwrite_image* 87
- hwrite_kern* 41
- hwrite_label* 10, 50, 101, 188
- hwrite_leaders_type* 69
- hwrite_ligature* 73
- hwrite_link* 105
- hwrite_list* 50, 62, 68, 76, 79–81, 83, 88, 108, 120, 127, 129
- hwrite_max_definitions* 162, 165
- hwrite_minus* 44, 68
- hwrite_named_param_list* 120, 172
- hwrite_nesting* 10, 57
- hwrite_order* 33, 62
- hwrite_param_list* 79, 129, 172
- hwrite_parameters* 169, 172
- hwrite_plus* 44, 68
- hwrite_range* 10, 132, 188
- hwrite_ref* 11, 28, 37, 41, 73, 79, 106, 115, 127–129, 177
- hwrite_ref_node* 44, 178
- hwrite_rule* 39
- hwrite_rule_dimension* 39
- hwrite_scaled* 26, 28
- hwrite_signed* 14, 36

hwrite_start 8–10, 29, 40, 44, 50, 64, 76,
99, 102, 108, 115, 119, 127, 132,
153, 165, 170, 172, 175, 178
hwrite_stretch 33, 44
hwrite_string 15, 130, 153, 169, 175
hwrite_txt_cc 57, 59, 73
hwrite_utf8 18, 58
hwrite_xdimen 29, 41, 44, 79, 87, 169
hwrite_xdimen_node 29, 68, 84, 120,
127, 130
hwritec 10, 15, 18, 24, 29, 57, 59, 73,
102, 115, 132
hxbbox_node 66
hyphen 55
hyphen character 53
hyphen_penalty_no 180

I

ia 90
IEEE754 20
ih 90
illustration 121
IMAGE 86, 164, 168, 174, 177
image 54, 137, 205, 216
image 86, 168, 174
image_aspect 86
image_height 86
image_kind 5, 58, 60, 87, 164, 168, 174,
177, 179, 205, 217, 220
image_spec 86
image_width 86
img_buf 91, 93–95
IMG_BUF_MAX 91
img_buf_size 91, 93, 96
IMG_HEAD_MAX 91
in 27
in 27
in_ext 189, 191, 230, 233
INCH 27
inch 27
indentation 54
inflate 144
inflate 145
inflateEnd 145
inflateInit 145
INFO 7–9, 28, 30, 40, 45, 50–52, 64, 76,
119, 127, 165, 169, 195, 199,
208, 211, 218, 233
info 4

info value 6
INITIAL 56
input file 193
insert node 121
insert_penalties 122
int_defaults 180, 222
int_kind 5, 35, 118, 164, 168, 171, 180,
183
INT16 221
INT32 221
int32_t 35
INT8 221
INTEGER 35, 164, 168
integer 13, 170, 179
integer 13, 35, 109, 168
inter_line_penalty_no 180
interactive 228
interword glue 55
isalpha 152
ITEM 82
item_kind 5, 83, 205, 219
iw 90

K

KERN 41, 174, 176
kern 40, 54, 74, 121, 182, 200, 210
kern 41, 174
kern_kind 5, 41, 58, 60, 175–177, 200,
210, 220
kind 4
kt 41, 229

L

LABEL 100, 164
Label 99
label 96, 184
LABEL_BOT 100–102
label_defaults 222
label_kind 6, 98–104, 106, 109, 163, 170,
179, 184
LABEL_MID 100, 103
LABEL_TOP 100–102
LABEL_UNDEF 100–106, 108
labels 99–104, 106, 108–110, 224, 226
LANGUAGE 37, 164, 168, 174, 177
language 54, 200, 210
language_kind 5, 37, 58, 60, 164, 168,
174, 177, 179, 200, 210, 220
LAST 126

last stream 123
 LEADERS 69, 164, 168, 177
 leaders 38, 68, 203, 214
 leaders 69, 168
 leaders_kind 69, 164, 168, 177, 179, 203,
 214, 220
 left_skip_no 182
 lex 2
 lex_debug 229
 lexer.l 228
 lg 73, 229
 lig_cc 73
 LIGATURE 73, 164, 168, 174, 176
 ligature 54, 72, 74, 203, 215
 ligature 73, 168, 174
 ligature_kind 5, 58, 60, 74, 164, 168,
 175–177, 179, 203, 215, 220
 line breaking 70, 77, 81
 line_penalty_no 180
 line skip glue 70
 line skip limit 70
 line_skip_limit_no 181
 line_skip_no 182
 linear function 28
 LINK 106
 link 96, 206
 link 131, 134
 link_color_no 185
 link_kind 6, 98, 105, 108, 206, 217
 list 47, 199, 218
 list 49, 56, 62, 66, 75, 78–83, 86, 108,
 118, 126, 128, 130
 list_end 52
 list_kind 5, 47, 49–51, 53, 56, 74, 76,
 120, 163, 165–167, 185, 215,
 218, 220, 233
 LittleEndian32 91
 LOG 138, 195
 log file 193
 LOG_PREFIX 195
 looseness_no 180
 lslimit 70
 ltype 69

M
 magic 137–139, 224
 main 191, 207, 223, 230, 233
 malloc 143
 MAP_FAILED 144

MAP_PRIVATE 144
 margin note 121
 mark node 121
 Match2 91, 94
 Match4 91, 93
 MATH 80
 math 80
 math_kind 5, 80, 204, 216
 math_quad_no 181
 Mathematics 79
 mathematics 79, 204, 216
 MAX 66, 164
 MAX_BANNER 137–139
 MAX_BASELINE_DEFAULT 168, 183, 222
 MAX_COLOR_DEFAULT 168, 185, 222
 max_default 163, 165, 169–171, 179–
 185, 222–224
 max_definitions 161, 164
 max_depth 65
 max_depth_no 181
 MAX_DIMEN 27, 65–68
 MAX_DIMEN_DEFAULT 168, 181, 222
 max_fixed 163–165, 167, 169–171, 179–
 185, 222–224
 MAX_FONT_PARAMS 173–175
 MAX_GLUE_DEFAULT 168, 182, 222
 MAX_HEX_DIGITS 24
 MAX_INT_DEFAULT 168, 180, 222
 MAX_LABEL_DEFAULT 184, 222
 max_list 164
 MAX_LIST_DEFAULT 167, 185
 max_outline 98, 107–110, 165, 222, 226
 MAX_PAGE_DEFAULT 168, 184
 max_range 131–133, 224, 226
 MAX_RANGE_DEFAULT 168, 185
 MAX_REF 163–165
 max_ref 99, 102, 104, 126, 131, 134,
 163–167, 170, 174, 222
 max_section_no 87, 140, 146, 150, 153–
 158, 175, 224, 226
 MAX_STR 14
 MAX_STREAM_DEFAULT 168, 184
 MAX_TAG_DISTANCE 141, 145–147, 157
 max_value 99, 164
 MAX_XDIMEN_DEFAULT 168, 182, 222
 maximum values 161, 163
 memcmp 115
 memcpy 116
 memmove 52, 109
 MESSAGE 195

message 195
 Microsoft Visual C 221
 MID 100
 millimeter 27
 MINUS 43, 62
 minus 43, 66
 mkdir 152
 mktables.c 223
 MM 27
 mm 27
 mmap 142, 144
 month_no 180
 munmap 143

N
 NAME 6, 8, 30, 40, 60, 64, 76, 119, 195,
 199, 208, 211
 name_type 151
 named_param_list 118, 171
 natural dimension 65
 nesting 10, 57, 102, 132
 never 228
 new_directory 149, 156, 224, 226
 new_output_buffers 147, 149, 226
 newline character 54, 137
 next 99–103
 next_in 145
 next_out 145
 next_range 131–134, 226
 NIGHT 111, 115
 NODE_HEAD 119, 197, 220
 node_pos 8, 30, 40, 51, 64, 76, 98, 103,
 108, 115, 165, 169, 175, 208,
 211, 218
 NODE_SIZE 118, 197, 199–206
 NODE_TAIL 119, 197, 220
 noinput 228
 NOREFERENCE 126
 normal_o 31–33, 182
 nounistd 228
 nounput 228
 noyy_top_state 228
 number 20, 27, 29, 32, 86

O

O_RDONLY 144
 OFF 81, 115, 132
 ON 81, 132
 on 106, 131–134, 226
 on_off 81, 106
 ONE 26, 92–96, 181–183
 opaque 145
 open 144
 option 189
 option 191
 option_aux 153, 158, 190, 192
 option_compress 157, 190, 192
 option_force 150, 190, 192
 option_global 153, 158, 190, 192
 option_hex 19, 190, 192
 option_log 191, 193
 option_utf8 19, 58, 190, 192
 optional section 137
 order 32
 out_ext 191, 193, 230, 233
 OUTLINE 99, 109
 Outline 104
 outline 96
 outline_kind 6, 98, 109
 outline_no 108
 outlines 107–110, 226
 output file 193
 output routine 121

P

PAGE 130, 164, 168
 page 130, 168
 page building 121
 page_depth 122
 page_goal 122
 page_kind 5, 131–133, 164, 168, 184
 page_max_depth 122
 page_on 131, 134, 226
 page_priority 130
 page range 130, 185
 page_shrink 122
 page_stretch 122
 page_total 122
 PAR 78
 par 78
 par_dimen 78
 par_fill_skip_no 182
 par_kind 5, 79, 204, 216, 233

paragraph 65, 77, 79, 81, 128, 204, 215
 PARAM 164, 168, 171
 param_kind 5, 47, 51, 53, 79, 118–120,
 129, 164, 167–169, 171, 176,
 179, 185, 198, 215, 218–220
 param_list 78–80, 128, 171
 param_ref 78, 80, 128, 176
 parameter 47
 parameter list 170
 parameters 199
 parameters 168, 171
 parse 229
 parser.y 228
 parsing 3, 8, 61, 228
 path 151
 path_end 152
 path_length 151, 192
 PENALTY 35, 174, 176
 penalty 35, 54, 121, 180, 200, 210
 penalty 35, 174
 penalty_kind 5, 35, 48, 58, 60, 175–177,
 200, 210, 220
 pg 131–135, 227
 placement 100
 PLUS 43, 62
 plus 43, 66
 point 27
 pos0 99, 101, 103, 147
 position 96, 140
 position 49, 56, 109, 171
 post_break 74
 post_display_penalty_no 180
 pre_break 74
 pre_display_penalty_no 180
 pretolerance_no 180
 PRINT_GLUE 183
 printers point 27
 printf 5, 181–185, 198, 223
 priority 129
 PRIx64 22–25, 143, 156, 221
 prog_name 189, 191–193
 PROT_READ 144
 PT 27, 32
 pt 27
 put_hint 140
 put.c 227
 put.h 225
 putc 11

Q

quad_no 181
 QUIT 195

R

radix point 20
 RANGE 132, 164
 range_kind 5, 53, 131, 133–135, 164,
 168, 170, 185
 range_pos 131–135, 226
 realloc 132
 REALLOCATE 132, 147
 REF 3, 16, 28, 37, 45, 57, 73, 79, 115,
 132, 167, 176
 Ref 60, 127, 171, 175
 ref 73, 113, 115, 126, 168, 174, 176
 REF_RNG 9, 51, 101, 103, 106, 108, 115,
 126–129, 163, 167, 170, 176
 REFERENCE 2–4, 16, 73, 86, 101, 106,
 109, 132, 177
 reference 176, 220
 reference point 61
 relative 151
 replace count 75
 replace_count 74
 resynchronization 47
 rf 55, 126, 168, 229
 right_skip_no 182
 RNG 195
 root 155
 ROUND 26
 round 91
 RULE 38, 164, 168, 174, 177
Rule 38
 rule 38, 54, 74, 121, 201, 210
 rule 38, 168, 174
 rule_dimension 38
 rule_kind 5, 39, 58, 60, 70, 164, 168,
 175, 177, 179, 201, 210, 214, 220
 rule_node 39, 69
 RUNNING 38
 RUNNING_DIMEN 38–40, 210
 running dimension 38

S

- S_IFDIR** 152
- scaled integer 26
- scaled point 27
- SCAN_** 3
- SCAN_DEC** 13
- SCAN_DECFLOAT** 20
- SCAN_END** 2, 53, 56
- SCAN_HEX** 13
- SCAN_HEXFLOAT** 21
- scan_level* 56
- SCAN_REF** 56
- SCAN_START** 2, 53, 56
- SCAN_STR** 14
- SCAN_TXT_END** 56
- SCAN_TXT_START** 55
- SCAN_UDEC** 3, 13
- SCAN_UTF8_1** 17
- SCAN_UTF8_2** 17
- SCAN_UTF8_3** 17
- SCAN_UTF8_4** 17
- scanning 2, 193, 228
- SECTION** 149
- section 137
- section_no* 10, 50, 104, 126, 134, 141, 147, 150, 153–157, 159, 162, 187, 224, 226
- SEEK_SET** 93
- SET_DBIT** 167, 170, 174
- shift amount 61
- SHIFTED** 62
- ship_out* v
- short format 140
- SHRINK** 189, 191, 230
- shrink.c** 229
- shrinkability 31, 42, 61, 65, 209
- SIGNED** 13, 21
- signed integer 13
- single quote 14–17
- size* 93–95, 104, 134, 145–148, 150, 153, 156–159, 162, 188, 224, 228
- SIZE_F** 195
- size_pos* 226
- size_t** 195
- SKIP** 191, 233
- skip** 207
- skip.c** 232
- space character 53–55
- span_count* 83
- split_max_depth* 128
- split_max_depth_no* 181
- split ratio 124
- split_top_skip* 128
- split_top_skip_no* 182
- st* 31, 43, 143, 209, 229
- st_mode* 152
- st_mtime* 143
- st_size* 143, 158
- stack* 228
- START** 2–4, 8, 101, 109, 113, 132, 149, 161, 164, 187
- start byte 4, 8, 208
- start_pos* 52
- stat* 143, 152, 158
- Stch** 31
- stderr* 191–193
- stdout* 189–191
- stem_length* 151–153, 190, 192–194
- stem_name* 151, 153, 190–194
- STR** 14, 18
- str* 11, 15, 139, 227
- STR_ADD** 14
- str_buffer* 14
- STR_END** 14
- str_length* 14
- STR_PUT** 14, 18
- STR_START** 14, 18
- strcat* 192
- strcmp* 191
- strcpy* 151, 192, 194
- strdup* 150, 153, 174
- STREAM** 125, 128
- stream 123, 125, 128, 184, 206, 219
- stream* 128
- stream_def_list* 130
- stream_def_node* 126, 130
- stream_info* 126
- stream_ins_node* 126
- stream_kind* 5, 126–129, 164, 168, 176, 184, 206, 219
- stream_link* 126
- stream_ref* 126, 128, 176
- stream_split* 126
- stream_type* 126
- STREAMDEF** 125, 164
- STRETCH** 189, 191, 231
- Stretch** 31
- stretch** v, 1, 8, 27, 49, 166, 179, 187, 230

stretch 32, 43, 62
stretch.c 230
stretchability 31, 42, 61, 65, 209
STRING 15, 18
string 14, 17, 142
string 18, 130, 150, 168, 174
strlen 151, 191–194
strncmp 137, 192
strncpy 192, 194
strtoul 13, 138, 192
strtol 13
strtol 13
suffix 24
symbol 2

T

tab character 54
tab_skip_no 182
TABLE 83
table 219
table 83
table_kind 5, 83, 205, 219
tables.c 223
TAG 6
tag 7, 226
TAGERR 195
template 121, 125, 129, 184
terminal symbol 3
text 47, 52, 142, 199
text 55–57, 72
time_no 180
TO 65
to 133–135
token 2
tolerance_no 180
TOP 66, 101, 126
top skip 122
top_skip_no 182
top stream 123
total_in 145
total_out 145
true 221
TXT 55
txt 57
TXT_CC 55, 57, 73
txt_cc 54, 59
TXT_END 55, 73
TXT_FONT 55, 57
txt_font 54, 58
TXT_FONT_GLUE 55, 57

TXT_FONT_HYPHEN 55, 57
TXT_GLOBAL 55, 57
txt_global 54, 58, 60
txt_glue 53, 55, 57, 59
txt_hyphen 53, 55, 57, 59
TXT_IGNORE 55, 57
txt_ignore 55, 57, 59
txt_length 54
TXT_LOCAL 55, 57
txt_local 54, 58–60
txt_node 53, 55, 57, 59
TXT_START 55, 73

U

UINT16 221
UINT32 221
UINT64 221
UINT8 221
uint8_t 31
union 229
unit 93–95
UNKNOWN 117
unknown_bytes 117
unknown_kind 6, 118, 170
unknown_node 118
unknown_nodes 117
UNSIGNED 2–4, 13, 21, 49, 73, 75, 83,
86, 99, 113, 117, 126, 130, 149,
164, 174
unsigned 13
USE_MMAP 142
used 99, 104, 106, 109
UTF8 16, 52

V

VBOX 62
vbox_dimen 65
vbox_kind 5, 62–64, 82, 202, 212
vbox_node 62, 69
verbose 229
VERSION_AS_STR 222
vertical box 61
vertical list 38
voidpf 145
VPACK 65
vpack 65
vpack_kind 5, 64–68, 82, 202, 213
VSET 65
vset_kind 5, 64–67, 82, 202, 213

vsize 28

vsize_dimen_no 181

vsize_xdimen_no 182

vxbox_node 66

W

warning 221, 228

whatsit node 121

where 99, 101–106, 108

widow_penalty_no 180

WIDTH 86

WIN32 151, 195, 221, 229, 231

wpx 92–95

wr 86–89, 216, 227

X

xd 29, 86, 229

XDIMEN 29, 82, 164, 168, 176

xdimen 29, 41, 43, 78, 86, 168

xdimen_defaults 86, 222

xdimen_kind 5, 30, 53, 68, 79, 84, 120,
164, 168, 176, 182, 200, 209,
212, 215, 219

xdimen_node 29, 66, 118, 126, 130

xdimen_ref 66, 78, 176

xppm 92

xppu 93–95

xs 154

xsize 144–146, 150, 156, 158, 224, 228

xtof 21

Y

yacc 3

year_no 180

yppm 92

yppu 93–95

yy_pop_state 56

yy_push_state 56

YYDEBUG 196, 230

yydebug 196, 230

yyerror 196

yyin 193, 229

yylex 229

yylineno 56, 109, 196, 228–230

yylval 13, 17, 20, 56

yyout 193, 229

yyparse 140, 229

yyset_debug 229

yytext 3, 13, 18, 20, 56, 228

yywrap 228

Z

Z_DEFAULT_COMPRESSION 145

Z_FINISH 145

Z_OK 145

Z_STREAM_END 145

zalloc 145

zero_baseline_no 183

zero_color_no 185

zero_dimen_no 41, 181

ZERO_GLUE 43–45, 72

zero_glue_no 169

zero_int_no 180

zero_label_no 184

zero_page_no 184

zero_range_no 185

zero_skip_no 44, 163, 177, 182

zero_stream_no 184

zero_xdimen_no 86, 89, 182

zfree 145

zlib 144

